

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 421 — Past Questions & Materials
by Renance

Course: CSC 421
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 8
Levels: 400

This study pack was compiled and restructured by Renance for CSC 421 students at Federal University of Petroleum Resources, Effurun. It brings together 8 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 8 — CSC 421 Database Design Concepts Blessing Okpenihwo 1786954185655 0

Level 400 • Second Semester • PastQuestion

DATABASE DESIGN CONCEPTS

Definitions, Examples and Diagrams

Author: Blessing Okpenihwo

2026

Yes. In fact, adding examples is one of the best ways to remember definitions for exams. Here's the improved version with examples.

1. Database Design

Definition

Database design is the process of planning how data will be stored, organized, and related before creating a database.

Example

Suppose you want to build a University Management System.

Instead of putting everything into one table, you design separate tables:

- Student
- Department
- Course
- Registration
- Result

This planning process is called database design.

2. Database Schema

Definition

A database schema is the logical blueprint that shows the tables, columns, keys, and relationships in a database.

Example

University Database Schema

Figure 1: University Database Schema — table relationships

The diagram above is an example of a database schema.

3. Entity

Definition

An entity is a real-world object or thing about which information is stored in a database.

Examples

- Student
- Lecturer
- Department
- Course
- Library Book

Each entity becomes a table.

4. Attribute

Definition

An attribute is a characteristic or property of an entity.

Example

Entity: Student

Attributes:

- Matric Number
- Name
- Gender
- Email
- Phone Number
- Level

Figure 2: The Student entity and its attributes

5. Primary Key

Definition

A primary key is an attribute that uniquely identifies every record in a table.

Example

Matric No Name

CSC001 John

CSC002 Mary

CSC003 David

Matric Number is the Primary Key because no two students have the same matric number.

Other examples:

- Department_ID
- Course_Code
- Registration_ID
- Result_ID

6. Foreign Key

Definition

A foreign key is a primary key copied from another table to create a relationship between tables.

Example

Department Table

Department_ID Department

CSC Computer Science

Student Table

Matric No Name Department_ID

CSC001 John CSC

The Department_ID inside the Student table is a Foreign Key.

Figure 3: Primary Key and Foreign Key relationship between Department and Student tables

7. Relationship

Definition

A relationship is a connection between two or more tables.

Examples

One Department → Many Students

Computer Science

- John
- Mary
- James

Figure 4: One-to-Many relationship — one Department, many Students

One Student → Many Courses

John registers:

- CSC421
- CSC304
- GST111

8. Data Redundancy

Definition

Data redundancy is the unnecessary repetition of the same information in different records.

Example

Student Department

John Computer Science

Mary Computer Science

David Computer Science

"Computer Science" is repeated three times.

That is data redundancy.

9. Functional Dependency

Definition

Functional dependency is a relationship where one attribute determines another attribute.

Examples

Matric_No → Student_Name

Knowing CSC001 tells us the student's name is John.

Department_ID → Department_Name

Knowing CSC tells us the department is Computer Science.

Course_Code → Course_Title

Knowing CSC421 tells us the course is Data Management II.

10. Update Anomaly

Definition

Update anomaly occurs when the same information must be updated in many places because it is repeated.

Example

Old HOD:

Prof. David

New HOD:

Prof. Musa

If there are 2,000 students, you must change the HOD's name in 2,000 records.

Missing one record creates inconsistent data.

11. Insertion Anomaly

Definition

Insertion anomaly occurs when you cannot insert valid information because some other information is missing.

Example

FUPRE creates a new Department of Artificial Intelligence.

No students have been admitted.

Since the department is stored only with student records, you cannot add the department until a student exists.

12. Deletion Anomaly

Definition

Deletion anomaly occurs when deleting one record accidentally deletes other important information.

Example

John is the only student in Computer Science.

If John's record is deleted after graduation,
the database also loses:

- Computer Science department
- HOD
- Office
- Telephone number

That is a deletion anomaly.

13. Normalization

Definition

Normalization is the process of organizing database tables to reduce redundancy and eliminate anomalies.

Example

Before normalization

One table contains:

- Student
- Department
- HOD
- Course
- Result

Everything is mixed together.

After normalization

The database is split into:

- Student Table
- Department Table
- Course Table
- Registration Table
- Result Table

This removes repeated data and makes the database easier to maintain.

Figure 5: Normalization — splitting one table into several

Exam Tip (Easy Way to Remember)

Term Easy Memory Trick

Database Design Planning before building

Schema Blueprint of the database

Entity A thing (Student, Course)

Attribute Information about the thing

Primary Key Unique ID

Term Easy Memory Trick

Foreign Key Link to another table

Relationship Connection between tables

Data Redundancy Repeating the same data

Functional Dependency One value determines another

Update Anomaly Update many records

Insertion Anomaly Can't add new data

Deletion Anomaly Lose important data when deleting

Normalization Split tables to remove repetition

This format (definition → example) is often the easiest to remember and is well suited for exam preparation.

Paper 2 of 8 — Csc 421

Level 400 • Second Semester • PastQuestion

part 3

The Relational Data

Model and SQL

This page intentionally left blank

chapter 5

The Relational Data Model and

Relational Database Constraints

This chapter opens Part 3 of the book, which covers relational databases. The relational data model was first introduced by Ted Codd of IBM Research in 1970 in a classic paper (Codd, 1970), and it attracted immediate attention due to its simplicity and mathematical foundation. The model uses the concept of a mathematical relation—which looks somewhat like a table of values—as its basic building block, and has its theoretical basis in set theory and first-order predicate logic. In this chapter we discuss the basic characteristics of the model and its constraints.

The first commercial implementations of the relational model became available in the early 1980s, such as the SQL/DS system on the MVS operating system by IBM and the Oracle DBMS. Since then, the model has been implemented in a large number of commercial systems, as well as a number of open source systems. Current popular commercial relational DBMSs (RDBMSs) include DB2 (from IBM), Oracle (from Oracle), Sybase DBMS (now from SAP), and SQLServer and Microsoft Access (from Microsoft). In addition, several open source systems, such as MySQL and PostgreSQL, are available.

Because of the importance of the relational model, all of Part 2 is devoted to this model and some of the languages associated with it. In Chapters 6 and 7, we describe some aspects of SQL, which is a comprehensive model and language that is the standard for commercial relational DBMSs. (Additional aspects of SQL will be covered in other chapters.) Chapter 8 covers the operations of the relational algebra and introduces the relational calculus—these are two formal languages associated with the relational model. The relational calculus is considered to be the basis for the SQL language, and the relational algebra is used in the internals of many database implementations for query processing and optimization (see Part 8 of the book).

179

180 Chapter 5 The Relational Data Model and Relational Database Constraints

Other features of the relational model are presented in subsequent parts of the book. Chapter 9 relates the relational model data structures to the constructs of the ER and EER models (presented in Chapters 3 and 4), and presents algorithms for designing a relational database schema by mapping a conceptual schema in the ER

or EER model into a relational representation. These mappings are incorporated into many database design and CASE1 tools. Chapters 10 and 11 in Part 4 discuss the programming techniques used to access database systems and the notion of connecting to relational databases via ODBC and JDBC standard protocols. We also introduce the topic of Web database programming in Chapter 11. Chapters 14 and 15 in Part 6 present another aspect of the relational model, namely the formal constraints of functional and multivalued dependencies; these dependencies are used to develop a relational database design theory based on the concept known as normalization.

In this chapter, we concentrate on describing the basic principles of the relational model of data. We begin by defining the modeling concepts and notation of the relational model in Section 5.1. Section 5.2 is devoted to a discussion of relational constraints that are considered an important part of the relational model and are automatically enforced in most relational DBMSs. Section 5.3 defines the update operations of the relational model, discusses how violations of integrity constraints are handled, and introduces the concept of a transaction. Section 5.4 summarizes the chapter.

This chapter and Chapter 8 focus on the formal foundations of the relational model, whereas Chapters 6 and 7 focus on the SQL practical relational model, which is the basis of most commercial and open source relational DBMSs. Many concepts are common between the formal and practical models, but a few differences exist that we shall point out.

5.1 Relational Model Concepts

The relational model represents the database as a collection of relations. Informally, each relation resembles a table of values or, to some extent, a flat file of records. It is called a flat file because each record has a simple linear or flat structure. For example, the database of files that was shown in Figure 1.2 is similar to the basic relational model representation. However, there are important differences between relations and files, as we shall soon see.

When a relation is thought of as a table of values, each row in the table represents a collection of related data values. A row represents a fact that typically corresponds to a real-world entity or relationship. The table name and column names are used to help to interpret the meaning of the values in each row. For example, the first table of Figure 1.2 is called STUDENT because each row represents facts about a particular student entity. The column names—Name, Student_number,

1

CASE stands for computer-aided software engineering.

5.1 Relational Model Concepts 181

Class, and Major—specify how to interpret the data values in each row, based on the column each value is in. All values in a column are of the same data type.

In the formal relational model terminology, a row is called a tuple, a column

header is called an attribute, and the table is called a relation. The data type describing the types of values that can appear in each column is represented by a domain of possible values. We now define these terms—domain, tuple, attribute, and relation—formally.

5.1.1 Domains, Attributes, Tuples, and Relations

A domain D is a set of atomic values. By atomic we mean that each value in the domain is indivisible as far as the formal relational model is concerned. A common method of specifying a domain is to specify a data type from which the data values forming the domain are drawn. It is also useful to specify a name for the domain, to help in interpreting its values. Some examples of domains follow:

■ ■ **Usa_phone_numbers.** The set of ten-digit phone numbers valid in the United States.

■ ■ **Local_phone_numbers.** The set of seven-digit phone numbers valid within a particular area code in the United States. The use of local phone numbers is quickly becoming obsolete, being replaced by standard ten-digit numbers.

■ ■ **Social_security_numbers.** The set of valid nine-digit Social Security numbers. (This is a unique identifier assigned to each person in the United States for employment, tax, and benefits purposes.)

■ ■ **Names:** The set of character strings that represent names of persons.

■ ■ **Grade_point_averages.** Possible values of computed grade point averages; each must be a real (floating-point) number between 0 and 4.

■ ■ **Employee_ages.** Possible ages of employees in a company; each must be an integer value between 15 and 80.

■ ■ **Academic_department_names.** The set of academic department names in a university, such as Computer Science, Economics, and Physics.

■ ■ **Academic_department_codes.** The set of academic department codes, such as 'CS', 'ECON', and 'PHYS'.

The preceding are called logical definitions of domains. A data type or format is also specified for each domain. For example, the data type for the domain **Usa_phone_numbers** can be declared as a character string of the form (ddd)ddd-dddd, where each d is a numeric (decimal) digit and the first three digits form a valid telephone area code. The data type for **Employee_ages** is an integer number between 15 and 80. For **Academic_department_names**, the data type is the set of all character strings that represent valid department names. A domain is thus given a name, data type, and format. Additional information for interpreting the values of a domain can also be given; for example, a numeric domain such as **Person_weights** should have the units of measurement, such as pounds or kilograms.

182 Chapter 5 The Relational Data Model and Relational Database Constraints

A relation schema² R , denoted by $R(A_1, A_2, \dots, A_n)$, is made up of a relation name R and a list of attributes, $A_1, A_2, \dots, A_n$. Each attribute A_i is the name of a role played by some domain D in the relation schema R . D is called the domain of A_i

and is denoted by $\text{dom}(A_i)$. A relation schema is used to describe a relation; R is called the name of this relation. The degree (or arity) of a relation is the number of attributes n of its relation schema.

A relation of degree seven, which stores information about university students, would contain seven attributes describing each student as follows:

STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)

Using the data type of each attribute, the definition is sometimes written as:

STUDENT(Name: string, Ssn: string, Home_phone: string, Address: string, Office_phone: string, Age: integer, Gpa: real)

For this relation schema, STUDENT is the name of the relation, which has seven attributes. In the preceding definition, we showed assignment of generic types such as string or integer to the attributes. More precisely, we can specify the following previously defined domains for some of the attributes of the STUDENT relation: $\text{dom}(\text{Name}) = \text{Names}$; $\text{dom}(\text{Ssn}) = \text{Social_security_numbers}$; $\text{dom}(\text{HomePhone}) = \text{USA_phone_numbers3}$, $\text{dom}(\text{Office_phone}) = \text{USA_phone_numbers}$, and $\text{dom}(\text{Gpa}) = \text{Grade_point_averages}$. It is also possible to refer to attributes of a relation schema by their position within the relation; thus, the second attribute of the STUDENT relation is Ssn, whereas the fourth attribute is Address.

A relation (or relation state)⁴ r of the relation schema $R(A_1, A_2, \dots, A_n)$, also denoted by $r(R)$, is a set of n -tuples $r = \{t_1, t_2, \dots, t_m\}$. Each n -tuple t is an ordered list of n values $t = \langle v_1, v_2, \dots, v_n \rangle$, where each value v_i , $1 \leq i \leq n$, is an element of $\text{dom}(A_i)$ or is a special NULL value. (NULL values are discussed further below and in Section 5.1.2.) The i th value in tuple t , which corresponds to the attribute A_i , is referred to as $t[A_i]$ or $t.A_i$ (or $t[i]$ if we use the positional notation). The terms relation intension for the schema R and relation extension for a relation state $r(R)$ are also commonly used. Figure 5.1 shows an example of a STUDENT relation, which corresponds to the STUDENT schema just specified. Each tuple in the relation represents a particular student entity (or object). We display the relation as a table, where each tuple is shown as a row and each attribute corresponds to a column header indicating a role or interpretation of the values in that column. NULL values represent attributes whose values are unknown or do not exist for some individual STUDENT tuple.

2

A relation schema is sometimes called a relation scheme.

3

With the large increase in phone numbers caused by the proliferation of mobile phones, most metropol-

itan areas in the United States now have multiple area codes, so seven-digit local dialing has been discontinued in most areas. We changed this domain to Usa_phone_numbers instead of Local_phone_

numbers, which would be a more general choice. This illustrates how database requirements can change

over time.

4

This has also been called a relation instance. We will not use this term because instance is also used

to refer to a single tuple or row.

5.1 Relational Model Concepts 183

Relation Name Attributes

STUDENT

Name Ssn Home_phone Address Office_phone Age Gpa

Benjamin Bayer 305-61-2435 (817)373-1616 2918 Bluebonnet Lane NULL 19 3.21

Chung-cha Kim 381-62-1245 (817)375-4409 125 Kirby Road NULL 18 2.89

Tuples Dick Davidson 422-11-2320 NULL 3452 Elgin Road (817)749-1253 25 3.53

Rohan Panchal 489-22-1100 (817)376-9821 265 Lark Lane (817)749-6492 28 3.93

Barbara Benson 533-69-1238 (817)839-8461 7384 Fontana Lane NULL 19 3.25

Figure 5.1

The attributes and tuples of a relation STUDENT.

The earlier definition of a relation can be restated more formally using set theory concepts as follows. A relation (or relation state) $r(R)$ is a mathematical relation of degree n on the domains $\text{dom}(A_1)$, $\text{dom}(A_2)$, ..., $\text{dom}(A_n)$, which is a subset of the Cartesian product (denoted by $\times$) of the domains that define R :

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

The Cartesian product specifies all possible combinations of values from the underlying domains. Hence, if we denote the total number of values, or cardinality, in a domain D by $|D|$ (assuming that all domains are finite), the total number of tuples in the Cartesian product is

$$|\text{dom}(A_1)| \times |\text{dom}(A_2)| \times \dots \times |\text{dom}(A_n)|$$

This product of cardinalities of all domains represents the total number of possible instances or tuples that can ever exist in any relation state $r(R)$. Of all these possible combinations, a relation state at a given time—the current relation state—reflects only the valid tuples that represent a particular state of the real world. In general, as the state of the real world changes, so does the relation state, by being transformed into another relation state. However, the schema R is relatively static and changes very infrequently—for example, as a result of adding an attribute to represent new information that was not originally stored in the relation.

It is possible for several attributes to have the same domain. The attribute names indicate different roles, or interpretations, for the domain. For example, in the STUDENT relation, the same domain `USA_phone_numbers` plays the role of `Home_phone`, referring to the home phone of a student, and the role of `Office_phone`, referring to the office phone of the student. A third possible attribute (not shown) with the same domain could be `Mobile_phone`.

5.1.2 Characteristics of Relations

The earlier definition of relations implies certain characteristics that make a relation different from a file or a table. We now discuss some of these characteristics.

184 Chapter 5 The Relational Data Model and Relational Database Constraints

Ordering of Tuples in a Relation. A relation is defined as a set of tuples. Mathematically, elements of a set have no order among them; hence, tuples in a relation do not have any particular order. In other words, a relation is not sensitive to the ordering of tuples. However, in a file, records are physically stored on disk (or in memory), so there always is an order among the records. This ordering indicates first, second, *i*th, and last records in the file. Similarly, when we display a relation as a table, the rows are displayed in a certain order.

Tuple ordering is not part of a relation definition because a relation attempts to represent facts at a logical or abstract level. Many tuple orders can be specified on the same relation. For example, tuples in the STUDENT relation in Figure 5.1 could be ordered by values of Name, Ssn, Age, or some other attribute. The definition of a relation does not specify any order: There is no preference for one ordering over another. Hence, the relation displayed in Figure 5.2 is considered identical to the one shown in Figure 5.1. When a relation is implemented as a file or displayed as a table, a particular ordering may be specified on the records of the file or the rows of the table.

Ordering of Values within a Tuple and an Alternative Definition of a Relation.

According to the preceding definition of a relation, an *n*-tuple is an ordered list of *n* values, so the ordering of values in a tuple—and hence of attributes in a relation schema—is important. However, at a more abstract level, the order of attributes and their values is not that important as long as the correspondence between attributes and values is maintained.

An alternative definition of a relation can be given, making the ordering of values in a tuple unnecessary. In this definition, a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes (instead of an ordered list of attributes), and a relation state $r(R)$ is a finite set of mappings $r = \{t_1, t_2, \dots, t_m\}$, where each tuple t_i is a mapping from R to D , and D is the union (denoted by $\cup$) of the attribute domains; that is, $D = \text{dom}(A_1) \cup \text{dom}(A_2) \cup \dots \cup \text{dom}(A_n)$. In this definition, $t[A_i]$ must be in $\text{dom}(A_i)$ for $1 \leq i \leq n$ for each mapping t in r . Each mapping t_i is called a tuple.

According to this definition of tuple as a mapping, a tuple can be considered as a set of (*<attribute>*, *<value>*) pairs, where each pair gives the value of the mapping from an attribute A_i to a value v_i from $\text{dom}(A_i)$. The ordering of attributes is not important, because the attribute name appears with its value. By this definition, the

Figure 5.2

The relation STUDENT from Figure 5.1 with a different order of tuples.

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53

Barbara Benson 533-69-1238 (817)839-8461 7384 Fontana Lane NULL 19 3.25
 Rohan Panchal 489-22-1100 (817)376-9821 265 Lark Lane (817)749-6492 28 3.93
 Chung-cha Kim 381-62-1245 (817)375-4409 125 Kirby Road NULL 18 2.89
 Benjamin Bayer 305-61-2435 (817)373-1616 2918 Bluebonnet Lane NULL 19 3.21

5.1 Relational Model Concepts 185

$t = \langle (\text{Name, Dick Davidson}), (\text{Ssn, 422-11-2320}), (\text{Home_phone, NULL}), (\text{Address, 3452 Elgin Road}), (\text{Office_phone, (817)749-1253}), (\text{Age, 25}), (\text{Gpa, 3.53}) \rangle$

$t = \langle (\text{Address, 3452 Elgin Road}), (\text{Name, Dick Davidson}), (\text{Ssn, 422-11-2320}), (\text{Age, 25}), (\text{Office_phone, (817)749-1253}), (\text{Gpa, 3.53}), (\text{Home_phone, NULL}) \rangle$

Figure 5.3

Two identical tuples when the order of attributes and values is not part of relation definition.

two tuples shown in Figure 5.3 are identical. This makes sense at an abstract level, since there really is no reason to prefer having one attribute value appear before another in a tuple. When the attribute name and value are included together in a tuple, it is known as self-describing data, because the description of each value (attribute name) is included in the tuple.

We will mostly use the first definition of relation, where the attributes are ordered in the relation schema and the values within tuples are similarly ordered, because it simplifies much of the notation. However, the alternative definition given here is more general.⁵

Values and NULLs in the Tuples. Each value in a tuple is an atomic value; that is, it is not divisible into components within the framework of the basic relational model. Hence, composite and multivalued attributes (see Chapter 3) are not allowed. This model is sometimes called the flat relational model. Much of the theory behind the relational model was developed with this assumption in mind, which is called the first normal form assumption.⁶ Hence, multivalued attributes must be represented by separate relations, and composite attributes are represented only by their simple component attributes in the basic relational model.⁷

An important concept is that of NULL values, which are used to represent the values of attributes that may be unknown or may not apply to a tuple. A special value, called NULL, is used in these cases. For example, in Figure 5.1, some STUDENT tuples have NULL for their office phones because they do not have an office (that is, office phone does not apply to these students). Another student has a NULL for home phone, presumably because either he does not have a home phone or he has one but we do not know it (value is unknown). In general, we can have several meanings for NULL values, such as value unknown, value exists but is not available, or attribute does not apply to this tuple (also known as value undefined). An example of the last type of NULL will occur if we add an attribute Visa_status to the STUDENT relation that applies only to tuples representing foreign students. It is possible to devise different codes for different meanings of

We will use the alternative definition of relation when we discuss query processing and optimization in

Chapter 18.

6

We discuss this assumption in more detail in Chapter 14.

7

Extensions of the relational model remove these restrictions. For example, object-relational systems

(Chapter 12) allow complex-structured attributes, as do the non-first normal form or nested relational models.

186 Chapter 5 The Relational Data Model and Relational Database Constraints

NULL values. Incorporating different types of NULL values into relational model operations has proven difficult and is outside the scope of our presentation.

The exact meaning of a NULL value governs how it fares during arithmetic aggregations or comparisons with other values. For example, a comparison of two NULL values leads to ambiguities—if both Customer A and B have NULL addresses, it does not mean they have the same address. During database design, it is best to avoid NULL values as much as possible. We will discuss this further in Chapters 7 and 8 in the context of operations and queries, and in Chapter 14 in the context of database design and normalization.

Interpretation (Meaning) of a Relation. The relation schema can be interpreted as a declaration or a type of assertion. For example, the schema of the STUDENT relation of Figure 5.1 asserts that, in general, a student entity has a Name, Ssn, Home_phone, Address, Office_phone, Age, and Gpa. Each tuple in the relation can then be interpreted as a fact or a particular instance of the assertion. For example, the first tuple in Figure 5.1 asserts the fact that there is a STUDENT whose Name is Benjamin Bayer, Ssn is 305-61-2435, Age is 19, and so on.

Notice that some relations may represent facts about entities, whereas other relations may represent facts about relationships. For example, a relation schema MAJORS (Student_ssn, Department_code) asserts that students major in academic disciplines. A tuple in this relation relates a student to his or her major discipline. Hence, the relational model represents facts about both entities and relationships uniformly as relations. This sometimes compromises understandability because one has to guess whether a relation represents an entity type or a relationship type. We introduced the entity-relationship (ER) model in detail in Chapter 3, where the entity and relationship concepts were described in detail. The mapping procedures in Chapter 9 show how different constructs of the ER/EER conceptual data models (see Part 2) get converted to relations.

An alternative interpretation of a relation schema is as a predicate; in this case, the values in each tuple are interpreted as values that satisfy the predicate. For example,

the predicate STUDENT (Name, Ssn, ...) is true for the five tuples in relation STUDENT of Figure 5.1. These tuples represent five different propositions or facts in the real world. This interpretation is quite useful in the context of logical programming languages, such as Prolog, because it allows the relational model to be used within these languages (see Section 26.5). An assumption called the closed world assumption states that the only true facts in the universe are those present within the extension (state) of the relation(s). Any other combination of values makes the predicate false. This interpretation is useful when we consider queries on relations based on relational calculus in Section 8.6.

5.1.3 Relational Model Notation

We will use the following notation in our presentation:

■ ■ A relation schema R of degree n is denoted by $R(A_1, A_2, \dots, A_n)$.

5.2 Relational Model Constraints and Relational Database Schemas 187

■ ■ The uppercase letters Q, R, S denote relation names.

■ ■ The lowercase letters q, r, s denote relation states.

■ ■ The letters t, u, v denote tuples.

■ ■ In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation—the current relation state—whereas STUDENT(Name, Ssn, ...) refers only to the relation schema.

■ ■ An attribute A can be qualified with the relation name R to which it belongs by using the dot notation $R.A$ —for example, STUDENT.Name or STUDENT.Age. This is because the same name may be used for two attributes in different relations. However, all attribute names in a particular relation must be distinct.

■ ■ An n -tuple t in a relation $r(R)$ is denoted by $t = \langle v_1, v_2, \dots, v_n \rangle$, where v_i is the value corresponding to attribute A_i . The following notation refers to component values of tuples:

Both $t[A_i]$ and $t.A_i$ (and sometimes $t[i]$) refer to the value v_i in t for attribute A_i .

Both $t[A_u, A_w, \dots, A_z]$ and $t.(A_u, A_w, \dots, A_z)$, where $A_u, A_w, \dots, A_z$ is a list of attributes from R , refer to the subtuple of values $\langle v_u, v_w, \dots, v_z \rangle$ from t corresponding to the attributes specified in the list.

As an example, consider the tuple $t = \langle \text{'Barbara Benson'}, \text{'533-69-1238'}, \text{'(817)839-8461'}, \text{'7384 Fontana Lane'}, \text{NULL}, 19, 3.25 \rangle$ from the STUDENT relation in Figure 5.1; we have $t[\text{Name}] = \langle \text{'Barbara Benson'} \rangle$, and $t[\text{Ssn}, \text{Gpa}, \text{Age}] = \langle \text{'533-69-1238'}, 3.25, 19 \rangle$.

5.2 Relational Model Constraints and Relational Database Schemas

So far, we have discussed the characteristics of single relations. In a relational database, there will typically be many relations, and the tuples in those relations are

usually related in various ways. The state of the whole database will correspond to the states of all its relations at a particular point in time. There are generally many restrictions or constraints on the actual values in a database state. These constraints are derived from the rules in the miniworld that the database represents, as we discussed in Section 1.6.8.

In this section, we discuss the various restrictions on data that can be specified on a relational database in the form of constraints. Constraints on databases can generally be divided into three main categories:

1. Constraints that are inherent in the data model. We call these inherent model-based constraints or implicit constraints.
2. Constraints that can be directly expressed in the schemas of the data model, typically by specifying them in the DDL (data definition language, see Section 2.3.1). We call these schema-based constraints or explicit constraints.

188 Chapter 5 The Relational Data Model and Relational Database Constraints

3. Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs or in some other way. We call these application-based or semantic constraints or business rules.

The characteristics of relations that we discussed in Section 5.1.2 are the inherent constraints of the relational model and belong to the first category. For example, the constraint that a relation cannot have duplicate tuples is an inherent constraint. The constraints we discuss in this section are of the second category, namely, constraints that can be expressed in the schema of the relational model via the DDL. Constraints in the third category are more general, relate to the meaning as well as behavior of attributes, and are difficult to express and enforce within the data model, so they are usually checked within the application programs that perform database updates. In some cases, these constraints can be specified as assertions in SQL (see Chapter 7). Another important category of constraints is data dependencies, which include functional dependencies and multivalued dependencies. They are used mainly for testing the “goodness” of the design of a relational database and are utilized in a process called normalization, which is discussed in Chapters 14 and 15.

The schema-based constraints include domain constraints, key constraints, constraints on NULLs, entity integrity constraints, and referential integrity constraints.

5.2.1 Domain Constraints

Domain constraints specify that within each tuple, the value of each attribute A must be an atomic value from the domain $\text{dom}(A)$. We have already discussed the ways in which domains can be specified in Section 5.1.1. The data types associated with domains typically include standard numeric data types for integers (such as short integer, integer, and long integer) and real numbers (float and double-precision float). Characters, Booleans, fixed-length strings, and variable-length strings are also available, as are date, time, timestamp, and other special data types. Domains can also be

described by a subrange of values from a data type or as an enumerated data type in which all possible values are explicitly listed. Rather than describe these in detail here, we discuss the data types offered by the SQL relational standard in Section 6.1.

5.2.2 Key Constraints and Constraints on NULL Values

In the formal relational model, a relation is defined as a set of tuples. By definition, all elements of a set are distinct; hence, all tuples in a relation must also be distinct. This means that no two tuples can have the same combination of values for all their attributes. Usually, there are other subsets of attributes of a relation schema R with the property that no two tuples in any relation state r of R should have the same combination of values for these attributes. Suppose that we denote one such subset of attributes by SK ; then for any two distinct tuples t_1 and t_2 in a relation state r of R , we have the constraint that:

$$t_1[SK] \neq t_2[SK]$$

5.2 Relational Model Constraints and Relational Database Schemas 189

Any such set of attributes SK is called a superkey of the relation schema R . A superkey SK specifies a uniqueness constraint that no two distinct tuples in any state r of R can have the same value for SK . Every relation has at least one default superkey—the set of all its attributes. A superkey can have redundant attributes, however, so a more useful concept is that of a key, which has no redundancy. A key k of a relation schema R is a superkey of R with the additional property that removing any attribute A from K leaves a set of attributes K' that is not a superkey of R any more.

Hence, a key satisfies two properties:

1. Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key. This uniqueness property also applies to a superkey.
2. It is a minimal superkey—that is, a superkey from which we cannot remove any attributes and still have the uniqueness constraint hold. This minimality property is required for a key but is optional for a superkey.

Hence, a key is a superkey but not vice versa. A superkey may be a key (if it is minimal) or may not be a key (if it is not minimal). Consider the STUDENT relation of Figure 5.1. The attribute set $\{Ssn\}$ is a key of STUDENT because no two student tuples can have the same value for Ssn .⁸ Any set of attributes that includes Ssn —for example, $\{Ssn, Name, Age\}$ —is a superkey. However, the superkey $\{Ssn, Name, Age\}$ is not a key of STUDENT because removing $Name$ or Age or both from the set still leaves us with a superkey. In general, any superkey formed from a single attribute is also a key. A key with multiple attributes must require all its attributes together to have the uniqueness property.

The value of a key attribute can be used to identify uniquely each tuple in the relation. For example, the Ssn value 305-61-2435 identifies uniquely the tuple corresponding to Benjamin Bayer in the STUDENT relation. Notice that a set of attributes constituting a key is a property of the relation schema; it is a constraint that should

hold on every valid relation state of the schema. A key is determined from the meaning of the attributes, and the property is time-invariant: It must continue to hold when we insert new tuples in the relation. For example, we cannot and should not designate the Name attribute of the STUDENT relation in Figure 5.1 as a key because it is possible that two students with identical names will exist at some point in a valid state.⁹

In general, a relation schema may have more than one key. In this case, each of the keys is called a candidate key. For example, the CAR relation in Figure 5.4 has two candidate keys: License_number and Engine_serial_number. It is common to designate one of the candidate keys as the primary key of the relation. This is the candidate key whose values are used to identify tuples in the relation. We use the convention that the attributes that form the primary key of a relation schema are underlined, as shown in Figure 5.4. Notice that when a relation schema has several candidate keys,

Note that Ssn is also a superkey.

Names are sometimes used as keys, but then some artifact—such as appending an ordinal number—must

be used to distinguish between persons with identical names.

190 Chapter 5 The Relational Data Model and Relational Database Constraints

CAR

License_number Engine_serial_number Make Model Year

Texas ABC-739 A69352 Ford Mustang 02

Florida TVP-347 B43696 Oldsmobile Cutlass 05

Figure 5.4 New York MPO-22 X83554 Oldsmobile Delta 01

The CAR relation, with California 432-TFY C43742 Mercedes 190-D 99

two candidate keys: California RSK-629 Y82935 Toyota Camry 04

License_number and

Engine_serial_number. Texas RSK-629 U028365 Jaguar XJS 04

the choice of one to become the primary key is somewhat arbitrary; however, it is usually better to choose a primary key with a single attribute or a small number of attributes. The other candidate keys are designated as unique keys and are not underlined.

Another constraint on attributes specifies whether NULL values are or are not permitted. For example, if every STUDENT tuple must have a valid, non-NULL value for the Name attribute, then Name of STUDENT is constrained to be NOT NULL.

5.2.3 Relational Databases and Relational

Database Schemas

The definitions and constraints we have discussed so far apply to single relations and their attributes. A relational database usually contains many relations, with tuples in relations that are related in various ways. In this section, we define a rela-

tional database and a relational database schema.

A relational database schema S is a set of relation schemas $S = \{R_1, R_2, \dots, R_m\}$ and a set of integrity constraints IC . A relational database state DB of S is a set of relation states $DB = \{r_1, r_2, \dots, r_m\}$ such that each r_i is a state of R_i and such that the r_i relation states satisfy the integrity constraints specified in IC . Figure 5.5 shows a relational database schema that we call $COMPANY = \{EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT\}$. In each relation schema, the underlined attribute represents the primary key. Figure 5.6 shows a relational database state corresponding to the $COMPANY$ schema. We will use this schema and database state in this chapter and in Chapters 4 through 6 for developing sample queries in different relational languages. (The data shown here is expanded and available for loading as a populated database from the Companion Website for the text, and can be used for the hands-on project exercises at the end of the chapters.)

When we refer to a relational database, we implicitly include both its schema and its current state. A database state that does not obey all the integrity constraints is

10

A relational database state is sometimes called a relational database snapshot or instance. However,

as we mentioned earlier, we will not use the term instance since it also applies to single tuples.

5.2 Relational Model Constraints and Relational Database Schemas 191

EMPLOYEE

Fname Minit Lname Ssn Bdate Address Sex Salary Super_ssn Dno

DEPARTMENT

Dname Dnumber Mgr_ssn Mgr_start_date

DEPT_LOCATIONS

Dnumber Dlocation

PROJECT

Pname Pnumber Plocation Dnum

WORKS_ON

Essn Pno Hours

Figure 5.5

DEPENDENT Schema diagram for the
COMPANY relational

Essn Dependent_name Sex Bdate Relationship database schema.

called not valid, and a state that satisfies all the constraints in the defined set of integrity constraints IC is called a valid state.

In Figure 5.5, the $Dnumber$ attribute in both $DEPARTMENT$ and $DEPT_LOCATIONS$ stands for the same real-world concept—the number given to a department. That same concept is called Dno in $EMPLOYEE$ and $Dnum$ in $PROJECT$. Attributes that

represent the same real-world concept may or may not have identical names in different relations. Alternatively, attributes that represent different concepts may have the same name in different relations. For example, we could have used the attribute name Name for both Pname of PROJECT and Dname of DEPARTMENT; in this case, we would have two attributes that share the same name but represent different real-world concepts—project names and department names.

In some early versions of the relational model, an assumption was made that the same real-world concept, when represented by an attribute, would have identical attribute names in all relations. This creates problems when the same real-world concept is used in different roles (meanings) in the same relation. For example, the concept of Social Security number appears twice in the EMPLOYEE relation of Figure 5.5: once in the role of the employee's SSN, and once in the role of the supervisor's SSN. We are required to give them distinct attribute names—Ssn and Super_ssn, respectively—because they appear in the same relation and in order to distinguish their meaning.

Each relational DBMS must have a data definition language (DDL) for defining a relational database schema. Current relational DBMSs are mostly using SQL for this purpose. We present the SQL DDL in Sections 6.1 and 6.2.

192 Chapter 5 The Relational Data Model and Relational Database Constraints

Figure 5.6

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT DEPT_LOCATIONS

Dname	Dnumber	Mgr_ssn	Mgr_start_date	Dnumber	Dlocation
Research	5	333445555	1988-05-22	1	Houston
Administration	4	987654321	1995-01-01	4	Stafford
Headquarters	1	888665555	1981-06-19	5	Bellaire
	5				Sugarland
	5				Houston

WORKS_ON PROJECT

Essn	Pno	Hours	Pname	Pnumber	Plocation	Dnum
123456789	1	32.5	ProductX	1	Bellaire	5

```

123456789 2 7.5 ProductY 2 Sugarland 5
666884444 3 40.0 ProductZ 3 Houston 5
453453453 1 20.0 Computerization 10 Stafford 4
453453453 2 20.0 Reorganization 20 Houston 1
333445555 2 10.0 Newbenefits 30 Stafford 4
333445555 3 10.0
333445555 10 10.0 DEPENDENT
333445555 20 10.0 Essn Dependent_name Sex Bdate Relationship
999887777 30 30.0 333445555 Alice F 1986-04-05 Daughter
999887777 10 10.0 333445555 Theodore M 1983-10-25 Son
987987987 10 35.0 333445555 Joy F 1958-05-03 Spouse
987987987 30 5.0 987654321 Abner M 1942-02-28 Spouse
987654321 30 20.0 123456789 Michael M 1988-01-04 Son
987654321 20 15.0 123456789 Alice F 1988-12-30 Daughter
888665555 20 NULL 123456789 Elizabeth F 1967-05-05 Spouse

```

5.2 Relational Model Constraints and Relational Database Schemas 193

Integrity constraints are specified on a database schema and are expected to hold on every valid database state of that schema. In addition to domain, key, and NOT NULL constraints, two other types of constraints are considered part of the relational model: entity integrity and referential integrity.

5.2.4 Entity Integrity, Referential Integrity, and Foreign Keys

The entity integrity constraint states that no primary key value can be NULL. This is because the primary key value is used to identify individual tuples in a relation. Having NULL values for the primary key implies that we cannot identify some tuples. For example, if two or more tuples had NULL for their primary keys, we may not be able to distinguish them if we try to reference them from other relations.

Key constraints and entity integrity constraints are specified on individual relations. The referential integrity constraint is specified between two relations and is used to maintain the consistency among tuples in the two relations. Informally, the referential integrity constraint states that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation. For example, in Figure 5.6, the attribute Dno of EMPLOYEE gives the department number for which each employee works; hence, its value in every EMPLOYEE tuple must match the Dnumber value of some tuple in the DEPARTMENT relation.

To define referential integrity more formally, first we define the concept of a foreign key. The conditions for a foreign key, given below, specify a referential integrity constraint between the two relation schemas R1 and R2. A set of attributes FK in relation schema R1 is a foreign key of R1 that references relation R2 if it satisfies the following rules:

1. The attributes in FK have the same domain(s) as the primary key attributes PK of R2; the attributes FK are said to reference or refer to the relation R2.

2. A value of FK in a tuple t_1 of the current state $r_1(R_1)$ either occurs as a value of PK for some tuple t_2 in the current state $r_2(R_2)$ or is NULL. In the former case, we have $t_1[FK] = t_2[PK]$, and we say that the tuple t_1 references or refers to the tuple t_2 .

In this definition, R_1 is called the referencing relation and R_2 is the referenced relation. If these two conditions hold, a referential integrity constraint from R_1 to R_2 is said to hold. In a database of many relations, there are usually many referential integrity constraints.

To specify these constraints, first we must have a clear understanding of the meaning or role that each attribute or set of attributes plays in the various relation schemas of the database. Referential integrity constraints typically arise from the relationships among the entities represented by the relation schemas. For example, consider the database shown in Figure 5.6. In the EMPLOYEE relation, the attribute Dno refers to the department for which an employee works; hence, we designate Dno to be a foreign key of EMPLOYEE referencing the DEPARTMENT relation. This means that a value of Dno in any tuple t_1 of the EMPLOYEE relation must match a value of

194 Chapter 5 The Relational Data Model and Relational Database Constraints

the primary key of DEPARTMENT—the Dnumber attribute—in some tuple t_2 of the DEPARTMENT relation, or the value of Dno can be NULL if the employee does not belong to a department or will be assigned to a department later. For example, in Figure 5.6 the tuple for employee ‘John Smith’ references the tuple for the ‘Research’ department, indicating that ‘John Smith’ works for this department.

Notice that a foreign key can refer to its own relation. For example, the attribute Super_ssn in EMPLOYEE refers to the supervisor of an employee; this is another employee, represented by a tuple in the EMPLOYEE relation. Hence, Super_ssn is a foreign key that references the EMPLOYEE relation itself. In Figure 5.6 the tuple for employee ‘John Smith’ references the tuple for employee ‘Franklin Wong,’ indicating that ‘Franklin Wong’ is the supervisor of ‘John Smith’.

We can diagrammatically display referential integrity constraints by drawing a directed arc from each foreign key to the relation it references. For clarity, the arrowhead may point to the primary key of the referenced relation. Figure 5.7 shows the schema in Figure 5.5 with the referential integrity constraints displayed in this manner.

All integrity constraints should be specified on the relational database schema (that is, specified as part of its definition) if we want the DBMS to enforce these constraints on Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.

EMPLOYEE

Fname Minit Lname Ssn Bdate Address Sex Salary Super_ssn Dno

DEPARTMENT

Dname Dnumber Mgr_ssn Mgr_start_date

DEPT_LOCATIONS

Dnumber Dlocation

PROJECT

Pname Pnumber Plocation Dnum

WORKS_ON

Essn Pno Hours

DEPENDENT

Essn Dependent_name Sex Bdate Relationship

5.3 Update Operations, Transactions, and Dealing with Constraint Violations 195

the database states. Hence, the DDL includes provisions for specifying the various types of constraints so that the DBMS can automatically enforce them. In SQL, the CREATE TABLE statement of the SQL DDL allows the definition of primary key, unique key, NOT NULL, entity integrity, and referential integrity constraints, among other constraints (see Sections 6.1 and 6.2) .

5.2.5 Other Types of Constraints

The preceding integrity constraints are included in the data definition language because they occur in most database applications. Another class of general constraints, sometimes called semantic integrity constraints, are not part of the DDL and have to be specified and enforced in a different way. Examples of such constraints are the salary of an employee should not exceed the salary of the employee's supervisor and the maximum number of hours an employee can work on all projects per week is 56. Such constraints can be specified and enforced within the application programs that update the database, or by using a general-purpose constraint specification language. Mechanisms called triggers and assertions can be used in SQL, through the CREATE ASSERTION and CREATE TRIGGER statements, to specify some of these constraints (see Chapter 7). It is more common to check for these types of constraints within the application programs than to use constraint specification languages because the latter are sometimes difficult and complex to use, as we discuss in Section 26.1.

The types of constraints we discussed so far may be called state constraints because they define the constraints that a valid state of the database must satisfy. Another type of constraint, called transition constraints, can be defined to deal with state changes in the database.¹¹ An example of a transition constraint is: “the salary of an employee can only increase.” Such constraints are typically enforced by the application programs or specified using active rules and triggers, as we discuss in Section 26.1.

5.3 Update Operations, Transactions, and Dealing with Constraint Violations

The operations of the relational model can be categorized into retrievals and updates. The relational algebra operations, which can be used to specify retrievals,

are discussed in detail in Chapter 8. A relational algebra expression forms a new relation after applying a number of algebraic operators to an existing set of relations; its main use is for querying a database to retrieve information. The user formulates a query that specifies the data of interest, and a new relation is formed by applying relational operators to retrieve this data. The result relation becomes the answer to (or result of) the user's query. Chapter 8 also introduces the language

11

State constraints are sometimes called static constraints, and transition constraints are sometimes called dynamic constraints.

196 Chapter 5 The Relational Data Model and Relational Database Constraints

called relational calculus, which is used to define a query declaratively without giving a specific order of operations.

In this section, we concentrate on the database modification or update operations. There are three basic operations that can change the states of relations in the database: Insert, Delete, and Update (or Modify). They insert new data, delete old data, or modify existing data records, respectively. Insert is used to insert one or more new tuples in a relation, Delete is used to delete tuples, and Update (or Modify) is used to change the values of some attributes in existing tuples. Whenever these operations are applied, the integrity constraints specified on the relational database schema should not be violated. In this section we discuss the types of constraints that may be violated by each of these operations and the types of actions that may be taken if an operation causes a violation. We use the database shown in Figure 5.6 for examples and discuss only domain constraints, key constraints, entity integrity constraints, and the referential integrity constraints shown in Figure 5.7. For each type of operation, we give some examples and discuss any constraints that each operation may violate.

5.3.1 The Insert Operation

The Insert operation provides a list of attribute values for a new tuple t that is to be inserted into a relation R . Insert can violate any of the four types of constraints. Domain constraints can be violated if an attribute value is given that does not appear in the corresponding domain or is not of the appropriate data type. Key constraints can be violated if a key value in the new tuple t already exists in another tuple in the relation $r(R)$. Entity integrity can be violated if any part of the primary key of the new tuple t is NULL. Referential integrity can be violated if the value of any foreign key in t refers to a tuple that does not exist in the referenced relation. Here are some examples to illustrate this discussion.

■■ Operation:

Insert <'Cecilia', 'F', 'Kolonsky', NULL, '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.

Result: This insertion violates the entity integrity constraint (NULL for the primary key Ssn), so it is rejected.

■ ■ Operation:

Insert <'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, '987654321', 4> into EMPLOYEE.

Result: This insertion violates the key constraint because another tuple with the same Ssn value already exists in the EMPLOYEE relation, and so it is rejected.

■ ■ Operation:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windswept, Katy, TX', F, 28000, '987654321', 7> into EMPLOYEE.

Result: This insertion violates the referential integrity constraint specified on Dno in EMPLOYEE because no corresponding referenced tuple exists in DEPARTMENT with Dnumber = 7.

5.3 Update Operations, Transactions, and Dealing with Constraint Violations 197

■ ■ Operation:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.

Result: This insertion satisfies all constraints, so it is acceptable.

If an insertion violates one or more constraints, the default option is to reject the insertion. In this case, it would be useful if the DBMS could provide a reason to the user as to why the insertion was rejected. Another option is to attempt to correct the reason for rejecting the insertion, but this is typically not used for violations caused by Insert; rather, it is used more often in correcting violations for Delete and Update. In the first operation, the DBMS could ask the user to provide a value for Ssn, and could then accept the insertion if a valid Ssn value is provided. In operation 3, the DBMS could either ask the user to change the value of Dno to some valid value (or set it to NULL), or it could ask the user to insert a DEPARTMENT tuple with Dnumber = 7 and could accept the original insertion only after such an operation was accepted. Notice that in the latter case the insertion violation can cascade back to the EMPLOYEE relation if the user attempts to insert a tuple for department 7 with a value for Mgr_ssn that does not exist in the EMPLOYEE relation.

5.3.2 The Delete Operation

The Delete operation can violate only referential integrity. This occurs if the tuple being deleted is referenced by foreign keys from other tuples in the database. To specify deletion, a condition on the attributes of the relation selects the tuple (or tuples) to be deleted. Here are some examples.

■ ■ Operation:

Delete the WORKS_ON tuple with Essn = '999887777' and Pno = 10.

Result: This deletion is acceptable and deletes exactly one tuple.

■ ■ Operation:

Delete the EMPLOYEE tuple with Ssn = '999887777'.

Result: This deletion is not acceptable, because there are tuples in

WORKS_ON that refer to this tuple. Hence, if the tuple in EMPLOYEE is deleted, referential integrity violations will result.

■■ Operation:

Delete the EMPLOYEE tuple with Ssn = '333445555'.

Result: This deletion will result in even worse referential integrity violations, because the tuple involved is referenced by tuples from the EMPLOYEE, DEPARTMENT, WORKS_ON, and DEPENDENT relations.

Several options are available if a deletion operation causes a violation. The first option, called restrict, is to reject the deletion. The second option, called cascade, is to attempt to cascade (or propagate) the deletion by deleting tuples that reference the tuple that is being deleted. For example, in operation 2, the DBMS could automatically delete the offending tuples from WORKS_ON with Essn = '999887777'. A third option, called set null or set default, is to modify the referencing attribute values that cause the violation; each such value is either set to NULL or changed to reference another default valid tuple. Notice that if a referencing attribute that causes a violation is part of the primary key, it cannot be set to NULL; otherwise, it would violate entity integrity.

Combinations of these three options are also possible. For example, to avoid having operation 3 cause a violation, the DBMS may automatically delete all tuples from WORKS_ON and DEPENDENT with Essn = '333445555'. Tuples in EMPLOYEE with Super_ssn = '333445555' and the tuple in DEPARTMENT with Mgr_ssn = '333445555' can have their Super_ssn and Mgr_ssn values changed to other valid values or to NULL. Although it may make sense to delete automatically the WORKS_ON and DEPENDENT tuples that refer to an EMPLOYEE tuple, it may not make sense to delete other EMPLOYEE tuples or a DEPARTMENT tuple.

In general, when a referential integrity constraint is specified in the DDL, the DBMS will allow the database designer to specify which of the options applies in case of a violation of the constraint. We discuss how to specify these options in the SQL DDL in Chapter 6.

5.3.3 The Update Operation

The Update (or Modify) operation is used to change the values of one or more attributes in a tuple (or tuples) of some relation R. It is necessary to specify a condition on the attributes of the relation to select the tuple (or tuples) to be modified.

Here are some examples.

■■ Operation:

Update the salary of the EMPLOYEE tuple with Ssn = '999887777' to 28000.

Result: Acceptable.

■■ Operation:

Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 1.

Result: Acceptable.

■■ Operation:

Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 7.

Result: Unacceptable, because it violates referential integrity.

■■ Operation:

Update the Ssn of the EMPLOYEE tuple with Ssn = '999887777' to '987654321'.

Result: Unacceptable, because it violates primary key constraint by repeating a value that already exists as a primary key in another tuple; it violates referential integrity constraints because there are other relations that refer to the existing value of Ssn.

Updating an attribute that is neither part of a primary key nor part of a foreign key usually causes no problems; the DBMS need only check to confirm that the new value is of the correct data type and domain. Modifying a primary key value is similar to deleting one tuple and inserting another in its place because we use the primary key to identify tuples. Hence, the issues discussed earlier in both Sections 5.3.1 (Insert) and 5.3.2 (Delete) come into play. If a foreign key attribute is modified, the

5.4 Summary 199

DBMS must make sure that the new value refers to an existing tuple in the referenced relation (or is set to NULL). Similar options exist to deal with referential integrity violations caused by Update as those options discussed for the Delete operation. In fact, when a referential integrity constraint is specified in the DDL, the DBMS will allow the user to choose separate options to deal with a violation caused by Delete and a violation caused by Update (see Section 6.2).

5.3.4 The Transaction Concept

A database application program running against a relational database typically executes one or more transactions. A transaction is an executing program that includes some database operations, such as reading from the database, or applying insertions, deletions, or updates to the database. At the end of the transaction, it must leave the database in a valid or consistent state that satisfies all the constraints specified on the database schema. A single transaction may involve any number of retrieval operations (to be discussed as part of relational algebra and calculus in Chapter 8, and as a part of the language SQL in Chapters 6 and 7) and any number of update operations. These retrievals and updates will together form an atomic unit of work against the database. For example, a transaction to apply a bank withdrawal will typically read the user account record, check if there is a sufficient balance, and then update the record by the withdrawal amount.

A large number of commercial applications running against relational databases in online transaction processing (OLTP) systems are executing transactions at rates that reach several hundred per second. Transaction processing concepts, concurrent execution of transactions, and recovery from failures will be discussed in Chapters 20 to 22.

5.4 Summary

In this chapter we presented the modeling concepts, data structures, and constraints provided by the relational model of data. We started by introducing the concepts of domains, attributes, and tuples. Then, we defined a relation schema as a list of attributes that describe the structure of a relation. A relation, or relation state, is a set of tuples that conforms to the schema.

Several characteristics differentiate relations from ordinary tables or files. The first is that a relation is not sensitive to the ordering of tuples. The second involves the ordering of attributes in a relation schema and the corresponding ordering of values within a tuple. We gave an alternative definition of relation that does not require ordering of attributes, but we continued to use the first definition, which requires attributes and tuple values to be ordered, for convenience. Then, we discussed values in tuples and introduced NULL values to represent missing or unknown information. We emphasized that NULL values should be avoided as much as possible. We classified database constraints into inherent model-based constraints, explicit schema-based constraints, and semantic constraints or business rules. Then, we

200 Chapter 5 The Relational Data Model and Relational Database Constraints

discussed the schema constraints pertaining to the relational model, starting with domain constraints, then key constraints (including the concepts of superkey, key, and primary key), and the NOT NULL constraint on attributes. We defined relational databases and relational database schemas. Additional relational constraints include the entity integrity constraint, which prohibits primary key attributes from being NULL. We described the interrelation referential integrity constraint, which is used to maintain consistency of references among tuples from various relations.

The modification operations on the relational model are Insert, Delete, and Update. Each operation may violate certain types of constraints (refer to Section 5.3). Whenever an operation is applied, the resulting database state must be a valid state. Finally, we introduced the concept of a transaction, which is important in relational DBMSs because it allows the grouping of several database operations into a single atomic action on the database.

Review Questions

- 5.1. Define the following terms as they apply to the relational model of data: domain, attribute, n-tuple, relation schema, relation state, degree of a relation, relational database schema, and relational database state.
- 5.2. Why are tuples in a relation not ordered?
- 5.3. Why are duplicate tuples not allowed in a relation?
- 5.4. What is the difference between a key and a superkey?
- 5.5. Why do we designate one of the candidate keys of a relation to be the primary key?
- 5.6. Discuss the characteristics of relations that make them different from ordi-

nary tables and files.

5.7. Discuss the various reasons that lead to the occurrence of NULL values in relations.

5.8. Discuss the entity integrity and referential integrity constraints. Why is each considered important?

5.9. Define foreign key. What is this concept used for?

5.10. What is a transaction? How does it differ from an Update operation?

Exercises

5.11. Suppose that each of the following Update operations is applied directly to the database state shown in Figure 5.6. Discuss all integrity constraints

Exercises 201

violated by each operation, if any, and the different ways of enforcing these constraints.

a. Insert <'Sophia', 'M', 'Wood', '973442298', '1974-05-21', '23 S Lamar Blvd. Rd, Austin, TX', 'F', 62000, '222445555', 5> into EMPLOYEE.

b. Insert <'6Sigma', 4, 'Austin', 4> into PROJECT.

c. Insert <'Information Technology', 2, '987987987', '2007-10-01'> into DEPARTMENT.

d. Insert <'777624972', 15, '40.0'> into WORKS_ON.

e. Insert <'888665555', 'John', 'M', null, 'Son'> into DEPENDENT.

f. Delete the DEPENDENT tuples with Essn = '987654321'.

g. Delete the DEPARTMENT tuples with Dnumber = 5.

h. Delete the WORKS_ON tuples with Pnoe = 30.

i. Modify the Plocation and Dnum of the PROJECT tuples with Dnum = 5 to 'Houston' and 1, respectively.

j. Modify the Super_ssn attribute of the EMPLOYEE tuple with Ssn = '888665555' to null.

k. Modify the Pnumber attribute of the PROJECT tuple with Pnumber = 30 to 40

5.12. Consider the AIRLINE relational database schema shown in Figure 5.8, which describes a database for airline flight information. Each FLIGHT is identified by a Flight_number, and consists of one or more FLIGHT_LEGs with Leg_numbers 1, 2, 3, and so on. Each FLIGHT_LEG has scheduled arrival and departure times, airports, and one or more LEG_INSTANCES— one for each Date on which the flight travels. FAREs are kept for each FLIGHT. For each FLIGHT_LEG instance, SEAT_RESERVATIONS are kept, as are the AIRPLANE used on the leg and the actual arrival and departure times and airports. An AIRPLANE is identified by an Airplane_id and is of a particular AIRPLANE_TYPE. CAN_LAND relates AIRPLANE_TYPES to the AIRPORTs at which they can land. An AIRPORT is identified by an Airport_code. Consider an update for the AIRLINE database to introduce a new leg instance tuple in the LEG_INSTANCE relation.

a. Give the operations for this update.

- b. What types of constraints would you expect to check?
- c. Which integrity constraints should be checked to insert a new flight leg tuple into the FLIGHT_LEG relation not?
- d. Specify all the referential integrity constraints that hold on the schema shown in Figure 5.8.

5.13. Consider the relation FACTORY(Product#, Production_Section#, Supervisor, Stage, Warehouse_code, Machine#, Machine_Time, Production_Time, Units_manufactured). This represents products manufactured in a factory, with unique Product_section#s. Identify what you think should be various candidate keys, and write in your own words the conditions or assumptions under which each candidate key would be valid.

202 Chapter 5 The Relational Data Model and Relational Database Constraints

AIRPORT

Airport_code Name City State

FLIGHT

Flight_number Airline Weekdays

FLIGHT_LEG

Flight_number Leg_number Departure_airport_code Scheduled_departure_time

Arrival_airport_code Scheduled_arrival_time

LEG_INSTANCE

Flight_number Leg_number Date Number_of_available_seats Airplane_id

Departure_airport_code Departure_time Arrival_airport_code Arrival_time

FARE

Flight_number Fare_code Amount Restrictions

AIRPLANE_TYPE

Airplane_type_name Max_seats Company

CAN_LAND

Airplane_type_name Airport_code

AIRPLANE

Airplane_id Total_number_of_seats Airplane_type

SEAT_RESERVATION

Flight_number Leg_number Date Seat_number Customer_name Customer_phone

Figure 5.8

The AIRLINE relational database schema.

5.14. Consider the following seven relations for a car-rental database application in a company:

CUSTOMER(Cust#, Cname, City)

RENTAL(Rental#, Rdate, Cust#, Time, Date, Hourly_rate)

RENTAL_CAR(Rental#, Car#, Driver#, Start_time, End_time, Amount_received)

CAR(Car#, Year, Model, Price, Depreciation, Last_service)

Exercises 203

SERVICING(Garager#, Car#, Service_date)

GARAGE(Garage#, GAddress, Owner_name)

DRIVER(Driver#, DName, DAddress)

Here, Hourly_rate refers to the per hour rate for a specific rental.

Amount_received is the total fare; Time and Date are the time and date when the car and driver is sent to the customer. Rdate is the date the rental order was placed.

Assume that a car can be driven by any driver and can go to more than one garage for servicing. Specify the foreign keys for this schema, stating any assumptions you make. What other constraints can you think of for this database?

5.15. Consider the following relations for a database that keeps track of visits of various doctors working for a group of hospitals, to various hospitals under the group:

DOCTORS(DSsn, Name, Start_year, Dept_no, location)

VISIT(DSsn, From_hospital, To_hospital, Departure_date, Return_date, Visit_id)

EXPENSE(Visit_id, Account#, Amount)

A visit can be charged to one or more accounts. Specify the foreign keys for this schema, stating any assumptions you make.

5.16. Consider the following relations for a library database that keeps track of users, suppliers, books, user registration and supply details:

USER(Ssn, Uname, Uaddress , Card#, Rdate)

BOOK(Book_isbn, Book_title, Publisher, Author)

BOOK_BORROWED(Ssn, Card#, Issue_date, Return_date, Book_isbn)

SUPPLIER(SSsn, Sname, Saddress , Account#)

SUPPLY(Book_isbn, SSsn, price, Sdate)

Specify the foreign keys for this schema, stating any assumptions you make.

5.17. Consider the following relations for a database that keeps track of booking of apartments by a constructor. (OPTION refers to some specific optional requirements/designs stated by the client to be implemented in the flat):

APARTMENT(Apartment#, Model, Address, Price_perSquareFt)

OPTION(Apartment#, Option_name, Extra_price)

BOOKING(Agent_id, Apartment#, Date, Booking_price)

AGENT(Agent_id, Name, Phone)

First, specify the foreign keys for this schema, stating any assumptions you make. Next, populate the relations with a few sample tuples, and then give an example of an insertion in the BOOKING and AGENT relations that violates the referential integrity constraints and of another insertion that does not.

5.18. Database design often involves decisions about the storage of attributes. For example, a phone number can be stored as one attribute or split into four attributes (one for each of the three hyphen-delineated groups of numbers

204 Chapter 5 The Relational Data Model and Relational Database Constraints

in a Phone number—XX -XX-XXXX-XXXX). However, phone numbers are usually represented as just one attribute. The decision is based on how the database will be used. This exercise asks you to think about specific situations where dividing the phone number is useful.

5.19. Consider a STUDENT relation in a UNIVERSITY database with the following attributes (Name, Ssn, Local_phone, Address, Cell_phone, Age, Gpa). Note that the cell phone may be from a different city and state (or province) from the local phone. A possible tuple of the relation is shown below:

Name	Ssn	Local_phone	Address	Cell_phone	Age	Gpa
George Shaw	123-45-6789	555-1234	123 Main St.,	555-4321	19	3.75
William Edwards		Anytown, CA	94539			

- Identify the critical missing information from the Local_phone and Cell_phone attributes. (Hint: How do you call someone who lives in a different state or province?)
- Would you store this additional information in the Local_phone and Cell_phone attributes or add new attributes to the schema for STUDENT?
- Consider the Name attribute. What are the advantages and disadvantages of splitting this field from one attribute into three attributes (first name, middle name, and last name)?
- What general guideline would you recommend for deciding when to store information in a single attribute and when to split the information?
- Suppose the student can have between 0 and 5 phones. Suggest two different designs that allow this type of information.

5.20. Recent changes in privacy laws have disallowed organizations from using Social Security numbers to identify individuals unless certain restrictions are satisfied. As a result, most U.S. universities cannot use SSNs as primary keys (except for financial data). In practice, Student_id, a unique identifier assigned to every student, is likely to be used as the primary key rather than SSN since Student_id can be used throughout the system.

- Some database designers are reluctant to use generated keys (also known as surrogate keys) for primary keys (such as Student_id) because they are artificial. Can you propose any natural choices of keys that can be used to identify the student record in a UNIVERSITY database?
- Suppose that you are able to guarantee uniqueness of a natural key that includes last name. Are you guaranteed that the last name will not change during the lifetime of the database? If last name can change, what solutions can you propose for creating a primary key that still includes last name but remains unique?
- What are the advantages and disadvantages of using generated (surrogate) keys?

Selected Bibliography 205

Selected Bibliography

The relational model was introduced by Codd (1970) in a classic paper. Codd also introduced relational algebra and laid the theoretical foundations for the relational model in a series of papers (Codd, 1971, 1972, 1972a, 1974); he was later given the Turing Award, the highest honor of the ACM (Association for Computing Machinery) for his work on the relational model. In a later paper, Codd (1979) discussed extending the relational model to incorporate more meta-data and semantics about the relations; he also proposed a three-valued logic to deal with uncertainty in relations and incorporating NULLs in the relational algebra. The resulting model is known as RM/T. Childs (1968) had earlier used set theory to model databases. Later, Codd (1990) published a book examining over 300 features of the relational data model and database systems. Date (2001) provides a retrospective review and analysis of the relational data model.

Since Codd's pioneering work, much research has been conducted on various aspects of the relational model. Todd (1976) describes an experimental DBMS called PRTV that directly implements the relational algebra operations. Schmidt and Swenson (1975) introduce additional semantics into the relational model by classifying different types of relations. Chen's (1976) entity-relationship model, which is discussed in Chapter 3, is a means to communicate the real-world semantics of a relational database at the conceptual level. Wiederhold and Elmasri (1979) introduce various types of connections between relations to enhance its constraints. Extensions of the relational model are discussed in Chapters 11 and 26. Additional bibliographic notes for other aspects of the relational model and its languages, systems, extensions, and theory are given in Chapters 6 to 9, 14, 15, 23, and 30. Maier (1983) and Atzeni and De Antonellis (1993) provide an extensive theoretical treatment of the relational data model.

This page intentionally left blank

chapter 6

Basic SQL

The SQL language may be considered one of the major reasons for the commercial success of relational databases. Because it became a standard for relational databases, users were less concerned about migrating their database applications from other types of database systems—for example, older network or hierarchical systems—to relational systems. This is because even if the users became dissatisfied with the particular relational DBMS product they were using, converting to another relational DBMS product was not expected to be too expensive and time-consuming because both systems followed the same language standards. In practice, of course, there are differences among various commercial relational DBMS packages. However, if the user is diligent in using only those features that are part of the standard,

and if two relational DBMSs faithfully support the standard, then conversion between two systems should be simplified. Another advantage of having such a standard is that users may write statements in a database application program that can access data stored in two or more relational DBMSs without having to change the database sublanguage (SQL), as long as both/all of the relational DBMSs support standard SQL.

This chapter presents the practical relational model, which is based on the SQL standard for commercial relational DBMSs, whereas Chapter 5 presented the most important concepts underlying the formal relational data model. In Chapter 8 (Sections 8.1 through 8.5), we shall discuss the relational algebra operations, which are very important for understanding the types of requests that may be specified on a relational database. They are also important for query processing and optimization in a relational DBMS, as we shall see in Chapters 18 and 19. However, the relational algebra operations are too low-level for most commercial DBMS users because a query in relational algebra is written as a sequence of operations that, when executed, produces the required result. Hence, the user must specify how—that is, in what order—to execute the query operations. On the other hand, the SQL language

207 Chapter 6 Basic SQL

provides a higher-level declarative language interface, so the user only specifies what the result is to be, leaving the actual optimization and decisions on how to execute the query to the DBMS. Although SQL includes some features from relational algebra, it is based to a greater extent on the tuple relational calculus, which we describe in Section 8.6. However, the SQL syntax is more user-friendly than either of the two formal languages.

The name SQL is presently expanded as Structured Query Language. Originally, SQL was called SEQUEL (Structured English QUery Language) and was designed and implemented at IBM Research as the interface for an experimental relational database system called SYSTEM R. SQL is now the standard language for commercial relational DBMSs. The standardization of SQL is a joint effort by the American National Standards Institute (ANSI) and the International Standards Organization (ISO), and the first SQL standard is called SQL-86 or SQL1. A revised and much expanded standard called SQL-92 (also referred to as SQL2) was subsequently developed. The next standard that is well-recognized is SQL:1999, which started out as SQL3. Additional updates to the standard are SQL:2003 and SQL:2006, which added XML features (see Chapter 13) among other updates to the language. Another update in 2008 incorporated more object database features into SQL (see Chapter 12), and a further update is SQL:2011. We will try to cover the latest version of SQL as much as possible, but some of the newer features are discussed in later chapters. It is also not possible to cover the language in its entirety in this text. It is important to note that when new features

are added to SQL, it usually takes a few years for some of these features to make it into the commercial SQL DBMSs.

SQL is a comprehensive database language: It has statements for data definitions, queries, and updates. Hence, it is both a DDL and a DML. In addition, it has facilities for defining views on the database, for specifying security and authorization, for defining integrity constraints, and for specifying transaction controls. It also has rules for embedding SQL statements into a general-purpose programming language such as Java or C/C++.

The later SQL standards (starting with SQL:1999) are divided into a core specification plus specialized extensions. The core is supposed to be implemented by all RDBMS vendors that are SQL compliant. The extensions can be implemented as optional modules to be purchased independently for specific database applications such as data mining, spatial data, temporal data, data warehousing, online analytical processing (OLAP), multimedia data, and so on.

Because the subject of SQL is both important and extensive, we devote two chapters to its basic features. In this chapter, Section 6.1 describes the SQL DDL commands for creating schemas and tables, and gives an overview of the basic data types in SQL. Section 6.2 presents how basic constraints such as key and referential integrity are specified. Section 6.3 describes the basic SQL constructs for

1

Originally, SQL had statements for creating and dropping indexes on the files that represent relations,

but these have been dropped from the SQL standard for some time.

6.1 SQL Data Definition and Data Types 209

specifying retrieval queries, and Section 6.4 describes the SQL commands for insertion, deletion, and update.

In Chapter 7, we will describe more complex SQL retrieval queries, as well as the ALTER commands for changing the schema. We will also describe the CREATE ASSERTION statement, which allows the specification of more general constraints on the database, and the concept of triggers, which is presented in more detail in Chapter 26. We discuss the SQL facility for defining views on the database in Chapter 7. Views are also called virtual or derived tables because they present the user with what appear to be tables; however, the information in those tables is derived from previously defined tables.

Section 6.5 lists some SQL features that are presented in other chapters of the book; these include object-oriented features in Chapter 12, XML in Chapter 13, transaction control in Chapter 20, active databases (triggers) in Chapter 26, online analytical processing (OLAP) features in Chapter 29, and security/authorization in Chapter 30. Section 6.6 summarizes the chapter. Chapters 10 and 11 discuss the various database programming techniques for programming with SQL.

6.1 SQL Data Definition and Data Types

SQL uses the terms table, row, and column for the formal relational model terms relation, tuple, and attribute, respectively. We will use the corresponding terms interchangeably. The main SQL command for data definition is the CREATE statement, which can be used to create schemas, tables (relations), types, and domains, as well as other constructs such as views, assertions, and triggers. Before we describe the relevant CREATE statements, we discuss schema and catalog concepts in Section 6.1.1 to place our discussion in perspective. Section 6.1.2 describes how tables are created, and Section 6.1.3 describes the most important data types available for attribute specification. Because the SQL specification is very large, we give a description of the most important features. Further details can be found in the various SQL standards documents (see end-of-chapter bibliographic notes).

6.1.1 Schema and Catalog Concepts in SQL

Early versions of SQL did not include the concept of a relational database schema; all tables (relations) were considered part of the same schema. The concept of an SQL schema was incorporated starting with SQL2 in order to group together tables and other constructs that belong to the same database application (in some systems, a schema is called a database). An SQL schema is identified by a schema name and includes an authorization identifier to indicate the user or account who owns the schema, as well as descriptors for each element in the schema. Schema elements include tables, types, constraints, views, domains, and other constructs (such as authorization grants) that describe the schema. A schema is created via the CREATE SCHEMA statement, which can include all the schema elements' definitions. Alternatively, the schema can be assigned a name and authorization identifier, and the

210 Chapter 6 Basic SQL

elements can be defined later. For example, the following statement creates a schema called COMPANY owned by the user with authorization identifier 'Jsmith'. Note that each statement in SQL ends with a semicolon.

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

In general, not all users are authorized to create schemas and schema elements. The privilege to create schemas, tables, and other constructs must be explicitly granted to the relevant user accounts by the system administrator or DBA.

In addition to the concept of a schema, SQL uses the concept of a catalog—a named collection of schemas.² Database installations typically have a default environment and schema, so when a user connects and logs in to that database installation, the user can refer directly to tables and other constructs within that schema without having to specify a particular schema name. A catalog always contains a special schema called INFORMATION_SCHEMA, which provides information on all the schemas in the catalog and all the element descriptors in these schemas. Integrity constraints such as referential integrity can be defined between relations only if they exist in schemas within the same catalog. Schemas within the same catalog can also share certain elements, such as type and domain definitions.

6.1.2 The CREATE TABLE Command in SQL

The CREATE TABLE command is used to specify a new relation by giving it a name and specifying its attributes and initial constraints. The attributes are specified first, and each attribute is given a name, a data type to specify its domain of values, and possibly attribute constraints, such as NOT NULL. The key, entity integrity, and referential integrity constraints can be specified within the CREATE TABLE statement after the attributes are declared, or they can be added later using the ALTER TABLE command (see Chapter 7). Figure 6.1 shows sample data definition statements in SQL for the COMPANY relational database schema shown in Figure 3.7.

Typically, the SQL schema in which the relations are declared is implicitly specified in the environment in which the CREATE TABLE statements are executed. Alternatively, we can explicitly attach the schema name to the relation name, separated by a period. For example, by writing

```
CREATE TABLE COMPANY.EMPLOYEE
```

rather than

```
CREATE TABLE EMPLOYEE
```

as in Figure 6.1, we can explicitly (rather than implicitly) make the EMPLOYEE table part of the COMPANY schema.

The relations declared through CREATE TABLE statements are called base tables (or base relations); this means that the table and its rows are actually created

2

SQL also includes the concept of a cluster of catalogs.

6.1 SQL Data Definition and Data Types 211

CREATE TABLE EMPLOYEE Figure 6.1

(Fname VARCHAR(15) NOT NULL, SQL CREATE

Minit CHAR, TABLE data

Lname VARCHAR(15) NOT NULL, definition statements

Ssn CHAR(9) NOT NULL, for defining the

Bdate DATE, COMPANY schema

Address VARCHAR(30), from Figure 5.7.

Sex CHAR,

Salary DECIMAL(10,2),

Super_ssn CHAR(9),

Dno INT NOT NULL,

PRIMARY KEY (Ssn),

CREATE TABLE DEPARTMENT

(Dname VARCHAR(15) NOT NULL,

Dnumber INT NOT NULL,

Mgr_ssn CHAR(9) NOT NULL,

Mgr_start_date DATE,

```

PRIMARY KEY (Dnumber),
UNIQUE (Dname),
FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
( Dnumber INT NOT NULL,
Dlocation VARCHAR(15) NOT NULL,
PRIMARY KEY (Dnumber, Dlocation),
FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE PROJECT
( Pname VARCHAR(15) NOT NULL,
Pnumber INT NOT NULL,
Plocation VARCHAR(15),
Dnum INT NOT NULL,
PRIMARY KEY (Pnumber),
UNIQUE (Pname),
FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE WORKS_ON
( Essn CHAR(9) NOT NULL,
Pno INT NOT NULL,
Hours DECIMAL(3,1) NOT NULL,
PRIMARY KEY (Essn, Pno),
FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );
CREATE TABLE DEPENDENT
( Essn CHAR(9) NOT NULL,
Dependent_name VARCHAR(15) NOT NULL,
Sex CHAR,
Bdate DATE,
Relationship VARCHAR(8),
PRIMARY KEY (Essn, Dependent_name),
FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );

```

212 Chapter 6 Basic SQL

and stored as a file by the DBMS. Base relations are distinguished from virtual relations, created through the CREATE VIEW statement (see Chapter 7), which may or may not correspond to an actual physical file. In SQL, the attributes in a base table are considered to be ordered in the sequence in which they are specified in the CREATE TABLE statement. However, rows (tuples) are not considered to be ordered within a table (relation).

It is important to note that in Figure 6.1, there are some foreign keys that may cause errors because they are specified either via circular references or because they refer to a table that has not yet been created. For example, the foreign key Super_ssn in

the EMPLOYEE table is a circular reference because it refers to the EMPLOYEE table itself. The foreign key Dno in the EMPLOYEE table refers to the DEPARTMENT table, which has not been created yet. To deal with this type of problem, these constraints can be left out of the initial CREATE TABLE statement, and then added later using the ALTER TABLE statement (see Chapter 7). We displayed all the foreign keys in Figure 6.1 to show the complete COMPANY schema in one place.

6.1.3 Attribute Data Types and Domains in SQL

The basic data types available for attributes include numeric, character string, bit string, Boolean, date, and time.

■ ■ Numeric data types include integer numbers of various sizes (INTEGER or INT, and SMALLINT) and floating-point (real) numbers of various precision (FLOAT or REAL, and DOUBLE PRECISION). Formatted numbers can be declared by using DECIMAL(i, j)—or DEC(i, j) or NUMERIC(i, j)—where i, the precision, is the total number of decimal digits and j, the scale, is the number of digits after the decimal point. The default for scale is zero, and the default for precision is implementation-defined.

■ ■ Character-string data types are either fixed length— CHAR (n) or CHARACTER(n), where n is the number of characters—or varying length— VARCHAR(n) or CHAR VARYING(n) or CHARACTER VARYING(n), where n is the maximum number of characters. When specifying a literal string value, it is placed between single quotation marks (apostrophes), and it is case sensitive (a distinction is made between uppercase and lowercase).³ For fixed-length strings, a shorter string is padded with blank characters to the right. For example, if the value 'Smith' is for an attribute of type CHAR(10), it is padded with five blank characters to become 'Smith' if needed. Padded blanks are generally ignored when strings are compared. For comparison purposes, strings are considered ordered in alphabetic (or lexicographic) order; if a string str1 appears before another string str2 in alphabetic order, then str1 is considered to be less than str2.⁴ There is also a concatenation operator denoted by || (double vertical bar) that can concatenate two strings

3

This is not the case with SQL keywords, such as CREATE or CHAR. With keywords, SQL is case insen-

sitive, meaning that SQL treats uppercase and lowercase letters as equivalent in keywords.

4

For nonalphabetic characters, there is a defined order.

6.1 SQL Data Definition and Data Types 213

in SQL. For example, 'abc' || 'XYZ' results in a single string 'abcXYZ'.

Another variable-length string data type called CHARACTER LARGE OBJECT or CLOB is also available to specify columns that have large text values, such as documents. The CLOB maximum length can be specified in kilobytes

(K), megabytes (M), or gigabytes (G). For example, CLOB(20M) specifies a maximum length of 20 megabytes.

■ ■ Bit-string data types are either of fixed length n —BIT(n)—or varying length—BIT VARYING(n), where n is the maximum number of bits. The default for n , the length of a character string or bit string, is 1. Literal bit strings are placed between single quotes but preceded by a B to distinguish them from character strings; for example, B'10101'.5 Another variable-length bitstring data type called BINARY LARGE OBJECT or BLOB is also available to specify columns that have large binary values, such as images. As for CLOB, the maximum length of a BLOB can be specified in kilobits (K), megabits (M), or gigabits (G). For example, BLOB(30G) specifies a maximum length of 30 gigabits.

■ ■ A Boolean data type has the traditional values of TRUE or FALSE. In SQL, because of the presence of NULL values, a three-valued logic is used, so a third possible value for a Boolean data type is UNKNOWN. We discuss the need for UNKNOWN and the three-valued logic in Chapter 7.

■ ■ The DATE data type has ten positions, and its components are YEAR, MONTH, and DAY in the form YYYY-MM-DD. The TIME data type has at least eight positions, with the components HOUR, MINUTE, and SECOND in the form HH:MM:SS. Only valid dates and times should be allowed by the SQL implementation. This implies that months should be between 1 and 12 and days must be between 01 and 31; furthermore, a day should be a valid day for the corresponding month. The < (less than) comparison can be used with dates or times—an earlier date is considered to be smaller than a later date, and similarly with time. Literal values are represented by single-quoted strings preceded by the keyword DATE or TIME; for example, DATE '2014-09-27' or TIME '09:12:47'. In addition, a data type TIME(i), where i is called time fractional seconds precision, specifies $i + 1$ additional positions for TIME—one position for an additional period (.) separator character, and i positions for specifying decimal fractions of a second. A TIME WITH TIME ZONE data type includes an additional six positions for specifying the displacement from the standard universal time zone, which is in the range +13:00 to -12:59 in units of HOURS:MINUTES. If WITH TIME ZONE is not included, the default is the local time zone for the SQL session.

Some additional data types are discussed below. The list of types discussed here is not exhaustive; different implementations have added more data types to SQL.

■ ■ A timestamp data type (TIMESTAMP) includes the DATE and TIME fields, plus a minimum of six positions for decimal fractions of seconds and an optional WITH TIME ZONE qualifier. Literal values are represented by single-quoted

5

Bit strings whose length is a multiple of 4 can be specified in hexadecimal notation, where the literal

string is preceded by X and each hexadecimal character represents 4 bits.

214 Chapter 6 Basic SQL

strings preceded by the keyword `TIMESTAMP`, with a blank space between data and time; for example, `TIMESTAMP '2014-09-27 09:12:47.648302'`.

■■ Another data type related to `DATE`, `TIME`, and `TIMESTAMP` is the `INTERVAL` data type. This specifies an interval—a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp. Intervals are qualified to be either `YEAR/MONTH` intervals or `DAY/TIME` intervals.

The format of `DATE`, `TIME`, and `TIMESTAMP` can be considered as a special type of string. Hence, they can generally be used in string comparisons by being cast (or coerced or converted) into the equivalent strings.

It is possible to specify the data type of each attribute directly, as in Figure 6.1; alternatively, a domain can be declared, and the domain name can be used with the attribute specification. This makes it easier to change the data type for a domain that is used by numerous attributes in a schema, and improves schema readability.

For example, we can create a domain `SSN_TYPE` by the following statement:

```
CREATE DOMAIN SSN_TYPE AS CHAR(9);
```

We can use `SSN_TYPE` in place of `CHAR(9)` in Figure 6.1 for the attributes `Ssn` and `Super_ssn` of `EMPLOYEE`, `Mgr_ssn` of `DEPARTMENT`, `Essn` of `WORKS_ON`, and `Essn` of `DEPENDENT`. A domain can also have an optional default specification via a `DEFAULT` clause, as we discuss later for attributes. Notice that domains may not be available in some implementations of SQL.

In SQL, there is also a `CREATE TYPE` command, which can be used to create user defined types or UDTs. These can then be used either as data types for attributes, or as the basis for creating tables. We shall discuss `CREATE TYPE` in detail in Chapter 12, because it is often used in conjunction with specifying object database features that have been incorporated into more recent versions of SQL.

6.2 Specifying Constraints in SQL

This section describes the basic constraints that can be specified in SQL as part of table creation. These include key and referential integrity constraints, restrictions on attribute domains and `NULLS`, and constraints on individual tuples within a relation using the `CHECK` clause. We discuss the specification of more general constraints, called assertions, in Chapter 7.

6.2.1 Specifying Attribute Constraints and Attribute Defaults

Because SQL allows `NULLS` as attribute values, a constraint `NOT NULL` may be specified if `NULL` is not permitted for a particular attribute. This is always implicitly specified for the attributes that are part of the primary key of each relation, but it can be specified for any other attributes whose values are required not to be `NULL`, as shown in Figure 6.1.

It is also possible to define a default value for an attribute by appending the clause `DEFAULT <value>` to an attribute definition. The default value is included in any

6.2 Specifying Constraints in SQL 215

```
CREATE TABLE EMPLOYEE
( ...,
Dno INT NOT NULL DEFAULT 1,
CONSTRAINT EMPPK
PRIMARY KEY (Ssn),
CONSTRAINT EMPSUPERFK
FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
ON DELETE SET NULL ON UPDATE CASCADE,
CONSTRAINT EMPDEPTFK
FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
ON DELETE SET DEFAULT ON UPDATE CASCADE);
CREATE TABLE DEPARTMENT
( ...,
Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555',
CONSTRAINT DEPTPK
PRIMARY KEY (Dnumber),
CONSTRAINT DEPTSK
UNIQUE (Dname),
CONSTRAINT DEPTMGRFK
FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
ON DELETE SET DEFAULT ON UPDATE CASCADE);
```

Figure 6.2 Example illustrating how default attribute values and referential integrity triggered actions are specified in SQL.

new tuple if an explicit value is not provided for that attribute. Figure 6.2 illustrates an example of specifying a default manager for a new department and a default department for a new employee. If no default clause is specified, the default value is NULL for attributes that do not have the NOT NULL constraint.

Another type of constraint can restrict attribute or domain values using the CHECK clause following an attribute or domain definition.⁶ For example, suppose that department numbers are restricted to integer numbers between 1 and 20; then, we can change the attribute declaration of Dnumber in the DEPARTMENT table (see Figure 6.1) to the following:

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
```

The CHECK clause can also be used in conjunction with the CREATE DOMAIN statement. For example, we can write the following statement:

```
CREATE DOMAIN D_NUM AS INTEGER
CHECK (D_NUM > 0 AND D_NUM < 21);
```

6

The CHECK clause can also be used for other purposes, as we shall see.

216 Chapter 6 Basic SQL

We can then use the created domain D_NUM as the attribute type for all attributes that refer to department numbers in Figure 6.1, such as Dnumber of DEPARTMENT, Dnum of PROJECT, Dno of EMPLOYEE, and so on.

6.2.2 Specifying Key and Referential Integrity Constraints

Because keys and referential integrity constraints are very important, there are special clauses within the CREATE TABLE statement to specify them. Some examples to illustrate the specification of keys and referential integrity are shown in Figure 6.1.7 The PRIMARY KEY clause specifies one or more attributes that make up the primary key of a relation. If a primary key has a single attribute, the clause can follow the attribute directly. For example, the primary key of DEPARTMENT can be specified as follows (instead of the way it is specified in Figure 6.1):

Dnumber INT PRIMARY KEY,

The UNIQUE clause specifies alternate (unique) keys, also known as candidate keys as illustrated in the DEPARTMENT and PROJECT table declarations in Figure 6.1.

The UNIQUE clause can also be specified directly for a unique key if it is a single attribute, as in the following example:

Dname VARCHAR(15) UNIQUE,

Referential integrity is specified via the FOREIGN KEY clause, as shown in Figure 6.1. As we discussed in Section 5.2.4, a referential integrity constraint can be violated when tuples are inserted or deleted, or when a foreign key or primary key attribute value is updated. The default action that SQL takes for an integrity violation is to reject the update operation that will cause a violation, which is known as the RESTRICT option. However, the schema designer can specify an alternative action to be taken by attaching a referential triggered action clause to any foreign key constraint. The options include SET NULL, CASCADE, and SET DEFAULT. An option must be qualified with either ON DELETE or ON UPDATE. We illustrate this with the examples shown in Figure 6.2. Here, the database designer chooses ON DELETE SET NULL and ON UPDATE CASCADE for the foreign key Super_ssn of EMPLOYEE. This means that if the tuple for a supervising employee is deleted, the value of Super_ssn is automatically set to NULL for all employee tuples that were referencing the deleted employee tuple. On the other hand, if the Ssn value for a supervising employee is updated (say, because it was entered incorrectly), the new value is cascaded to Super_ssn for all employee tuples referencing the updated employee tuple.⁸

In general, the action taken by the DBMS for SET NULL or SET DEFAULT is the same for both ON DELETE and ON UPDATE: The value of the affected referencing attributes is changed to NULL for SET NULL and to the specified default value of the

7

Key and referential integrity constraints were not included in early versions of SQL.

8

Notice that the foreign key `Super_ssn` in the `EMPLOYEE` table is a circular reference and hence may have to be added later as a named constraint using the `ALTER TABLE` statement as we discussed at the end of Section 6.1.2.

6.3 Basic Retrieval Queries in SQL 217

referencing attribute for `SET DEFAULT`. The action for `CASCADE ON DELETE` is to delete all the referencing tuples, whereas the action for `CASCADE ON UPDATE` is to change the value of the referencing foreign key attribute(s) to the updated (new) primary key value for all the referencing tuples. It is the responsibility of the database designer to choose the appropriate action and to specify it in the database schema. As a general rule, the `CASCADE` option is suitable for “relationship” relations (see Section 9.1), such as `WORKS_ON`; for relations that represent multivalued attributes, such as `DEPT_LOCATIONS`; and for relations that represent weak entity types, such as `DEPENDENT`.

6.2.3 Giving Names to Constraints

Figure 6.2 also illustrates how a constraint may be given a constraint name, following the keyword `CONSTRAINT`. The names of all constraints within a particular schema must be unique. A constraint name is used to identify a particular constraint in case the constraint must be dropped later and replaced with another constraint, as we discuss in Chapter 7. Giving names to constraints is optional. It is also possible to temporarily defer a constraint until the end of a transaction, as we shall discuss in Chapter 20 when we present transaction concepts.

6.2.4 Specifying Constraints on Tuples Using `CHECK`

In addition to key and referential integrity constraints, which are specified by special keywords, other table constraints can be specified through additional `CHECK` clauses at the end of a `CREATE TABLE` statement. These can be called row-based constraints because they apply to each row individually and are checked whenever a row is inserted or modified. For example, suppose that the `DEPARTMENT` table in Figure 6.1 had an additional attribute `Dept_create_date`, which stores the date when the department was created. Then we could add the following `CHECK` clause at the end of the `CREATE TABLE` statement for the `DEPARTMENT` table to make sure that a manager’s start date is later than the department creation date.

```
CHECK (Dept_create_date <= Mgr_start_date);
```

The `CHECK` clause can also be used to specify more general constraints using the `CREATE ASSERTION` statement of SQL. We discuss this in Chapter 7 because it requires the full power of queries, which are discussed in Sections 6.3 and 7.1.

6.3 Basic Retrieval Queries in SQL

SQL has one basic statement for retrieving information from a database: the SELECT statement. The SELECT statement is not the same as the SELECT operation of relational algebra, which we shall discuss in Chapter 8. There are many options and flavors to the SELECT statement in SQL, so we will introduce its features gradually. We will use example queries specified on the schema of Figure 5.5 and will

218 Chapter 6 Basic SQL

refer to the sample database state shown in Figure 5.6 to show the results of some of these queries. In this section, we present the features of SQL for simple retrieval queries. Features of SQL for specifying more complex retrieval queries are presented in Section 7.1.

Before proceeding, we must point out an important distinction between the practical SQL model and the formal relational model discussed in Chapter 5: SQL allows a table (relation) to have two or more tuples that are identical in all their attribute values. Hence, in general, an SQL table is not a set of tuples, because a set does not allow two identical members; rather, it is a multiset (sometimes called a bag) of tuples. Some SQL relations are constrained to be sets because a key constraint has been declared or because the DISTINCT option has been used with the SELECT statement (described later in this section). We should be aware of this distinction as we discuss the examples.

6.3.1 The SELECT-FROM-WHERE Structure of Basic SQL Queries

Queries in SQL can be very complex. We will start with simple queries, and then progress to more complex ones in a step-by-step manner. The basic form of the SELECT statement, sometimes called a mapping or a select-from-where block, is formed of the three clauses SELECT, FROM, and WHERE and has the following form:

SELECT <attribute list>

FROM <table list>

WHERE <condition>;

where

■ ■ <attribute list> is a list of attribute names whose values are to be retrieved by the query.

■ ■ <table list> is a list of the relation names required to process the query.

■ ■ <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query.

In SQL, the basic logical comparison operators for comparing attribute values with one another and with literal constants are =, <, <=, >, >=, and <>. These correspond to the relational algebra operators =, <, ≤, >, ≥, and ≠, respectively, and to the C/C++ programming language operators =, <, <=, >, >=, and !=. The main syntactic difference is the not equal operator. SQL has additional comparison operators that we will present gradually.

We illustrate the basic SELECT statement in SQL with some sample queries. The

queries are labeled here with the same query numbers used in Chapter 8 for easy cross-reference.

9

The SELECT and FROM clauses are required in all SQL queries. The WHERE is optional (see Section 6.3.3).

6.3 Basic Retrieval Queries in SQL 219

Query 0. Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

Q0: SELECT Bdate, Address

FROM EMPLOYEE

WHERE Fname = 'John' AND Minit = 'B' AND Lname = 'Smith';

This query involves only the EMPLOYEE relation listed in the FROM clause. The query selects the individual EMPLOYEE tuples that satisfy the condition of the WHERE clause, then projects the result on the Bdate and Address attributes listed in the SELECT clause.

The SELECT clause of SQL specifies the attributes whose values are to be retrieved, which are called the projection attributes in relational algebra (see Chapter 8) and the WHERE clause specifies the Boolean condition that must be true for any retrieved tuple, which is known as the selection condition in relational algebra.

Figure 6.3(a) shows the result of query Q0 on the database of Figure 5.6.

We can think of an implicit tuple variable or iterator in the SQL query ranging or looping over each individual tuple in the EMPLOYEE table and evaluating the condition in the WHERE clause. Only those tuples that satisfy the condition—that is, those tuples for which the condition evaluates to TRUE after substituting their corresponding attribute values—are selected.

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

Q1: SELECT Fname, Lname, Address

FROM EMPLOYEE, DEPARTMENT

WHERE Dname = 'Research' AND Dnumber = Dno;

In the WHERE clause of Q1, the condition Dname = 'Research' is a selection condition that chooses the particular tuple of interest in the DEPARTMENT table, because Dname is an attribute of DEPARTMENT. The condition Dnumber = Dno is called a join condition, because it combines two tuples: one from DEPARTMENT and one from EMPLOYEE, whenever the value of Dnumber in DEPARTMENT is equal to the value of Dno in EMPLOYEE. The result of query Q1 is shown in Figure 6.3(b). In general, any number of selection and join conditions may be specified in a single SQL query.

A query that involves only selection and join conditions plus projection attributes is known as a select-project-join query. The next example is a select-project-join query with two join conditions.

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
Q2: SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND
Plocation = 'Stafford'
```

220 Chapter 6 Basic SQL

Figure 6.3

Results of SQL queries when applied to the COMPANY database state shown in Figure 5.6. (a) Q0. (b) Q1. (c) Q2. (d) Q8. (e) Q9. (f) Q10. (g) Q1C.

(a) Bdate Address (b) Fname Lname Address

1965-01-09 731Fondren, Houston, TX John Smith 731 Fondren, Houston, TX
Franklin Wong 638 Voss, Houston, TX
Ramesh Narayan 975 Fire Oak, Humble, TX
Joyce English 5631 Rice, Houston, TX

(c) Pnumber Dnum Lname Address Bdate (f) Ssn Dname

10 4 Wallace 291Berry, Bellaire, TX 1941-06-20 123456789 Research
30 4 Wallace 291Berry, Bellaire, TX 1941-06-20 333445555 Research
999887777 Research
987654321 Research

(d) E.Fname E.Lname S.Fname S.Lname

666884444 Research
John Smith Franklin Wong
453453453 Research
Franklin Wong James Borg
987987987 Research
Alicia Zelaya Jennifer Wallace
888665555 Research
Jennifer Wallace James Borg
123456789 Administration
Ramesh Narayan Franklin Wong
333445555 Administration
Joyce English Franklin Wong
999887777 Administration
Ahmad Jabbar Jennifer Wallace
987654321 Administration
666884444 Administration
(e) E.Fname 453453453 Administration
123456789 987987987 Administration
333445555 888665555 Administration

999887777 123456789 Headquarters
 987654321
 333445555 Headquarters
 666884444 999887777 Headquarters
 453453453 987654321 Headquarters
 987987987
 666884444 Headquarters
 888665555
 453453453 Headquarters
 987987987 Headquarters
 888665555 Headquarters
 (g)

Fname Minit Lname Ssn Bdate Address Sex Salary Super_ssn Dno
 John B Smith 123456789 1965-09-01 731 Fondren, Houston, TX M 30000 333445555 5
 Franklin T Wong 333445555 1955-12-08 638 Voss, Houston, TX M 40000 888665555 5
 Ramesh K Narayan 666884444 1962-09-15 975 Fire Oak, Humble, TX M 38000 333445555 5
 Joyce A English 453453453 1972-07-31 5631 Rice, Houston, TX F 25000 333445555 5

6.3 Basic Retrieval Queries in SQL 221

The join condition $Dnum = Dnumber$ relates a project tuple to its controlling department tuple, whereas the join condition $Mgr_ssn = Ssn$ relates the controlling department tuple to the employee tuple who manages that department. Each tuple in the result will be a combination of one project, one department (that controls the project), and one employee (that manages the department). The projection attributes are used to choose the attributes to be displayed from each combined tuple. The result of query Q2 is shown in Figure 6.3(c).

6.3.2 Ambiguous Attribute Names, Aliasing, Renaming, and Tuple Variables

In SQL, the same name can be used for two (or more) attributes as long as the attributes are in different tables. If this is the case, and a multitable query refers to two or more attributes with the same name, we must qualify the attribute name with the relation name to prevent ambiguity. This is done by prefixing the relation name to the attribute name and separating the two by a period. To illustrate this, suppose that in Figures 5.5 and 5.6 the Dno and Lname attributes of the EMPLOYEE relation were called Dnumber and Name, and the Dname attribute of DEPARTMENT was also called Name; then, to prevent ambiguity, query Q1 would be rephrased as shown in Q1A. We must prefix the attributes Name and Dnumber in Q1A to specify which ones we are referring to, because the same attribute names are used in both relations:

Q1A: SELECT Fname, EMPLOYEE.Name, Address
 FROM EMPLOYEE, DEPARTMENT
 WHERE DEPARTMENT.Name = 'Research' AND

DEPARTMENT.Dnumber = EMPLOYEE.Dnumber;

Fully qualified attribute names can be used for clarity even if there is no ambiguity in attribute names. Q1 can be rewritten as Q1' below with fully qualified attribute names. We can also rename the table names to shorter names by creating an alias for each table name to avoid repeated typing of long table names (see Q8 below).

```
Q1': SELECT EMPLOYEE.Fname, EMPLOYEE.LName,
EMPLOYEE.Address
FROM EMPLOYEE, DEPARTMENT
WHERE DEPARTMENT.DName = 'Research' AND
DEPARTMENT.Dnumber = EMPLOYEE.Dno;
```

The ambiguity of attribute names also arises in the case of queries that refer to the same relation twice, as in the following example.

Query 8. For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
Q8: SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

222 Chapter 6 Basic SQL

In this case, we are required to declare alternative relation names E and S, called aliases or tuple variables, for the EMPLOYEE relation. An alias can follow the keyword AS, as shown in Q8, or it can directly follow the relation name—for example, by writing EMPLOYEE E, EMPLOYEE S in the FROM clause of Q8. It is also possible to rename the relation attributes within the query in SQL by giving them aliases.

For example, if we write

```
EMPLOYEE AS E(Fn, Mi, Ln, Ssn, Bd, Addr, Sex, Sal, Sssn, Dno)
```

in the FROM clause, Fn becomes an alias for Fname, Mi for Minit, Ln for Lname, and so on.

In Q8, we can think of E and S as two different copies of the EMPLOYEE relation; the first, E, represents employees in the role of supervisees or subordinates; the second, S, represents employees in the role of supervisors. We can now join the two copies. Of course, in reality there is only one EMPLOYEE relation, and the join condition is meant to join the relation with itself by matching the tuples that satisfy the join condition E.Super_ssn = S.Ssn. Notice that this is an example of a one-level recursive query, as we will discuss in Section 8.4.2. In earlier versions of SQL, it was not possible to specify a general recursive query, with an unknown number of levels, in a single SQL statement. A construct for specifying recursive queries has been incorporated into SQL:1999 (see Chapter 7).

The result of query Q8 is shown in Figure 6.3(d). Whenever one or more aliases are given to a relation, we can use these names to represent different references

to that same relation. This permits multiple references to the same relation within a query.

We can use this alias-naming or renaming mechanism in any SQL query to specify tuple variables for every table in the WHERE clause, whether or not the same relation needs to be referenced more than once. In fact, this practice is recommended since it results in queries that are easier to comprehend. For example, we could specify query Q1 as in Q1B:

```
Q1B: SELECT E.Fname, E.LName, E.Address  
FROM EMPLOYEE AS E, DEPARTMENT AS D  
WHERE D.DName = 'Research' AND D.Dnumber = E.Dno;
```

6.3.3 Unspecified WHERE Clause and Use of the Asterisk

We discuss two more features of SQL here. A missing WHERE clause indicates no condition on tuple selection; hence, all tuples of the relation specified in the FROM clause qualify and are selected for the query result. If more than one relation is specified in the FROM clause and there is no WHERE clause, then the CROSS PRODUCT—all possible tuple combinations—of these relations is selected. For example, Query 9 selects all EMPLOYEE Ssns (Figure 6.3(e)), and Query 10 selects all combinations of an EMPLOYEE Ssn and a DEPARTMENT Dname, regardless of whether the employee works for the department or not (Figure 6.3(f)).

6.3 Basic Retrieval Queries in SQL 223

Queries 9 and 10. Select all EMPLOYEE Ssns (Q9) and all combinations of EMPLOYEE Ssn and DEPARTMENT Dname (Q10) in the database.

```
Q9: SELECT Ssn  
FROM EMPLOYEE;
```

```
Q10: SELECT Ssn, Dname  
FROM EMPLOYEE, DEPARTMENT;
```

It is extremely important to specify every selection and join condition in the WHERE clause; if any such condition is overlooked, incorrect and very large relations may result. Notice that Q10 is similar to a CROSS PRODUCT operation followed by a PROJECT operation in relational algebra (see Chapter 8). If we specify all the attributes of EMPLOYEE and DEPARTMENT in Q10, we get the actual CROSS PRODUCT (except for duplicate elimination, if any).

To retrieve all the attribute values of the selected tuples, we do not have to list the attribute names explicitly in SQL; we just specify an asterisk (*), which stands for all the attributes. The * can also be prefixed by the relation name or alias; for example, EMPLOYEE.* refers to all attributes of the EMPLOYEE table.

Query Q1C retrieves all the attribute values of any EMPLOYEE who works in DEPARTMENT number 5 (Figure 6.3(g)), query Q1D retrieves all the attributes of an EMPLOYEE and the attributes of the DEPARTMENT in which he or she works for

every employee of the 'Research' department, and Q10A specifies the CROSS PRODUCT of the EMPLOYEE and DEPARTMENT relations.

Q1C: SELECT *
FROM EMPLOYEE
WHERE Dno = 5;

Q1D: SELECT *
FROM EMPLOYEE, DEPARTMENT
WHERE Dname = 'Research' AND Dno = Dnumber;

Q10A: SELECT *
FROM EMPLOYEE, DEPARTMENT;

6.3.4 Tables as Sets in SQL

As we mentioned earlier, SQL usually treats a table not as a set but rather as a multiset; duplicate tuples can appear more than once in a table, and in the result of a query. SQL does not automatically eliminate duplicate tuples in the results of queries, for the following reasons:

- Duplicate elimination is an expensive operation. One way to implement it is to sort the tuples first and then eliminate duplicates.

- The user may want to see duplicate tuples in the result of a query.

- When an aggregate function (see Section 7.1.7) is applied to tuples, in most cases we do not want to eliminate duplicates.

224 Chapter 6 Basic SQL

(a) Salary (b) Salary (c) Fname Lname

30000 30000

Figure 6.4 40000 40000

Results of additional 25000 25000

SQL queries when

43000 43000

applied to the

COMPANY database 38000 38000 (d) Fname Lname

state shown in 25000 55000 James Borg

Figure 5.6. (a) Q11. 25000

(b) Q11A. (c) Q16.

(d) Q18. 55000

An SQL table with a key is restricted to being a set, since the key value must be distinct in each tuple.¹⁰ If we do want to eliminate duplicate tuples from the result of an SQL query, we use the keyword DISTINCT in the SELECT clause, meaning that only distinct tuples should remain in the result. In general, a query with SELECT DISTINCT eliminates duplicates, whereas a query with SELECT ALL does not. Specifying SELECT with neither ALL nor DISTINCT—as in our previous examples—is equivalent to SELECT ALL. For example, Q11 retrieves the salary of every employee;

if several employees have the same salary, that salary value will appear as many times in the result of the query, as shown in Figure 6.4(a). If we are interested only in distinct salary values, we want each value to appear only once, regardless of how many employees earn that salary. By using the keyword `DISTINCT` as in Q11A, we accomplish this, as shown in Figure 6.4(b).

Query 11. Retrieve the salary of every employee (Q11) and all distinct salary values (Q11A).

Q11: `SELECT ALL Salary`
`FROM EMPLOYEE;`

Q11A: `SELECT DISTINCT Salary`
`FROM EMPLOYEE;`

SQL has directly incorporated some of the set operations from mathematical set theory, which are also part of relational algebra (see Chapter 8). There are set union (`UNION`), set difference (`EXCEPT`),¹¹ and set intersection (`INTERSECT`) operations. The relations resulting from these set operations are sets of tuples; that is, duplicate tuples are eliminated from the result. These set operations apply only to type-compatible relations, so we must make sure that the two relations on which we apply the operation have the same attributes and that the attributes appear in the same order in both relations. The next example illustrates the use of `UNION`.

10

In general, an SQL table is not required to have a key, although in most cases there will be one.

11

In some systems, the keyword `MINUS` is used for the set difference operation instead of `EXCEPT`.

6.3 Basic Retrieval Queries in SQL 225

(a) R S (b) T (c) T

A A A A

a1 a1 a1 a2

a2 a2 a1 a3

a2 a4 a2 Figure 6.5

The results of SQL multiset

a3 a5 a2 operations. (a) Two tables,

a2 (d) T R(A) and S(A).

a3 A (b) R(A) `UNION ALL` S(A).

(c) R(A) `EXCEPT ALL` S(A).

a4 a1

(d) R(A) `INTERSECT ALL`

a5 a2 S(A).

Query 4. Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```

Q4A: ( SELECT DISTINCT Pnumber
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn
AND Lname = 'Smith' )
UNION
( SELECT DISTINCT Pnumber
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber = Pno AND Essn = Ssn
AND Lname = 'Smith' );

```

The first SELECT query retrieves the projects that involve a 'Smith' as manager of the department that controls the project, and the second retrieves the projects that involve a 'Smith' as a worker on the project. Notice that if several employees have the last name 'Smith', the project names involving any of them will be retrieved. Applying the UNION operation to the two SELECT queries gives the desired result. SQL also has corresponding multiset operations, which are followed by the keyword ALL (UNION ALL, EXCEPT ALL, INTERSECT ALL). Their results are multisets (duplicates are not eliminated). The behavior of these operations is illustrated by the examples in Figure 6.5. Basically, each tuple—whether it is a duplicate or not—is considered as a different tuple when applying these operations.

6.3.5 Substring Pattern Matching and Arithmetic Operators

In this section we discuss several more features of SQL. The first feature allows comparison conditions on only parts of a character string, using the LIKE comparison operator. This can be used for string pattern matching. Partial strings are specified using two reserved characters: % replaces an arbitrary number of zero or more characters, and the underscore (_) replaces a single character. For example, consider the following query.

226 Chapter 6 Basic SQL

Query 12. Retrieve all employees whose address is in Houston, Texas.

```

Q12: SELECT Fname, Lname
FROM EMPLOYEE
WHERE Address LIKE '%Houston,TX%';

```

To retrieve all employees who were born during the 1970s, we can use Query Q12A. Here, '7' must be the third character of the string (according to our format for date), so we use the value '__ 7 _____', with each underscore serving as a placeholder for an arbitrary character.

Query 12A. Find all employees who were born during the 1950s.

```

Q12: SELECT Fname, Lname
FROM EMPLOYEE
WHERE Bdate LIKE '__ 7 _____';

```

If an underscore or % is needed as a literal character in the string, the character

should be preceded by an escape character, which is specified after the string using the keyword ESCAPE. For example, 'AB_CD\%EF' ESCAPE '\' represents the literal string 'AB_CD%EF' because \ is specified as the escape character. Any character not used in the string can be chosen as the escape character. Also, we need a rule to specify apostrophes or single quotation marks (' ') if they are to be included in a string because they are used to begin and end strings. If an apostrophe (') is needed, it is represented as two consecutive apostrophes (") so that it will not be interpreted as ending the string. Notice that substring comparison implies that attribute values are not atomic (indivisible) values, as we had assumed in the formal relational model (see Section 5.1) .

Another feature allows the use of arithmetic in queries. The standard arithmetic operators for addition (+), subtraction (−), multiplication (*), and division (/) can be applied to numeric values or attributes with numeric domains. For example, suppose that we want to see the effect of giving all employees who work on the 'ProductX' project a 10% raise; we can issue Query 13 to see what their salaries would become. This example also shows how we can rename an attribute in the query result using AS in the SELECT clause.

Query 13. Show the resulting salaries if every employee working on the 'ProductX' project is given a 10% raise.

```
Q13: SELECT E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
FROM EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE E.Ssn = W.Essn AND W.Pno = P.Pnumber AND
P.Pname = 'ProductX';
```

For string data types, the concatenate operator || can be used in a query to append two string values. For date, time, timestamp, and interval data types, operators include incrementing (+) or decrementing (−) a date, time, or timestamp by an interval. In addition, an interval value is the result of the difference between two date, time, or timestamp values. Another comparison operator, which can be used for convenience, is BETWEEN, which is illustrated in Query 14.

6.3 Basic Retrieval Queries in SQL 227

Query 14. Retrieve all employees in department 5 whose salary is between \$30,000 and \$40,000.

```
Q14: SELECT *
FROM EMPLOYEE
WHERE (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

The condition (Salary BETWEEN 30000 AND 40000) in Q14 is equivalent to the condition ((Salary >= 30000) AND (Salary <= 40000)).

6.3.6 Ordering of Query Results

SQL allows the user to order the tuples in the result of a query by the values of one or more of the attributes that appear in the query result, by using the ORDER BY

clause. This is illustrated by Query 15.

Query 15. Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, then first name.

```
Q15: SELECT D.Dname, E.Lname, E.Fname, P.Pname
FROM DEPARTMENT AS D, EMPLOYEE AS E, WORKS_ON AS W,
PROJECT AS P
WHERE D.Dnumber = E.Dno AND E.Ssn = W.Essn AND W.Pno =
P.Pnumber
ORDER BY D.Dname, E.Lname, E.Fname;
```

The default order is in ascending order of values. We can specify the keyword DESC if we want to see the result in a descending order of values. The keyword ASC can be used to specify ascending order explicitly. For example, if we want descending alphabetical order on Dname and ascending order on Lname, Fname, the ORDER BY clause of Q15 can be written as

```
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC
```

6.3.7 Discussion and Summary of

Basic SQL Retrieval Queries

A simple retrieval query in SQL can consist of up to four clauses, but only the first two—SELECT and FROM—are mandatory. The clauses are specified in the following order, with the clauses between square brackets [...] being optional:

```
SELECT <attribute list>
FROM <table list>
[ WHERE <condition> ]
[ ORDER BY <attribute list> ];
```

The SELECT clause lists the attributes to be retrieved, and the FROM clause specifies all relations (tables) needed in the simple query. The WHERE clause identifies the conditions for selecting the tuples from these relations, including

228 Chapter 6 Basic SQL

join conditions if needed. ORDER BY specifies an order for displaying the results of a query. Two additional clauses GROUP BY and HAVING will be described in Section 7.1.8.

In Chapter 7, we will present more complex features of SQL retrieval queries. These include the following: nested queries that allow one query to be included as part of another query; aggregate functions that are used to provide summaries of the information in the tables; two additional clauses (GROUP BY and HAVING) that can be used to provide additional power to aggregate functions; and various types of joins that can combine records from various tables in different ways.

6.4 INSERT, DELETE, and UPDATE

Statements in SQL

In SQL, three commands can be used to modify the database: INSERT, DELETE, and UPDATE. We discuss each of these in turn.

6.4.1 The INSERT Command

In its simplest form, INSERT is used to add a single tuple (row) to a relation (table). We must specify the relation name and a list of values for the tuple. The values should be listed in the same order in which the corresponding attributes were specified in the CREATE TABLE command. For example, to add a new tuple to the EMPLOYEE relation shown in Figure 5.5 and specified in the CREATE TABLE EMPLOYEE ... command in Figure 6.1, we can use U1:

U1: INSERT INTO EMPLOYEE

VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
Oak Forest, Katy, TX', 'M', 37000, '653298653', 4);

A second form of the INSERT statement allows the user to specify explicit attribute names that correspond to the values provided in the INSERT command. This is useful if a relation has many attributes but only a few of those attributes are assigned values in the new tuple. However, the values must include all attributes with NOT NULL specification and no default value. Attributes with NULL allowed or DEFAULT values are the ones that can be left out. For example, to enter a tuple for a new EMPLOYEE for whom we know only the Fname, Lname, Dno, and Ssn attributes, we can use U1A:

U1A: INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)

VALUES ('Richard', 'Marini', 4, '653298653');

Attributes not specified in U1A are set to their DEFAULT or to NULL, and the values are listed in the same order as the attributes are listed in the INSERT command itself. It is also possible to insert into a relation multiple tuples separated by commas in a single INSERT command. The attribute values forming each tuple are enclosed in parentheses.

6.4 INSERT, DELETE, and UPDATE Statements in SQL 229

A DBMS that fully implements SQL should support and enforce all the integrity constraints that can be specified in the DDL. For example, if we issue the command in U2 on the database shown in Figure 5.6, the DBMS should reject the operation because no DEPARTMENT tuple exists in the database with Dnumber = 2. Similarly, U2A would be rejected because no Ssn value is provided and it is the primary key, which cannot be NULL.

U2: INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno)

VALUES ('Robert', 'Hatcher', '980760540', 2);

(U2 is rejected if referential integrity checking is provided by DBMS.)

U2A: INSERT INTO EMPLOYEE (Fname, Lname, Dno)

VALUES ('Robert', 'Hatcher', 5);

(U2A is rejected if NOT NULL checking is provided by DBMS.)

A variation of the INSERT command inserts multiple tuples into a relation in con-

junction with creating the relation and loading it with the result of a query. For example, to create a temporary table that has the employee last name, project name, and hours per week for each employee working on a project, we can write the statements in U3A and U3B:

```
U3A: CREATE TABLE WORKS_ON_INFO
```

```
( Emp_name VARCHAR(15),
```

```
Proj_name VARCHAR(15),
```

```
Hours_per_week DECIMAL(3,1) );
```

```
U3B: INSERT INTO WORKS_ON_INFO ( Emp_name, Proj_name,
```

```
Hours_per_week )
```

```
SELECT E.Lname, P.Pname, W.Hours
```

```
FROM PROJECT P, WORKS_ON W, EMPLOYEE E
```

```
WHERE P.Pnumber = W.Pno AND W.Essn = E.Ssn;
```

A table WORKS_ON_INFO is created by U3A and is loaded with the joined information retrieved from the database by the query in U3B. We can now query WORKS_ON_INFO as we would any other relation; when we do not need it anymore, we can remove it by using the DROP TABLE command (see Chapter 7). Notice that the WORKS_ON_INFO table may not be up to date; that is, if we update any of the PROJECT, WORKS_ON, or EMPLOYEE relations after issuing U3B, the information in WORKS_ON_INFO may become outdated. We have to create a view (see Chapter 7) to keep such a table up to date.

Most DBMSs have bulk loading tools that allow a user to load formatted data from a file into a table without having to write a large number of INSERT commands. The user can also write a program to read each record in the file, format it as a row in the table, and insert it using the looping constructs of a programming language (see Chapters 10 and 11, where we discuss database programming techniques). Another variation for loading data is to create a new table TNEW that has the same attributes as an existing table T, and load some of the data currently in T into TNEW. The syntax for doing this uses the LIKE clause. For example, if we

230 Chapter 6 Basic SQL

want to create a table D5EMPS with a similar structure to the EMPLOYEE table and load it with the rows of employees who work in department 5, we can write the following SQL:

```
CREATE TABLE D5EMPS LIKE EMPLOYEE
```

```
(SELECT E.*
```

```
FROM EMPLOYEE AS E
```

```
WHERE E.Dno = 5) WITH DATA;
```

The clause WITH DATA specifies that the table will be created and loaded with the data specified in the query, although in some implementations it may be left out.

6.4.2 The DELETE Command

The DELETE command removes tuples from a relation. It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted. Tuples are explicitly deleted from only one table at a time. However, the deletion may propagate to tuples in other relations if referential triggered actions are specified in the referential integrity constraints of the DDL (see Section 6.2.2).¹² Depending on the number of tuples selected by the condition in the WHERE clause, zero, one, or several tuples can be deleted by a single DELETE command. A missing WHERE clause specifies that all tuples in the relation are to be deleted; however, the table remains in the database as an empty table. We must use the DROP TABLE command to remove the table definition (see Chapter 7). The DELETE commands in U4A to U4D, if applied independently to the database state shown in Figure 5.6, will delete zero, one, four, and all tuples, respectively, from the EMPLOYEE relation:

U4A: DELETE FROM EMPLOYEE

WHERE Lname = 'Brown';

U4B: DELETE FROM EMPLOYEE

WHERE Ssn = '123456789';

U4C: DELETE FROM EMPLOYEE

WHERE Dno = 5;

U4D: DELETE FROM EMPLOYEE;

6.4.3 The UPDATE Command

The UPDATE command is used to modify attribute values of one or more selected tuples. As in the DELETE command, a WHERE clause in the UPDATE command selects the tuples to be modified from a single relation. However, updating a primary key value may propagate to the foreign key values of tuples in other relations if such a referential triggered action is specified in the referential integrity

12

Other actions can be automatically applied through triggers (see Section 26.1) and other mechanisms.

6.5 Additional Features of SQL 231

constraints of the DDL (see Section 6.2.2). An additional SET clause in the UPDATE command specifies the attributes to be modified and their new values. For example, to change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively, we use U5:

U5: UPDATE PROJECT

SET Plocation = 'Bellaire', Dnum = 5

WHERE Pnumber = 10;

Several tuples can be modified with a single UPDATE command. An example is to give all employees in the 'Research' department a 10% raise in salary, as shown in U6. In this request, the modified Salary value depends on the original Salary value

in each tuple, so two references to the Salary attribute are needed. In the SET clause, the reference to the Salary attribute on the right refers to the old Salary value before modification, and the one on the left refers to the new Salary value after modification:

```
U6: UPDATE EMPLOYEE
```

```
SET Salary = Salary * 1.1
```

```
WHERE Dno = 5;
```

It is also possible to specify NULL or DEFAULT as the new attribute value. Notice that each UPDATE command explicitly refers to a single relation only. To modify multiple relations, we must issue several UPDATE commands.

6.5 Additional Features of SQL

SQL has a number of additional features that we have not described in this chapter but that we discuss elsewhere in the book. These are as follows:

■■ In Chapter 7, which is a continuation of this chapter, we will present the following SQL features: various techniques for specifying complex retrieval queries, including nested queries, aggregate functions, grouping, joined tables, outer joins, case statements, and recursive queries; SQL views, triggers, and assertions; and commands for schema modification.

■■ SQL has various techniques for writing programs in various programming languages that include SQL statements to access one or more databases. These include embedded (and dynamic) SQL, SQL/CLI (Call Level Interface) and its predecessor ODBC (Open Data Base Connectivity), and SQL/PSM (Persistent Stored Modules). We discuss these techniques in Chapter 10. We also describe how to access SQL databases through the Java programming language using JDBC and SQLJ.

■■ Each commercial RDBMS will have, in addition to the SQL commands, a set of commands for specifying physical database design parameters, file structures for relations, and access paths such as indexes. We called these commands a storage definition language (SDL) in Chapter 2. Earlier versions of SQL had commands for creating indexes, but these were removed from the

232 Chapter 6 Basic SQL

language because they were not at the conceptual schema level. Many systems still have the CREATE INDEX commands; but they require a special privilege. We describe this in Chapter 17.

■■ SQL has transaction control commands. These are used to specify units of database processing for concurrency control and recovery purposes. We discuss these commands in Chapter 20 after we discuss the concept of transactions in more detail.

■■ SQL has language constructs for specifying the granting and revoking of privileges to users. Privileges typically correspond to the right to use certain SQL commands to access certain relations. Each relation is assigned an

owner, and either the owner or the DBA staff can grant to selected users the privilege to use an SQL statement—such as SELECT, INSERT, DELETE, or UPDATE—to access the relation. In addition, the DBA staff can grant the privileges to create schemas, tables, or views to certain users. These SQL commands—called GRANT and REVOKE—are discussed in Chapter 20, where we discuss database security and authorization.

■ ■ SQL has language constructs for creating triggers. These are generally referred to as active database techniques, since they specify actions that are automatically triggered by events such as database updates. We discuss these features in Section 26.1, where we discuss active database concepts.

■ ■ SQL has incorporated many features from object-oriented models to have more powerful capabilities, leading to enhanced relational systems known as object-relational. Capabilities such as creating complex-structured attributes, specifying abstract data types (called UDTs or user-defined types) for attributes and tables, creating object identifiers for referencing tuples, and specifying operations on types are discussed in Chapter 12.

■ ■ SQL and relational databases can interact with new technologies such as XML (see Chapter 13) and OLAP/data warehouses (Chapter 29).

6.6 Summary

In this chapter, we introduced the SQL database language. This language and its variations have been implemented as interfaces to many commercial relational DBMSs, including Oracle's Oracle; ibm's DB2; Microsoft's SQL Server; and many other systems including Sybase and INGRES. Some open source systems also provide SQL, such as MySQL and PostgreSQL. The original version of SQL was implemented in the experimental DBMS called SYSTEM R, which was developed at IBM Research. SQL is designed to be a comprehensive language that includes statements for data definition, queries, updates, constraint specification, and view definition. We discussed the following features of SQL in this chapter: the data definition commands for creating tables, SQL basic data types, commands for constraint specification, simple retrieval queries, and database update commands. In the next chapter, we will present the following features of SQL: complex retrieval queries; views; triggers and assertions; and schema modification commands.

Exercises 233

Review Questions

- 6.1. How do the relations (tables) in SQL differ from the relations defined formally in Chapter 3? Discuss the other differences in terminology. Why does SQL allow duplicate tuples in a table or in a query result?
- 6.2. List the data types that are allowed for SQL attributes.
- 6.3. How does SQL allow implementation of the entity integrity and referential integrity constraints described in Chapter 3? What about referential triggered actions?

6.4. Describe the four clauses in the syntax of a simple SQL retrieval query. Show what type of constructs can be specified in each of the clauses. Which are required and which are optional?

Exercises

6.5. Consider the seven relations for a car-rental database application provided in exercise 5.14. What are the referential integrity constraints that should hold on the schema? Write appropriate SQL DDL statements to define the database.

6.6. Repeat Exercise 6.5, but use the AIRLINE database schema of Figure 5.8.

6.7. Consider the following Hotel, Room, Booking and Guest schemas in a DBMS. The hotelNo is the primary key for Hotel table and roomNo is the primary key for the Room relation. Booking stores the details of room reservations and bookingNo is the primary key. Guest stores the guests details and guestNo is the primary key.

Hotel (hotelNo, hotelName, hotelType, hotelAddress, hotelCity, numRoom)

Room (roomNo, hotelNo, roomPrice)

Booking (bookingNo, hotelNo, guestNo, checkIn, checkOut, totalGuest, roomNo)

Guest (guestNo, firstName, lastName, guestAddress)

1. Write the SQL to list full details of all the hotels.

2. Write the SQL to list full details of all the hotels in New York.

3. Write the SQL to list the guests in New York in descending order by last name.

6.8. Write appropriate SQL DDL statements for declaring the LIBRARY relational database schema of Figure 6.6. Specify the keys and referential triggered actions.

6.9. How can the key and foreign key constraints be enforced by the DBMS? Is the enforcement technique you suggest difficult to implement? Can the constraint checks be executed efficiently when updates are applied to the database?

234 Chapter 6 Basic SQL

BOOK

Book_id Title Publisher_name

BOOK_AUTHORS

Book_id Author_name

PUBLISHER

Name Address Phone

BOOK_COPIES

Book_id Branch_id No_of_copies

BOOK_LOANS

Book_id Branch_id Card_no Date_out Due_date

LIBRARY_BRANCH

Branch_id Branch_name Address

Figure 6.6 BORROWER

A relational database Card_no Name Address Phone

schema for a

LIBRARY database.

6.10. Specify the following queries in SQL on the COMPANY relational database schema shown in Figure 5.5. Show the result of each query if it is applied to the COMPANY database in Figure 5.6.

- Retrieve the names of all employees in department 5 who earns more than 3000 and works on ProductZ project.
- List the names of all employees who are from Houston, Texas and works under manager 333445555.
- Find the names of all employees who are working in the project Computerisation.

6.11. Specify the updates of Exercise 5.11 using the SQL update commands.

Exercises 235

6.12. Specify the following queries in SQL on the database schema of Figure 1.2.

- Retrieve the course names of all the courses that comes under the department of 'cs' (computer science).
- Retrieve the names of all courses along with the name of the instructor taught during the fall of 2008.
- For each section taught by Professor Anderson, retrieve the course number, semester, year, and number of students who took the section.
- Retrieve the name and transcript of each junior student (Class = 1) majoring in mathematics (MATH). A transcript includes course name, course number, credit hours, semester, year, and grade for each course completed by the student.

6.13. Write SQL update statements to do the following on the database schema shown in Figure 1.2.

- Insert a new course, <'Financial Accounting', 'fac4390', 5, 'BUSINESS'>.
- Insert a new section, <145, 'fac4390', 'Fall', '17', 'Hanif' >.
- Insert a new student, <'Robin', 34, 3, 'BUSINESS'>.
- Update the record for the student whose student number is 17 and change his class from 1 to 3.

6.14. Referring to Exercise 6.7, insert the following values into the hotel table and update the price of all rooms by 15%.

hotelNo hotelName hotelType hotelAddress hotelCity numRoom

H1001 Hilton Hong Luxury Beach Street Hong 200

Kong Kong

6.15. Consider that the EMPLOYEE table's constraint EMPSUPERFK as specified in Figure 6.2 is changed to read as follows:

```
CONSTRAINT EMPSUPERFK
FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
ON DELETE CASCADE ON UPDATE CASCADE,
```

Answer the following questions:

a. What happens when the following command is run on the database state shown in Figure 5.6?

```
DELETE EMPLOYEE WHERE Lname = 'Borg'
```

b. Is it better to CASCADE or SET NULL in case of EMPSUPERFK constraint ON DELETE?

6.16. Write SQL statements to create a table EMPLOYEE_BACKUP to back up the EMPLOYEE table shown in Figure 5.6.

236 Chapter 6 Basic SQL

Selected Bibliography

The SQL language, originally named SEQUEL, was based on the language SQUARE (Specifying Queries as Relational Expressions) described by Boyce et al. (1975). The syntax of SQUARE was modified into SEQUEL (Chamberlin & Boyce, 1974) and then into SEQUEL 2 (Chamberlin et al., 1976), on which SQL is based. The original implementation of SEQUEL was done at IBM Research, San Jose, California. We will give additional references to various aspects of SQL at the end of Chapter 7.

chapter 7

More SQL: Complex Queries,
Triggers, Views, and
Schema Modification

This chapter describes more advanced features of the SQL language for relational databases. We start in Section 7.1 by presenting more complex features of SQL retrieval queries, such as nested queries, joined tables, outer joins, aggregate functions, and grouping, and case statements. In Section 7.2, we describe the CREATE ASSERTION statement, which allows the specification of more general constraints on the database. We also introduce the concept of triggers and the CREATE TRIGGER statement, which will be presented in more detail in Section 26.1 when we present the principles of active databases. Then, in Section 7.3, we describe the SQL facility for defining views on the database. Views are also called virtual or derived tables because they present the user with what appear to be tables; however, the information in those tables is derived from previously defined tables. Section 7.4 introduces the SQL ALTER TABLE statement, which is used for modifying the database tables and constraints. Section 7.5 is the chapter summary.

This chapter is a continuation of Chapter 6. The instructor may skip parts of this

chapter if a less detailed introduction to SQL is intended.

7.1 More Complex SQL Retrieval Queries

In Section 6.3, we described some basic types of retrieval queries in SQL. Because of the generality and expressive power of the language, there are many additional features that allow users to specify more complex retrievals from the database. We discuss several of these features in this section.

237

238 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

7.1.1 Comparisons Involving NULL and Three-Valued Logic

SQL has various rules for dealing with NULL values. Recall from Section 5.1.2 that NULL is used to represent a missing value, but that it usually has one of three different interpretations—value unknown (value exists but is not known, or it is not known whether or not the value exists), value not available (value exists but is purposely withheld), or value not applicable (the attribute does not apply to this tuple or is undefined for this tuple). Consider the following examples to illustrate each of the meanings of NULL.

1. Unknown value. A person's date of birth is not known, so it is represented by NULL in the database. An example of the other case of unknown would be NULL for a person's home phone because it is not known whether or not the person has a home phone.

2. Unavailable or withheld value. A person has a home phone but does not want it to be listed, so it is withheld and represented as NULL in the database.

3. Not applicable attribute. An attribute LastCollegeDegree would be NULL for a person who has no college degrees because it does not apply to that person.

It is often not possible to determine which of the meanings is intended; for example, a NULL for the home phone of a person can have any of the three meanings. Hence, SQL does not distinguish among the different meanings of NULL.

In general, each individual NULL value is considered to be different from every other NULL value in the various database records. When a record with NULL in one of its attributes is involved in a comparison operation, the result is considered to be UNKNOWN (it may be TRUE or it may be FALSE). Hence, SQL uses a three-valued logic with values TRUE, FALSE, and UNKNOWN instead of the standard two-valued (Boolean) logic with values TRUE or FALSE. It is therefore necessary to define the results (or truth values) of three-valued logical expressions when the logical connectives AND, OR, and NOT are used. Table 7.1 shows the resulting values.

Table 7.1 Logical Connectives in Three-Valued Logic

(a) AND TRUE FALSE UNKNOWN

TRUE TRUE FALSE UNKNOWN

FALSE FALSE FALSE FALSE

UNKNOWN UNKNOWN FALSE UNKNOWN

(b) OR TRUE FALSE UNKNOWN

TRUE TRUE TRUE TRUE

FALSE TRUE FALSE UNKNOWN

UNKNOWN TRUE UNKNOWN UNKNOWN

(c) NOT

TRUE FALSE

FALSE TRUE

UNKNOWN UNKNOWN

7.1 More Complex SQL Retrieval Queries 239

In Tables 7.1(a) and 7.1(b), the rows and columns represent the values of the results of comparison conditions, which would typically appear in the WHERE clause of an SQL query. Each expression result would have a value of TRUE, FALSE, or UNKNOWN. The result of combining the two values using the AND logical connective is shown by the entries in Table 7.1(a). Table 7.1(b) shows the result of using the OR logical connective. For example, the result of (FALSE AND UNKNOWN) is FALSE, whereas the result of (FALSE OR UNKNOWN) is UNKNOWN. Table 7.1(c) shows the result of the NOT logical operation. Notice that in standard Boolean logic, only TRUE or FALSE values are permitted; there is no UNKNOWN value.

In select-project-join queries, the general rule is that only those combinations of tuples that evaluate the logical expression in the WHERE clause of the query to TRUE are selected. Tuple combinations that evaluate to FALSE or UNKNOWN are not selected. However, there are exceptions to that rule for certain operations, such as outer joins, as we shall see in Section 7.1.6.

SQL allows queries that check whether an attribute value is NULL. Rather than using = or <> to compare an attribute value to NULL, SQL uses the comparison operators IS or IS NOT. This is because SQL considers each NULL value as being distinct from every other NULL value, so equality comparison is not appropriate. It follows that when a join condition is specified, tuples with NULL values for the join attributes are not included in the result (unless it is an OUTER JOIN; see Section 7.1.6). Query 18 illustrates NULL comparison by retrieving any employees who do not have a supervisor.

Query 18. Retrieve the names of all employees who do not have supervisors.

Q18: SELECT Fname, Lname

FROM EMPLOYEE

WHERE Super_ssn IS NULL;

7.1.2 Nested Queries, Tuples, and Set/Multiset Comparisons

Some queries require that existing values in the database be fetched and then used in a comparison condition. Such queries can be conveniently formulated by using nested queries, which are complete select-from-where blocks within another SQL query. That other query is called the outer query. These nested queries can also appear in the WHERE clause or the FROM clause or the SELECT clause or other

SQL clauses as needed. Query 4 is formulated in Q4 without a nested query, but it can be rephrased to use nested queries as shown in Q4A. Q4A introduces the comparison operator IN, which compares a value v with a set (or multiset) of values V and evaluates to TRUE if v is one of the elements in V.

In Q4A, the first nested query selects the project numbers of projects that have an employee with last name 'Smith' involved as manager, whereas the second nested query selects the project numbers of projects that have an employee with last name 'Smith' involved as worker. In the outer query, we use the OR logical connective to retrieve a PROJECT tuple if the PNUMBER value of that tuple is in the result of either nested query.

240 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

```
Q4A: SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN
( SELECT Pnumber
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Pnumber AND
Mgr_ssn = Ssn AND Lname = 'Smith' )
OR
Pnumber IN
( SELECT Pno
FROM WORKS_ON, EMPLOYEE
WHERE Essn = Ssn AND Lname = 'Smith' );
```

If a nested query returns a single attribute and a single tuple, the query result will be a single (scalar) value. In such cases, it is permissible to use = instead of IN for the comparison operator. In general, the nested query will return a table (relation), which is a set or multiset of tuples.

SQL allows the use of tuples of values in comparisons by placing them within parentheses. To illustrate this, consider the following query:

```
SELECT DISTINCT Essn
FROM WORKS_ON
WHERE (Pno, Hours) IN ( SELECT Pno, Hours
FROM WORKS_ON
WHERE Essn = '123456789' );
```

This query will select the Essns of all employees who work the same (project, hours) combination on some project that employee 'John Smith' (whose Ssn = '123456789') works on. In this example, the IN operator compares the subtuple of values in parentheses (Pno, Hours) within each tuple in WORKS_ON with the set of type-compatible tuples produced by the nested query.

In addition to the IN operator, a number of other comparison operators can be used to compare a single value v (typically an attribute name) to a set or multiset v (typically a nested query). The = ANY (or = SOME) operator returns TRUE if the value v

is equal to some value in the set V and is hence equivalent to IN . The two keywords ANY and $SOME$ have the same effect. Other operators that can be combined with ANY (or $SOME$) include $>$, $>=$, $<$, $<=$, and $<>$. The keyword ALL can also be combined with each of these operators. For example, the comparison condition $(v > ALL V)$ returns $TRUE$ if the value v is greater than all the values in the set (or multiset) V . An example is the following query, which returns the names of employees whose salary is greater than the salary of all the employees in department 5:

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ALL ( SELECT Salary
FROM EMPLOYEE
WHERE Dno = 5 );
```

7.1 More Complex SQL Retrieval Queries 241

Notice that this query can also be specified using the MAX aggregate function (see Section 7.1.7).

In general, we can have several levels of nested queries. We can once again be faced with possible ambiguity among attribute names if attributes of the same name exist—one in a relation in the $FROM$ clause of the outer query, and another in a relation in the $FROM$ clause of the nested query. The rule is that a reference to an unqualified attribute refers to the relation declared in the innermost nested query. For example, in the $SELECT$ clause and $WHERE$ clause of the first nested query of Q4A, a reference to any unqualified attribute of the $PROJECT$ relation refers to the $PROJECT$ relation specified in the $FROM$ clause of the nested query. To refer to an attribute of the $PROJECT$ relation specified in the outer query, we specify and refer to an alias (tuple variable) for that relation. These rules are similar to scope rules for program variables in most programming languages that allow nested procedures and functions. To illustrate the potential ambiguity of attribute names in nested queries, consider Query 16.

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
Q16: SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN ( SELECT D.Essn
FROM DEPENDENT AS D
WHERE E.Fname = D.Dependent_name
AND E.Sex = D.Sex );
```

In the nested query of Q16, we must qualify $E.Sex$ because it refers to the Sex attribute of $EMPLOYEE$ from the outer query, and $DEPENDENT$ also has an attribute called Sex . If there were any unqualified references to Sex in the nested query, they would refer to the Sex attribute of $DEPENDENT$. However, we would not have to qualify the attributes $Fname$ and Ssn of $EMPLOYEE$ if they appeared in the nested

query because the DEPENDENT relation does not have attributes called Fname and Ssn, so there is no ambiguity.

It is generally advisable to create tuple variables (aliases) for all the tables referenced in an SQL query to avoid potential errors and ambiguities, as illustrated in Q16.

7.1.3 Correlated Nested Queries

Whenever a condition in the WHERE clause of a nested query references some attribute of a relation declared in the outer query, the two queries are said to be correlated. We can understand a correlated query better by considering that the nested query is evaluated once for each tuple (or combination of tuples) in the outer query. For example, we can think of Q16 as follows: For each EMPLOYEE tuple, evaluate the nested query, which retrieves the Essn values for all DEPENDENT tuples with the same sex and name as that EMPLOYEE tuple; if the Ssn value of the EMPLOYEE tuple is in the result of the nested query, then select that EMPLOYEE tuple.

242 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

In general, a query written with nested select-from-where blocks and using the = or IN comparison operators can always be expressed as a single block query. For example, Q16 may be written as in Q16A:

```
Q16A: SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn AND E.Sex = D.Sex
AND E.Fname = D.Dependent_name;
```

7.1.4 The EXISTS and UNIQUE Functions in SQL

EXISTS and UNIQUE are Boolean functions that return TRUE or FALSE; hence, they can be used in a WHERE clause condition. The EXISTS function in SQL is used to check whether the result of a nested query is empty (contains no tuples) or not. The result of EXISTS is a Boolean value TRUE if the nested query result contains at least one tuple, or FALSE if the nested query result contains no tuples. We illustrate the use of EXISTS—and NOT EXISTS—with some examples. First, we formulate Query 16 in an alternative form that uses EXISTS as in Q16B:

```
Q16B: SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE EXISTS ( SELECT *
FROM DEPENDENT AS D
WHERE E.Ssn = D.Essn AND E.Sex = D.Sex
AND E.Fname = D.Dependent_name);
```

EXISTS and NOT EXISTS are typically used in conjunction with a correlated nested query. In Q16B, the nested query references the Ssn, Fname, and Sex attributes of the EMPLOYEE relation from the outer query. We can think of Q16B as follows: For each EMPLOYEE tuple, evaluate the nested query, which retrieves all DEPENDENT tuples with the same Essn, Sex, and Dependent_name as the EMPLOYEE tuple; if at

least one tuple EXISTS in the result of the nested query, then select that EMPLOYEE tuple. EXISTS(Q) returns TRUE if there is at least one tuple in the result of the nested query Q, and returns FALSE otherwise. On the other hand, NOT EXISTS(Q) returns TRUE if there are no tuples in the result of nested query Q, and returns FALSE otherwise. Next, we illustrate the use of NOT EXISTS.

Query 6. Retrieve the names of employees who have no dependents.

```
Q6: SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS ( SELECT *
FROM DEPENDENT
WHERE Ssn = Essn );
```

In Q6, the correlated nested query retrieves all DEPENDENT tuples related to a particular EMPLOYEE tuple. If none exist, the EMPLOYEE tuple is selected because the WHERE-clause condition will evaluate to TRUE in this case. We can explain Q6 as follows: For each EMPLOYEE tuple, the correlated nested query selects all

7.1 More Complex SQL Retrieval Queries 243

DEPENDENT tuples whose Essn value matches the EMPLOYEE Ssn; if the result is empty, no dependents are related to the employee, so we select that EMPLOYEE tuple and retrieve its Fname and Lname.

Query 7. List the names of managers who have at least one dependent.

```
Q7: SELECT Fname, Lname
FROM EMPLOYEE
WHERE EXISTS ( SELECT *
FROM DEPENDENT
WHERE Ssn = Essn )
AND
EXISTS ( SELECT *
FROM DEPARTMENT
WHERE Ssn = Mgr_ssn );
```

One way to write this query is shown in Q7, where we specify two nested correlated queries; the first selects all DEPENDENT tuples related to an EMPLOYEE, and the second selects all DEPARTMENT tuples managed by the EMPLOYEE. If at least one of the first and at least one of the second exists, we select the EMPLOYEE tuple. Can you rewrite this query using only a single nested query or no nested queries?

The query Q3: Retrieve the name of each employee who works on all the projects controlled by department number 5 can be written using EXISTS and NOT EXISTS in SQL systems. We show two ways of specifying this query Q3 in SQL as Q3A and Q3B. This is an example of certain types of queries that require universal quantification, as we will discuss in Section 8.6.7. One way to write this query is to use the

construct (S2 EXCEPT S1) as explained next, and checking whether the result is empty.¹ This option is shown as Q3A.

```
Q3A: SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS ( ( SELECT Pnumber
FROM PROJECT
WHERE Dnum = 5)
EXCEPT ( SELECT Pno
FROM WORKS_ON
WHERE Ssn = Essn) );
```

In Q3A, the first subquery (which is not correlated with the outer query) selects all projects controlled by department 5, and the second subquery (which is correlated) selects all projects that the particular employee being considered works on. If the set difference of the first subquery result MINUS (EXCEPT) the second subquery result is empty, it means that the employee works on all the projects and is therefore selected.

1

Recall that EXCEPT is the set difference operator. The keyword MINUS is also sometimes used, for example, in Oracle.

244 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

The second option is shown as Q3B. Notice that we need two-level nesting in Q3B and that this formulation is quite a bit more complex than Q3A.

```
Q3B: SELECT Lname, Fname
FROM EMPLOYEE
WHERE NOT EXISTS ( SELECT *
FROM WORKS_ON B
WHERE ( B.Pno IN ( SELECT Pnumber
FROM PROJECT
WHERE Dnum = 5 )
AND
NOT EXISTS ( SELECT *
FROM WORKS_ON C
WHERE C.Essn = Ssn
AND C.Pno = B.Pno )));
```

In Q3B, the outer nested query selects any WORKS_ON (B) tuples whose Pno is of a project controlled by department 5, if there is not a WORKS_ON (C) tuple with the same Pno and the same Ssn as that of the EMPLOYEE tuple under consideration in the outer query. If no such tuple exists, we select the EMPLOYEE tuple. The form of Q3B matches the following rephrasing of Query 3: Select each employee such that there does not exist a project controlled by department 5 that the employee does

not work on. It corresponds to the way we will write this query in tuple relation calculus (see Section 8.6.7).

There is another SQL function, `UNIQUE(Q)`, which returns `TRUE` if there are no duplicate tuples in the result of query `Q`; otherwise, it returns `FALSE`. This can be used to test whether the result of a nested query is a set (no duplicates) or a multiset (duplicates exist).

7.1.5 Explicit Sets and Renaming in SQL

We have seen several queries with a nested query in the `WHERE` clause. It is also possible to use an explicit set of values in the `WHERE` clause, rather than a nested query. Such a set is enclosed in parentheses in SQL.

Query 17. Retrieve the Social Security numbers of all employees who work on project numbers 1, 2, or 3.

```
Q17: SELECT DISTINCT Essn
FROM WORKS_ON
WHERE Pno IN (1, 2, 3);
```

In SQL, it is possible to rename any attribute that appears in the result of a query by adding the qualifier `AS` followed by the desired new name. Hence, the `AS` construct can be used to alias both attribute and relation names in general, and it can be used in appropriate parts of a query. For example, Q8A shows how query Q8 from Section 4.3.2 can be slightly changed to retrieve the last name of each employee and his or her supervisor while renaming the resulting attribute names

7.1 More Complex SQL Retrieval Queries 245

as `Employee_name` and `Supervisor_name`. The new names will appear as column headers for the query result.

```
Q8A: SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

7.1.6 Joined Tables in SQL and Outer Joins

The concept of a joined table (or joined relation) was incorporated into SQL to permit users to specify a table resulting from a join operation in the `FROM` clause of a query. This construct may be easier to comprehend than mixing together all the select and join conditions in the `WHERE` clause. For example, consider query Q1, which retrieves the name and address of every employee who works for the 'Research' department. It may be easier to specify the join of the `EMPLOYEE` and `DEPARTMENT` relations in the `WHERE` clause, and then to select the desired tuples and attributes. This can be written in SQL as in Q1A:

```
Q1A: SELECT Fname, Lname, Address
FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE Dname = 'Research';
```

The `FROM` clause in Q1A contains a single joined table. The attributes of such a table are all the attributes of the first table, `EMPLOYEE`, followed by all the attributes of

the second table, DEPARTMENT. The concept of a joined table also allows the user to specify different types of join, such as NATURAL JOIN and various types of OUTER JOIN. In a NATURAL JOIN on two relations R and S, no join condition is specified; an implicit EQUIJOIN condition for each pair of attributes with the same name from R and S is created. Each such pair of attributes is included only once in the resulting relation (see Sections 8.3.2 and 8.4.4 for more details on the various types of join operations in relational algebra).

If the names of the join attributes are not the same in the base relations, it is possible to rename the attributes so that they match, and then to apply NATURAL JOIN. In this case, the AS construct can be used to rename a relation and all its attributes in the FROM clause. This is illustrated in Q1B, where the DEPARTMENT relation is renamed as DEPT and its attributes are renamed as Dname, Dno (to match the name of the desired join attribute Dno in the EMPLOYEE table), Mssn, and Msdate. The implied join condition for this NATURAL JOIN is $EMPLOYEE.Dno = DEPT.Dno$, because this is the only pair of attributes with the same name after renaming:

```
Q1B: SELECT Fname, Lname, Address
FROM (EMPLOYEE NATURAL JOIN
(DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))
WHERE Dname = 'Research';
```

The default type of join in a joined table is called an inner join, where a tuple is included in the result only if a matching tuple exists in the other relation. For example, in query Q8A, only employees who have a supervisor are included in the result; 246 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

an EMPLOYEE tuple whose value for Super_ssn is NULL is excluded. If the user requires that all employees be included, a different type of join called OUTER JOIN must be used explicitly (see Section 8.4.4 for the definition of OUTER JOIN in relational algebra). There are several variations of OUTER JOIN, as we shall see. In the SQL standard, this is handled by explicitly specifying the keyword OUTER JOIN in a joined table, as illustrated in Q8B:

```
Q8B: SELECT E.Lname AS Employee_name,
S.Lname AS Supervisor_name
FROM (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
ON E.Super_ssn = S.Ssn);
```

In SQL, the options available for specifying joined tables include INNER JOIN (only pairs of tuples that match the join condition are retrieved, same as JOIN), LEFT OUTER JOIN (every tuple in the left table must appear in the result; if it does not have a matching tuple, it is padded with NULL values for the attributes of the right table), RIGHT OUTER JOIN (every tuple in the right table must appear in the result; if it does not have a matching tuple, it is padded with NULL values for the attributes of the left table), and FULL OUTER JOIN. In the latter three options, the keyword OUTER may be omitted. If the join attributes have the same name, one can also specify the natural

join variation of outer joins by using the keyword `NATURAL` before the operation (for example, `NATURAL LEFT OUTER JOIN`). The keyword `CROSS JOIN` is used to specify the `CARTESIAN PRODUCT` operation (see Section 8.2.2), although this should be used only with the utmost care because it generates all possible tuple combinations. It is also possible to nest join specifications; that is, one of the tables in a join may itself be a joined table. This allows the specification of the join of three or more tables as a single joined table, which is called a multiway join. For example, Q2A is a different way of specifying query Q2 from Section 6.3.1 using the concept of a joined table:

```
Q2A: SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM ((PROJECT JOIN DEPARTMENT ON Dnum = Dnumber)
JOIN EMPLOYEE ON Mgr_ssn = Ssn)
WHERE Plocation = 'Stafford';
```

Not all SQL implementations have implemented the new syntax of joined tables. In some systems, a different syntax was used to specify outer joins by using the comparison operators `+`, `=`, `= +`, and `+ = +` for left, right, and full outer join, respectively, when specifying the join condition. For example, this syntax is available in Oracle. To specify the left outer join in Q8B using this syntax, we could write the query Q8C as follows:

```
Q8C: SELECT E.Lname, S.Lname
FROM EMPLOYEE E, EMPLOYEE S
WHERE E.Super_ssn + = S.Ssn;
```

7.1.7 Aggregate Functions in SQL

Aggregate functions are used to summarize information from multiple tuples into a single-tuple summary. Grouping is used to create subgroups of tuples before summarization. Grouping and aggregation are required in many database

7.1 More Complex SQL Retrieval Queries 247

applications, and we will introduce their use in SQL through examples. A number of built-in aggregate functions exist: `COUNT`, `SUM`, `MAX`, `MIN`, and `AVG`.² The `COUNT` function returns the number of tuples or values as specified in a query. The functions `SUM`, `MAX`, `MIN`, and `AVG` can be applied to a set or multiset of numeric values and return, respectively, the sum, maximum value, minimum value, and average (mean) of those values. These functions can be used in the `SELECT` clause or in a `HAVING` clause (which we introduce later). The functions `MAX` and `MIN` can also be used with attributes that have nonnumeric domains if the domain values have a total ordering among one another.³ We illustrate the use of these functions with several queries.

Query 19. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

```
Q19: SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
FROM EMPLOYEE;
```

This query returns a single-row summary of all the rows in the `EMPLOYEE` table.

We could use AS to rename the column names in the resulting single-row table; for example, as in Q19A.

```
Q19A: SELECT SUM (Salary) AS Total_Sal, MAX (Salary) AS Highest_Sal,
MIN (Salary) AS Lowest_Sal, AVG (Salary) AS Average_Sal
FROM EMPLOYEE;
```

If we want to get the preceding aggregate function values for employees of a specific department—say, the ‘Research’ department—we can write Query 20, where the EMPLOYEE tuples are restricted by the WHERE clause to those employees who work for the ‘Research’ department.

Query 20. Find the sum of the salaries of all employees of the ‘Research’ department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
Q20: SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE Dname = ‘Research’;
```

Queries 21 and 22. Retrieve the total number of employees in the company (Q21) and the number of employees in the ‘Research’ department (Q22).

```
Q21: SELECT COUNT (*)
FROM EMPLOYEE;
```

```
Q22: SELECT COUNT (*)
FROM EMPLOYEE, DEPARTMENT
WHERE DNO = DNUMBER AND DNAME = ‘Research’;
```

2

Additional aggregate functions for more advanced statistical calculation were added in SQL-99.

3

Total order means that for any two values in the domain, it can be determined that one appears before

the other in the defined order; for example, DATE, TIME, and TIMESTAMP domains have total orderings

on their values, as do alphabetic strings.

248 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

Here the asterisk (*) refers to the rows (tuples), so COUNT (*) returns the number of rows in the result of the query. We may also use the COUNT function to count values in a column rather than tuples, as in the next example.

Query 23. Count the number of distinct salary values in the database.

```
Q23: SELECT COUNT (DISTINCT Salary)
FROM EMPLOYEE;
```

If we write COUNT(SALARY) instead of COUNT(DISTINCT SALARY) in Q23, then duplicate values will not be eliminated. However, any tuples with NULL for SALARY will not be counted. In general, NULL values are discarded when aggregate func-

tions are applied to a particular column (attribute); the only exception is for COUNT(*) because tuples instead of values are counted. In the previous examples, any Salary values that are NULL are not included in the aggregate function calculation. The general rule is as follows: when an aggregate function is applied to a collection of values, NULLs are removed from the collection before the calculation; if the collection becomes empty because all values are NULL, the aggregate function will return NULL (except in the case of COUNT, where it will return 0 for an empty collection of values).

The preceding examples summarize a whole relation (Q19, Q21, Q23) or a selected subset of tuples (Q20, Q22), and hence all produce a table with a single row or a single value. They illustrate how functions are applied to retrieve a summary value or summary tuple from a table. These functions can also be used in selection conditions involving nested queries. We can specify a correlated nested query with an aggregate function, and then use the nested query in the WHERE clause of an outer query. For example, to retrieve the names of all employees who have two or more dependents (Query 5), we can write the following:

```
Q5: SELECT Lname, Fname
FROM EMPLOYEE
WHERE ( SELECT COUNT (*)
FROM DEPENDENT
WHERE Ssn = Essn ) >= 2;
```

The correlated nested query counts the number of dependents that each employee has; if this is greater than or equal to two, the employee tuple is selected.

SQL also has aggregate functions SOME and ALL that can be applied to a collection of Boolean values; SOME returns TRUE if at least one element in the collection is TRUE, whereas ALL returns TRUE if all elements in the collection are TRUE.

7.1.8 Grouping: The GROUP BY and HAVING Clauses

In many cases we want to apply the aggregate functions to subgroups of tuples in a relation, where the subgroups are based on some attribute values. For example, we may want to find the average salary of employees in each department or the number

7.1 More Complex SQL Retrieval Queries 249

of employees who work on each project. In these cases we need to partition the relation into nonoverlapping subsets (or groups) of tuples. Each group (partition) will consist of the tuples that have the same value of some attribute(s), called the grouping attribute(s). We can then apply the function to each such group independently to produce summary information about each group. SQL has a GROUP BY clause for this purpose. The GROUP BY clause specifies the grouping attributes, which should also appear in the SELECT clause, so that the value resulting from applying each aggregate function to a group of tuples appears along with the value of the grouping attribute(s).

Query 24. For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
Q24: SELECT Dno, COUNT (*), AVG (Salary)
FROM EMPLOYEE
GROUP BY Dno;
```

In Q24, the EMPLOYEE tuples are partitioned into groups—each group having the same value for the GROUP BY attribute Dno. Hence, each group contains the employees who work in the same department. The COUNT and AVG functions are applied to each such group of tuples. Notice that the SELECT clause includes only the grouping attribute and the aggregate functions to be applied on each group of tuples. Figure 7.1(a) illustrates how grouping works and shows the result of Q24.

If NULLs exist in the grouping attribute, then a separate group is created for all tuples with a NULL value in the grouping attribute. For example, if the EMPLOYEE table had some tuples that had NULL for the grouping attribute Dno, there would be a separate group for those tuples in the result of Q24.

Query 25. For each project, retrieve the project number, the project name, and the number of employees who work on that project.

```
Q25: SELECT Pnumber, Pname, COUNT (*)
FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno
GROUP BY Pnumber, Pname;
```

Q25 shows how we can use a join condition in conjunction with GROUP BY. In this case, the grouping and functions are applied after the joining of the two relations in the WHERE clause.

Sometimes we want to retrieve the values of these functions only for groups that satisfy certain conditions. For example, suppose that we want to modify Query 25 so that only projects with more than two employees appear in the result. SQL provides a HAVING clause, which can appear in conjunction with a GROUP BY clause, for this purpose. HAVING provides a condition on the summary information regarding the group of tuples associated with each value of the grouping attributes. Only the groups that satisfy the condition are retrieved in the result of the query. This is illustrated by Query 26.

250 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

Figure 7.1

Results of GROUP BY and HAVING. (a) Q24. (b) Q26.

```
(a) Fname Minit Lname Ssn . . . Salary Super_ssn Dno Dno Count (*) Avg (Salary)
John B Smith 123456789 30000 333445555 5 5 4 33250
Franklin T Wong 333445555 40000 888665555 5 4 3 31000
Ramesh K Narayan 666884444 38000 333445555 5 1 1 55000
```

Joyce A English 453453453 . . . 25000 333445555 5 Result of Q24

Alicia J Zelaya 999887777 25000 987654321 4

Jennifer S Wallace 987654321 43000 888665555 4

Ahmad V Jabbar 987987987 25000 987654321 4

James E Bong 888665555 55000 NULL 1

Grouping EMPLOYEE tuples by the value of Dno

(b) Pname Pnumber ... Essn Pno Hours These groups are not selected by the HAVING condition of Q26.

ProductX 1 123456789 1 32.5

ProductX 1 453453453 1 20.0

ProductY 2 123456789 2 7.5

ProductY 2 453453453 2 20.0

ProductY 2 333445555 2 10.0

ProductZ 3 666884444 3 40.0

ProductZ 3 333445555 3 10.0

Computerization 10 ... 333445555 10 10.0

Computerization 10 999887777 10 10.0

Computerization 10 987987987 10 35.0

Reorganization 20 333445555 20 10.0

Reorganization 20 987654321 20 15.0

Reorganization 20 888665555 20 NULL

Newbenefits 30 987987987 30 5.0

Newbenefits 30 987654321 30 20.0

Newbenefits 30 999887777 30 30.0

After applying the WHERE clause but before applying HAVING

Pname Pnumber ... Essn Pno Hours Pname Count (*)

ProductY 2 123456789 2 7.5 ProductY 3

ProductY 2 453453453 2 20.0 Computerization 3

ProductY 2 333445555 2 10.0 Reorganization 3

Computerization 10 333445555 10 10.0 Newbenefits 3

Computerization 10 ... 999887777 10 10.0 Result of Q26

(Pnumber not shown)

Computerization 10 987987987 10 35.0

Reorganization 20 333445555 20 10.0

Reorganization 20 987654321 20 15.0

Reorganization 20 888665555 20 NULL

Newbenefits 30 987987987 30 5.0

Newbenefits 30 987654321 30 20.0

Newbenefits 30 999887777 30 30.0

After applying the HAVING clause condition

7.1 More Complex SQL Retrieval Queries 251

Query 26. For each project on which more than two employees work, retrieve the project number, the project name, and the number of employees who work on the project.

```
Q26: SELECT Pnumber, Pname, COUNT (*)
FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno
GROUP BY Pnumber, Pname
HAVING COUNT (*) > 2;
```

Notice that although selection conditions in the WHERE clause limit the tuples to which functions are applied, the HAVING clause serves to choose whole groups. Figure 7.1(b) illustrates the use of HAVING and displays the result of Q26.

Query 27. For each project, retrieve the project number, the project name, and the number of employees from department 5 who work on the project.

```
Q27: SELECT Pnumber, Pname, COUNT (*)
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber = Pno AND Ssn = Essn AND Dno = 5
GROUP BY Pnumber, Pname;
```

In Q27, we restrict the tuples in the relation (and hence the tuples in each group) to those that satisfy the condition specified in the WHERE clause—namely, that they work in department number 5. Notice that we must be extra careful when two different conditions apply (one to the aggregate function in the SELECT clause and another to the function in the HAVING clause). For example, suppose that we want to count the total number of employees whose salaries exceed \$40,000 in each department, but only for departments where more than five employees work. Here, the condition (SALARY > 40000) applies only to the COUNT function in the SELECT clause. Suppose that we write the following incorrect query:

```
SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000
GROUP BY Dno
HAVING COUNT (*) > 5;
```

This is incorrect because it will select only departments that have more than five employees who each earn more than \$40,000. The rule is that the WHERE clause is executed first, to select individual tuples or joined tuples; the HAVING clause is applied later, to select individual groups of tuples. In the incorrect query, the tuples are already restricted to employees who earn more than \$40,000 before the function in the HAVING clause is applied. One way to write this query correctly is to use a nested query, as shown in Query 28.

Query 28. For each department that has more than five employees, retrieve the department number and the number of its employees who are making more

than \$40,000.

252 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

```
Q28: SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000 AND Dno IN
( SELECT Dno
FROM EMPLOYEE
GROUP BY Dno
HAVING COUNT (*) > 5)
GROUP BY Dno;
```

7.1.9 Other SQL Constructs: WITH and CASE

In this section, we illustrate two additional SQL constructs. The WITH clause allows a user to define a table that will only be used in a particular query; it is somewhat similar to creating a view (see Section 7.3) that will be used only in one query and then dropped. This construct was introduced as a convenience in SQL:99 and may not be available in all SQL based DBMSs. Queries using WITH can generally be written using other SQL constructs. For example, we can rewrite Q28 as Q28':

```
Q28': WITH BIGDEPTS (Dno) AS
( SELECT Dno
FROM EMPLOYEE
GROUP BY Dno
HAVING COUNT (*) > 5)
SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000 AND Dno IN BIGDEPTS
GROUP BY Dno;
```

In Q28', we defined in the WITH clause a temporary table BIG_DEPTS whose result holds the Dno's of departments with more than five employees, then used this table in the subsequent query. Once this query is executed, the temporary table BIGDEPTS is discarded.

SQL also has a CASE construct, which can be used when a value can be different based on certain conditions. This can be used in any part of an SQL query where a value is expected, including when querying, inserting or updating tuples. We illustrate this with an example. Suppose we want to give employees different raise amounts depending on which department they work for; for example, employees in department 5 get a \$2,000 raise, those in department 4 get \$1,500 and those in department 1 get \$3,000 (see Figure 5.6 for the employee tuples). Then we could re-write the update operation U6 from Section 6.4.3 as U6':

```
U6': UPDATE EMPLOYEE
SET Salary =
CASE WHEN Dno = 5 THEN Salary + 2000
```

```

WHEN Dno = 4 THEN Salary + 1500
WHEN Dno = 1 THEN Salary + 3000
ELSE Salary + 0 ;

```

7.1 More Complex SQL Retrieval Queries 253

In U6', the salary raise value is determined through the CASE construct based on the department number for which each employee works. The CASE construct can also be used when inserting tuples that can have different attributes being NULL depending on the type of record being inserted into a table, as when a specialization (see Chapter 4) is mapped into a single table (see Chapter 9) or when a union type is mapped into relations.

7.1.10 Recursive Queries in SQL

In this section, we illustrate how to write a recursive query in SQL. This syntax was added in SQL:99 to allow users the capability to specify a recursive query in a declarative manner. An example of a recursive relationship between tuples of the same type is the relationship between an employee and a supervisor. This relationship is described by the foreign key Super_ssn of the EMPLOYEE relation in Figures 5.5 and 5.6, and it relates each employee tuple (in the role of supervisee) to another employee tuple (in the role of supervisor). An example of a recursive operation is to retrieve all supervisees of a supervisory employee e at all levels—that is, all employees e' directly supervised by e , all employees e' directly supervised by each employee e' , all employees e'' directly supervised by each employee e'' , and so on.

In SQL:99, this query can be written as follows:

```

Q29: WITH RECURSIVE SUP_EMP (SupSsn, EmpSsn) AS
( SELECT SupervisorSsn, Ssn
FROM EMPLOYEE
UNION
SELECT E.Ssn, S.SupSsn
FROM EMPLOYEE AS E, SUP_EMP AS S
WHERE E.SupervisorSsn = S.EmpSsn)
SELECT*
FROM SUP_EMP;

```

In Q29, we are defining a view SUP_EMP that will hold the result of the recursive query. The view is initially empty. It is first loaded with the first level (supervisor, supervisee) Ssn combinations via the first part (SELECT SupervisorSsn, Ssn FROM EMPLOYEE), which is called the base query. This will be combined via UNION with each successive level of supervisees through the second part, where the view contents are joined again with the base values to get the second level combinations, which are UNIONed with the first level. This is repeated with successive levels until a fixed point is reached, where no more tuples are added to the view. At this point, the result of the recursive query is in the view SUP_EMP.

7.1.11 Discussion and Summary of SQL Queries

A retrieval query in SQL can consist of up to six clauses, but only the first two—SELECT and FROM—are mandatory. The query can span several lines, and is ended by a semicolon. Query terms are separated by spaces, and parentheses can be used to group relevant parts of a query in the standard way. The clauses are specified in the following order, with the clauses between square brackets [...] being optional:

```
SELECT <attribute and function list>
FROM <table list>
[ WHERE <condition> ]
[ GROUP BY <grouping attribute(s)> ]
[ HAVING <group condition> ]
[ ORDER BY <attribute list> ];
```

The SELECT clause lists the attributes or functions to be retrieved. The FROM clause specifies all relations (tables) needed in the query, including joined relations, but not those in nested queries. The WHERE clause specifies the conditions for selecting the tuples from these relations, including join conditions if needed. GROUP BY specifies grouping attributes, whereas HAVING specifies a condition on the groups being selected rather than on the individual tuples. The built-in aggregate functions COUNT, SUM, MIN, MAX, and AVG are used in conjunction with grouping, but they can also be applied to all the selected tuples in a query without a GROUP BY clause. Finally, ORDER BY specifies an order for displaying the result of a query.

In order to formulate queries correctly, it is useful to consider the steps that define the meaning or semantics of each query. A query is evaluated conceptually⁴ by first applying the FROM clause (to identify all tables involved in the query or to materialize any joined tables), followed by the WHERE clause to select and join tuples, and then by GROUP BY and HAVING. Conceptually, ORDER BY is applied at the end to sort the query result. If none of the last three clauses (GROUP BY, HAVING, and ORDER BY) are specified, we can think conceptually of a query as being executed as follows: For each combination of tuples—one from each of the relations specified in the FROM clause—evaluate the WHERE clause; if it evaluates to TRUE, place the values of the attributes specified in the SELECT clause from this tuple combination in the result of the query. Of course, this is not an efficient way to implement the query in a real system, and each DBMS has special query optimization routines to decide on an execution plan that is efficient to execute. We discuss query processing and optimization in Chapters 18 and 19.

In general, there are numerous ways to specify the same query in SQL. This flexibility in specifying queries has advantages and disadvantages. The main advantage is that users can choose the technique with which they are most comfortable when specifying a query. For example, many queries may be specified with join conditions in the WHERE clause, or by using joined relations in the FROM clause, or with some form of

nested queries and the IN comparison operator. Some users may be more comfortable with one approach, whereas others may be more comfortable with another. From the programmer's and the system's point of view regarding query optimization, it is generally preferable to write a query with as little nesting and implied ordering as possible. The disadvantage of having numerous ways of specifying the same query is that this may confuse the user, who may not know which technique to use to specify

4

The actual order of query evaluation is implementation dependent; this is just a way to conceptually

view a query in order to correctly formulate it.

7.2 Specifying Constraints as Assertions and Actions as Triggers 255

particular types of queries. Another problem is that it may be more efficient to execute a query specified in one way than the same query specified in an alternative way. Ideally, this should not be the case: The DBMS should process the same query in the same way regardless of how the query is specified. But this is quite difficult in practice, since each DBMS has different methods for processing queries specified in different ways. Thus, an additional burden on the user is to determine which of the alternative specifications is the most efficient to execute. Ideally, the user should worry only about specifying the query correctly, whereas the DBMS would determine how to execute the query efficiently. In practice, however, it helps if the user is aware of which types of constructs in a query are more expensive to process than others.

7.2 Specifying Constraints as Assertions and Actions as Triggers

In this section, we introduce two additional features of SQL: the CREATE ASSERTION statement and the CREATE TRIGGER statement. Section 7.2.1 discusses CREATE ASSERTION, which can be used to specify additional types of constraints that are outside the scope of the built-in relational model constraints (primary and unique keys, entity integrity, and referential integrity) that we presented in Section 5.2. These built-in constraints can be specified within the CREATE TABLE statement of SQL (see Sections 6.1 and 6.2).

In Section 7.2.2 we introduce CREATE TRIGGER, which can be used to specify automatic actions that the database system will perform when certain events and conditions occur. This type of functionality is generally referred to as active databases. We only introduce the basics of triggers in this chapter, and present a more complete discussion of active databases in Section 26.1.

7.2.1 Specifying General Constraints as Assertions in SQL

In SQL, users can specify general constraints—those that do not fall into any of the categories described in Sections 6.1 and 6.2—via declarative assertions, using the CREATE ASSERTION statement. Each assertion is given a constraint name and is specified via a condition similar to the WHERE clause of an SQL query. For exam-

ple, to specify the constraint that the salary of an employee must not be greater than the salary of the manager of the department that the employee works for in SQL, we can write the following assertion:

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT *
FROM EMPLOYEE E, EMPLOYEE M,
DEPARTMENT D
WHERE E.Salary>M.Salary
AND E.Dno = D.Dnumber
AND D.Mgr_ssn = M.Ssn ) );
```

256 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

The constraint name SALARY_CONSTRAINT is followed by the keyword CHECK , which is followed by a condition in parentheses that must hold true on every database state for the assertion to be satisfied. The constraint name can be used later to disable the constraint or to modify or drop it. The DBMS is responsible for ensuring that the condition is not violated. Any WHERE clause condition can be used, but many constraints can be specified using the EXISTS and NOT EXISTS style of SQL conditions. Whenever some tuples in the database cause the condition of an ASSERTION statement to evaluate to FALSE, the constraint is violated. The constraint is satisfied by a database state if no combination of tuples in that database state violates the constraint.

The basic technique for writing such assertions is to specify a query that selects any tuples that violate the desired condition. By including this query inside a NOT EXISTS clause, the assertion will specify that the result of this query must be empty so that the condition will always be TRUE. Thus, the assertion is violated if the result of the query is not empty. In the preceding example, the query selects all employees whose salaries are greater than the salary of the manager of their department. If the result of the query is not empty, the assertion is violated.

Note that the CHECK clause and constraint condition can also be used to specify constraints on individual attributes and domains (see Section 6.2.1) and on individual tuples (see Section 6.2.4). A major difference between CREATE ASSERTION and the individual domain constraints and tuple constraints is that the CHECK clauses on individual attributes, domains, and tuples are checked in SQL only when tuples are inserted or updated in a specific table. Hence, constraint checking can be implemented more efficiently by the DBMS in these cases. The schema designer should use CHECK on attributes, domains, and tuples only when he or she is sure that the constraint can only be violated by insertion or updating of tuples. On the other hand, the schema designer should use CREATE ASSERTION only in cases where it is not possible to use CHECK on attributes, domains, or tuples, so that simple checks are implemented more efficiently by the DBMS.

7.2.2 Introduction to Triggers in SQL

Another important statement in SQL is CREATE TRIGGER. In many cases it is convenient to specify the type of action to be taken when certain events occur and when certain conditions are satisfied. For example, it may be useful to specify a condition that, if violated, causes some user to be informed of the violation. A manager may want to be informed if an employee's travel expenses exceed a certain limit by receiving a message whenever this occurs. The action that the DBMS must take in this case is to send an appropriate message to that user. The condition is thus used to monitor the database. Other actions may be specified, such as executing a specific stored procedure or triggering other updates. The CREATE TRIGGER statement is used to implement such actions in SQL. We discuss triggers in detail in Section 26.1 when we describe active databases. Here we just give a simple example of how triggers may be used.

7.2 Specifying Constraints as Assertions and Actions as Triggers 257

Suppose we want to check whenever an employee's salary is greater than the salary of his or her direct supervisor in the COMPANY database (see Figures 5.5 and 5.6). Several events can trigger this rule: inserting a new employee record, changing an employee's salary, or changing an employee's supervisor. Suppose that the action to take would be to call an external stored procedure SALARY_VIOLATION,⁵ which will notify the supervisor. The trigger could then be written as in R5 below. Here we are using the syntax of the Oracle database system.

```
R5: CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
ON EMPLOYEE
FOR EACH ROW
WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
WHERE SSN = NEW.SUPERVISOR_SSN ) )
INFORM_SUPERVISOR(NEW.Supervisor_ssn,
NEW.Ssn );
```

The trigger is given the name SALARY_VIOLATION, which can be used to remove or deactivate the trigger later. A typical trigger which is regarded as an ECA (Event, Condition, Action) rule has three components:

1. The event(s): These are usually database update operations that are explicitly applied to the database. In this example the events are: inserting a new employee record, changing an employee's salary, or changing an employee's supervisor. The person who writes the trigger must make sure that all possible events are accounted for. In some cases, it may be necessary to write more than one trigger to cover all possible cases. These events are specified after the keyword BEFORE in our example, which means that the trigger should be executed before the triggering operation is executed. An alternative is to use the keyword AFTER, which specifies that the trigger should be

executed after the operation specified in the event is completed.

2. The condition that determines whether the rule action should be executed:

Once the triggering event has occurred, an optional condition may be evaluated. If no condition is specified, the action will be executed once the event occurs. If a condition is specified, it is first evaluated, and only if it evaluates to true will the rule action be executed. The condition is specified in the WHEN clause of the trigger.

3. The action to be taken: The action is usually a sequence of SQL statements, but it could also be a database transaction or an external program that will be automatically executed. In this example, the action is to execute the stored procedure `INFORM_SUPERVISOR`.

Triggers can be used in various applications, such as maintaining database consistency, monitoring database updates, and updating derived data automatically. A complete discussion is given in Section 26.1.

5

Assuming that an appropriate external procedure has been declared. We discuss stored procedures in

Chapter 10.

258 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

7.3 Views (Virtual Tables) in SQL

In this section we introduce the concept of a view in SQL. We show how views are specified, and then we discuss the problem of updating views and how views can be implemented by the DBMS.

7.3.1 Concept of a View in SQL

A view in SQL terminology is a single table that is derived from other tables.⁶ These other tables can be base tables or previously defined views. A view does not necessarily exist in physical form; it is considered to be a virtual table, in contrast to base tables, whose tuples are always physically stored in the database. This limits the possible update operations that can be applied to views, but it does not provide any limitations on querying a view.

We can think of a view as a way of specifying a table that we need to reference frequently, even though it may not exist physically. For example, referring to the `COMPANY` database in Figure 5.5, we may frequently issue queries that retrieve the employee name and the project names that the employee works on. Rather than having to specify the join of the three tables `EMPLOYEE`, `WORKS_ON`, and `PROJECT` every time we issue this query, we can define a view that is specified as the result of these joins. Then we can issue queries on the view, which are specified as single-table retrievals rather than as retrievals involving two joins on three tables. We call the `EMPLOYEE`, `WORKS_ON`, and `PROJECT` tables the defining tables of the view.

7.3.2 Specification of Views in SQL

In SQL, the command to specify a view is `CREATE VIEW`. The view is given a (vir-

tual) table name (or view name), a list of attribute names, and a query to specify the contents of the view. If none of the view attributes results from applying functions or arithmetic operations, we do not have to specify new attribute names for the view, since they would be the same as the names of the attributes of the defining tables in the default case. The views in V1 and V2 create virtual tables whose schemas are illustrated in Figure 7.2 when applied to the database schema of Figure 5.5.

V1: CREATE VIEW WORKS_ON1

AS SELECT Fname, Lname, Pname, Hours

FROM EMPLOYEE, PROJECT, WORKS_ON

WHERE Ssn = Essn AND Pno = Pnumber;

V2: CREATE VIEW DEPT_INFO(Dept_name, No_of_emps, Total_sal)

AS SELECT Dname, COUNT (*), SUM (Salary)

FROM DEPARTMENT, EMPLOYEE

WHERE Dnumber = Dno

GROUP BY Dname;

6

As used in SQL, the term view is more limited than the term user view discussed in Chapters 1 and 2,

since a user view would possibly include many relations.

7.3 Views (Virtual Tables) in SQL 259

WORKS_ON1

Fname Lname Pname Hours

Figure 7.2

DEPT_INFO Two views specified on

the database schema of

Dept_name No_of_emps Total_sal Figure 5.5.

In V1, we did not specify any new attribute names for the view WORKS_ON1 (although we could have); in this case, WORKS_ON1 inherits the names of the view attributes from the defining tables EMPLOYEE, PROJECT, and WORKS_ON. View V2 explicitly specifies new attribute names for the view DEPT_INFO, using a one-to-one correspondence between the attributes specified in the CREATE VIEW clause and those specified in the SELECT clause of the query that defines the view.

We can now specify SQL queries on a view—or virtual table—in the same way we specify queries involving base tables. For example, to retrieve the last name and first name of all employees who work on the 'ProductX' project, we can utilize the WORKS_ON1 view and specify the query as in QV1:

QV1: SELECT Fname, Lname

FROM WORKS_ON1

WHERE Pname = 'ProductX';

The same query would require the specification of two joins if specified on the base relations directly; one of the main advantages of a view is to simplify the specification of certain queries. Views are also used as a security and authorization mechanism (see Section 7.3.4 and Chapter 30).

A view is supposed to be always up-to-date; if we modify the tuples in the base tables on which the view is defined, the view must automatically reflect these changes. Hence, the view does not have to be realized or materialized at the time of view definition but rather at the time when we specify a query on the view. It is the responsibility of the DBMS and not the user to make sure that the view is kept up-to-date. We will discuss various ways the DBMS can utilize to keep a view up-to-date in the next subsection.

If we do not need a view anymore, we can use the DROP VIEW command to dispose of it. For example, to get rid of the view V1, we can use the SQL statement in V1A:

```
V1A: DROP VIEW WORKS_ON1;
```

7.3.3 View Implementation, View Update, and Inline Views

The problem of how a DBMS can efficiently implement a view for efficient querying is complex. Two main approaches have been suggested. One strategy, called query modification, involves modifying or transforming the view query (submitted by the user) into a query on the underlying base tables. For example, the query QV1 would be automatically modified to the following query by the DBMS:

```
SELECT Fname, Lname  
FROM EMPLOYEE, PROJECT, WORKS_ON  
WHERE Ssn = Essn AND Pno = Pnumber  
AND Pname = 'ProductX';
```

The disadvantage of this approach is that it is inefficient for views defined via complex queries that are time-consuming to execute, especially if multiple view queries are going to be applied to the same view within a short period of time. The second strategy, called view materialization, involves physically creating a temporary or permanent view table when the view is first queried or created and keeping that table on the assumption that other queries on the view will follow. In this case, an efficient strategy for automatically updating the view table when the base tables are updated must be developed in order to keep the view up-to-date. Techniques using the concept of incremental update have been developed for this purpose, where the DBMS can determine what new tuples must be inserted, deleted, or modified in a materialized view table when a database update is applied to one of the defining base tables. The view is generally kept as a materialized (physically stored) table as long as it is being queried. If the view is not queried for a certain period of time, the system may then automatically remove the physical table and recompute it from scratch when future queries reference the view.

Different strategies as to when a materialized view is updated are possible. The

immediate update strategy updates a view as soon as the base tables are changed; the lazy update strategy updates the view when needed by a view query; and the periodic update strategy updates the view periodically (in the latter strategy, a view query may get a result that is not up-to-date).

A user can always issue a retrieval query against any view. However, issuing an INSERT, DELETE, or UPDATE command on a view table is in many cases not possible. In general, an update on a view defined on a single table without any aggregate functions can be mapped to an update on the underlying base table under certain conditions. For a view involving joins, an update operation may be mapped to update operations on the underlying base relations in multiple ways. Hence, it is often not possible for the DBMS to determine which of the updates is intended. To illustrate potential problems with updating a view defined on multiple tables, consider the WORKS_ON1 view, and suppose that we issue the command to update the PNAME attribute of 'John Smith' from 'ProductX' to 'ProductY'. This view update is shown in UV1:

```
UV1: UPDATE WORKS_ON1
```

```
SET Pname = 'ProductY'
```

```
WHERE Lname = 'Smith' AND Fname = 'John'
```

```
AND Pname = 'ProductX';
```

This query can be mapped into several updates on the base relations to give the desired update effect on the view. In addition, some of these updates will create

7.3 Views (Virtual Tables) in SQL 261

additional side effects that affect the result of other queries. For example, here are two possible updates, (a) and (b), on the base relations corresponding to the view update operation in UV1:

```
(a): UPDATE WORKS_ON
```

```
SET Pno = ( SELECT Pnumber
```

```
FROM PROJECT
```

```
WHERE Pname = 'ProductY' )
```

```
WHERE Essn IN ( SELECT Ssn
```

```
FROM EMPLOYEE
```

```
WHERE Lname = 'Smith' AND Fname = 'John' )
```

```
AND
```

```
Pno = ( SELECT Pnumber
```

```
FROM PROJECT
```

```
WHERE Pname = 'ProductX' );
```

```
(b): UPDATE PROJECT SET Pname = 'ProductY'
```

```
WHERE Pname = 'ProductX';
```

Update (a) relates 'John Smith' to the 'ProductY' PROJECT tuple instead of the 'ProductX' PROJECT tuple and is the most likely desired update. However, (b) would

also give the desired update effect on the view, but it accomplishes this by changing the name of the 'ProductX' tuple in the PROJECT relation to 'ProductY'. It is quite unlikely that the user who specified the view update UV1 wants the update to be interpreted as in (b), since it also has the side effect of changing all the view tuples with Pname = 'ProductX'.

Some view updates may not make much sense; for example, modifying the Total_sal attribute of the DEPT_INFO view does not make sense because Total_sal is defined to be the sum of the individual employee salaries. This incorrect request is shown as UV2:

```
UV2: UPDATE DEPT_INFO
SET Total_sal = 100000
WHERE Dname = 'Research';
```

Generally, a view update is feasible when only one possible update on the base relations can accomplish the desired update operation on the view. Whenever an update on the view can be mapped to more than one update on the underlying base relations, it is usually not permitted. Some researchers have suggested that the DBMS have a certain procedure for choosing one of the possible updates as the most likely one. Some researchers have developed methods for choosing the most likely update, whereas other researchers prefer to have the user choose the desired update mapping during view definition. But these options are generally not available in most commercial DBMSs.

In summary, we can make the following observations:

- ■ A view with a single defining table is updatable if the view attributes contain the primary key of the base relation, as well as all attributes with the NOT NULL constraint that do not have default values specified.

262 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

- ■ Views defined on multiple tables using joins are generally not updatable.

- ■ Views defined using grouping and aggregate functions are not updatable.

In SQL, the clause WITH CHECK OPTION should be added at the end of the view definition if a view is to be updated by INSERT, DELETE, or UPDATE statements. This allows the system to reject operations that violate the SQL rules for view updates. The full set of SQL rules for when a view may be modified by the user are more complex than the rules stated earlier.

It is also possible to define a view table in the FROM clause of an SQL query. This is known as an in-line view. In this case, the view is defined within the query itself.

7.3.4 Views as Authorization Mechanisms

We describe SQL query authorization statements (GRANT and REVOKE) in detail in Chapter 30, when we present database security and authorization mechanisms. Here, we will just give a couple of simple examples to illustrate how views can be used to hide certain attributes or tuples from unauthorized users. Suppose a certain user is only allowed to see employee information for employees who work for

department 5; then we can create the following view DEPT5EMP and grant the user the privilege to query the view but not the base table EMPLOYEE itself. This user will only be able to retrieve employee information for employee tuples whose Dno = 5, and will not be able to see other employee tuples when the view is queried.

```
CREATE VIEW DEPT5EMP AS
SELECT *
FROM EMPLOYEE
WHERE Dno = 5;
```

In a similar manner, a view can restrict a user to only see certain columns; for example, only the first name, last name, and address of an employee may be visible as follows:

```
CREATE VIEW BASIC_EMP_DATA AS
SELECT Fname, Lname, Address
FROM EMPLOYEE;
```

Thus by creating an appropriate view and granting certain users access to the view and not the base tables, they would be restricted to retrieving only the data specified in the view. Chapter 30 discusses security and authorization in detail, including the GRANT and REVOKE statements of SQL.

7.4 Schema Change Statements in SQL

In this section, we give an overview of the schema evolution commands available in SQL, which can be used to alter a schema by adding or dropping tables, attributes, constraints, and other schema elements. This can be done while the database is operational and does not require recompilation of the database schema. Certain

7.4 Schema Change Statements in SQL 263

checks must be done by the DBMS to ensure that the changes do not affect the rest of the database and make it inconsistent.

7.4.1 The DROP Command

The DROP command can be used to drop named schema elements, such as tables, domains, types, or constraints. One can also drop a whole schema if it is no longer needed by using the DROP SCHEMA command. There are two drop behavior options: CASCADE and RESTRICT. For example, to remove the COMPANY database schema and all its tables, domains, and other elements, the CASCADE option is used as follows:

```
DROP SCHEMA COMPANY CASCADE;
```

If the RESTRICT option is chosen in place of CASCADE, the schema is dropped only if it has no elements in it; otherwise, the DROP command will not be executed. To use the RESTRICT option, the user must first individually drop each element in the schema, then drop the schema itself.

If a base relation within a schema is no longer needed, the relation and its definition can be deleted by using the DROP TABLE command. For example, if we no longer

wish to keep track of dependents of employees in the COMPANY database of Figure 6.1, we can get rid of the DEPENDENT relation by issuing the following command:

```
DROP TABLE DEPENDENT CASCADE;
```

If the RESTRICT option is chosen instead of CASCADE, a table is dropped only if it is not referenced in any constraints (for example, by foreign key definitions in another relation) or views (see Section 7.3) or by any other elements. With the CASCADE option, all such constraints, views, and other elements that reference the table being dropped are also dropped automatically from the schema, along with the table itself. Notice that the DROP TABLE command not only deletes all the records in the table if successful, but also removes the table definition from the catalog. If it is desired to delete only the records but to leave the table definition for future use, then the DELETE command (see Section 6.4.2) should be used instead of DROP TABLE. The DROP command can also be used to drop other types of named schema elements, such as constraints or domains.

7.4.2 The ALTER Command

The definition of a base table or of other named schema elements can be changed by using the ALTER command. For base tables, the possible alter table actions include adding or dropping a column (attribute), changing a column definition, and adding or dropping table constraints. For example, to add an attribute for keeping track of jobs of employees to the EMPLOYEE base relation in the COMPANY schema (see Figure 6.1), we can use the command

```
ALTER TABLE COMPANY.EMPLOYEE ADD COLUMN Job VARCHAR(12);
```

264 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

We must still enter a value for the new attribute Job for each individual EMPLOYEE tuple. This can be done either by specifying a default clause or by using the UPDATE command individually on each tuple (see Section 6.4.3). If no default clause is specified, the new attribute will have NULLs in all the tuples of the relation immediately after the command is executed; hence, the NOT NULL constraint is not allowed in this case.

To drop a column, we must choose either CASCADE or RESTRICT for drop behavior. If CASCADE is chosen, all constraints and views that reference the column are dropped automatically from the schema, along with the column. If RESTRICT is chosen, the command is successful only if no views or constraints (or other schema elements) reference the column. For example, the following command removes the attribute Address from the EMPLOYEE base table:

```
ALTER TABLE COMPANY.EMPLOYEE DROP COLUMN Address CASCADE;
```

It is also possible to alter a column definition by dropping an existing default clause or by defining a new default clause. The following examples illustrate this clause:

```
ALTER TABLE COMPANY.DEPARTMENT ALTER COLUMN Mgr_ssn  
DROP DEFAULT;
```

```
ALTER TABLE COMPANY.DEPARTMENT ALTER COLUMN Mgr_ssn  
SET DEFAULT '333445555';
```

One can also change the constraints specified on a table by adding or dropping a named constraint. To be dropped, a constraint must have been given a name when it was specified. For example, to drop the constraint named EMPSUPERFK in Figure 6.2 from the EMPLOYEE relation, we write:

```
ALTER TABLE COMPANY.EMPLOYEE
DROP CONSTRAINT EMPSUPERFK CASCADE;
```

Once this is done, we can redefine a replacement constraint by adding a new constraint to the relation, if needed. This is specified by using the ADD CONSTRAINT keyword in the ALTER TABLE statement followed by the new constraint, which can be named or unnamed and can be of any of the table constraint types discussed. The preceding subsections gave an overview of the schema evolution commands of SQL. It is also possible to create new tables and views within a database schema using the appropriate commands. There are many other details and options; we refer the interested reader to the SQL documents listed in the Selected Bibliography at the end of this chapter.

7.5 Summary

In this chapter we presented additional features of the SQL database language. We started in Section 7.1 by presenting more complex features of SQL retrieval queries, including nested queries, joined tables, outer joins, aggregate functions, and grouping. In Section 7.2, we described the CREATE ASSERTION statement, which allows the specification of more general constraints on the database, and introduced the

7.5 Summary 265

concept of triggers and the CREATE TRIGGER statement. Then, in Section 7.3, we described the SQL facility for defining views on the database. Views are also called virtual or derived tables because they present the user with what appear to be tables; however, the information in those tables is derived from previously defined tables. Section 7.4 introduced the SQL ALTER TABLE statement, which is used for modifying the database tables and constraints.

Table 7.2 summarizes the syntax (or structure) of various SQL statements. This summary is not meant to be comprehensive or to describe every possible SQL construct; rather, it is meant to serve as a quick reference to the major types of constructs available in SQL. We use BNF notation, where nonterminal symbols

Table 7.2 Summary of SQL Syntax

```
CREATE TABLE <table name> ( <column name> <column type> [ <attribute constraint> ]
{ , <column name> <column type> [ <attribute constraint> ] }
[ <table constraint> { , <table constraint> } ] )
DROP TABLE <table name>
ALTER TABLE <table name> ADD <column name> <column type>
SELECT [ DISTINCT ] <attribute list>
```

```

FROM ( <table name> { <alias> } | <joined table> ) { , ( <table name> { <alias> } | <joined
table> ) }
[ WHERE <condition> ]
[ GROUP BY <grouping attributes> [ HAVING <group selection condition> ] ]
[ ORDER BY <column name> [ <order> ] { , <column name> [ <order> ] } ]
<attribute list> ::= ( * | ( <column name> | <function> ( ( [ DISTINCT ] <column name> | * ) ) )
{ , ( <column name> | <function> ( ( [ DISTINCT ] <column name> | * ) ) ) } )
<grouping attributes> ::= <column name> { , <column name> }
<order> ::= ( ASC | DESC )
INSERT INTO <table name> [ ( <column name> { , <column name> } ) ]
( VALUES ( <constant value> , { <constant value> } ) { , ( <constant value> { , <constant value>
} ) }
| <select statement> )
DELETE FROM <table name>
[ WHERE <selection condition> ]
UPDATE <table name>
SET <column name> = <value expression> { , <column name> = <value expression> }
[ WHERE <selection condition> ]
CREATE [ UNIQUE ] INDEX <index name>
ON <table name> ( <column name> [ <order> ] { , <column name> [ <order> ] } )
[ CLUSTER ]
DROP INDEX <index name>
CREATE VIEW <view name> [ ( <column name> { , <column name> } ) ]
AS <select statement>
DROP VIEW <view name>

```

NOTE: The commands for creating and dropping indexes are not part of standard SQL.

266 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

are shown in angled brackets < ... >, optional parts are shown in square brackets [...], repetitions are shown in braces { ... }, and alternatives are shown in parentheses (... | ... | ...).⁷

Review Questions

- 7.1. Describe the six clauses in the syntax of an SQL retrieval query. Show what type of constructs can be specified in each of the six clauses. Which of the six clauses are required and which are optional?
- 7.2. Describe conceptually how an SQL retrieval query will be executed by specifying the conceptual order of executing each of the six clauses.
- 7.3. Discuss how NULLs are treated in comparison operators in SQL. How are NULLs treated when aggregate functions are applied in an SQL query? How are NULLs treated if they exist in grouping attributes?
- 7.4. Discuss how each of the following constructs is used in SQL, and discuss the various options for each construct. Specify what each construct is

useful for.

- a. Nested queries
- b. Joined tables and outer joins
- c. Aggregate functions and grouping
- d. Triggers
- e. Assertions and how they differ from triggers
- f. The SQL WITH clause
- g. SQL CASE construct
- h. Views and their updatability
- i. Schema change commands

Exercises

7.5. Specify the following queries on the database in Figure 5.5 in SQL. Show the query results if each query is applied to the database state in Figure 5.6.

- a. For each department whose average employee salary is more than \$30,000, retrieve the department name and the number of employees working for that department.
- b. Suppose that we want the number of male employees in each department making more than \$30,000, rather than all employees (as in Exercise 7.5a). Can we specify this query in SQL? Why or why not?

7

The full syntax of SQL is described in many voluminous documents of hundreds of pages.

Exercises 267

7.6. Specify the following queries in SQL on the database schema in Figure 1.2.

- a. Retrieve the names and major departments of all straight-A students (students who have a grade of A in all their courses).
- b. Retrieve the names and major departments of all students who do not have a grade of A in any of their courses.

7.7. In SQL, specify the following queries on the database in Figure 5.5 using the concept of nested queries and other concepts described in this chapter.

- a. Retrieve the names of all employees who work in the department that has the employee with the highest salary among all employees.
- b. Retrieve the names of all employees whose supervisor's supervisor has '888665555' for Ssn.
- c. Retrieve the names of employees who make at least \$10,000 more than the employee who is paid the least in the company.

7.8. Specify the following views in SQL on the COMPANY database schema shown in Figure 5.5.

- a. A view that has the department name, manager name, and manager salary for every department
- b. A view that has the employee name, supervisor name, and employee salary for each employee who works in the 'Research' department

- c. A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project
- d. A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project with more than one employee working on it

7.9. Consider the following view, DEPT_SUMMARY, defined on the COMPANY database in Figure 5.6:

```
CREATE VIEW DEPT_SUMMARY (D, C, Total_s, Average_s)
AS SELECT Dno, COUNT (*), SUM (Salary), AVG (Salary)
FROM EMPLOYEE
GROUP BY Dno;
```

State which of the following queries and updates would be allowed on the view. If a query or update would be allowed, show what the corresponding query or update on the base relations would look like, and give its result when applied to the database in Figure 5.6.

- a. SELECT *
- FROM DEPT_SUMMARY;
- b. SELECT D, C
- FROM DEPT_SUMMARY
- WHERE TOTAL_S > 100000;

268 Chapter 7 More SQL: Complex Queries, Triggers, Views, and Schema Modification

- c. SELECT D, AVERAGE_S
- FROM DEPT_SUMMARY
- WHERE C > (SELECT C FROM DEPT_SUMMARY WHERE D = 4);
- d. UPDATE DEPT_SUMMARY
- SET D=3
- WHERE D = 4;
- e. DELETE FROM DEPT_SUMMARY
- WHERE C > 4;

Selected Bibliography

Reisner (1977) describes a human factors evaluation of SEQUEL, a precursor of SQL, in which she found that users have some difficulty with specifying join conditions and grouping correctly. Date (1984) contains a critique of the SQL language that points out its strengths and shortcomings. Date and Darwen (1993) describes SQL2. ANSI (1986) outlines the original SQL standard. Various vendor manuals describe the characteristics of SQL as implemented on DB2, SQL/DS, Oracle, INGRES, Informix, and other commercial DBMS products. Melton and Simon (1993) give a comprehensive treatment of the ANSI 1992 standard called SQL2. Horowitz (1992) discusses some of the problems related to referential integrity and

propagation of updates in SQL2.

The question of view updates is addressed by Dayal and Bernstein (1978), Keller (1982), and Langerak (1990), among others. View implementation is discussed in Blakeley et al. (1989). Negri et al. (1991) describes formal semantics of SQL queries. There are many books that describe various aspects of SQL. For example, two references that describe SQL-99 are Melton and Simon (2002) and Melton (2003). Further SQL standards—SQL 2006 and SQL 2008—are described in a variety of technical reports; but no standard references exist.

chapter 8

The Relational Algebra and Relational Calculus

In this chapter we discuss the two formal languages for the relational model: the relational algebra and the relational calculus. In contrast, Chapters 6 and 7 described the practical language for the relational model, namely the SQL standard. Historically, the relational algebra and calculus were developed before the SQL language. SQL is primarily based on concepts from relational calculus and has been extended to incorporate some concepts from relational algebra as well. Because most relational DBMSs use SQL as their language, we presented the SQL language first.

Recall from Chapter 2 that a data model must include a set of operations to manipulate the database, in addition to the data model's concepts for defining the database's structure and constraints. We presented the structures and constraints of the formal relational model in Chapter 5. The basic set of operations for the formal relational model is the relational algebra. These operations enable a user to specify basic retrieval requests as relational algebra expressions. The result of a retrieval query is a new relation. The algebra operations thus produce new relations, which can be further manipulated using operations of the same algebra. A sequence of relational algebra operations forms a relational algebra expression, whose result will also be a relation that represents the result of a database query (or retrieval request).

The relational algebra is very important for several reasons. First, it provides a formal foundation for relational model operations. Second, and perhaps more important, it is used as a basis for implementing and optimizing queries in the query processing and optimization modules that are integral parts of relational database management systems (RDBMSs), as we shall discuss in Chapters 18 and 19. Third, some of its concepts are incorporated into the SQL standard

269

270 Chapter 8 The Relational Algebra and Relational Calculus

query language for RDBMSs. Although most commercial RDBMSs in use today do not provide user interfaces for relational algebra queries, the core operations and functions in the internal modules of most relational systems are based on

relational algebra operations. We will define these operations in detail in Sections 8.1 through 8.4 of this chapter.

Whereas the algebra defines a set of operations for the relational model, the relational calculus provides a higher-level declarative language for specifying relational queries. In a relational calculus expression, there is no order of operations to specify how to retrieve the query result—only what information the result should contain. This is the main distinguishing feature between relational algebra and relational calculus. The relational calculus is important because it has a firm basis in mathematical logic and because the standard query language (SQL) for RDBMSs has some of its foundations in a variation of relational calculus known as the tuple relational calculus.¹

The relational algebra is often considered to be an integral part of the relational data model. Its operations can be divided into two groups. One group includes set operations from mathematical set theory; these are applicable because each relation is defined to be a set of tuples in the formal relational model (see Section 5.1). Set operations include UNION, INTERSECTION, SET DIFFERENCE, and CARTESIAN PRODUCT (also known as CROSS PRODUCT). The other group consists of operations developed specifically for relational databases—these include SELECT, PROJECT, and JOIN, among others. First, we describe the SELECT and PROJECT operations in Section 8.1 because they are unary operations that operate on single relations. Then we discuss set operations in Section 8.2. In Section 8.3, we discuss JOIN and other complex binary operations, which operate on two tables by combining related tuples (records) based on join conditions. The COMPANY relational database shown in Figure 5.6 is used for our examples.

Some common database requests cannot be performed with the original relational algebra operations, so additional operations were created to express these requests. These include aggregate functions, which are operations that can summarize data from the tables, as well as additional types of JOIN and UNION operations, known as OUTER JOINS and OUTER UNIONS. These operations, which were added to the original relational algebra because of their importance to many database applications, are described in Section 8.4. We give examples of specifying queries that use relational operations in Section 8.5. Some of these same queries were used in Chapters 6 and 7. By using the same query numbers in this chapter, the reader can contrast how the same queries are written in the various query languages.

In Sections 8.6 and 8.7 we describe the other main formal language for relational databases, the relational calculus. There are two variations of relational calculus. The tuple relational calculus is described in Section 8.6 and the domain relational calculus is described in Section 8.7. Some of the SQL constructs discussed in

1

SQL is based on tuple relational calculus, but also incorporates some of the operations from the relational algebra and its extensions, as illustrated in Chapters 6, 7, and 9.

8.1 Unary Relational Operations: SELECT and PROJECT 271

Chapters 6 and 7 are based on the tuple relational calculus. The relational calculus is a formal language, based on the branch of mathematical logic called predicate calculus.² In tuple relational calculus, variables range over tuples, whereas in domain relational calculus, variables range over the domains (values) of attributes. In Appendix C we give an overview of the Query-By-Example (QBE) language, which is a graphical user-friendly relational language based on domain relational calculus. Section 8.8 summarizes the chapter.

For the reader who is interested in a less detailed introduction to formal relational languages, Sections 8.4, 8.6, and 8.7 may be skipped.

8.1 Unary Relational Operations:

SELECT and PROJECT

8.1.1 The SELECT Operation

The SELECT operation is used to choose a subset of the tuples from a relation that satisfies a selection condition.³ We can consider the SELECT operation to be a filter that keeps only those tuples that satisfy a qualifying condition. Alternatively, we can consider the SELECT operation to restrict the tuples in a relation to only those tuples that satisfy the condition. The SELECT operation can also be visualized as a horizontal partition of the relation into two sets of tuples—those tuples that satisfy the condition and are selected, and those tuples that do not satisfy the condition and are filtered out. For example, to select the EMPLOYEE tuples whose department is 4, or those whose salary is greater than \$30,000, we can individually specify each of these two conditions with a SELECT operation as follows:

$\sigma_{Dno=4}(EMPLOYEE)$

$\sigma_{Salary>30000}(EMPLOYEE)$

In general, the SELECT operation is denoted by

$\sigma_{\langle \text{selection condition} \rangle}(R)$

where the symbol σ (sigma) is used to denote the SELECT operator and the selection condition is a Boolean expression (condition) specified on the attributes of relation R . Notice that R is generally a relational algebra expression whose result is a relation—the simplest such expression is just the name of a database relation. The relation resulting from the SELECT operation has the same attributes as R .

The Boolean expression specified in $\langle \text{selection condition} \rangle$ is made up of a number of clauses of the form

$\langle \text{attribute name} \rangle \langle \text{comparison op} \rangle \langle \text{constant value} \rangle$

2

In this chapter no familiarity with first-order predicate calculus—which deals with quantified variables

and values—is assumed.

3

The SELECT operation is different from the SELECT clause of SQL. The SELECT operation chooses

tuples from a table, and is sometimes called a RESTRICT or FILTER operation.

272 Chapter 8 The Relational Algebra and Relational Calculus

or

<attribute name> <comparison op> <attribute name>

where <attribute name> is the name of an attribute of R, <comparison op> is normally one of the operators $\{=, <, \leq, >, \geq, \neq\}$, and <constant value> is a constant value from the attribute domain. Clauses can be connected by the standard Boolean operators and, or, and not to form a general selection condition. For example, to select the tuples for all employees who either work in department 4 and make over \$25,000 per year, or work in department 5 and make over \$30,000, we can specify the following SELECT operation:

$\sigma(\text{Dno}=4 \text{ AND } \text{Salary}>25000) \text{ OR } (\text{Dno}=5 \text{ AND } \text{Salary}>30000)(\text{EMPLOYEE})$

The result is shown in Figure 8.1(a).

Notice that all the comparison operators in the set $\{=, <, \leq, >, \geq, \neq\}$ can apply to attributes whose domains are ordered values, such as numeric or date domains.

Domains of strings of characters are also considered to be ordered based on the collating sequence of the characters. If the domain of an attribute is a set of unordered values, then only the comparison operators in the set $\{=, \neq\}$ can be used. An example of an unordered domain is the domain Color = { 'red', 'blue', 'green', 'white', 'yellow', ... }, where no order is specified among the various colors. Some domains allow additional types of comparison operators; for example, a domain of character strings may allow the comparison operator SUBSTRING_OF.

Figure 8.1

Results of SELECT and PROJECT operations. (a) $\sigma(\text{Dno}=4 \text{ AND } \text{Salary}>25000) \text{ OR } (\text{Dno}=5 \text{ AND } \text{Salary}>30000) (\text{EMPLOYEE})$.

(b) $\pi_{\text{Lname, Fname, Salary}}(\text{EMPLOYEE})$. (c) $\pi_{\text{Sex, Salary}}(\text{EMPLOYEE})$.

(a)

Fname Minit Lname Ssn Bdate Address Sex Salary Super_ssn Dno

Franklin T Wong 333445555 1955-12-08 638 Voss, Houston, TX M 40000 888665555 5

Jennifer S Wallace 987654321 1941-06-20 291 Berry, Bellaire, TX F 43000 888665555 4

Ramesh K Narayan 666884444 1962-09-15 975 Fire Oak, Humble, TX M 38000 333445555 5

(b) (c)

Lname Fname Salary Sex Salary

Smith John 30000 M 30000

Wong Franklin 40000 M 40000

Zelaya Alicia 25000 F 25000

Wallace Jennifer 43000 F 43000

Narayan Ramesh 38000 M 38000

English Joyce 25000 M 25000

Jabbar Ahmad 25000 M 55000

Borg James 55000

8.1 Unary Relational Operations: SELECT and PROJECT 273

In general, the result of a SELECT operation can be determined as follows. The $\langle \text{selection condition} \rangle$ is applied independently to each individual tuple t in R . This is done by substituting each occurrence of an attribute A_i in the selection condition with its value in the tuple $t[A_i]$. If the condition evaluates to TRUE, then tuple t is selected. All the selected tuples appear in the result of the SELECT operation. The Boolean conditions AND, OR, and NOT have their normal interpretation, as follows:

■■ (cond1 AND cond2) is TRUE if both (cond1) and (cond2) are TRUE; otherwise, it is FALSE.

■■ (cond1 OR cond2) is TRUE if either (cond1) or (cond2) or both are TRUE; otherwise, it is FALSE.

■■ (NOT cond) is TRUE if cond is FALSE; otherwise, it is FALSE.

The SELECT operator is unary; that is, it is applied to a single relation. Moreover, the selection operation is applied to each tuple individually; hence, selection conditions cannot involve more than one tuple. The degree of the relation resulting from a SELECT operation—its number of attributes—is the same as the degree of R . The number of tuples in the resulting relation is always less than or equal to the number of tuples in R . That is, $|\sigma_C(R)| \leq |R|$ for any condition C . The fraction of tuples selected by a selection condition is referred to as the selectivity of the condition.

Notice that the SELECT operation is commutative; that is,

$$\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(R)) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R))$$

Hence, a sequence of SELECTs can be applied in any order. In addition, we can always combine a cascade (or sequence) of SELECT operations into a single SELECT operation with a conjunctive (AND) condition; that is,

$$\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\dots (\sigma_{\langle \text{condn} \rangle}(R)) \dots)) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \dots \text{ AND } \langle \text{condn} \rangle}(R)$$

In SQL, the SELECT condition is typically specified in the WHERE clause of a query.

For example, the following operation:

$\sigma_{\text{Dno}=4 \text{ AND Salary} > 25000}(\text{EMPLOYEE})$

would correspond to the following SQL query:

SELECT *

FROM EMPLOYEE

WHERE Dno=4 AND Salary>25000;

8.1.2 The PROJECT Operation

If we think of a relation as a table, the SELECT operation chooses some of the rows from the table while discarding other rows. The PROJECT operation, on the other hand, selects certain columns from the table and discards the other columns. If we are interested in only certain attributes of a relation, we use the PROJECT operation to project the relation over these attributes only. Therefore, the result of the PROJECT operation can be visualized as a vertical partition of the relation into two relations:

one has the needed columns (attributes) and contains the result of the operation, and the other contains the discarded columns. For example, to list each employee's first and last name and salary, we can use the PROJECT operation as follows:

$\pi_{\text{LName, FName, Salary}}(\text{EMPLOYEE})$

The resulting relation is shown in Figure 8.1(b). The general form of the PROJECT operation is

$\pi_{\langle \text{attribute list} \rangle}(R)$

where π (pi) is the symbol used to represent the PROJECT operation, and $\langle \text{attribute list} \rangle$ is the desired sublist of attributes from the attributes of relation R . Again, notice that R is, in general, a relational algebra expression whose result is a relation, which in the simplest case is just the name of a database relation. The result of the PROJECT operation has only the attributes specified in $\langle \text{attribute list} \rangle$ in the same order as they appear in the list. Hence, its degree is equal to the number of attributes in $\langle \text{attribute list} \rangle$.

If the attribute list includes only nonkey attributes of R , duplicate tuples are likely to occur. The PROJECT operation removes any duplicate tuples, so the result of the PROJECT operation is a set of distinct tuples, and hence a valid relation. This is known as duplicate elimination. For example, consider the following PROJECT operation:

$\pi_{\text{Sex, Salary}}(\text{EMPLOYEE})$

The result is shown in Figure 8.1(c). Notice that the tuple $\langle \text{'F'}, 25000 \rangle$ appears only once in Figure 8.1(c), even though this combination of values appears twice in the EMPLOYEE relation. Duplicate elimination involves sorting or some other technique to detect duplicates and thus adds more processing. If duplicates are not eliminated, the result would be a multiset or bag of tuples rather than a set. This was not permitted in the formal relational model but is allowed in SQL (see Section 6.3).

The number of tuples in a relation resulting from a PROJECT operation is always less than or equal to the number of tuples in R . If the projection list is a superkey of R —that is, it includes some key of R —the resulting relation has the same number of tuples as R . Moreover,

$\pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) = \pi_{\langle \text{list1} \rangle}(R)$

as long as $\langle \text{list2} \rangle$ contains the attributes in $\langle \text{list1} \rangle$; otherwise, the left-hand side is an incorrect expression. It is also noteworthy that commutativity does not hold on PROJECT.

In SQL, the PROJECT attribute list is specified in the SELECT clause of a query. For example, the following operation:

$\pi_{\text{Sex, Salary}}(\text{EMPLOYEE})$

would correspond to the following SQL query:

```
SELECT DISTINCT Sex, Salary
FROM EMPLOYEE
```

8.1 Unary Relational Operations: SELECT and PROJECT 275

Notice that if we remove the keyword DISTINCT from this SQL query, then duplicates will not be eliminated. This option is not available in the formal relational algebra, but the algebra can be extended to include this operation and allow relations to be multisets; we do not discuss these extensions here.

8.1.3 Sequences of Operations and the RENAME Operation

The relations shown in Figure 8.1 that depict operation results do not have any names. In general, for most queries, we need to apply several relational algebra operations one after the other. Either we can write the operations as a single relational algebra expression by nesting the operations, or we can apply one operation at a time and create intermediate result relations. In the latter case, we must give names to the relations that hold the intermediate results. For example, to retrieve the first name, last name, and salary of all employees who work in department number 5, we must apply a SELECT and a PROJECT operation. We can write a single relational algebra expression, also known as an in-line expression, as follows:

$$\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$$

Figure 8.2(a) shows the result of this in-line relational algebra expression. Alternatively, we can explicitly show the sequence of operations, giving a name to each intermediate relation, and using the assignment operation, denoted by $\leftarrow$ (left arrow), as follows:

$$\text{DEP5_EMPS} \leftarrow \sigma_{\text{Dno}=5}(\text{EMPLOYEE})$$

$$\text{RESULT} \leftarrow \pi_{\text{Fname, Lname, Salary}}(\text{DEP5_EMPS})$$

It is sometimes simpler to break down a complex sequence of operations by specifying intermediate result relations than to write a single relational algebra expression. We can also use this technique to rename the attributes in the intermediate and result relations. This can be useful in connection with more complex operations such as UNION and JOIN, as we shall see. To rename the attributes in a relation, we simply list the new attribute names in parentheses, as in the following example:

$$\text{TEMP} \leftarrow \sigma_{\text{Dno}=5}(\text{EMPLOYEE})$$

$$\text{R}(\text{First_name, Last_name, Salary}) \leftarrow \pi_{\text{Fname, Lname, Salary}}(\text{TEMP})$$

These two operations are illustrated in Figure 8.2(b).

If no renaming is applied, the names of the attributes in the resulting relation of a SELECT operation are the same as those in the original relation and in the same order. For a PROJECT operation with no renaming, the resulting relation has the same attribute names as those in the projection list and in the same order in which they appear in the list.

We can also define a formal RENAME operation—which can rename either the relation name or the attribute names, or both—as a unary operator. The general RENAME operation when applied to a relation R of degree n is denoted by any of the following three forms:

$$\rho_S(B_1, B_2, \dots, B_n)(R) \text{ or } \rho_S(R) \text{ or } \rho(B_1, B_2, \dots, B_n)(R)$$

(a)

Fname Lname Salary

John Smith 30000

Franklin Wong 40000

Ramesh Narayan 38000

Joyce English 25000

(b)

TEMP

Fname Minit Lname Ssn Bdate Address Sex Salary Super_ssn Dno

John B Smith 123456789 1965-01-09 731 Fondren, Houston,TX M 30000 333445555 5

Franklin T Wong 333445555 1955-12-08 638 Voss, Houston,TX M 40000 888665555 5

Ramesh K Narayan 666884444 1962-09-15 975 Fire Oak, Humble,TX M 38000 333445555 5

Joyce A English 453453453 1972-07-31 5631 Rice, Houston, TX F 25000 333445555 5

R

First_name Last_name Salary

John Smith 30000

Franklin Wong 40000

Ramesh Narayan 38000

Joyce English 25000

Figure 8.2

Results of a sequence of operations. (a) $\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$.

(b) Using intermediate relations and renaming of attributes.

where the symbol ρ (rho) is used to denote the RENAME operator, S is the new relation name, and $B_1, B_2, \dots, B_n$ are the new attribute names. The first expression renames both the relation and its attributes, the second renames the relation only, and the third renames the attributes only. If the attributes of R are $(A_1, A_2, \dots, A_n)$ in that order, then each A_i is renamed as B_i .

In SQL, a single query typically represents a complex relational algebra expression.

Renaming in SQL is accomplished by aliasing using AS, as in the following example:

```
SELECT E.Fname AS First_name, E.Lname AS Last_name, E.Salary AS Salary
FROM EMPLOYEE AS E
WHERE E.Dno=5,
```

8.2 Relational Algebra Operations

from Set Theory

8.2.1 The UNION, INTERSECTION, and MINUS Operations

The next group of relational algebra operations are the standard mathematical operations on sets. For example, to retrieve the Social Security numbers of all

8.2 Relational Algebra Operations from Set Theory 277

employees who either work in department 5 or directly supervise an employee who works in department 5, we can use the UNION operation as follows:4

```

DEP5_EMPS ← σDno=5(EMPLOYEE)
RESULT1 ← πSsn(DEP5_EMPS)
RESULT2(Ssn) ← πSuper_ssn(DEP5_EMPS)
RESULT ← RESULT1 ∪ RESULT2

```

The relation RESULT1 has the Ssn of all employees who work in department 5, whereas RESULT2 has the Ssn of all employees who directly supervise an employee who works in department 5. The UNION operation produces the tuples that are in either RESULT1 or RESULT2 or both (see Figure 8.3) while eliminating any duplicates. Thus, the Ssn value '333445555' appears only once in the result.

Several set theoretic operations are used to merge the elements of two sets in various ways, including UNION, INTERSECTION, and SET DIFFERENCE (also called MINUS or EXCEPT). These are binary operations; that is, each is applied to two sets (of tuples). When these operations are adapted to relational databases, the two relations on which any of these three operations are applied must have the same type of tuples; this condition has been called union compatibility or type compatibility.

Two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_n)$ are said to be union compatible (or type compatible) if they have the same degree n and if $\text{dom}(A_i) = \text{dom}(B_i)$ for $1 \leq i \leq n$. This means that the two relations have the same number of attributes and each corresponding pair of attributes has the same domain.

We can define the three operations UNION, INTERSECTION, and SET DIFFERENCE on two union-compatible relations R and S as follows:

■■ UNION: The result of this operation, denoted by $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S . Duplicate tuples are eliminated.

■■ INTERSECTION: The result of this operation, denoted by $R \cap S$, is a relation that includes all tuples that are in both R and S .

■■ SET DIFFERENCE (or MINUS): The result of this operation, denoted by $R - S$, is a relation that includes all tuples that are in R but not in S .

RESULT1 RESULT2 RESULT Figure 8.3

Result of the UNION operation

Ssn Ssn Ssn RESULT ← RESULT1 ∪ RESULT2.

123456789 333445555 123456789

333445555 888665555 333445555

666884444 666884444

453453453 453453453

888665555

4

As a single relational algebra expression, this becomes $\text{Result} \leftarrow \pi_{\text{Ssn}} (\sigma_{\text{Dno}=5} (\text{EMPLOYEE})) \cup \pi_{\text{Super_ssn}} (\sigma_{\text{Dno}=5} (\text{EMPLOYEE}))$.

278 Chapter 8 The Relational Algebra and Relational Calculus

We will adopt the convention that the resulting relation has the same attribute names as the first relation R. It is always possible to rename the attributes in the result using the rename operator.

Figure 8.4 illustrates the three operations. The relations STUDENT and INSTRUCTOR in Figure 8.4(a) are union compatible and their tuples represent the names of students and the names of instructors, respectively. The result of the UNION operation in Figure 8.4(b) shows the names of all students and instructors. Note that duplicate tuples appear only once in the result. The result of the INTERSECTION operation (Figure 8.4(c)) includes only those who are both students and instructors.

Notice that both UNION and INTERSECTION are commutative operations; that is, $R \cup S = S \cup R$ and $R \cap S = S \cap R$

Both UNION and INTERSECTION can be treated as n-ary operations applicable to any number of relations because both are also associative operations; that is, $R \cup (S \cup T) = (R \cup S) \cup T$ and $(R \cap S) \cap T = R \cap (S \cap T)$

Figure 8.4

The set operations UNION, INTERSECTION, and MINUS. (a) Two union-compatible relations. (b) $\text{STUDENT} \cup \text{INSTRUCTOR}$. (c) $\text{STUDENT} \cap \text{INSTRUCTOR}$. (d) $\text{STUDENT} - \text{INSTRUCTOR}$. (e) $\text{INSTRUCTOR} - \text{STUDENT}$.

(a) STUDENT INSTRUCTOR

Fn Ln Fname Lname (b) Fn Ln

Susan Yao John Smith Susan Yao

Ramesh Shah Ricardo Browne Ramesh Shah

Johnny Kohler Susan Yao Johnny Kohler

Barbara Jones Francis Johnson Barbara Jones

Amy Ford Ramesh Shah Amy Ford

Jimmy Wang Jimmy Wang

Ernest Gilbert Ernest Gilbert

John Smith

Ricardo Browne

Francis Johnson

(c) Fn Ln (d) Fn Ln (e) Fname Lname

Susan Yao Johnny Kohler John Smith

Ramesh Shah Barbara Jones Ricardo Browne

Amy Ford Francis Johnson

Jimmy Wang

Ernest Gilbert

8.2 Relational Algebra Operations from Set Theory 279

The MINUS operation is not commutative; that is, in general,

$R - S \neq S - R$

Figure 8.4(d) shows the names of students who are not instructors, and Fig-

ure 8.4(e) shows the names of instructors who are not students.

Note that INTERSECTION can be expressed in terms of union and set difference as follows:

$$R \cap S = ((R \cup S) - (R - S)) - (S - R)$$

In SQL, there are three operations—UNION, INTERSECT, and EXCEPT—that correspond to the set operations described here. In addition, there are multiset operations (UNION ALL, INTERSECT ALL, and EXCEPT ALL) that do not eliminate duplicates (see Section 6.3.4).

8.2.2 The CARTESIAN PRODUCT (CROSS PRODUCT)

Operation

Next, we discuss the CARTESIAN PRODUCT operation—also known as CROSS PRODUCT or CROSS JOIN—which is denoted by $\times$. This is also a binary set operation, but the relations on which it is applied do not have to be union compatible. In its binary form, this set operation produces a new element by combining every member (tuple) from one relation (set) with every member (tuple) from the other relation (set). In general, the result of $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$ is a relation Q with degree $n + m$ attributes $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order. The resulting relation Q has one tuple for each combination of tuples—one from R and one from S . Hence, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R * n_S$ tuples.

The n -ary CARTESIAN PRODUCT operation is an extension of the above concept, which produces new tuples by concatenating all possible combinations of tuples from n underlying relations. The CARTESIAN PRODUCT operation applied by itself is generally meaningless. It is mostly useful when followed by a selection that matches values of attributes coming from the component relations. For example, suppose that we want to retrieve a list of names of each female employee's dependents. We can do this as follows:

```
FEMALE_EMPS ← σSex='F'(EMPLOYEE)
EMP_NAMES ← πFname, Lname, Ssn(FEMALE_EMPS)
EMP_DEPENDENTS ← EMP_NAMES × DEPENDENT
ACTUAL_DEPENDENTS ← σSsn=Essn(EMP_DEPENDENTS)
RESULT ← πFname, Lname, Dependent_name(ACTUAL_DEPENDENTS)
```

The resulting relations from this sequence of operations are shown in Figure 8.5. The EMP_DEPENDENTS relation is the result of applying the CARTESIAN PRODUCT operation to EMP_NAMES from Figure 8.5 with DEPENDENT from Figure 5.6. In EMP_DEPENDENTS, every tuple from EMP_NAMES is combined with every tuple from DEPENDENT, giving a result that is not very meaningful (every dependent is

280 Chapter 8 The Relational Algebra and Relational Calculus

Figure 8.5

The CARTESIAN PRODUCT (CROSS PRODUCT) operation.

FEMALE_EMPS

```
Fname Minit Lname Ssn Bdate Address Sex Salary Super_ssn Dno
Alicia J Zelaya 999887777 1968-07-19 3321Castle, Spring, TX F 25000 987654321 4
Jennifer S Wallace 987654321 1941-06-20 291Berry, Bellaire, TX F 43000 888665555 4
Joyce A English 453453453 1972-07-31 5631 Rice, Houston, TX F 25000 333445555 5
```

EMPNames

```
Fname Lname Ssn
Alicia Zelaya 999887777
Jennifer Wallace 987654321
Joyce English 453453453
```

EMP_DEPENDENTS

```
Fname Lname Ssn Essn Dependent_name Sex Bdate ...
Alicia Zelaya 999887777 333445555 Alice F 1986-04-05 ...
Alicia Zelaya 999887777 333445555 Theodore M 1983-10-25 ...
Alicia Zelaya 999887777 333445555 Joy F 1958-05-03 ...
Alicia Zelaya 999887777 987654321 Abner M 1942-02-28 ...
Alicia Zelaya 999887777 123456789 Michael M 1988-01-04 ...
Alicia Zelaya 999887777 123456789 Alice F 1988-12-30 ...
Alicia Zelaya 999887777 123456789 Elizabeth F 1967-05-05 ...
Jennifer Wallace 987654321 333445555 Alice F 1986-04-05 ...
Jennifer Wallace 987654321 333445555 Theodore M 1983-10-25 ...
Jennifer Wallace 987654321 333445555 Joy F 1958-05-03 ...
Jennifer Wallace 987654321 987654321 Abner M 1942-02-28 ...
Jennifer Wallace 987654321 123456789 Michael M 1988-01-04 ...
Jennifer Wallace 987654321 123456789 Alice F 1988-12-30 ...
Jennifer Wallace 987654321 123456789 Elizabeth F 1967-05-05 ...
Joyce English 453453453 333445555 Alice F 1986-04-05 ...
Joyce English 453453453 333445555 Theodore M 1983-10-25 ...
Joyce English 453453453 333445555 Joy F 1958-05-03 ...
Joyce English 453453453 987654321 Abner M 1942-02-28 ...
Joyce English 453453453 123456789 Michael M 1988-01-04 ...
Joyce English 453453453 123456789 Alice F 1988-12-30 ...
Joyce English 453453453 123456789 Elizabeth F 1967-05-05 ...
```

ACTUAL_DEPENDENTS

```
Fname Lname Ssn Essn Dependent_name Sex Bdate ...
Jennifer Wallace 987654321 987654321 Abner M 1942-02-28 ...
```

RESULT

```
Fname Lname Dependent_name
Jennifer Wallace Abner
```

8.3 Binary Relational Operations: JOIN and DIVISION 281

combined with every female employee). We want to combine a female employee

tuple only with her particular dependents—namely, the `DEPENDENT` tuples whose `Essn` value match the `Ssn` value of the `EMPLOYEE` tuple. The `ACTUAL_DEPENDENTS` relation accomplishes this. The `EMP_DEPENDENTS` relation is a good example of the case where relational algebra can be correctly applied to yield results that make no sense at all. It is the responsibility of the user to make sure to apply only meaningful operations to relations.

The `CARTESIAN PRODUCT` creates tuples with the combined attributes of two relations. We can `SELECT` related tuples only from the two relations by specifying an appropriate selection condition after the Cartesian product, as we did in the preceding example. Because this sequence of `CARTESIAN PRODUCT` followed by `SELECT` is quite commonly used to combine related tuples from two relations, a special operation, called `JOIN`, was created to specify this sequence as a single operation. We discuss the `JOIN` operation next.

In SQL, `CARTESIAN PRODUCT` can be realized by using the `CROSS JOIN` option in joined tables (see Section 7.1.6). Alternatively, if there are two tables in the `FROM` clause and there is no corresponding join condition in the `WHERE` clause of the SQL query, the result will also be the `CARTESIAN PRODUCT` of the two tables (see Q10 in Section 6.3.3).

8.3 Binary Relational Operations:

`JOIN` and `DIVISION`

8.3.1 The `JOIN` Operation

The `JOIN` operation, denoted by $\bowtie$, is used to combine related tuples from two relations into single “longer” tuples. This operation is very important for any relational database with more than a single relation because it allows us to process relationships among relations. To illustrate `JOIN`, suppose that we want to retrieve the name of the manager of each department. To get the manager’s name, we need to combine each department tuple with the employee tuple whose `Ssn` value matches the `Mgr_ssn` value in the department tuple. We do this by using the `JOIN` operation and then projecting the result over the necessary attributes, as follows:

```
DEPT_MGR  $\leftarrow$  DEPARTMENT  $\bowtie$  Mgr_ssn=Ssn EMPLOYEE
```

```
RESULT  $\leftarrow$   $\pi$ Dname, Lname, Fname(DEPT_MGR)
```

The first operation is illustrated in Figure 8.6. Note that `Mgr_ssn` is a foreign key of the `DEPARTMENT` relation that references `Ssn`, the primary key of the `EMPLOYEE` relation. This referential integrity constraint plays a role in having matching tuples in the referenced relation `EMPLOYEE`.

The `JOIN` operation can be specified as a `CARTESIAN PRODUCT` operation followed by a `SELECT` operation. However, `JOIN` is very important because it is used frequently when specifying database queries. Consider the earlier example

282 Chapter 8 The Relational Algebra and Relational Calculus

Figure 8.6

Result of the `JOIN` operation `DEPT_MGR  $\leftarrow$  DEPARTMENT  $\bowtie$  Mgr_ssn=Ssn EMPLOYEE`.

DEPT_MGR

Dname Dnumber Mgr_ssn ... Fname Minit Lname Ssn ...

Research 5 333445555 ... Franklin T Wong 333445555 ...

Administration 4 987654321 ... Jennifer S Wallace 987654321 ...

Headquarters 1 888665555 ... James E Borg 888665555 ...

illustrating CARTESIAN PRODUCT, which included the following sequence of operations:

$EMP_DEPENDENTS \leftarrow EMPNAMES \times DEPENDENT$

$ACTUAL_DEPENDENTS \leftarrow \sigma_{Ssn=Essn}(EMP_DEPENDENTS)$

These two operations can be replaced with a single JOIN operation as follows:

$ACTUAL_DEPENDENTS \leftarrow EMPNAMES \Join_{Ssn=Essn} DEPENDENT$

The general form of a JOIN operation on two relations⁵ $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is

$R \Join_{\text{join condition}} S$

The result of the JOIN is a relation Q with $n + m$ attributes $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$ in that order; Q has one tuple for each combination of tuples—one from R and one from S —whenever the combination satisfies the join condition. This is the main difference between CARTESIAN PRODUCT and JOIN. In JOIN, only combinations of tuples satisfying the join condition appear in the result, whereas in the CARTESIAN PRODUCT all combinations of tuples are included in the result. The join condition is specified on attributes from the two relations R and S and is evaluated for each combination of tuples. Each tuple combination for which the join condition evaluates to TRUE is included in the resulting relation Q as a single combined tuple.

A general join condition is of the form

$\langle \text{condition} \rangle \text{ AND } \langle \text{condition} \rangle \text{ AND } \dots \text{ AND } \langle \text{condition} \rangle$

where each $\langle \text{condition} \rangle$ is of the form $A_i \theta B_j$, A_i is an attribute of R , B_j is an attribute of S , A_i and B_j have the same domain, and θ (theta) is one of the comparison operators $\{=, <, \leq, >, \geq, \neq\}$. A JOIN operation with such a general join condition is called a THETA JOIN. Tuples whose join attributes are NULL or for which the join condition is FALSE do not appear in the result. In that sense, the JOIN operation does not necessarily preserve all of the information in the participating relations, because tuples that do not get combined with matching ones in the other relation do not appear in the result.

⁵

Again, notice that R and S can be any relations that result from general relational algebra expressions.

8.3 Binary Relational Operations: JOIN and DIVISION 283

8.3.2 Variations of JOIN: The EQUIJOIN and NATURAL JOIN

The most common use of JOIN involves join conditions with equality comparisons

only. Such a JOIN, where the only comparison operator used is $=$, is called an EQUIJOIN. Both previous examples were EQUIJOINS. Notice that in the result of an EQUIJOIN we always have one or more pairs of attributes that have identical values in every tuple. For example, in Figure 8.6, the values of the attributes *Mgr_ssn* and *Ssn* are identical in every tuple of *DEPT_MGR* (the EQUIJOIN result) because the equality join condition specified on these two attributes requires the values to be identical in every tuple in the result. Because one of each pair of attributes with identical values is superfluous, a new operation called NATURAL JOIN—denoted by $*$ —was created to get rid of the second (superfluous) attribute in an EQUIJOIN condition.⁶ The standard definition of NATURAL JOIN requires that the two join attributes (or each pair of join attributes) have the same name in both relations. If this is not the case, a renaming operation is applied first.

Suppose we want to combine each *PROJECT* tuple with the *DEPARTMENT* tuple that controls the project. In the following example, first we rename the *Dnumber* attribute of *DEPARTMENT* to *Dnum*—so that it has the same name as the *Dnum* attribute in *PROJECT*—and then we apply NATURAL JOIN:

```
PROJ_DEPT ← PROJECT * ρ(Dname, Dnum, Mgr_ssn, Mgr_start_date)(DEPARTMENT)
```

The same query can be done in two steps by creating an intermediate table *DEPT* as follows:

```
DEPT ← ρ(Dname, Dnum, Mgr_ssn, Mgr_start_date)(DEPARTMENT)
```

```
PROJ_DEPT ← PROJECT * DEPT
```

The attribute *Dnum* is called the join attribute for the NATURAL JOIN operation, because it is the only attribute with the same name in both relations. The resulting relation is illustrated in Figure 8.7(a). In the *PROJ_DEPT* relation, each tuple combines a *PROJECT* tuple with the *DEPARTMENT* tuple for the department that controls the project, but only one join attribute value is kept.

If the attributes on which the natural join is specified already have the same names in both relations, renaming is unnecessary. For example, to apply a natural join on the *Dnumber* attributes of *DEPARTMENT* and *DEPT_LOCATIONS*, it is sufficient to write

```
DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS
```

The resulting relation is shown in Figure 8.7(b), which combines each department with its locations and has one tuple for each location. In general, the join condition for NATURAL JOIN is constructed by equating each pair of join attributes that have the same name in the two relations and combining these conditions with AND.

There can be a list of join attributes from each relation, and each corresponding pair must have the same name.

6

NATURAL JOIN is basically an EQUIJOIN followed by the removal of the superfluous attributes.

284 Chapter 8 The Relational Algebra and Relational Calculus

(a)

PROJ_DEPT

Pname Pnumber Plocation Dnum Dname Mgr_ssn Mgr_start_date

ProductX 1 Bellaire 5 Research 333445555 1988-05-22

ProductY 2 Sugarland 5 Research 333445555 1988-05-22

ProductZ 3 Houston 5 Research 333445555 1988-05-22

Computerization 10 Stafford 4 Administration 987654321 1995-01-01

Reorganization 20 Houston 1 Headquarters 888665555 1981-06-19

Newbenefits 30 Stafford 4 Administration 987654321 1995-01-01

(b)

DEPT_LOCS

Dname Dnumber Mgr_ssn Mgr_start_date Location

Headquarters 1 888665555 1981-06-19 Houston

Administration 4 987654321 1995-01-01 Stafford

Research 5 333445555 1988-05-22 Bellaire

Research 5 333445555 1988-05-22 Sugarland

Research 5 333445555 1988-05-22 Houston

Figure 8.7

Results of two natural join operations. (a) $\text{proj_dept} \leftarrow \text{project} * \text{dept}$.(b) $\text{dept_locs} \leftarrow \text{department} * \text{dept_locations}$.

Notice that if no combination of tuples satisfies the join condition, the result of a JOIN is an empty relation with zero tuples. In general, if R has n_R tuples and S has n_S tuples, the result of a JOIN operation $R \lt \text{join condition} \gt S$ will have between zero and $n_R * n_S$ tuples. The expected size of the join result divided by the maximum size $n_R * n_S$ leads to a ratio called join selectivity, which is a property of each join condition. If there is no join condition, all combinations of tuples qualify and the JOIN degenerates into a CARTESIAN PRODUCT, also called CROSS PRODUCT or CROSS JOIN.

As we can see, a single JOIN operation is used to combine data from two relations so that related information can be presented in a single table. These operations are also known as inner joins, to distinguish them from a different join variation called outer joins (see Section 8.4.4). Informally, an inner join is a type of match-and-combine operation defined formally as a combination of CARTESIAN PRODUCT and SELECTION. Note that sometimes a join may be specified between a relation and itself, as we will illustrate in Section 8.4.3. The NATURAL JOIN or EQUIJOIN operation can also be specified among multiple tables, leading to an n-way join. For example, consider the following three-way join:

$$((\text{PROJECT Dnum}=\text{DnumberDEPARTMENT}) \text{ Mgr_ssn}=\text{SsnEMPLOYEE})$$

8.3 Binary Relational Operations: JOIN and DIVISION 285

This combines each project tuple with its controlling department tuple into a single

tuple, and then combines that tuple with an employee tuple that is the department manager. The net result is a consolidated relation in which each tuple contains this project-department-manager combined information.

In SQL, JOIN can be realized in several different ways. The first method is to specify the <join conditions> in the WHERE clause, along with any other selection conditions. This is very common and is illustrated by queries Q1, Q1A, Q1B, Q2, and Q8 in Sections 6.3.1 and 6.3.2, as well as by many other query examples in Chapters 6 and 7. The second way is to use a nested relation, as illustrated by queries Q4A and Q16 in Section 7.1.2. Another way is to use the concept of joined tables, as illustrated by the queries Q1A, Q1B, Q8B, and Q2A in Section 7.1.6. The construct of joined tables was added to SQL2 to allow the user to specify explicitly all the various types of joins, because the other methods were more limited. It also allows the user to clearly distinguish join conditions from the selection conditions in the WHERE clause.

8.3.3 A Complete Set of Relational Algebra Operations

It has been shown that the set of relational algebra operations $\{\sigma, \pi, \cup, \rho, -, \times\}$ is a complete set; that is, any of the other original relational algebra operations can be expressed as a sequence of operations from this set. For example, the INTERSECTION operation can be expressed by using UNION and MINUS as follows:

$$R \cap S \equiv (R \cup S) - ((R - S) \cup (S - R))$$

Although, strictly speaking, INTERSECTION is not required, it is inconvenient to specify this complex expression every time we wish to specify an intersection. As another example, a JOIN operation can be specified as a CARTESIAN PRODUCT followed by a SELECT operation, as we discussed:

$$R \text{ <condition> } S \equiv \sigma \text{ <condition> } (R \times S)$$

Similarly, a NATURAL JOIN can be specified as a CARTESIAN PRODUCT preceded by RENAME and followed by SELECT and PROJECT operations. Hence, the various JOIN operations are also not strictly necessary for the expressive power of the relational algebra. However, they are important to include as separate operations because they are convenient to use and are very commonly applied in database applications. Other operations have been included in the basic relational algebra for convenience rather than necessity. We discuss one of these—the DIVISION operation—in the next section.

8.3.4 The DIVISION Operation

The DIVISION operation, denoted by $\div$, is useful for a special kind of query that sometimes occurs in database applications. An example is Retrieve the names of employees who work on all the projects that 'John Smith' works on. To express this query using the DIVISION operation, proceed as follows. First, retrieve the

286 Chapter 8 The Relational Algebra and Relational Calculus

list of project numbers that 'John Smith' works on in the intermediate relation

SMITH_PNOS:

$SMITH \leftarrow \sigma_{Fname='John' \text{ AND } Lname='Smith'}(EMPLOYEE)$

$SMITH_PNOS \leftarrow \pi_{Pno}(WORKS_ON \text{ Essn}=SsnSMITH)$

Next, create a relation that includes a tuple $\langle Pno, Essn \rangle$ whenever the employee whose Ssn is Essn works on the project whose number is Pno in the intermediate relation SSN_PNOS:

$SSN_PNOS \leftarrow \pi_{Essn, Pno}(WORKS_ON)$

Finally, apply the DIVISION operation to the two relations, which gives the desired employees' Social Security numbers:

$SSNS(Ssn) \leftarrow SSN_PNOS \div SMITH_PNOS$

$RESULT \leftarrow \pi_{Fname, Lname}(SSNS * EMPLOYEE)$

The preceding operations are shown in Figure 8.8(a).

In general, the DIVISION operation is applied to two relations $R(Z) \div S(X)$, where the attributes of S are a subset of the attributes of R; that is, $X \subseteq Z$. Let Y be the set of attributes of R that are not attributes of S; that is, $Y = Z - X$ (and hence $Z = X \cup Y$).

Figure 8.8

The DIVISION operation. (a) Dividing SSN_PNOS by SMITH_PNOS. (b) $T \leftarrow R \div S$.

(a) (b)

SSN_PNOS SMITH_PNOS R S

Essn Pno Pno A B A

123456789 1 1 a1 b1 a1

123456789 2 2 a2 b1 a2

666884444 3 a3 b1 a3

453453453 1 a4 b1

453453453 2 SSNS a1 b2 T

333445555 2 Ssn a3 b2 B

333445555 3 123456789 a2 b3 b1

333445555 10 453453453 a3 b3 b4

333445555 20 a4 b3

999887777 30 a1 b4

999887777 10 a2 b4

987987987 10 a3 b4

987987987 30

987654321 30

987654321 20

888665555 20

8.3 Binary Relational Operations: JOIN and DIVISION 287

The result of DIVISION is a relation $T(Y)$ that includes a tuple t if tuples t_R appear in R with $t_R[Y] = t$, and with $t_R[X] = t_S$ for every tuple t_S in S . This means that, for a tuple t to appear in the result T of the DIVISION, the values in t must appear in R in combination with every tuple in S . Note that in the formulation of the DIVISION

operation, the tuples in the denominator relation S restrict the numerator relation R by selecting those tuples in the result that match all values present in the denominator. It is not necessary to know what those values are as they can be computed by another operation, as illustrated in the $SMITH_PNOS$ relation in the previous example.

Figure 8.8(b) illustrates a $DIVISION$ operation where $X = \{A\}$, $Y = \{B\}$, and $Z = \{A, B\}$.

Notice that the tuples (values) $b1$ and $b4$ appear in R in combination with all three tuples in S ; that is why they appear in the resulting relation T . All other values of B in R do not appear with all the tuples in S and are not selected: $b2$ does not appear with $a2$, and $b3$ does not appear with $a1$.

The $DIVISION$ operation can be expressed as a sequence of π , $\times$, and $-$ operations as follows:

$T1 \leftarrow \pi_Y(R)$

$T2 \leftarrow \pi_Y((S \times T1) - R)$

$T \leftarrow T1 - T2$

The $DIVISION$ operation is defined for convenience for dealing with queries that involve universal quantification (see Section 8.6.7) or the all condition. Most RDBMS implementations with SQL as the primary query language do not directly implement division. SQL has a roundabout way of dealing with the type of query just illustrated (see Section 7.1.4, queries Q3A and Q3B). Table 8.1 lists the various basic relational algebra operations we have discussed.

8.3.5 Notation for Query Trees

In this section we describe a notation typically used in relational DBMSs (RDBMSs) to represent queries internally. The notation is called a query tree or sometimes it is known as a query evaluation tree or query execution tree. It includes the relational algebra operations being executed and is used as a possible data structure for the internal representation of the query in an RDBMS.

A query tree is a tree data structure that corresponds to a relational algebra expression. It represents the input relations of the query as leaf nodes of the tree, and represents the relational algebra operations as internal nodes. An execution of the query tree consists of executing an internal node operation whenever its operands (represented by its child nodes) are available, and then replacing that internal node by the relation that results from executing the operation. The execution terminates when the root node is executed and produces the result relation for the query.

Figure 8.9 shows a query tree for Query 2 (see Section 6.3.1): For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date. This query is specified

288 Chapter 8 The Relational Algebra and Relational Calculus

Table 8.1 Operations of Relational Algebra

OPERATION PURPOSE NOTATION

SELECT Selects all tuples that satisfy the selection $\sigma_{\langle \text{selection condition} \rangle}(R)$

condition from a relation R.

PROJECT Produces a new relation with only some of the $\pi_{\langle \text{attribute list} \rangle}(R)$ attributes of R, and removes duplicate tuples.

THETA JOIN Produces all combinations of tuples from R1 and R2 that satisfy the join condition $R1 \bowtie_{\langle \text{join condition} \rangle} R2$.

EQUIJOIN Produces all the combinations of tuples from R1 and R2 that satisfy a join condition with R1 ($\bowtie_{\langle \text{join attributes 1} \rangle}$), only equality comparisons. ($\bowtie_{\langle \text{join attributes 2} \rangle}$) R2

NATURAL JOIN Same as EQUIJOIN except that the join attributes of R1 and R2 are not included in the resulting relation; OR $R1 \bowtie_{\langle \text{join attributes 1} \rangle} R2$ if the join attributes have the same names, they ($\bowtie_{\langle \text{join attributes 2} \rangle}$) do not have to be specified at all. $R2 \text{ OR } R1 * R2$

UNION Produces a relation that includes all the tuples $R1 \cup R2$ in R1 or R2 or both R1 and R2; R1 and R2 must be union compatible.

INTERSECTION Produces a relation that includes all the tuples $R1 \cap R2$ in both R1 and R2; R1 and R2 must be union compatible.

DIFFERENCE Produces a relation that includes all the tuples $R1 - R2$ in R1 that are not in R2; R1 and R2 must be union compatible.

CARTESIAN PRODUCT Produces a relation that has the attributes of $R1 \times R2$ R1 and R2 and includes as tuples all possible combinations of tuples from R1 and R2.

DIVISION Produces a relation $R(X)$ that includes all tuples $R1(Z) \div R2(Y)$ $t[X]$ in $R1(Z)$ that appear in R1 in combination with every tuple from $R2(Y)$, where $Z = X \cup Y$.

on the relational schema of Figure 5.5 and corresponds to the following relational algebra expression:

$\pi_{\text{Pnumber, Dnum, Lname, Address, Bdate}}(((\sigma_{\text{Plocation}='Stafford'}(\text{PROJECT}))$
 $\text{Dnum}=\text{Dnumber}(\text{DEPARTMENT})) \text{ Mgr_ssn}=\text{Ssn}(\text{EMPLOYEE}))$

In Figure 8.9, the three leaf nodes P, D, and E represent the three relations PROJECT, DEPARTMENT, and EMPLOYEE. The relational algebra operations in the expression are represented by internal tree nodes. The query tree signifies an explicit order of execution in the following sense. In order to execute Q2, the node marked (1) in Figure 8.9 must begin execution before node (2) because some resulting tuples of operation (1) must be available before we can begin to execute operation (2). Similarly,

8.4 Additional Relational Operations 289

π

P.Pnumber,P.Dnum,E.Lname,E.Address,E.Bdate

(3)

D.Mgr_ssn=E.Ssn

(2)

P.Dnum=D.Dnumber E EMPLOYEE

(1)

σ P.Plocation= 'Stafford' D DEPARTMENT

Figure 8.9

Query tree corresponding

to the relational algebra

P PROJECT

expression for Q2.

node (2) must begin to execute and produce results before node (3) can start execution, and so on. In general, a query tree gives a good visual representation and understanding of the query in terms of the relational operations it uses and is recommended as an additional means for expressing queries in relational algebra. We will revisit query trees when we discuss query processing and optimization in Chapters 18 and 19.

8.4 Additional Relational Operations

Some common database requests—which are needed in commercial applications for RDBMSs—cannot be performed with the original relational algebra operations described in Sections 8.1 through 8.3. In this section we define additional operations to express these requests. These operations enhance the expressive power of the original relational algebra.

8.4.1 Generalized Projection

The generalized projection operation extends the projection operation by allowing functions of attributes to be included in the projection list. The generalized form can be expressed as:

$\pi_{F1, F2, \dots, F_n}(R)$

where $F1, F2, \dots, F_n$ are functions over the attributes in relation R and may involve arithmetic operations and constant values. This operation is helpful when developing reports where computed values have to be produced in the columns of a query result.

290 Chapter 8 The Relational Algebra and Relational Calculus

As an example, consider the relation

EMPLOYEE (Ssn, Salary, Deduction, Years_service)

A report may be required to show

Net Salary = Salary - Deduction,

Bonus = 2000 * Years_service, and

Tax = 0.25 * Salary

Then a generalized projection combined with renaming may be used as follows:

REPORT $\leftarrow \rho(\text{Ssn}, \text{Net_salary}, \text{Bonus}, \text{Tax})(\pi_{\text{Ssn}, \text{Salary} - \text{Deduction}, 2000 * \text{Years_service}, 0.25 * \text{Salary}}(\text{EMPLOYEE}))$

8.4.2 Aggregate Functions and Grouping

Another type of request that cannot be expressed in the basic relational algebra is to specify mathematical aggregate functions on collections of values from the database. Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples. These functions are used in simple statistical queries that summarize information from the database tuples. Common functions applied to collections of numeric values include SUM, AVERAGE, MAXIMUM, and MINIMUM. The COUNT function is used for counting tuples or values.

Another common type of request involves grouping the tuples in a relation by the value of some of their attributes and then applying an aggregate function independently to each group. An example would be to group EMPLOYEE tuples by Dno, so that each group includes the tuples for employees working in the same department. We can then list each Dno value along with, say, the average salary of employees within the department, or the number of employees who work in the department. We can define an AGGREGATE FUNCTION operation, using the symbol $\bowtie$ (pronounced script F)⁷, to specify these types of requests as follows:

$\langle \text{grouping attributes} \rangle \bowtie \langle \text{function list} \rangle (R)$

where $\langle \text{grouping attributes} \rangle$ is a list of attributes of the relation specified in R, and $\langle \text{function list} \rangle$ is a list of ($\langle \text{function} \rangle \langle \text{attribute} \rangle$) pairs. In each such pair, $\langle \text{function} \rangle$ is one of the allowed functions—such as SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT—and $\langle \text{attribute} \rangle$ is an attribute of the relation specified by R. The resulting relation has the grouping attributes plus one attribute for each element in the function list. For example, to retrieve each department number, the number of employees in the department, and their average salary, while renaming the resulting attributes as indicated below, we write:

$\rho R(\text{Dno}, \text{No_of_employees}, \text{Average_sal}) (\text{Dno} \bowtie \text{COUNT Ssn}, \text{AVERAGE Salary} (\text{EMPLOYEE}))$

⁷

There is no single agreed-upon notation for specifying aggregate functions. In some cases a “script A” is used.

8.4 Additional Relational Operations 291

R

(a) Dno No_of_employees Average_sal (b) Dno Count_ssn Average_salary

5 4 33250 5 4 33250

4 3 31000 4 3 31000

1 1 55000 1 1 55000

(c) Count_ssn Average_salary

8 35125

Figure 8.10

The aggregate function operation.

- a. $\rho R(\text{Dno}, \text{No_of_employees}, \text{Average_sal})(\text{Dno} \bowtie \text{COUNT Ssn}, \text{AVERAGE Salary}(\text{EMPLOYEE}))$.
- b. $\text{Dno} \bowtie \text{COUNT Ssn}, \text{AVERAGE Salary}(\text{EMPLOYEE})$.
- c. $\bowtie \text{COUNT Ssn}, \text{AVERAGE Salary}(\text{EMPLOYEE})$.

The result of this operation on the EMPLOYEE relation of Figure 5.6 is shown in Figure 8.10(a).

In the preceding example, we specified a list of attribute names—between parentheses in the RENAME operation—for the resulting relation R. If no renaming is applied, then the attributes of the resulting relation that correspond to the function list will each be the concatenation of the function name with the attribute name in the form `<function>_<attribute>`.⁸ For example, Figure 8.10(b) shows the result of the following operation:

$\text{Dno} \bowtie \text{COUNT Ssn}, \text{AVERAGE Salary}(\text{EMPLOYEE})$

If no grouping attributes are specified, the functions are applied to all the tuples in the relation, so the resulting relation has a single tuple only. For example, Figure 8.10(c) shows the result of the following operation:

$\bowtie \text{COUNT Ssn}, \text{AVERAGE Salary}(\text{EMPLOYEE})$

It is important to note that, in general, duplicates are not eliminated when an aggregate function is applied; this way, the normal interpretation of functions such as SUM and AVERAGE is computed.⁹ However, NULL values are not considered in the aggregation, as we discussed in Section 7.1.7. It is worth emphasizing that the result of applying an aggregate function is a relation, not a scalar number—even if it has a single value. This makes the relational algebra a closed mathematical system.

8

Note that this is an arbitrary notation, consistent with what SQL would do.

9

In SQL, the option of eliminating duplicates before applying the aggregate function is available by including the keyword DISTINCT (see Section 4.4.4).

292 Chapter 8 The Relational Algebra and Relational Calculus

8.4.3 Recursive Closure Operations

Another type of operation that, in general, cannot be specified in the basic original relational algebra is recursive closure. This operation is applied to a recursive relationship between tuples of the same type, such as the relationship between an employee and a supervisor. This relationship is described by the foreign key Super_ssn of the EMPLOYEE relation in Figures 5.5 and 5.6, and it relates each employee tuple (in the role of supervisee) to another employee tuple (in the role of supervisor). An example of a recursive operation is to retrieve all supervisees of an

employee e at all levels—that is, all employees e' directly supervised by e , all employees e'' directly supervised by each employee e' , all employees e''' directly supervised by each employee e'' , and so on.

It is relatively straightforward in the relational algebra to specify all employees supervised by e at a specific level by joining the table with itself one or more times. However, it is difficult to specify all supervisees at all levels. For example, to specify the Ssns of all employees e' directly supervised—at level one—by the employee e whose name is 'James Borg' (see Figure 5.6), we can apply the following operation:

```
BORG_SSN ← πSsn(σFname='James' AND Lname='Borg'(EMPLOYEE))
```

```
SUPERVISION(Ssn1, Ssn2) ← πSsn,Super_ssn(EMPLOYEE)
```

```
RESULT1(Ssn) ← πSsn1(SUPERVISION Ssn2=SsnBORG_SSN)
```

To retrieve all employees supervised by Borg at level 2—that is, all employees e'' supervised by some employee e' who is directly supervised by Borg—we can apply another JOIN to the result of the first query, as follows:

```
RESULT2(Ssn) ← πSsn1(SUPERVISION Ssn2=SsnRESULT1)
```

To get both sets of employees supervised at levels 1 and 2 by 'James Borg', we can apply the UNION operation to the two results, as follows:

```
RESULT ← RESULT2 ∪ RESULT1
```

The results of these queries are illustrated in Figure 8.11. Although it is possible to retrieve employees at each level and then take their UNION, we cannot, in general, specify a query such as “retrieve the supervisees of 'James Borg' at all levels” without utilizing a looping mechanism unless we know the maximum number of levels.¹⁰ An operation called the transitive closure of relations has been proposed to compute the recursive relationship as far as the recursion proceeds.

8.4.4 OUTER JOIN Operations

Next, we discuss some additional extensions to the JOIN operation that are necessary to specify certain types of queries. The JOIN operations described earlier match tuples that satisfy the join condition. For example, for a NATURAL JOIN

10

The SQL3 standard includes syntax for recursive closure.

8.4 Additional Relational Operations 293

SUPERVISION

(Borg's Ssn is 888665555)

(Ssn) (Super_ssn)

Ssn1 Ssn2

123456789 333445555

333445555 888665555

999887777 987654321

987654321 888665555

```

666884444 333445555
453453453 333445555
987987987 987654321
888665555 null
RESULT1 RESULT2 RESULT
Ssn Ssn Ssn
333445555 123456789 123456789
987654321 999887777 999887777
(Supervised by Borg) 666884444 666884444
453453453 453453453
987987987 987987987
(Supervised by 333445555
Borg's subordinates) 987654321 Figure 8.11
A two-level recursive
(RESULT1  $\cup$  RESULT2) query.

```

operation $R * S$, only tuples from R that have matching tuples in S —and vice versa—appear in the result. Hence, tuples without a matching (or related) tuple are eliminated from the JOIN result. Tuples with NULL values in the join attributes are also eliminated. This type of join, where tuples with no match are eliminated, is known as an inner join. The join operations we described earlier in Section 8.3 are all inner joins. This amounts to the loss of information if the user wants the result of the JOIN to include all the tuples in one or more of the component relations.

A set of operations, called outer joins, were developed for the case where the user wants to keep all the tuples in R , or all those in S , or all those in both relations in the result of the JOIN, regardless of whether or not they have matching tuples in the other relation. This satisfies the need of queries in which tuples from two tables are to be combined by matching corresponding rows, but without losing any tuples for lack of matching values. For example, suppose that we want a list of all employee names as well as the name of the departments they manage if they happen to manage a department; if they do not manage one, we can indicate it

294 Chapter 8 The Relational Algebra and Relational Calculus

Figure 8.12 RESULT

The result of a LEFT

Fname Minit Lname Dname

OUTER JOIN operation.

John B Smith NULL

Franklin T Wong Research

Alicia J Zelaya NULL

Jennifer S Wallace Administration

Ramesh K Narayan NULL

Joyce A English NULL

Ahmad V Jabbar NULL

James E Borg Headquarters

with a NULL value. We can apply an operation LEFT OUTER JOIN, denoted by $\bowtie$, to retrieve the result as follows:

$TEMP \leftarrow (EMPLOYEE \text{ Ssn=Mgr_ssn } DEPARTMENT)$

$RESULT \leftarrow \pi_{Fname, Minit, Lname, Dname}(TEMP)$

The LEFT OUTER JOIN operation keeps every tuple in the first, or left, relation R in R S; if no matching tuple is found in S, then the attributes of S in the join result are filled or padded with NULL values. The result of these operations is shown in Figure 8.12.

A similar operation, RIGHT OUTER JOIN, denoted by $\bowtie$, keeps every tuple in the second, or right, relation S in the result of R S. A third operation, FULL OUTER JOIN, denoted by $\bowtie$, keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with NULL values as needed. The three outer join operations are part of the SQL2 standard (see Section 7.1.6). These operations were provided later as an extension of relational algebra in response to the typical need in business applications to show related information from multiple tables exhaustively. Sometimes a complete reporting of data from multiple tables is required whether or not there are matching values.

8.4.5 The OUTER UNION Operation

The OUTER UNION operation was developed to take the union of tuples from two relations that have some common attributes, but are not union (type) compatible. This operation will take the UNION of tuples in two relations $R(X, Y)$ and $S(X, Z)$ that are partially compatible, meaning that only some of their attributes, say X, are union compatible. The attributes that are union compatible are represented only once in the result, and those attributes that are not union compatible from either relation are also kept in the result relation $T(X, Y, Z)$. It is therefore the same as a FULL OUTER JOIN on the common attributes.

Two tuples t_1 in R and t_2 in S are said to match if $t_1[X] = t_2[X]$. These will be combined (unioned) into a single tuple in t. Tuples in either relation that have no matching tuple in the other relation are padded with NULL values. For example, an

8.5 Examples of Queries in Relational Algebra 295

OUTER UNION can be applied to two relations whose schemas are STUDENT(Name, Ssn, Department, Advisor) and INSTRUCTOR(Name, Ssn, Department, Rank). Tuples from the two relations are matched based on having the same combination of values of the shared attributes—Name, Ssn, Department. The resulting relation, STUDENT_OR_INSTRUCTOR, will have the following attributes:

STUDENT_OR_INSTRUCTOR(Name, Ssn, Department, Advisor, Rank)

All the tuples from both relations are included in the result, but tuples with the same (Name, Ssn, Department) combination will appear only once in the result.

Tuples appearing only in STUDENT will have a NULL for the Rank attribute, whereas tuples appearing only in INSTRUCTOR will have a NULL for the Advisor attribute. A tuple that exists in both relations, which represent a student who is also an instructor, will have values for all its attributes.¹¹

Notice that the same person may still appear twice in the result. For example, we could have a graduate student in the Mathematics department who is an instructor in the Computer Science department. Although the two tuples representing that person in STUDENT and INSTRUCTOR will have the same (Name, Ssn) values, they will not agree on the Department value, and so will not be matched. This is because Department has two different meanings in STUDENT (the department where the person studies) and INSTRUCTOR (the department where the person is employed as an instructor). If we wanted to apply the OUTER UNION based on the same (Name, Ssn) combination only, we should rename the Department attribute in each table to reflect that they have different meanings and designate them as not being part of the union-compatible attributes. For example, we could rename the attributes as MajorDept in STUDENT and WorkDept in INSTRUCTOR.

8.5 Examples of Queries in Relational Algebra

The following are additional examples to illustrate the use of the relational algebra operations. All examples refer to the database in Figure 5.6. In general, the same query can be stated in numerous ways using the various operations. We will state each query in one way and leave it to the reader to come up with equivalent formulations.

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

```
RESEARCH_DEPT ← σDname='Research'(DEPARTMENT)
RESEARCH_EMPS ← (RESEARCH_DEPT Dnumber=DnoEMPLOYEE)
RESULT ← πFname, Lname, Address(RESEARCH_EMPS)
```

As a single in-line expression, this query becomes:

```
πFname, Lname, Address (σDname='Research'(DEPARTMENT Dnumber=Dno(EMPLOYEE)))
```

11

Note that OUTER UNION is equivalent to a FULL OUTER JOIN if the join attributes are all the common attributes of the two relations.

296 Chapter 8 The Relational Algebra and Relational Calculus

This query could be specified in other ways; for example, the order of the JOIN and SELECT operations could be reversed, or the JOIN could be replaced by a NATURAL JOIN after renaming one of the join attributes to match the other join attribute name.

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
STAFFORD_PROJS ← σPlocation='Stafford'(PROJECT)
```

```

CONTR_DEPTS ← (STAFFORD_PROJS Dnum=DnumberDEPARTMENT)
PROJ_DEPT_MGRS ← (CONTR_DEPTS Mgr_ssn=SsnEMPLOYEE)
RESULT ← πPnumber, Dnum, Lname, Address, Bdate(PROJ_DEPT_MGRS)

```

In this example, we first select the projects located in Stafford, then join them with their controlling departments, and then join the result with the department managers. Finally, we apply a project operation on the desired attributes.

Query 3. Find the names of employees who work on all the projects controlled by department number 5.

```

DEPT5_PROJS ← ρ(Pno)(πPnumber(σDnum=5(PROJECT)))
EMP_PROJ ← ρ(Ssn, Pno)(πEssn, Pno(WORKS_ON))
RESULT_EMP_SSNS ← EMP_PROJ ÷ DEPT5_PROJS
RESULT ← πLname, Fname(RESULT_EMP_SSNS * EMPLOYEE)

```

In this query, we first create a table DEPT5_PROJS that contains the project numbers of all projects controlled by department 5. Then we create a table EMP_PROJ that holds (Ssn, Pno) tuples, and apply the division operation. Notice that we renamed the attributes so that they will be correctly used in the division operation. Finally, we join the result of the division, which holds only Ssn values, with the EMPLOYEE table to retrieve the Fname, Lname attributes from EMPLOYEE.

Query 4. Make a list of project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```

SMITHS(Essn) ← πSsn (σLname='Smith'(EMPLOYEE))
SMITH_WORKER_PROJS ← πPno(WORKS_ON * SMITHS)
MGRS ← πLname, Dnumber(EMPLOYEE Ssn=Mgr_ssnDEPARTMENT)
SMITH_MANAGED_DEPTS(Dnum) ← πDnumber (σLname='Smith'(MGRS))
SMITH_MGR_PROJS(Pno) ← πPnumber(SMITH_MANAGED_DEPTS * PROJECT)
RESULT ← (SMITH_WORKER_PROJS ∪ SMITH_MGR_PROJS)

```

In this query, we retrieved the project numbers for projects that involve an employee named Smith as a worker in SMITH_WORKER_PROJS. Then we retrieved the project numbers for projects that involve an employee named Smith as manager of the department that controls the project in SMITH_MGR_PROJS. Finally, we applied the

8.5 Examples of Queries in Relational Algebra 297

UNION operation on SMITH_WORKER_PROJS and SMITH_MGR_PROJS. As a single in-line expression, this query becomes:

```

πPno (WORKS_ON Essn=Ssn (πSsn (σLname='Smith'(EMPLOYEE))) ∪ πPno
((πDnumber (σLname='Smith'(πLname, Dnumber(EMPLOYEE)))
Ssn=Mgr_ssnDEPARTMENT)) Dnum=DnumberPROJECT)

```

Query 5. List the names of all employees with two or more dependents.

Strictly speaking, this query cannot be done in the basic (original) relational

algebra. We have to use the AGGREGATE FUNCTION operation with the COUNT aggregate function. We assume that dependents of the same employee have distinct Dependent_name values.

```
T1(Ssn, No_of_dependents) ← Essn ⋈ COUNT Dependent_name(DEPENDENT)
T2 ← σNo_of_dependents>2(T1)
RESULT ← πLname, Fname(T2 * EMPLOYEE)
```

Query 6. Retrieve the names of employees who have no dependents.

This is an example of the type of query that uses the MINUS (SET DIFFERENCE) operation.

```
ALL_EMPS ← πSsn(EMPLOYEE)
EMPS_WITH_DEPS(Ssn) ← πEssn(DEPENDENT)
EMPS_WITHOUT_DEPS ← (ALL_EMPS - EMPS_WITH_DEPS)
RESULT ← πLname, Fname(EMPS_WITHOUT_DEPS * EMPLOYEE)
```

We first retrieve a relation with all employee Ssns in ALL_EMPS. Then we create a table with the Ssn s of employees who have at least one dependent in EMPS_WITH_DEPS. Then we apply the SET DIFFERENCE operation to retrieve employees Ssns with no dependents in EMPS_WITHOUT_DEPS, and finally join this with EMPLOYEE to retrieve the desired attributes. As a single in-line expression, this query becomes:

```
πLname, Fname((πSsn(EMPLOYEE) - ρSsn(πEssn(DEPENDENT))) * EMPLOYEE)
```

Query 7. List the names of managers who have at least one dependent.

```
MGRS(Ssn) ← πMgr_ssn(DEPARTMENT)
EMPS_WITH_DEPS(Ssn) ← πEssn(DEPENDENT)
MGRS_WITH_DEPS ← (MGRS ⋈ EMPS_WITH_DEPS)
RESULT ← πLname, Fname(MGRS_WITH_DEPS * EMPLOYEE)
```

In this query, we retrieve the Ssns of managers in MGRS, and the Ssns of employees with at least one dependent in EMPS_WITH_DEPS, then we apply the SET INTERSECTION operation to get the Ssn s of managers who have at least one dependent.

As we mentioned earlier, the same query can be specified in many different ways in relational algebra. In particular, the operations can often be applied in various orders. In addition, some operations can be used to replace others; for example, the

INTERSECTION operation in Q7 can be replaced by a NATURAL JOIN. As an exercise, try to do each of these sample queries using different operations.¹² We showed how to write queries as single relational algebra expressions for queries Q1, Q4, and Q6. Try to write the remaining queries as single expressions. In Chapters 6 and 7 and in Sections 8.6 and 8.7, we show how these queries are written in other relational languages.

8.6 The Tuple Relational Calculus

In this and the next section, we introduce another formal query language for the relational model called relational calculus. This section introduces the language

known as tuple relational calculus, and Section 8.7 introduces a variation called domain relational calculus. In both variations of relational calculus, we write one declarative expression to specify a retrieval request; hence, there is no description of how, or in what order, to evaluate a query. A calculus expression specifies what is to be retrieved rather than how to retrieve it. Therefore, the relational calculus is considered to be a nonprocedural language. This differs from relational algebra, where we must write a sequence of operations to specify a retrieval request in a particular order of applying the operations; thus, it can be considered as a procedural way of stating a query. It is possible to nest algebra operations to form a single expression; however, a certain order among the operations is always explicitly specified in a relational algebra expression. This order also influences the strategy for evaluating the query. A calculus expression may be written in different ways, but the way it is written has no bearing on how a query should be evaluated.

It has been shown that any retrieval that can be specified in the basic relational algebra can also be specified in relational calculus, and vice versa; in other words, the expressive power of the languages is identical. This led to the definition of the concept of a relationally complete language. A relational query language L is considered relationally complete if we can express in L any query that can be expressed in relational calculus. Relational completeness has become an important basis for comparing the expressive power of high-level query languages. However, as we saw in Section 8.4, certain frequently required queries in database applications cannot be expressed in basic relational algebra or calculus. Most relational query languages are relationally complete but have more expressive power than relational algebra or relational calculus because of additional operations such as aggregate functions, grouping, and ordering. As we mentioned in the introduction to this chapter, the relational calculus is important for two reasons. First, it has a firm basis in mathematical logic. Second, the standard query language (SQL) for RDBMSs has its basic foundation in the tuple relational calculus.

Our examples refer to the database shown in Figures 5.6 and 5.7. We will use the same queries that were used in Section 8.5. Sections 8.6.6, 8.6.7, and 8.6.8 discuss dealing with universal quantifiers and safety of expression issues. Students interested in a basic introduction to tuple relational calculus may skip these sections.

12

When queries are optimized (see Chapters 18 and 19), the system will choose a particular sequence

of operations that corresponds to an execution strategy that can be executed efficiently.

8.6 The Tuple Relational Calculus 299

8.6.1 Tuple Variables and Range Relations

The tuple relational calculus is based on specifying a number of tuple variables.

Each tuple variable usually ranges over a particular database relation, meaning that the variable may take as its value any individual tuple from that relation. A simple

tuple relational calculus query is of the form:

$\{t \mid \text{COND}(t)\}$

where t is a tuple variable and $\text{COND}(t)$ is a conditional (Boolean) expression involving t that evaluates to either TRUE or FALSE for different assignments of tuples to the variable t . The result of such a query is the set of all tuples t that evaluate $\text{COND}(t)$ to TRUE. These tuples are said to satisfy $\text{COND}(t)$. For example, to find all employees whose salary is above \$50,000, we can write the following tuple calculus expression:

$\{t \mid \text{EMPLOYEE}(t) \text{ AND } t.\text{Salary} > 50000\}$

The condition $\text{EMPLOYEE}(t)$ specifies that the range relation of tuple variable t is EMPLOYEE . Each EMPLOYEE tuple t that satisfies the condition $t.\text{Salary} > 50000$ will be retrieved. Notice that $t.\text{Salary}$ references attribute Salary of tuple variable t ; this notation resembles how attribute names are qualified with relation names or aliases in SQL, as we saw in Chapter 6. In the notation of Chapter 5, $t.\text{Salary}$ is the same as writing $t[\text{Salary}]$.

The previous query retrieves all attribute values for each selected EMPLOYEE tuple t . To retrieve only some of the attributes—say, the first and last names—we write $t.\text{Fname}, t.\text{Lname} \mid \text{EMPLOYEE}(t) \text{ AND } t.\text{Salary} > 50000\}$

Informally, we need to specify the following information in a tuple relational calculus expression:

- ■ For each tuple variable t , the range relation R of t . This value is specified by a condition of the form $R(t)$. If we do not specify a range relation, then the variable t will range over all possible tuples “in the universe” as it is not restricted to any one relation.

- ■ A condition to select particular combinations of tuples. As tuple variables range over their respective range relations, the condition is evaluated for every possible combination of tuples to identify the selected combinations for which the condition evaluates to TRUE.

- ■ A set of attributes to be retrieved, the requested attributes. The values of these attributes are retrieved for each selected combination of tuples.

Before we discuss the formal syntax of tuple relational calculus, consider another query.

Query 0. Retrieve the birth date and address of the employee (or employees) whose name is John B. Smith.

Q0: $\{t.\text{Bdate}, t.\text{Address} \mid \text{EMPLOYEE}(t) \text{ AND } t.\text{Fname} = \text{'John'} \text{ AND } t.\text{Minit} = \text{'B'} \text{ AND } t.\text{Lname} = \text{'Smith'}\}$

300 Chapter 8 The Relational Algebra and Relational Calculus

In tuple relational calculus, we first specify the requested attributes $t.\text{Bdate}$ and $t.\text{Address}$ for each selected tuple t . Then we specify the condition for selecting a tuple following the bar ($\mid$)—namely, that t be a tuple of the EMPLOYEE relation whose Fname , Minit , and Lname attribute values are ‘John’, ‘B’, and ‘Smith’, respectively.

8.6.2 Expressions and Formulas in Tuple Relational Calculus

A general expression of the tuple relational calculus is of the form

$$\{t1.A_j, t2.A_k, \dots, tn.A_m \mid \text{COND}(t1, t2, \dots, tn, tn+1, tn+2, \dots, tn+m)\}$$

where $t1, t2, \dots, tn, tn+1, \dots, tn+m$ are tuple variables, each A_i is an attribute of the relation on which t_i ranges, and COND is a condition or formula¹³ of the tuple relational calculus. A formula is made up of predicate calculus atoms, which can be one of the following:

1. An atom of the form $R(t_i)$, where R is a relation name and t_i is a tuple variable. This atom identifies the range of the tuple variable t_i as the relation whose name is R . It evaluates to TRUE if t_i is a tuple in the relation R , and evaluates to FALSE otherwise.
2. An atom of the form $t_i.A \text{ op } t_j.B$, where op is one of the comparison operators in the set $\{=, <, \leq, >, \geq, \neq\}$, t_i and t_j are tuple variables, A is an attribute of the relation on which t_i ranges, and B is an attribute of the relation on which t_j ranges.
3. An atom of the form $t_i.A \text{ op } c$ or $c \text{ op } t_j.B$, where op is one of the comparison operators in the set $\{=, <, \leq, >, \geq, \neq\}$, t_i and t_j are tuple variables, A is an attribute of the relation on which t_i ranges, B is an attribute of the relation on which t_j ranges, and c is a constant value.

Each of the preceding atoms evaluates to either TRUE or FALSE for a specific combination of tuples; this is called the truth value of an atom. In general, a tuple variable t ranges over all possible tuples in the universe. For atoms of the form $R(t)$, if t is assigned to a tuple that is a member of the specified relation R , the atom is TRUE; otherwise, it is FALSE. In atoms of types 2 and 3, if the tuple variables are assigned to tuples such that the values of the specified attributes of the tuples satisfy the condition, then the atom is TRUE.

A formula (Boolean condition) is made up of one or more atoms connected via the logical operators AND, OR, and NOT and is defined recursively by Rules 1 and 2 as follows:

■ ■ Rule 1: Every atom is a formula.

■ ■ Rule 2: If $F1$ and $F2$ are formulas, then so are $(F1 \text{ AND } F2)$, $(F1 \text{ OR } F2)$, $\text{NOT } (F1)$, and $\text{NOT } (F2)$. The truth values of these formulas are derived from their component formulas $F1$ and $F2$ as follows:

13

Also called a well-formed formula, or WFF, in mathematical logic.

8.6 The Tuple Relational Calculus 301

- a. $(F1 \text{ AND } F2)$ is TRUE if both $F1$ and $F2$ are TRUE; otherwise, it is FALSE.
- b. $(F1 \text{ OR } F2)$ is FALSE if both $F1$ and $F2$ are FALSE; otherwise, it is TRUE.
- c. $\text{NOT } (F1)$ is TRUE if $F1$ is FALSE; it is FALSE if $F1$ is TRUE.
- d. $\text{NOT } (F2)$ is TRUE if $F2$ is FALSE; it is FALSE if $F2$ is TRUE.

8.6.3 The Existential and Universal Quantifiers

In addition, two special symbols called quantifiers can appear in formulas; these are the universal quantifier ($\forall$) and the existential quantifier ($\exists$). Truth values for formulas with quantifiers are described in Rules 3 and 4 below; first, however, we need to define the concepts of free and bound tuple variables in a formula. Informally, a tuple variable t is bound if it is quantified, meaning that it appears in an $(\exists t)$ or $(\forall t)$ clause; otherwise, it is free. Formally, we define a tuple variable in a formula as free or bound according to the following rules:

■ ■ An occurrence of a tuple variable in a formula F that is an atom is free in F .

■ ■ An occurrence of a tuple variable t is free or bound in a formula made up of logical connectives— $(F1 \text{ AND } F2)$, $(F1 \text{ OR } F2)$, $\text{NOT}(F1)$, and $\text{NOT}(F2)$ —depending on whether it is free or bound in $F1$ or $F2$ (if it occurs in either).

Notice that in a formula of the form $F = (F1 \text{ AND } F2)$ or $F = (F1 \text{ OR } F2)$, a tuple variable may be free in $F1$ and bound in $F2$, or vice versa; in this case, one occurrence of the tuple variable is bound and the other is free in F .

■ ■ All free occurrences of a tuple variable t in F are bound in a formula F' of the form $F' = (\exists t)(F)$ or $F' = (\forall t)(F)$. The tuple variable is bound to the quantifier specified in F' . For example, consider the following formulas:

$F1: d.Dname = \text{'Research'}$

$F2: (\exists t)(d.Dnumber = t.Dno)$

$F3: (\forall d)(d.Mgr_ssn = \text{'333445555'})$

The tuple variable d is free in both $F1$ and $F2$, whereas it is bound to the $(\forall)$ quantifier in $F3$. Variable t is bound to the $(\exists)$ quantifier in $F2$.

We can now give Rules 3 and 4 for the definition of a formula we started earlier:

■ ■ Rule 3: If F is a formula, then so is $(\exists t)(F)$, where t is a tuple variable. The formula $(\exists t)(F)$ is TRUE if the formula F evaluates to TRUE for some (at least one) tuple assigned to free occurrences of t in F ; otherwise, $(\exists t)(F)$ is FALSE.

■ ■ Rule 4: If F is a formula, then so is $(\forall t)(F)$, where t is a tuple variable. The formula $(\forall t)(F)$ is TRUE if the formula F evaluates to TRUE for every tuple (in the universe) assigned to free occurrences of t in F ; otherwise, $(\forall t)(F)$ is FALSE.

The $(\exists)$ quantifier is called an existential quantifier because a formula $(\exists t)(F)$ is TRUE if there exists some tuple that makes F TRUE. For the universal quantifier, $(\forall t)(F)$ is TRUE if every possible tuple that can be assigned to free occurrences of t in F is substituted for t , and F is TRUE for every such substitution. It is called the universal or for all quantifier because every tuple in the universe of tuples must make F TRUE to make the quantified formula TRUE.

302 Chapter 8 The Relational Algebra and Relational Calculus

8.6.4 Sample Queries in Tuple Relational Calculus

We will use some of the same queries from Section 8.5 to give a flavor of how the same queries are specified in relational algebra and in relational calculus. Notice that some queries are easier to specify in the relational algebra than in the relational calculus, and vice versa.

Query 1. List the name and address of all employees who work for the 'Research' department.

Q1: {t.Fname, t.Lname, t.Address | EMPLOYEE(t) AND ($\exists d$)(DEPARTMENT(d) AND d.Dname='Research' AND d.Dnumber=t.Dno)}

The only free tuple variables in a tuple relational calculus expression should be those that appear to the left of the bar (|). In Q1, t is the only free variable; it is then bound successively to each tuple. If a tuple satisfies the conditions specified after the bar in Q1, the attributes Fname, Lname, and Address are retrieved for each such tuple. The conditions EMPLOYEE(t) and DEPARTMENT(d) specify the range relations for t and d. The condition d.Dname = 'Research' is a selection condition and corresponds to a SELECT operation in the relational algebra, whereas the condition d.Dnumber = t.Dno is a join condition and is similar in purpose to the (INNER) JOIN operation (see Section 8.3).

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, birth date, and address.

Q2: {p.Pnumber, p.Dnum, m.Lname, m.Bdate, m.Address | PROJECT(p) AND EMPLOYEE(m) AND p.Plocation='Stafford' AND ($\exists d$)(DEPARTMENT(d) AND p.Dnum=d.Dnumber AND d.Mgr_ssn=m.Ssn)}

In Q2 there are two free tuple variables, p and m. Tuple variable d is bound to the existential quantifier. The query condition is evaluated for every combination of tuples assigned to p and m, and out of all possible combinations of tuples to which p and m are bound, only the combinations that satisfy the condition are selected. Several tuple variables in a query can range over the same relation. For example, to specify Q8—for each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor—we specify two tuple variables e and s that both range over the EMPLOYEE relation:

Q8: {e.Fname, e.Lname, s.Fname, s.Lname | EMPLOYEE(e) AND EMPLOYEE(s) AND e.Super_ssn=s.Ssn}

Query 3'. List the name of each employee who works on some project controlled by department number 5. This is a variation of Q3 in which all is changed to some. In this case we need two join conditions and two existential quantifiers.

Q0': {e.Lname, e.Fname | EMPLOYEE(e) AND ($\exists x$)($\exists w$)(PROJECT(x) AND WORKS_ON(w) AND x.Dnum=5 AND w.Essn=e.Ssn AND x.Pnumber=w.Pno)}

8.6 The Tuple Relational Calculus 303

Query 4. Make a list of project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as manager of the controlling department for the project.

Q4: { p.Pnumber | PROJECT(p) AND ($\exists e$)($\exists w$)(EMPLOYEE(e)

```

AND WORKS_ON(w) AND w.Pno=p.Pnumber
AND e.Lname='Smith' AND e.Ssn=w.Essn) )
OR
((∃m)(∃d)(EMPLOYEE(m) AND DEPARTMENT(d)
AND p.Dnum=d.Dnumber AND d.Mgr_ssn=m.Ssn
AND m.Lname='Smith'))))}

```

Compare this with the relational algebra version of this query in Section 8.5. The UNION operation in relational algebra can usually be substituted with an OR connective in relational calculus.

8.6.5 Notation for Query Graphs

In this section, we describe a notation that has been proposed to represent relational calculus queries that do not involve complex quantification in a graphical form.

These types of queries are known as select-project-join queries because they only involve these three relational algebra operations. The notation may be expanded to more general queries, but we do not discuss these extensions here. This graphical representation of a query is called a query graph. Figure 8.13 shows the query graph for Q2. Relations in the query are represented by relation nodes, which are displayed as single circles. Constant values, typically from the query selection conditions, are represented by constant nodes, which are displayed as double circles or ovals. Selection and join conditions are represented by the graph edges (the lines that connect the nodes), as shown in Figure 8.13. Finally, the attributes to be retrieved from each relation are displayed in square brackets above each relation.

The query graph representation does not indicate a particular order to specify which operations to perform first, and is hence a more neutral representation of a select-project-join query than the query tree representation (see Section 8.3.5), where the order of execution is implicitly specified. There is only a single query graph corresponding to each query. Although some query optimization techniques were based on query graphs, it is now generally accepted that query trees are preferable because,

[P.Pnumber,P.Dnum] [E.Lname,E.address,E.Bdate] Figure 8.13

Query graph for Q2.

P.Dnum=D.Dnumber D.Mgr_ssn=E.Ssn

P D E

P.Plocation='Stafford'

'Stafford'

304 Chapter 8 The Relational Algebra and Relational Calculus

in practice, the query optimizer needs to show the order of operations for query execution, which is not possible in query graphs.

In the next section we discuss the relationship between the universal and existential quantifiers and show how one can be transformed into the other.

8.6.6 Transforming the Universal and Existential Quantifiers

We now introduce some well-known transformations from mathematical logic that relate the universal and existential quantifiers. It is possible to transform a universal quantifier into an existential quantifier, and vice versa, to get an equivalent expression. One general transformation can be described informally as follows: Transform one type of quantifier into the other with negation (preceded by NOT); AND and OR replace one another; a negated formula becomes unnegated; and an unnegated formula becomes negated. Some special cases of this transformation can be stated as follows, where the $\equiv$ symbol stands for equivalent to:

$$(\forall x) (P(x)) \equiv \text{NOT } (\exists x) (\text{NOT } (P(x)))$$

$$(\exists x) (P(x)) \equiv \text{NOT } (\forall x) (\text{NOT } (P(x)))$$

$$(\forall x) (P(x) \text{ AND } Q(x)) \equiv \text{NOT } (\exists x) (\text{NOT } (P(x)) \text{ OR } \text{NOT } (Q(x)))$$

$$(\forall x) (P(x) \text{ OR } Q(x)) \equiv \text{NOT } (\exists x) (\text{NOT } (P(x)) \text{ AND } \text{NOT } (Q(x)))$$

$$(\exists x) (P(x)) \text{ OR } Q(x) \equiv \text{NOT } (\forall x) (\text{NOT } (P(x)) \text{ AND } \text{NOT } (Q(x)))$$

$$(\exists x) (P(x) \text{ AND } Q(x)) \equiv \text{NOT } (\forall x) (\text{NOT } (P(x)) \text{ OR } \text{NOT } (Q(x)))$$

Notice also that the following is TRUE, where the $\Rightarrow$ symbol stands for implies:

$$(\forall x)(P(x)) \Rightarrow (\exists x)(P(x))$$

$$\text{NOT } (\exists x)(P(x)) \Rightarrow \text{NOT } (\forall x)(P(x))$$

8.6.7 Using the Universal Quantifier in Queries

Whenever we use a universal quantifier, it is quite judicious to follow a few rules to ensure that our expression makes sense. We discuss these rules with respect to the query Q3.

Query 3. List the names of employees who work on all the projects controlled by department number 5. One way to specify this query is to use the universal quantifier as shown:

Q3: {e.Lname, e.Fname | EMPLOYEE(e) AND (($\forall x$)(NOT(PROJECT(x)) OR NOT (x.Dnum=5) OR (($\exists w$)(WORKS_ON(w) AND w.Essn=e.Ssn AND x.Pnumber=w.Pno))))}

We can break up Q3 into its basic components as follows:

Q3: {e.Lname, e.Fname | EMPLOYEE(e) AND F'}

F' = (($\forall x$)(NOT(PROJECT(x)) OR F1))

F1 = NOT(x.Dnum=5) OR F2

F2 = (($\exists w$)(WORKS_ON(w) AND w.Essn=e.Ssn AND x.Pnumber=w.Pno))

8.6 The Tuple Relational Calculus 305

We want to make sure that a selected employee e works on all the projects controlled by department 5, but the definition of universal quantifier says that to make the quantified formula TRUE, the inner formula must be TRUE for all tuples in the universe. The trick is to exclude from the universal quantification all tuples that we are not interested in by making the condition TRUE for all such tuples. This is necessary because a universally quantified tuple variable, such as x in Q3, must evaluate to TRUE for every possible tuple assigned to it to make the quantified

formula TRUE.

The first tuples to exclude (by making them evaluate automatically to TRUE) are those that are not in the relation R of interest. In Q3, using the expression $\text{NOT}(\text{PROJECT}(x))$ inside the universally quantified formula evaluates to TRUE all tuples x that are not in the PROJECT relation. Then we exclude the tuples we are not interested in from R itself. In Q3, using the expression $\text{NOT}(x.\text{Dnum}=5)$ evaluates to TRUE all tuples x that are in the PROJECT relation but are not controlled by department 5. Finally, we specify a condition F2 that must hold on all the remaining tuples in R. Hence, we can explain Q3 as follows:

1. For the formula $F' = (\forall x)(F)$ to be TRUE, we must have the formula F be TRUE for all tuples in the universe that can be assigned to x. However, in Q3 we are only interested in F being TRUE for all tuples of the PROJECT relation that are controlled by department 5. Hence, the formula F is of the form $(\text{NOT}(\text{PROJECT}(x)) \text{ OR } F1)$. The 'NOT (PROJECT(x)) OR ...' condition is TRUE for all tuples not in the PROJECT relation and has the effect of eliminating these tuples from consideration in the truth value of F1. For every tuple in the PROJECT relation, F1 must be TRUE if F' is to be TRUE.

2. Using the same line of reasoning, we do not want to consider tuples in the PROJECT relation that are not controlled by department number 5, since we are only interested in PROJECT tuples whose $\text{Dnum}=5$. Therefore, we can write:

IF $(x.\text{Dnum}=5)$ THEN F2

which is equivalent to

$(\text{NOT } (x.\text{Dnum}=5) \text{ OR } F2)$

3. Formula F1, hence, is of the form $\text{NOT}(x.\text{Dnum}=5) \text{ OR } F2$. In the context of Q3, this means that, for a tuple x in the PROJECT relation, either its $\text{Dnum} \neq 5$ or it must satisfy F2.

4. Finally, F2 gives the condition that we want to hold for a selected EMPLOYEE tuple: that the employee works on every PROJECT tuple that has not been excluded yet. Such employee tuples are selected by the query.

In English, Q3 gives the following condition for selecting an EMPLOYEE tuple e:

For every tuple x in the PROJECT relation with $x.\text{Dnum}=5$, there must exist a tuple w in WORKS_ON such that $w.\text{Essn}=e.\text{Ssn}$ and $w.\text{Pno}=x.\text{Pnumber}$. This is equivalent to saying that EMPLOYEE e works on every PROJECT x in DEPARTMENT number 5. (Whew!)

306 Chapter 8 The Relational Algebra and Relational Calculus

Using the general transformation from universal to existential quantifiers given in Section 8.6.6, we can rephrase the query in Q3 as shown in Q3A, which uses a negated existential quantifier instead of the universal quantifier:

Q3A: $\{e.\text{Lname}, e.\text{Fname} \mid \text{EMPLOYEE}(e) \text{ AND } (\text{NOT } (\exists x) (\text{PROJECT}(x) \text{ AND } (x.\text{Dnum}=5) \text{ and } (\text{NOT } (\exists w)(\text{WORKS_ON}(w) \text{ AND } w.\text{Essn}=e.\text{Ssn}$

AND x.Pnumber=w.Pno))))}

We now give some additional examples of queries that use quantifiers.

Query 6. List the names of employees who have no dependents.

Q6: {e.Fname, e.Lname | EMPLOYEE(e) AND (NOT (∃d)(DEPENDENT(d) AND e.Ssn=d.Essn))}

Using the general transformation rule, we can rephrase Q6 as follows:

Q6A: {e.Fname, e.Lname | EMPLOYEE(e) AND ((∀d)(NOT(DEPENDENT(d) OR NOT(e.Ssn=d.Essn)))}

Query 7. List the names of managers who have at least one dependent.

Q7: {e.Fname, e.Lname | EMPLOYEE(e) AND ((∃d)(∃p)(DEPARTMENT(d) AND DEPENDENT(p) AND e.Ssn=d.Mgr_ssn AND p.Essn=e.Ssn))}

This query is handled by interpreting managers who have at least one dependent as managers for whom there exists some dependent.

8.6.8 Safe Expressions

Whenever we use universal quantifiers, existential quantifiers, or negation of predicates in a calculus expression, we must make sure that the resulting expression makes sense. A safe expression in relational calculus is one that is guaranteed to yield a finite number of tuples as its result; otherwise, the expression is called unsafe.

For example, the expression

{t | NOT (EMPLOYEE(t))}

is unsafe because it yields all tuples in the universe that are not EMPLOYEE tuples, which are infinitely numerous. If we follow the rules for Q3 discussed earlier, we will get a safe expression when using universal quantifiers. We can define safe expressions more precisely by introducing the concept of the domain of a tuple relational calculus expression: This is the set of all values that either appear as constant values in the expression or exist in any tuple in the relations referenced in the expression. For example, the domain of {t | NOT(EMPLOYEE(t))} is the set of all attribute values appearing in some tuple of the EMPLOYEE relation (for any attribute). The domain of the expression Q3A would include all values appearing in EMPLOYEE, PROJECT, and WORKS_ON (unioned with the value 5 appearing in the query itself).

An expression is said to be safe if all values in its result are from the domain of the expression. Notice that the result of {t | NOT(EMPLOYEE(t))} is unsafe, since it will,

8.7 The Domain Relational Calculus 307

in general, include tuples (and hence values) from outside the EMPLOYEE relation; such values are not in the domain of the expression. All of our other examples are safe expressions.

8.7 The Domain Relational Calculus

There is another type of relational calculus called the domain relational calculus, or

simply domain calculus. Historically, while SQL (see Chapters 6 and 7), which was based on tuple relational calculus, was being developed by IBM Research at San Jose, California, another language called QBE (Query-By-Example), which is related to domain calculus, was being developed almost concurrently at the IBM T. J. Watson Research Center in Yorktown Heights, New York. The formal specification of the domain calculus was proposed after the development of the QBE language and system.

Domain calculus differs from tuple calculus in the type of variables used in formulas: Rather than having variables range over tuples, the variables range over single values from domains of attributes. To form a relation of degree n for a query result, we must have n of these domain variables—one for each attribute. An expression of the domain calculus is of the form

$$\{x_1, x_2, \dots, x_n \mid \text{COND}(x_1, x_2, \dots, x_n, x_{n+1}, x_{n+2}, \dots, x_{n+m})\}$$

where $x_1, x_2, \dots, x_n, x_{n+1}, x_{n+2}, \dots, x_{n+m}$ are domain variables that range over domains (of attributes), and COND is a condition or formula of the domain relational calculus.

A formula is made up of atoms. The atoms of a formula are slightly different from those for the tuple calculus and can be one of the following:

1. An atom of the form $R(x_1, x_2, \dots, x_j)$, where R is the name of a relation of degree j and each x_i , $1 \leq i \leq j$, is a domain variable. This atom states that a list of values of $\langle x_1, x_2, \dots, x_j \rangle$ must be a tuple in the relation whose name is R , where x_i is the value of the i th attribute value of the tuple. To make a domain calculus expression more concise, we can drop the commas in a list of variables; thus, we can write:

$$\{x_1, x_2, \dots, x_n \mid R(x_1 \ x_2 \ x_3) \text{ AND } \dots\}$$

instead of:

$$\{x_1, x_2, \dots, x_n \mid R(x_1, x_2, x_3) \text{ AND } \dots\}$$

2. An atom of the form $x_i \text{ op } x_j$, where op is one of the comparison operators in the set $\{=, <, \leq, >, \geq, \neq\}$, and x_i and x_j are domain variables.

3. An atom of the form $x_i \text{ op } c$ or $c \text{ op } x_j$, where op is one of the comparison operators in the set $\{=, <, \leq, >, \geq, \neq\}$, x_i and x_j are domain variables, and c is a constant value.

As in tuple calculus, atoms evaluate to either TRUE or FALSE for a specific set of values, called the truth values of the atoms. In case 1, if the domain variables are

308 Chapter 8 The Relational Algebra and Relational Calculus

assigned values corresponding to a tuple of the specified relation R , then the atom is TRUE. In cases 2 and 3, if the domain variables are assigned values that satisfy the condition, then the atom is TRUE.

In a similar way to the tuple relational calculus, formulas are made up of atoms, variables, and quantifiers, so we will not repeat the specifications for formulas here. Some examples of queries specified in the domain calculus follow. We will use low-

ercase letters l, m, n, ... , x, y, z for domain variables.

Query 0. List the birth date and address of the employee whose name is 'John B. Smith'.

Q0: {u, v | (∃q) (∃r) (∃s) (∃t) (∃w) (∃x) (∃y) (∃z)

(EMPLOYEE(qrstuvwxyz) AND q='John' AND r='B' AND s='Smith'))}

We need ten variables for the EMPLOYEE relation, one to range over each of the domains of attributes of EMPLOYEE in order. Of the ten variables q, r, s, ... , z, only u and v are free, because they appear to the left of the bar and hence should not be bound to a quantifier. We first specify the requested attributes, Bdate and Address, by the free domain variables u for BDATE and v for ADDRESS. Then we specify the condition for selecting a tuple following the bar (|)—namely, that the sequence of values assigned to the variables qrstuvwxyz be a tuple of the EMPLOYEE relation and that the values for q (Fname), r (Minit), and s (Lname) be equal to 'John', 'B', and 'Smith', respectively. For convenience, we will quantify only those variables actually appearing in a condition (these would be q, r, and s in Q0) in the rest of our examples.¹⁴

An alternative shorthand notation, used in QBE, for writing this query is to assign the constants 'John', 'B', and 'Smith' directly as shown in Q0A. Here, all variables not appearing to the left of the bar are implicitly existentially quantified:¹⁵

Q0A: {u, v | EMPLOYEE('John', 'B', 'Smith', t, u, v, w, x, y, z)}

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

Q1: {q, s, v | (∃z) (∃l) (∃m) (EMPLOYEE(qrstuvwxyz) AND DEPARTMENT(lmno) AND l='Research' AND m=z)}

A condition relating two domain variables that range over attributes from two relations, such as m = z in Q1, is a join condition, whereas a condition that relates a domain variable to a constant, such as l = 'Research', is a selection condition.

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, birth date, and address.

14

Quantifying only the domain variables actually used in conditions and specifying a predicate such as

EMPLOYEE(qrstuvwxyz) without separating domain variables with commas is an abbreviated notation

used for convenience; it is not the correct formal notation.

15

Again, this is not a formally accurate notation.

8.8 Summary 309

Q2: {i, k, s, u, v | (∃j)(∃m)(∃n)(∃t)(PROJECT(hijk) AND EMPLOYEE(qrstuvwxyz) AND DEPARTMENT(lmno) AND k=m AND

$n=t \text{ AND } j='Stafford')\}$

Query 6. List the names of employees who have no dependents.

Q6: $\{q, s \mid (\exists t)(\text{EMPLOYEE}(qrstuvwxyz) \text{ AND } (\text{NOT}(\exists l)(\text{DEPENDENT}(lmnop) \text{ AND } t=l)))\}$

Q6 can be restated using universal quantifiers instead of the existential quantifiers, as shown in Q6A:

Q6A: $\{q, s \mid (\exists t)(\text{EMPLOYEE}(qrstuvwxyz) \text{ AND } ((\forall l)(\text{NOT}(\text{DEPENDENT}(lmnop)) \text{ OR } \text{NOT}(t=l))))\}$

Query 7. List the names of managers who have at least one dependent.

Q7: $\{s, q \mid (\exists t)(\exists j)(\exists l)(\text{EMPLOYEE}(qrstuvwxyz) \text{ AND } \text{DEPARTMENT}(hijk) \text{ AND } \text{DEPENDENT}(lmnop) \text{ AND } t=j \text{ AND } l=t)\}$

As we mentioned earlier, it can be shown that any query that can be expressed in the basic relational algebra can also be expressed in the domain or tuple relational calculus. Also, any safe expression in the domain or tuple relational calculus can be expressed in the basic relational algebra.

The QBE language was based on the domain relational calculus, although this was realized later, after the domain calculus was formalized. QBE was one of the first graphical query languages with minimum syntax developed for database systems. It was developed at IBM Research and is available as an IBM commercial product as part of the Query Management Facility (QMF) interface option to DB2. The basic ideas used in QBE have been applied in several other commercial products. Because of its important place in the history of relational languages, we have included an overview of QBE in Appendix C.

8.8 Summary

In this chapter we presented two formal languages for the relational model of data. They are used to manipulate relations and produce new relations as answers to queries. We discussed the relational algebra and its operations, which are used to specify a sequence of operations to specify a query. Then we introduced two types of relational calculi called tuple calculus and domain calculus.

In Sections 8.1 through 8.3, we introduced the basic relational algebra operations and illustrated the types of queries for which each is used. First, we discussed the unary relational operators SELECT and PROJECT, as well as the RENAME operation. Then, we discussed binary set theoretic operations requiring that relations on which they are applied be union (or type) compatible; these include UNION, INTERSECTION, and SET DIFFERENCE. The CARTESIAN PRODUCT operation is a set operation that can be used to combine tuples from two relations, producing all possible combinations. It is rarely used in practice; however, we showed how

310 Chapter 8 The Relational Algebra and Relational Calculus

CARTESIAN PRODUCT followed by SELECT can be used to define matching tuples from two relations and leads to the JOIN operation. Different JOIN operations called

THETA JOIN, EQUIJOIN, and NATURAL JOIN were introduced. Query trees were introduced as a graphical representation of relational algebra queries, which can also be used as the basis for internal data structures that the DBMS can use to represent a query. We discussed some important types of queries that cannot be stated with the basic relational algebra operations but are important for practical situations. We introduced GENERALIZED PROJECTION to use functions of attributes in the projection list and the AGGREGATE FUNCTION operation to deal with aggregate types of statistical requests that summarize the information in the tables. We discussed recursive queries, for which there is no direct support in the algebra but which can be handled in a step-by-step approach, as we demonstrated. Then we presented the OUTER JOIN and OUTER UNION operations, which extend JOIN and UNION and allow all information in source relations to be preserved in the result.

The last two sections described the basic concepts behind relational calculus, which is based on the branch of mathematical logic called predicate calculus. There are two types of relational calculi: (1) the tuple relational calculus, which uses tuple variables that range over tuples (rows) of relations, and (2) the domain relational calculus, which uses domain variables that range over domains (columns of relations). In relational calculus, a query is specified in a single declarative statement, without specifying any order or method for retrieving the query result. Hence, relational calculus is often considered to be a higher-level declarative language than the relational algebra, because a relational calculus expression states what we want to retrieve regardless of how the query may be executed.

We introduced query graphs as an internal representation for queries in relational calculus. We also discussed the existential quantifier ($\exists$) and the universal quantifier ($\forall$). We discussed the problem of specifying safe queries whose results are finite. We also discussed rules for transforming universal into existential quantifiers, and vice versa. It is the quantifiers that give expressive power to the relational calculus, making it equivalent to the basic relational algebra. There is no analog to grouping and aggregation functions in basic relational calculus, although some extensions have been suggested.

Review Questions

- 8.1. List the operations of relational algebra and the purpose of each.
- 8.2. What is union compatibility? Why do the UNION, INTERSECTION, and DIFFERENCE operations require that the relations on which they are applied be union compatible?
- 8.3. Discuss some types of queries for which renaming of attributes is necessary in order to specify the query unambiguously.
- 8.4. Discuss the various types of inner join operations. Why is theta join required?

Exercises 311

- 8.5. What role does the concept of foreign key play when specifying the most common types of meaningful join operations?

- 8.6. What is the FUNCTION operation? For what is it used?
- 8.7. How are the OUTER JOIN operations different from the INNER JOIN operations? How is the OUTER UNION operation different from UNION?
- 8.8. In what sense does relational calculus differ from relational algebra, and in what sense are they similar?
- 8.9. How does tuple relational calculus differ from domain relational calculus?
- 8.10. Discuss the meanings of the existential quantifier ($\exists$) and the universal quantifier ($\forall$).
- 8.11. Define the following terms with respect to the tuple calculus: tuple variable, range relation, atom, formula, and expression.
- 8.12. Define the following terms with respect to the domain calculus: domain variable, range relation, atom, formula, and expression.
- 8.13. What is meant by a safe expression in relational calculus?
- 8.14. When is a query language called relationally complete?

Exercises

- 8.15. Show the result of each of the sample queries in Section 8.5 as it would apply to the database state in Figure 5.6.
- 8.16. Specify the following queries on the COMPANY relational database schema shown in Figure 5.5 using the relational operators discussed in this chapter. Also show the result of each query as it would apply to the database state in Figure 5.6.
- Retrieve the names of all employees in department 5 who work more than 10 hours per week on the ProductX project.
 - List the names of all employees who have a dependent with the same first name as themselves.
 - Find the names of all employees who are directly supervised by 'Franklin Wong'.
 - For each project, list the project name and the total hours per week (by all employees) spent on that project.
 - Retrieve the names of all employees who work on every project.
 - Retrieve the names of all employees who do not work on any project.
 - For each department, retrieve the department name and the average salary of all employees working in that department.
 - Retrieve the average salary of all female employees.
- 312 Chapter 8 The Relational Algebra and Relational Calculus
- Find the names and addresses of all employees who work on at least one project located in Houston but whose department has no location in Houston.
 - List the last names of all department managers who have no dependents.
- 8.17. Consider the AIRLINE relational database schema shown in Figure 5.8, which

was described in Exercise 5.12. Specify the following queries in relational algebra:

- For each flight, list the flight number, airline, date, departure airport and arrival airport along with the number of available seats, for each leg.
- For all the airplane types available in the database, list the airplane type, name of the company, maximum seats available and the airports (or airport codes) where these planes can land.
- List the name, seat number and phone numbers of all the customers of all flights or flight legs that departed from Houston Intercontinental Airport (airport code 'iah') and arrived in Los Angeles International Airport (airport code 'lax') on '2016-03-16'.
- List fare information for all the flights run by the airline 'Delta Airlines'.
- Retrieve the number of available seats on all flights run by Delta Airline, on '2016-04-09'.

8.18. Consider the LIBRARY relational database schema shown in Figure 8.14, which is used to keep track of books, borrowers, and book loans. Referential integrity constraints are shown as directed arcs in Figure 8.14, as in the notation of Figure 5.7. Write down relational expressions for the following queries:

- How many copies of the book titled 'Gone with the Wind' are owned by the library branch whose name is 'Georgetown'?
- How many copies of the book titled 'Gone with the Wind' are owned by each library branch?
- Retrieve the card number, names and phone numbers of all borrowers who have a due date on '20-03-16'.
- For each book that is loaned out from the 'Georgetown' branch retrieve the book title, the author's name, and number of copies remained.
- For all the authors, retrieve name, book_id, title of the book and name of publisher of the books written by them.
- Retrieve the names, addresses, phone numbers of all the borrowers who have borrowed the book 'Gone with the Wind'.
- For each book borrowed by a borrower Simon Hill, retrieve the title and the number of copies owned by the library branch whose name is 'Georgetown'.

8.19. Specify the following queries in relational algebra on the database schema given in Exercise 5.14:

Exercises 313

BOOK

Book_id Title Publisher_name

BOOK_AUTHORS

Book_id Author_name

PUBLISHER

Name Address Phone

BOOK_COPIES

Book_id Branch_id No_of_copies

BOOK_LOANS

Book_id Branch_id Card_no Date_out Due_date

LIBRARY_BRANCH

Branch_id Branch_name Address

BORROWER Figure 8.14

Card_no Name Address Phone A relational database
schema for a LIBRARY
database.

- List all information about the cars that were serviced at the garage number '001' on today's date.
- List the RENTAL information about the rental services where the DRIVER named Jose Lopez had provided service. Produce a listing: Rental#, Car#, Start_time, End_time and Amount_received.
- Produce a listing Cname, No_of_rentals, Tot_rental_amt, where the middle column is the total number of rental services taken by the customer and the last column is the total rental amount received for that customer.
- List the rentals where the duration of service was more than 10 hours.
- List the driver number, names and addresses of all those drivers who are yet to be employed.

8.20. Specify the following queries in relational algebra on the database schema given in Exercise 5.15:

- Give the details (all attributes of VISIT relation) for visits that exceeded \$5,000 in expenses.

314 Chapter 8 The Relational Algebra and Relational Calculus

- Print the DSSns of doctors who visited the hospital 'Heritage'.
- Print the total visit expenses incurred by the doctor with DSSN = '234-56-7890'.

8.21. Specify the following queries in relational algebra on the database schema given in Exercise 5.16:

- Write down the ISBN number, title and author of all the books borrowed by a user named John Smith.
- Produce a list that would show details (include Book_isbn, Book_title, Publisher, Author) of all the books in the library supplied by a supplier with SSsn '234-57-8901'.
- List all the books that have been borrowed on the date '2016-03-22' and published by the Pearson Publication.

8.22. Consider the two tables T1 and T2 shown in Figure 8.15. Show the results of the following operations:

- $T1 \bowtie_{T1.P = T2.A} T2$
- $T1 \bowtie_{T1.Q = T2.B} T2$
- $T1 \bowtie_{T1.P = T2.A} T2$
- $T1 \bowtie_{T1.Q = T2.B} T2$
- $T1 \cup T2$

8.23. Specify the following queries in relational algebra on the database schema in Exercise 5.17:

- For the agent named 'Jane Doe', list the following information for all the apartments she sold: Apartment#, Model, Price_per_sqft.
- List the Apartment# and Model of apartments that have no options.
- Consider the NATURAL JOIN operation between AGENT and BOOKING.

What is the meaning of a left outer join for these tables (do not change the order of relations)? Explain with an example.

- Write a query in relational algebra involving selection and one set operation and say in words what the query does.

8.24. Specify queries a, b, c, e, f, i, and j of Exercise 8.16 in both tuple and domain relational calculus.

8.25. Specify queries a, b, c, and d of Exercise 8.17 in both tuple and domain relational calculus.

Figure 8.15 TABLE T1 TABLE T2

A database state for the

P Q R A B C

relations T1 and T2.

10 a 5 10 b 6

15 b 8 25 c 3

25 a 6 10 b 5

Exercises 315

8.26. Specify queries c, e, and f of Exercise 8.18 in both tuple and domain relational calculus.

- Retrieve the names and phone numbers of all borrowers who has a due date on '20-03-16'.
- For all the authors, retrieve name, book_id, title of the book and name of publisher of the books written by them.
- Retrieve the names, addresses, phone numbers of all the borrowers who have borrowed the book 'Gone with the Wind'.

8.27. In a tuple relational calculus query with n tuple variables, what would be the typical minimum number of join conditions? Why? What is the effect of having a smaller number of join conditions?

8.28. Rewrite the domain relational calculus queries that followed Q0 in Sec-

tion 8.7 in the style of the abbreviated notation of Q0A, where the objective is to minimize the number of domain variables by writing constants in place of variables wherever possible.

8.29. Consider this query: Retrieve the Ssns of employees who work on at least those projects on which the employee with Ssn=123456789 works. This may be stated as (FORALL x) (IF P THEN Q), where

■ ■ x is a tuple variable that ranges over the PROJECT relation.

■ ■ P $\equiv$ employee with Ssn=123456789 works on project x.

■ ■ Q $\equiv$ employee e works on project x.

Express the query in tuple relational calculus, using the rules

■ ■ $(\forall x)(P(x)) \equiv \text{NOT}(\exists x)(\text{NOT}(P(x)))$.

■ ■ $(\text{IF } P \text{ THEN } Q) \equiv (\text{NOT}(P) \text{ OR } Q)$.

8.30. Show how you can specify the following relational algebra operations in both tuple and domain relational calculus.

a. $\sigma_A=C(R(A, B, C))$ e. $R(A, B, C) \cap S(A, B, C)$

b. $\pi_{\langle A, B \rangle}(R(A, B, C))$ f. $R(A, B, C) = S(A, B, C)$

c. $R(A, B, C) * S(C, D, E)$ g. $R(A, B, C) \times S(D, E, F)$

d. $R(A, B, C) \cup S(A, B, C)$ h. $R(A, B) \div S(A)$

8.31. Suggest extensions to the relational calculus so that it may express the following types of operations that were discussed in Section 8.4: (a) aggregate functions and grouping; (b) OUTER JOIN operations; (c) recursive closure queries.

8.32. A nested query is a query within a query. More specifically, a nested query is a parenthesized query whose result can be used as a value in a number of places, such as instead of a relation. Specify the following queries on the database specified in Figure 5.5 using the concept of nested queries and the relational operators discussed in this chapter. Also show the result of each query as it would apply to the database state in Figure 5.6.

a. List the names of all employees who work in the department that has the employee with the highest salary among all employees.

316 Chapter 8 The Relational Algebra and Relational Calculus

b. List the names of all employees whose supervisor's supervisor has '888665555' for Ssn.

c. List the names of employees who make at least \$10,000 more than the employee who is paid the least in the company.

8.33. State whether the following conclusions are true or false:

a. $\text{NOT}(P(x) \text{ OR } Q(x)) \rightarrow (\text{NOT}(P(x)) \text{ AND } (\text{NOT}(Q(x))))$

b. $\text{NOT}(\exists x)(P(x)) \rightarrow \forall x(\text{NOT}(P(x)))$

c. $(\exists x)(P(x)) \rightarrow \forall x((P(x))$

Laboratory Exercises

8.34. Specify and execute the following queries in relational algebra (RA) using

the RA interpreter on the COMPANY database schema in Figure 5.5.

- List the names of all employees in department 5 who work more than 10 hours per week on the ProductX project.
- List the names of all employees who have a dependent with the same first name as themselves.
- List the names of employees who are directly supervised by Franklin Wong.
- List the names of employees who work on every project.
- List the names of employees who do not work on any project.
- List the names and addresses of employees who work on at least one project located in Houston but whose department has no location in Houston.
- List the names of department managers who have no dependents.

8.35. Consider the following MAILORDER relational schema describing the data for a mail order company.

PARTS(Pno, Pname, Qoh, Price, Olevel)

CUSTOMERS(Cno, Cname, Street, Zip, Phone)

EMPLOYEES(Eno, Ename, Zip, Hdate)

ZIP_CODES(Zip, City)

ORDERS(Ono, Cno, Eno, Received, Shipped)

ODETAILS(Ono, Pno, Qty)

Qoh stands for quantity on hand: the other attribute names are self-explanatory. Specify and execute the following queries using the RA interpreter on the MAILORDER database schema.

- Retrieve the names of parts that cost less than \$20.00.
- Retrieve the names and cities of employees who have taken orders for parts costing more than \$50.00.
- Retrieve the pairs of customer number values of customers who live in the same ZIP Code.

Laboratory Exercises 317

- Retrieve the names of customers who have ordered parts from employees living in Wichita.
- Retrieve the names of customers who have ordered parts costing less than \$20.00.
- Retrieve the names of customers who have not placed an order.
- Retrieve the names of customers who have placed exactly two orders.

8.36. Consider the following GRADEBOOK relational schema describing the data for a grade book of a particular instructor. (Note: The attributes A, B, C, and D of COURSES store grade cutoffs.)

CATALOG(Cno, Ctitle)

STUDENTS(Sid, Fname, Lname, Minit)

COURSES(Term, Sec_no, Cno, A, B, C, D)

ENROLLS(Sid, Term, Sec_no)

Specify and execute the following queries using the RA interpreter on the GRADEBOOK database schema.

- a. Retrieve the names of students enrolled in the Automata class during the fall 2009 term.
- b. Retrieve the Sid values of students who have enrolled in CSc226 and CSc227.
- c. Retrieve the Sid values of students who have enrolled in CSc226 or CSc227.
- d. Retrieve the names of students who have not enrolled in any class.
- e. Retrieve the names of students who have enrolled in all courses in the CATALOG table.

8.37. Consider a database that consists of the following relations.

SUPPLIER(Sno, Sname)

PART(Pno, Pname)

PROJECT(Jno, Jname)

SUPPLY(Sno, Pno, Jno)

The database records information about suppliers, parts, and projects and includes a ternary relationship between suppliers, parts, and projects. This relationship is a many-many-many relationship. Specify and execute the following queries using the RA interpreter.

- a. Retrieve the part numbers that are supplied to exactly two projects.
- b. Retrieve the names of suppliers who supply more than two parts to project 'J1'.
- c. Retrieve the part numbers that are supplied by every supplier.
- d. Retrieve the project names that are supplied by supplier 'S1' only.
- e. Retrieve the names of suppliers who supply at least two different parts each to at least two different projects.

318 Chapter 8 The Relational Algebra and Relational Calculus

8.38. Specify and execute the following queries for the database in Exercise 5.16 using the RA interpreter.

- a. Retrieve the names of students who have enrolled in a course that uses a textbook published by Addison-Wesley-Longman.
- b. Retrieve the names of courses in which the textbook has been changed at least once.
- c. Retrieve the names of departments that adopt textbooks published by Addison-Wesley only.
- d. Retrieve the names of departments that adopt textbooks written by Navathe and published by Addison-Wesley.
- e. Retrieve the names of students who have never used a book (in a course) written by Navathe and published by Addison-Wesley.

8.39. Repeat Laboratory Exercises 8.34 through 8.38 in domain relational calculus (DRC) by using the DRC interpreter.

Selected Bibliography

Codd (1970) defined the basic relational algebra. Date (1983a) discusses outer joins. Work on extending relational operations is discussed by Carlis (1986) and Ozsoyoglu et al. (1985). Cammarata et al. (1989) extends the relational model integrity constraints and joins.

Codd (1971) introduced the language Alpha, which is based on concepts of tuple relational calculus. Alpha also includes the notion of aggregate functions, which goes beyond relational calculus. The original formal definition of relational calculus was given by Codd (1972), which also provided an algorithm that transforms any tuple relational calculus expression to relational algebra. The QUEL (Stonebraker et al., 1976) is based on tuple relational calculus, with implicit existential quantifiers, but no universal quantifiers, and was implemented in the INGRES system as a commercially available language. Codd defined relational completeness of a query language to mean at least as powerful as relational calculus. Ullman (1988) describes a formal proof of the equivalence of relational algebra with the safe expressions of tuple and domain relational calculus. Abiteboul et al. (1995) and Atzeni and deAntonellis (1993) give a detailed treatment of formal relational languages.

Although ideas of domain relational calculus were initially proposed in the QBE language (Zloof, 1975), the concept was formally defined by Lacroix and Pirotte (1977a). The experimental version of the Query-By-Example system is described in Zloof (1975). The ILL (Lacroix & Pirotte, 1977b) is based on domain relational calculus. Whang et al. (1990) extends QBE with universal quantifiers. Visual query languages, of which QBE is an example, are being proposed as a means of querying databases; conferences such as the Visual Database Systems Working Conference (e.g., Arisawa & Catarci (2000) or Zhou & Pu (2002)) present a number of proposals for such languages.

chapter 9

Relational Database

Design by ER- and

EER-to-Relational Mapping

This chapter discusses how to design a relational database schema based on a conceptual schema design. Figure 3.1 presented a high-level view of the database design process. In this chapter we focus on the logical database design step of database design, which is also known as data model mapping. We present the procedures to create a relational schema from an entity-relationship (ER) or an enhanced ER (EER) schema. Our discussion relates the constructs of the ER and EER models, presented in Chapters 3 and 4, to the constructs of the relational model, presented in Chapters 5 through 8. Many computer-aided software engineering (CASE) tools are based on

the ER or EER models, or other similar models, as we have discussed in Chapters 3 and 4. Many tools use ER or EER diagrams or variations to develop the schema graphically and collect information about the data types and constraints, then convert the ER/EER schema automatically into a relational database schema in the DDL of a specific relational DBMS. The design tools employ algorithms similar to the ones presented in this chapter.

We outline a seven-step algorithm in Section 9.1 to convert the basic ER model constructs—entity types (strong and weak), binary relationships (with various structural constraints), n-ary relationships, and attributes (simple, composite, and multivalued)—into relations. Then, in Section 9.2, we continue the mapping algorithm by describing how to map EER model constructs—specialization/generalization and union types (categories)—into relations. Section 9.3 summarizes the chapter.

319

320 Chapter 9 Relational Database Design by ER- and EER-to-Relational Mapping

9.1 Relational Database Design Using
ER-to-Relational Mapping

9.1.1 ER-to-Relational Mapping Algorithm

In this section we describe the steps of an algorithm for ER-to-relational mapping. We use the COMPANY database example to illustrate the mapping procedure. The COMPANY ER schema is shown again in Figure 9.1, and the corresponding COMPANY relational database schema is shown in Figure 9.2 to illustrate the

Figure 9.1

The ER conceptual schema diagram for the COMPANY database.

Fname Minit Lname

Bdate Name Address Salary

Ssn Sex Locations

N 1

WORKS_FOR

Name Number

EMPLOYEE Start_date Number_of_employees DEPARTMENT

1

1 1

MANAGES

CONTROLS

Hours

N

M N

WORKS_ON PROJECT

Supervisor Supervisee 1

1

SUPERVISION N Name

Location

DEPENDENTS_OF Number

N

DEPENDENT

Name Sex Birth_date Relationship

9.1 Relational Database Design Using ER-to-Relational Mapping 321

EMPLOYEE

Fname Minit Lname Ssn Bdate Address Sex Salary Super_ssn Dno

DEPARTMENT

Dname Dnumber Mgr_ssn Mgr_start_date

DEPT_LOCATIONS

Dnumber Dlocation

PROJECT

Pname Pnumber Plocation Dnum

WORKS_ON

Essn Pno Hours

Figure 9.2

Result of mapping the

DEPENDENT

COMPANY ER schema

Essn Dependent_name Sex Bdate Relationship into a relational database schema.

mapping steps. We assume that the mapping will create tables with simple single-valued attributes. The relational model constraints defined in Chapter 5, which include primary keys, unique keys (if any), and referential integrity constraints on the relations, will also be specified in the mapping results.

Step 1: Mapping of Regular Entity Types. For each regular (strong) entity type *E* in the ER schema, create a relation *R* that includes all the simple attributes of *E*. Include only the simple component attributes of a composite attribute. Choose one of the key attributes of *E* as the primary key for *R*. If the chosen key of *E* is a composite, then the set of simple attributes that form it will together form the primary key of *R*.

If multiple keys were identified for *E* during the conceptual design, the information describing the attributes that form each additional key is kept in order to specify additional (unique) keys of relation *R*. Knowledge about keys is also kept for indexing purposes and other types of analyses.

In our example, we create the relations EMPLOYEE, DEPARTMENT, and PROJECT in Figure 9.2 to correspond to the regular entity types EMPLOYEE, DEPARTMENT, and PROJECT from Figure 9.1. The foreign key and relationship attributes, if any, are not included yet; they will be added during subsequent steps. These include the attributes Super_ssn and Dno of EMPLOYEE, Mgr_ssn and Mgr_start_date of DEPARTMENT, and Dnum of PROJECT. In our example, we choose Ssn, Dnumber, and Pnumber as primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT, respectively. Knowledge that Dname of DEPARTMENT and Pname of PROJECT are unique keys is kept for possible use later in the design.

The relations that are created from the mapping of entity types are sometimes called entity relations because each tuple represents an entity instance. The result after this mapping step is shown in Figure 9.3(a).

Step 2: Mapping of Weak Entity Types. For each weak entity type *W* in the ER schema with owner entity type *E*, create a relation *R* and include all simple attributes (or simple components of composite attributes) of *W* as attributes of *R*. In addition, include as foreign key attributes of *R*, the primary key attribute(s) of the relation(s) that correspond to the owner entity type(s); this takes care of mapping the identifying relationship type of *W*. The primary key of *R* is the combination of the primary key(s) of the owner(s) and the partial key of the weak entity type *W*, if any. If there is a weak entity type *E2* whose owner is also a weak entity type *E1*, then *E1* should be mapped before *E2* to determine its primary key first.

In our example, we create the relation DEPENDENT in this step to correspond to the weak entity type DEPENDENT (see Figure 9.3(b)). We include the primary key Ssn of the EMPLOYEE relation—which corresponds to the owner entity type—as a foreign key attribute of DEPENDENT; we rename it Essn, although this is not

Figure 9.3 (a) EMPLOYEE

Illustration of some

Fname Minit Lname Ssn Bdate Address Sex Salary

mapping steps.

(a) Entity relations

DEPARTMENT

after step 1.

(b) Additional weak entity Dname Dnumber
relation after step 2.

(c) Relationship relations PROJECT

after step 5. Pname Pnumber Plocation

(d) Relation representing

multivalued attribute

after step 6. (b) DEPENDENT

Essn Dependent_name Sex Bdate Relationship

(c) WORKS_ON

Essn Pno Hours

(d) DEPT_LOCATIONS

Dnumber Dlocation

9.1 Relational Database Design Using ER-to-Relational Mapping 323

necessary. The primary key of the DEPENDENT relation is the combination {Essn, Dependent_name}, because Dependent_name (also renamed from Name in Figure 9.1) is the partial key of DEPENDENT.

It is common to choose the propagate (CASCADE) option for the referential triggered action (see Section 6.2) on the foreign key in the relation corresponding to the weak entity type, since a weak entity has an existence dependency on its owner entity. This can be used for both ON UPDATE and ON DELETE.

Step 3: Mapping of Binary 1:1 Relationship Types. For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R. There are three possible approaches: (1) the foreign key approach, (2) the merged relationship approach, and (3) the cross-reference or relationship relation approach. The first approach is the most useful and should be followed unless special conditions exist, as we discuss below.

1. Foreign key approach: Choose one of the relations—S, say—and include as a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S. Include all the simple attributes (or simple components of composite attributes) of the 1:1 relationship type R as attributes of S.

In our example, we map the 1:1 relationship type MANAGES from Figure 9.1 by choosing the participating entity type DEPARTMENT to serve in the role of S because its participation in the MANAGES relationship type is total (every department has a manager). We include the primary key of the EMPLOYEE relation as foreign key in the DEPARTMENT relation and rename it to Mgr_ssn. We also include the simple attribute Start_date of the MANAGES relationship type in the DEPARTMENT relation and rename it Mgr_start_date (see Figure 9.2).

Note that it is possible to include the primary key of S as a foreign key in T instead. In our example, this amounts to having a foreign key attribute, say Department_managed in the EMPLOYEE relation, but it will have a NULL value for employee tuples who do not manage a department. This would be a bad choice, because if only 2% of employees manage a department, then 98% of the foreign keys would be NULL in this case. Another possibility is to have foreign keys in both relations S and T redundantly, but this creates redundancy and incurs a penalty for consistency maintenance.

2. Merged relation approach: An alternative mapping of a 1:1 relationship type is to merge the two entity types and the relationship into a single relation. This is possible when both participations are total, as this would indicate that the two tables will have the exact same number of tuples at all times.

3. Cross-reference or relationship relation approach: The third option is to set up a third relation *R* for the purpose of cross-referencing the primary keys of the two relations *S* and *T* representing the entity types. As we will see, this approach is required for binary *M:N* relationships. The relation *R* is called a relationship relation (or sometimes a lookup table), because each

324 Chapter 9 Relational Database Design by ER- and EER-to-Relational Mapping

tuple in *R* represents a relationship instance that relates one tuple from *S* with one tuple from *T*. The relation *R* will include the primary key attributes of *S* and *T* as foreign keys to *S* and *T*. The primary key of *R* will be one of the two foreign keys, and the other foreign key will be a unique key of *R*. The drawback is having an extra relation, and requiring extra join operations when combining related tuples from the tables.

Step 4: Mapping of Binary 1:N Relationship Types. There are two possible approaches: (1) the foreign key approach and (2) the cross-reference or relationship relation approach. The first approach is generally preferred as it reduces the number of tables.

1. The foreign key approach: For each regular binary 1:N relationship type *R*, identify the relation *S* that represents the participating entity type at the *N*-side of the relationship type. Include as foreign key in *S* the primary key of the relation *T* that represents the other entity type participating in *R*; we do this because each entity instance on the *N*-side is related to at most one entity instance on the 1-side of the relationship type. Include any simple attributes (or simple components of composite attributes) of the 1:N relationship type as attributes of *S*.

To apply this approach to our example, we map the 1:N relationship types *WORKS_FOR*, *CONTROLS*, and *SUPERVISION* from Figure 9.1. For *WORKS_FOR* we include the primary key *Dnumber* of the *DEPARTMENT* relation as foreign key in the *EMPLOYEE* relation and call it *Dno*. For *SUPERVISION* we include the primary key of the *EMPLOYEE* relation as foreign key in the *EMPLOYEE* relation itself—because the relationship is recursive—and call it *Super_ssn*. The *CONTROLS* relationship is mapped to the foreign key attribute *Dnum* of *PROJECT*, which references the primary key *Dnumber* of the *DEPARTMENT* relation. These foreign keys are shown in Figure 9.2.

2. The relationship relation approach: An alternative approach is to use the relationship relation (cross-reference) option as in the third option for binary 1:1 relationships. We create a separate relation *R* whose attributes are the primary keys of *S* and *T*, which will also be foreign keys to *S* and *T*. The

primary key of R is the same as the primary key of S. This option can be used if few tuples in S participate in the relationship to avoid excessive NULL values in the foreign key.

Step 5: Mapping of Binary M:N Relationship Types. In the traditional relational model with no multivalued attributes, the only option for M:N relationships is the relationship relation (cross-reference) option. For each binary M:N relationship type R, create a new relation S to represent R. Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; their combination will form the primary key of S. Also include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S. Notice that we cannot represent an M:N relationship type by a single foreign key attribute in one of the participating relations (as we did for 9.1 Relational Database Design Using ER-to-Relational Mapping 325

1:1 or 1:N relationship types) because of the M:N cardinality ratio; we must create a separate relationship relation S.

In our example, we map the M:N relationship type WORKS_ON from Figure 9.1 by creating the relation WORKS_ON in Figure 9.2. We include the primary keys of the PROJECT and EMPLOYEE relations as foreign keys in WORKS_ON and rename them Pno and Essn, respectively (renaming is not required; it is a design choice). We also include an attribute Hours in WORKS_ON to represent the Hours attribute of the relationship type. The primary key of the WORKS_ON relation is the combination of the foreign key attributes { Essn, Pno}. This relationship relation is shown in Figure 9.3(c).

The propagate (CASCADE) option for the referential triggered action (see Section 4.2) should be specified on the foreign keys in the relation corresponding to the relationship R, since each relationship instance has an existence dependency on each of the entities it relates. This can be used for both ON UPDATE and ON DELETE. Although we can map 1:1 or 1:N relationships in a manner similar to M:N relationships by using the cross-reference (relationship relation) approach, as we discussed earlier, this is only recommended when few relationship instances exist, in order to avoid NULL values in foreign keys. In this case, the primary key of the relationship relation will be only one of the foreign keys that reference the participating entity relations. For a 1:N relationship, the primary key of the relationship relation will be the foreign key that references the entity relation on the N-side. For a 1:1 relationship, either foreign key can be used as the primary key of the relationship relation.

Step 6: Mapping of Multivalued Attributes. For each multivalued attribute A, create a new relation R. This relation R will include an attribute corresponding to A, plus the primary key attribute K—as a foreign key in R—of the relation that represents the entity type or relationship type that has A as a multivalued attribute. The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.

In our example, we create a relation DEPT_LOCATIONS (see Figure 9.3(d)).

The attribute Dlocation represents the multivalued attribute LOCATIONS of DEPARTMENT, whereas Dnumber—as foreign key—represents the primary key of the DEPARTMENT relation. The primary key of DEPT_LOCATIONS is the combination of {Dnumber, Dlocation}. A separate tuple will exist in DEPT_LOCATIONS for each location that a department has. It is important to note that in more recent versions of the relational model that allow array data types, the multivalued attribute can be mapped to an array attribute rather than requiring a separate table.

The propagate (CASCADE) option for the referential triggered action (see Section 6.2) should be specified on the foreign key in the relation R corresponding to the multivalued attribute for both ON UPDATE and ON DELETE. We should also note that the key of R when mapping a composite, multivalued attribute requires some analysis of the meaning of the component attributes. In some cases, when a multivalued attribute is composite, only some of the component attributes are required to be part of the key of R; these attributes are similar to a partial key of a weak entity type that corresponds to the multivalued attribute (see Section 3.5).

Figure 9.2 shows the COMPANY relational database schema obtained with steps 1 through 6, and Figure 5.6 shows a sample database state. Notice that we did not yet discuss the mapping of n-ary relationship types ($n > 2$) because none exist in Figure 9.1; these are mapped in a similar way to M:N relationship types by including the following additional step in the mapping algorithm.

Step 7: Mapping of N-ary Relationship Types. We use the relationship relation option. For each n-ary relationship type R, where $n > 2$, create a new relationship relation S to represent R. Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types. Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S. The primary key of S is usually a combination of all the foreign keys that reference the relations representing the participating entity types. However, if the cardinality constraints on any of the entity types E participating in R is 1, then the primary key of S should not include the foreign key attribute that references the relation E' corresponding to E (see the discussion in Section 3.9.2 concerning constraints on n-ary relationships).

Consider the ternary relationship type SUPPLY in Figure 3.17, which relates a SUPPLIER s, PART p, and PROJECT j whenever s is currently supplying p to j; this can be mapped to the relation SUPPLY shown in Figure 9.4, whose primary key is the combination of the three foreign keys {Sname, Part_no, Proj_name}.

9.1.2 Discussion and Summary of Mapping

for ER Model Constructs

Table 9.1 summarizes the correspondences between ER and relational model constructs and constraints.

Figure 9.4 SUPPLIER

Mapping the n-ary

Sname ...

relationship type

SUPPLY from

Figure 3.17(a). PROJECT

Proj_name ...

PART

Part_no ...

SUPPLY

Sname Proj_name Part_no Quantity

9.1 Relational Database Design Using ER-to-Relational Mapping 327

Table 9.1 Correspondence between ER and Relational Models

ER MODEL	RELATIONAL MODEL
Entity type	Entity relation
1:1 or 1:N relationship type	Foreign key (or relationship relation)
M:N relationship type	Relationship relation and two foreign keys
n-ary relationship type	Relationship relation and n foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key

One of the main points to note in a relational schema, in contrast to an ER schema, is that relationship types are not represented explicitly; instead, they are represented by having two attributes A and B, one a primary key and the other a foreign key (over the same domain) included in two relations S and T. Two tuples in S and T are related when they have the same value for A and B. By using the EQUIJOIN operation (or NATURAL JOIN if the two join attributes have the same name) over S.A and T.B, we can combine all pairs of related tuples from S and T and materialize the relationship. When a binary 1:1 or 1:N relationship type is involved and the foreign key mapping is used, a single join operation is usually needed. When the relationship relation approach is used, such as for a binary M:N relationship type, two join operations are needed, whereas for n-ary relationship types, n joins are needed to fully materialize the relationship instances.

For example, to form a relation that includes the employee name, project name, and hours that the employee works on each project, we need to connect each EMPLOYEE tuple to the related PROJECT tuples via the WORKS_ON relation in Figure 9.2. Hence, we must apply the EQUIJOIN operation to

the EMPLOYEE and WORKS_ON relations with the join condition $EMPLOYEE.Ssn = WORKS_ON.Essn$, and then apply another EQUIJOIN operation to the resulting relation and the PROJECT relation with join condition $WORKS_ON.Pno = PROJECT.Pnumber$. In general, when multiple relationships need to be traversed, numerous join operations must be specified. The user must always be aware of the foreign key attributes in order to use them correctly in combining related tuples from two or more relations. This is sometimes considered to be a drawback of the relational data model, because the foreign key/primary key correspondences are not always obvious upon inspection of relational schemas. If an EQUIJOIN is performed among attributes of two relations that do not represent a foreign key/primary key relationship, the result can often be meaningless and may lead to spurious data. For example, the reader can try joining the PROJECT and DEPT_LOCATIONS relations on the condition $Dlocation = Plocation$ and examine the result.

328 Chapter 9 Relational Database Design by ER- and EER-to-Relational Mapping

In the relational schema we create a separate relation for each multivalued attribute. For a particular entity with a set of values for the multivalued attribute, the key attribute value of the entity is repeated once for each value of the multivalued attribute in a separate tuple because the basic relational model does not allow multiple values (a list, or a set of values) for an attribute in a single tuple. For example, because department 5 has three locations, three tuples exist in the DEPT_LOCATIONS relation in Figure 3.6; each tuple specifies one of the locations. In our example, we apply EQUIJOIN to DEPT_LOCATIONS and DEPARTMENT on the Dnumber attribute to get the values of all locations along with other DEPARTMENT attributes. In the resulting relation, the values of the other DEPARTMENT attributes are repeated in separate tuples for every location that a department has.

The basic relational algebra does not have a NEST or COMPRESS operation that would produce a set of tuples of the form $\{ \langle '1', 'Houston' \rangle, \langle '4', 'Stafford' \rangle, \langle '5', 'Bellaire', 'Sugarland', 'Houston' \rangle \}$ from the DEPT_LOCATIONS relation in Figure 3.6. This is a serious drawback of the basic normalized or flat version of the relational model. The object data model and object-relational systems (see Chapter 12) do allow multivalued attributes by using the array type for the attribute.

9.2 Mapping EER Model Constructs to Relations

In this section, we discuss the mapping of EER model constructs to relations by extending the ER-to-relational mapping algorithm that was presented in Section 9.1.1.

9.2.1 Mapping of Specialization or Generalization

There are several options for mapping a number of subclasses that together form a specialization (or alternatively, that are generalized into a superclass), such as the {SECRETARY, TECHNICIAN, ENGINEER} subclasses of EMPLOYEE in Figure 4.4. The

two main options are to map the whole specialization into a single table, or to map it into multiple tables. Within each option are variations that depend on the constraints on the specialization/generalization.

We can add a further step to our ER-to-relational mapping algorithm from Section 9.1.1, which has seven steps, to handle the mapping of specialization. Step 8, which follows, gives the most common options; other mappings are also possible. We discuss the conditions under which each option should be used. We use $\text{Attrs}(R)$ to denote the attributes of a relation R , and $\text{PK}(R)$ to denote the primary key of R . First we describe the mapping formally, then we illustrate it with examples.

Step 8: Options for Mapping Specialization or Generalization. Convert each specialization with m subclasses $\{S_1, S_2, \dots, S_m\}$ and (generalized) superclass C , where the attributes of C are $\{k, a_1, \dots, a_n\}$ and k is the (primary) key, into relation schemas using one of the following options:

9.2 Mapping EER Model Constructs to Relations 329

■■ Option 8A: Multiple relations—superclass and subclasses. Create a relation L for C with attributes $\text{Attrs}(L) = \{k, a_1, \dots, a_n\}$ and $\text{PK}(L) = k$. Create a relation L_i for each subclass S_i , $1 \leq i \leq m$, with the attributes $\text{Attrs}(L_i) = \{k\} \cup \{\text{attributes of } S_i\}$ and $\text{PK}(L_i) = k$. This option works for any specialization (total or partial, disjoint or overlapping).

■■ Option 8B: Multiple relations—subclass relations only. Create a relation L_i for each subclass S_i , $1 \leq i \leq m$, with the attributes $\text{Attrs}(L_i) = \{\text{attributes of } S_i\} \cup \{k, a_1, \dots, a_n\}$ and $\text{PK}(L_i) = k$. This option only works for a specialization whose subclasses are total (every entity in the superclass must belong to (at least) one of the subclasses). Additionally, it is only recommended if the specialization has the disjointedness constraint (see Section 4.3.1). If the specialization is overlapping, the same entity may be duplicated in several relations.

■■ Option 8C: Single relation with one type attribute. Create a single relation L with attributes $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t\}$ and $\text{PK}(L) = k$. The attribute t is called a type (or discriminating) attribute whose value indicates the subclass to which each tuple belongs, if any. This option works only for a specialization whose subclasses are disjoint, and has the potential for generating many NULL values if many specific (local) attributes exist in the subclasses.

■■ Option 8D: Single relation with multiple type attributes. Create a single relation schema L with attributes $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t_1, t_2, \dots, t_m\}$ and $\text{PK}(L) = k$. Each t_i , $1 \leq i \leq m$, is a Boolean type attribute indicating whether or not a tuple belongs to subclass S_i . This option is used for a specialization whose subclasses are overlapping (but will also work for a disjoint specialization).

Options 8A and 8B are the multiple-relation options, whereas options 8C and 8D are

the single-relation options. Option 8A creates a relation L for the superclass C and its attributes, plus a relation L_i for each subclass S_i ; each L_i includes the specific (local) attributes of S_i , plus the primary key of the superclass C , which is propagated to L_i and becomes its primary key. It also becomes a foreign key to the superclass relation. An EQUIJOIN operation on the primary key between any L_i and L produces all the specific and inherited attributes of the entities in S_i . This option is illustrated in Figure 9.5(a) for the EER schema in Figure 4.4. Option 8A works for any constraints on the specialization: disjoint or overlapping, total or partial. Notice that the constraint

$$\pi_{\langle k \rangle}(L_i) \subseteq \pi_{\langle k \rangle}(L)$$

must hold for each L_i . This specifies a foreign key from each L_i to L .

In option 8B, the EQUIJOIN operation between each subclass and the superclass is built into the schema and the superclass relation L is done away with, as illustrated in Figure 9.5(b) for the EER specialization in Figure 4.3(b). This option works well only when both the disjoint and total constraints hold. If the specialization is not total, an entity that does not belong to any of the subclasses S_i is lost. If the specialization is not disjoint, an entity belonging to more than one subclass will have its

330 Chapter 9 Relational Database Design by ER- and EER-to-Relational Mapping

(a) EMPLOYEE

Ssn Fname Minit Lname Birth_date Address Job_type

SECRETARY TECHNICIAN ENGINEER

Ssn Typing_speed Ssn Tgrade Ssn Eng_type

(b) CAR

Vehicle_id License_plate_no Price Max_speed No_of_passengers

TRUCK

Vehicle_id License_plate_no Price No_of_axles Tonnage

(c) EMPLOYEE

Ssn Fname Minit Lname Birth_date Address Job_type Typing_speed Tgrade Eng_type

(d) PART

Part_no Description Mflag Drawing_no Manufacture_date Batch_no Pflag Supplier_name List_price

Figure 9.5

Options for mapping specialization or generalization. (a) Mapping the EER schema in Figure 4.4 using option 8A.

(b) Mapping the EER schema in Figure 4.3(b) using option 8B. (c) Mapping the EER schema in Figure 4.4 using

option 8C. (d) Mapping Figure 4.5 using option 8D with Boolean type fields Mflag and Pflag.

inherited attributes from the superclass C stored redundantly in more than one table L_i .

With option 8B, no relation holds all the entities in the superclass C ; consequently, we must apply an OUTER UNION (or FULL OUTER JOIN) operation (see Section 6.4) to the L_i relations to retrieve all the entities in C . The result of the outer union will be similar to the relations under options 8C and 8D except that the type fields will be missing. When-

ever we search for an arbitrary entity in C , we must search all the m relations L_i . Options 8C and 8D create a single relation to represent the superclass C and all its subclasses. An entity that does not belong to some of the subclasses will have NULL values for the specific (local) attributes of these subclasses. These options are not recommended if many specific attributes are defined for the subclasses. If few local subclass attributes exist, however, these mappings are preferable to options 8A and 8B because they do away with the need to specify JOIN operations; therefore, they can yield a more efficient implementation for queries.

Option 8C is used to handle disjoint subclasses by including a single type (or image or discriminating) attribute t to indicate to which of the m subclasses each tuple belongs; hence, the domain of t could be $\{1, 2, \dots, m\}$. If the specialization is partial, t can have NULL values in tuples that do not belong to any subclass. If the specialization is attribute-defined, that attribute itself serves the purpose of t and t is not needed; this option is illustrated in Figure 9.5(c) for the EER specialization in Figure 4.4.

Option 8D is designed to handle overlapping subclasses by including m Boolean type (or flag) fields, one for each subclass. It can also be used for disjoint subclasses.

9.2 Mapping EER Model Constructs to Relations 331

Each type field t_i can have a domain $\{\text{yes}, \text{no}\}$, where a value of yes indicates that the tuple is a member of subclass S_i . If we use this option for the EER specialization in Figure 4.4, we would include three type attributes—`Is_a_secretary`, `Is_a_engineer`, and `Is_a_technician`—instead of the `Job_type` attribute in Figure 9.5(c). Figure 9.5(d) shows the mapping of the specialization from Figure 4.5 using option 8D.

For a multilevel specialization (or generalization) hierarchy or lattice, we do not have to follow the same mapping option for all the specializations. Instead, we can use one mapping option for part of the hierarchy or lattice and other options for other parts. Figure 9.6 shows one possible mapping into relations for the EER lattice in Figure 4.6. Here we used option 8A for `PERSON/{EMPLOYEE, ALUMNUS, STUDENT}`, and option 8C for `EMPLOYEE/{STAFF, FACULTY, STUDENT_ASSISTANT}` by including the type attribute `Employee_type`. We then used the single-table option 8D for `STUDENT_ASSISTANT/{RESEARCH_ASSISTANT, TEACHING_ASSISTANT}` by including the type attributes `Ta_flag` and `Ra_flag` in `EMPLOYEE`. We also used option 8D for `STUDENT/STUDENT_ASSISTANT` by including the type attributes `Student_assist_flag` in `STUDENT`, and for `STUDENT/{GRADUATE_STUDENT, UNDERGRADUATE_STUDENT}` by including the type attributes `Grad_flag` and `Undergrad_flag` in `STUDENT`. In Figure 9.6, all attributes whose names end with type or flag are type fields.

9.2.2 Mapping of Shared Subclasses (Multiple Inheritance)

A shared subclass, such as `ENGINEERING_MANAGER` in Figure 4.6, is a subclass of several superclasses, indicating multiple inheritance. These classes must all have the same key attribute; otherwise, the shared subclass would be modeled as a category (union type) as we discussed in Section 4.4. We can apply any of the options discussed in step 8 to a shared subclass, subject to the restrictions discussed in step 8 of

the mapping algorithm. In Figure 9.6, options 8C and 8D are used for the shared subclass STUDENT_ASSISTANT. Option 8C is used in the EMPLOYEE relation (Employee_type attribute) and option 8D is used in the STUDENT relation (Student_assist_flag attribute).

PERSON Figure 9.6

Mapping the EER specialization

Ssn Name Birth_date Sex Address

lattice in Figure 4.8 using

multiple options.

EMPLOYEE

Ssn Salary Employee_type Position Rank Percent_time Ra_flag Ta_flag Project Course

ALUMNUS ALUMNUS_DEGREES

Ssn Ssn Year Degree Major

STUDENT

Ssn Major_dept Grad_flag Undergrad_flag Degree_program Class Student_assist_flag

332 Chapter 9 Relational Database Design by ER- and EER-to-Relational Mapping

9.2.3 Mapping of Categories (Union Types)

We add another step to the mapping procedure—step 9—to handle categories. A category (or union type) is a subclass of the union of two or more superclasses that can have different keys because they can be of different entity types (see Section 4.4). An example is the OWNER category shown in Figure 4.8, which is a subset of the union of three entity types PERSON, BANK, and COMPANY. The other category in that figure, REGISTERED_VEHICLE, has two superclasses that have the same key attribute.

Step 9: Mapping of Union Types (Categories). For mapping a category whose defining superclasses have different keys, it is customary to specify a new key attribute, called a surrogate key, when creating a relation to correspond to the union type. The keys of the defining classes are different, so we cannot use any one of them exclusively to identify all entities in the relation. In our example in Figure 4.8, we create a relation OWNER to correspond to the OWNER category, as illustrated in Figure 9.7, and include any attributes of the category in this relation. The primary key of the OWNER relation is the surrogate key, which we called Owner_id. We also

Figure 9.7 PERSON

Mapping the EER categories

Ssn Driver_license_no Name Address Owner_id

(union types) in Figure 4.8 to

relations.

BANK

Bname Baddress Owner_id

COMPANY

Cname Caddress Owner_id

OWNER

Owner_id

REGISTERED_VEHICLE

Vehicle_id License_plate_number

CAR

Vehicle_id Cstyle Cmake Cmodel Cyear

TRUCK

Vehicle_id Tmake Tmodel Tonnage Tyear

OWNS

Owner_id Vehicle_id Purchase_date Lien_or_regular

Exercises 333

include the surrogate key attribute Owner_id as foreign key in each relation corresponding to a superclass of the category, to specify the correspondence in values between the surrogate key and the original key of each superclass. Notice that if a particular PERSON (or BANK or COMPANY) entity is not a member of OWNER, it would have a NULL value for its Owner_id attribute in its corresponding tuple in the PERSON (or BANK or COMPANY) relation, and it would not have a tuple in the OWNER relation. It is also recommended to add a type attribute (not shown in Figure 9.7) to the OWNER relation to indicate the particular entity type to which each tuple belongs (PERSON or BANK or COMPANY).

For a category whose superclasses have the same key, such as VEHICLE in Figure 4.8, there is no need for a surrogate key. The mapping of the REGISTERED_VEHICLE category, which illustrates this case, is also shown in Figure 9.7.

9.3 Summary

In Section 9.1, we showed how a conceptual schema design in the ER model can be mapped to a relational database schema. An algorithm for ER-to-relational mapping was given and illustrated by examples from the COMPANY database. Table 9.1 summarized the correspondences between the ER and relational model constructs and constraints. Next, we added additional steps to the algorithm in Section 9.2 for mapping the constructs from the EER model into the relational model. Similar algorithms are incorporated into graphical database design tools to create a relational schema from a conceptual schema design automatically.

Review Questions

- 9.1. (a) What do you mean by entity relations? Discuss in brief about the seven-step algorithm to convert the basic ER model constructs into relations. Discuss the steps involved in the mapping of Weak Entity Types in ER model.
- (b) What is the need of Enhanced ER (EER) model? Multiple Inheritance concepts can be applied to the EER Model. Justify.

Exercises

9.2. Given the entities:

student (s_roll,s_name,s_phone,s_address)

course (c_id,c_name,c_startdate,c_fees)

Draw an ER diagram to represent a relation between the two entities. Map the ER diagram into relational schema using a foreign-key approach.

9.3. Try to map the relational schema in Figure 6.14 into an ER schema. This is part of a process known as reverse engineering, where a conceptual schema is created for an existing implemented database. State any assumptions you make.

334 Chapter 9 Relational Database Design by ER- and EER-to-Relational Mapping

Date

Time_stamp

Time

SHIP_MOVEMENT

Longitude

N

Latitude

HISTORY

1 Type Tonnage Hull

Sname

N 1

SHIP TYPE SHIP_TYPE

Owner

(0,*)

Start_date End_date

N

(1,1)

HOME_PORT SHIP_AT PORT_VISIT

_PORT Continent

Name

1 (0,*)

N 1

IN STATE/COUNTRY

Pname PORT Name

N 1

ON SEA/OCEAN/LAKE

Figure 9.8

An ER schema for a SHIP_TRACKING database.

9.4. Figure 9.8 shows an ER schema for a database that can be used to keep track of

transport ships and their locations for maritime authorities. Map this schema into a relational schema and specify all primary keys and foreign keys.

9.5. Map the BANK ER schema of Exercise 3.23 (shown in Figure 3.21) into a relational schema. Specify all primary keys and foreign keys. Repeat for the AIRLINE schema (Figure 3.20) of Exercise 3.19 and for the other schemas for Exercises 3.16 through 3.24.

9.6. Explain in brief with an example how the EER model introduced the concepts of super-class and sub-class entity types in ER model?

9.7. Draw an EER diagram of an example depicting the concept of super-class and sub-class entity types in ER Model.

Laboratory Exercises 335

CAR Engine_size

Vin Price

VEHICLE d TRUCK Tonnage

Model N

Date SUV No_seats

SALE

1 1

SALESPERSON CUSTOMER Address City

Figure 9.9

Street EER diagram for

Sid Name Ssn Name State

a car dealer.

9.8. Consider the EER diagram in Figure 9.9 for a car dealer.

Map the EER schema into a set of relations. For the VEHICLE to CAR/TRUCK/SUV generalization, consider the four options presented in Section 9.2.1 and show the relational schema design under each of those options.

9.9. Using the attributes you provided for the EER diagram in Exercise 4.27, map the complete schema into a set of relations. Choose an appropriate option out of 8A thru 8D from Section 9.2.1 in doing the mapping of generalizations and defend your choice.

Laboratory Exercises

9.10. Consider the ER design for the UNIVERSITY database that was modeled using a tool like ERwin or Rational Rose in Laboratory Exercise 3.31. Using the SQL schema generation feature of the modeling tool, generate the SQL schema for an Oracle database.

9.11. Consider the ER design for the MAIL_ORDER database that was modeled using a tool like ERwin or Rational Rose in Laboratory Exercise 3.32. Using the SQL schema generation feature of the modeling tool, generate the SQL schema for an Oracle database.

9.12. Consider the ER design for the CONFERENCE_REVIEW database that was modeled using a tool like ERwin or Rational Rose in Laboratory Exercise 3.34. Using the SQL schema generation feature of the modeling tool, generate the SQL schema for an Oracle database.

336 Chapter 9 Relational Database Design by ER- and EER-to-Relational Mapping

9.13. Consider the EER design for the GRADE_BOOK database that was modeled using a tool like ERwin or Rational Rose in Laboratory Exercise 4.28. Using the SQL schema generation feature of the modeling tool, generate the SQL schema for an Oracle database.

9.14. Consider the EER design for the ONLINE_AUCTION database that was modeled using a tool like ERwin or Rational Rose in Laboratory Exercise 4.29. Using the SQL schema generation feature of the modeling tool, generate the SQL schema for an Oracle database.

Selected Bibliography

The original ER-to-relational mapping algorithm was described in Chen's classic paper (Chen, 1976). Batini et al. (1992) discuss a variety of mapping algorithms from ER and EER models to legacy models and vice versa.

part 4

Database Programming

Techniques

This page intentionally left blank

chapter 10

Introduction to SQL

Programming Techniques

In Chapters 6 and 7, we described several aspects of the SQL language, which is the standard for relational databases. We described the SQL statements for data definition, schema modification, queries, views, and updates. We also described how various constraints on the database contents, such as key and referential integrity constraints, are specified. In this chapter and the next, we discuss some of the methods that have been developed for accessing databases from programs. Most database access in practical applications is accomplished through software programs that implement database applications. This software is usually developed in a general-purpose programming language such as Java, C/C++/C#, COBOL (historically), or some other programming language. In addition, many scripting languages, such as PHP, Python, and JavaScript, are also being used for programming of database access within Web applications. In this chapter, we focus on how databases can be accessed from the traditional programming languages C/C++ and Java, whereas in the next chapter we introduce how databases are accessed from scripting languages such as PHP. Recall from Section 2.3.1 that when database statements are included in a program,

the general-purpose programming language is called the host language, whereas the database language—SQL, in our case—is called the data sublanguage. In some cases, special database programming languages are developed specifically for writing database applications. Although many of these were developed as research prototypes, some notable database programming languages have widespread use, such as Oracle’s PL/SQL (Programming Language/SQL).

It is important to note that database programming is a very broad topic. There are whole textbooks devoted to each database programming technique and how that technique is realized in a specific system. New techniques are developed all the

340 Chapter 10 Introduction to SQL Programming Techniques

time, and changes to existing techniques are incorporated into newer system versions and languages. An additional difficulty in presenting this topic is that although there are SQL standards, these standards themselves are continually evolving, and each DBMS vendor may have some variations from the standard. Because of this, we have chosen to give an introduction to some of the main types of database programming techniques and to compare these techniques, rather than study one particular method or system in detail. The examples we give serve to illustrate the main differences that a programmer would face when using each of these database programming techniques. We will try to use the SQL standards in our examples rather than describe a specific system. When using a specific system, the materials in this chapter can serve as an introduction, but should be augmented with the system manuals or with books describing the specific system.

We start our presentation of database programming in Section 10.1 with an overview of the different techniques developed for accessing a database from programs. Then, in Section 10.2, we discuss the rules for embedding SQL statements into a general-purpose programming language, generally known as embedded SQL. This section also briefly discusses dynamic SQL, in which queries can be dynamically constructed at runtime, and presents the basics of the SQLJ variation of embedded SQL that was developed specifically for the programming language Java. In Section 10.3, we discuss the technique known as SQL/CLI (Call Level Interface), in which a library of procedures and functions is provided for accessing the database. Various sets of library functions have been proposed. The SQL/CLI set of functions is the one given in the SQL standard. Another widely used library of functions is ODBC (Open Data Base Connectivity), which has many similarities to SQL/CLI; in fact, SQL/CLI can be thought of as the standardized version of ODBC. A third library of classes—which we do describe—is JDBC; this was developed specifically for accessing databases from the Java object-oriented programming language (OOPL). In OOPL, a library of classes is used instead of a library of functions and procedures, and each class has its own operations and functions. In Section 10.4 we discuss SQL/PSM (Persistent Stored Modules), which is a part of the SQL standard that

allows program modules—procedures and functions—to be stored by the DBMS and accessed through SQL; this also specifies a procedural database programming language for writing the persistent stored modules. We briefly compare the three approaches to database programming in Section 10.5, and provide a chapter summary in Section 10.6.

10.1 Overview of Database Programming

Techniques and Issues

We now turn our attention to the techniques that have been developed for accessing databases from programs and, in particular, to the issue of how to access SQL databases from application programs. Our presentation of SQL in Chapters 6 and 7 focused on the language constructs for various database operations—from schema definition and constraint specification to querying, updating, and specifying views.

10.1 Overview of Database Programming Techniques and Issues 341

Most database systems have an interactive interface where these SQL commands can be typed directly into a monitor for execution by the database system.

For example, in a computer system where the Oracle RDBMS is installed, the command SQLPLUS starts the interactive interface. The user can type SQL commands or queries directly over several lines, ended by a semicolon and the Enter key (that is, "`;<cr>`"). Alternatively, a file of commands can be created and executed through the interactive interface by typing `@<filename>`. The system will execute the commands written in the file and display the results, if any.

The interactive interface is quite convenient for schema and constraint creation or for occasional ad hoc queries. However, in practice, the majority of database interactions are executed through programs that have been carefully designed and tested. These programs are generally known as application programs or database applications, and are used as canned transactions by the end users, as discussed in Section 1.4.3. Another common use of database programming is to access a database through an application program that implements a Web interface, for example, when making airline reservations or online purchases. In fact, the vast majority of Web electronic commerce applications include some database access commands. Chapter 11 gives an overview of Web database programming using PHP, a scripting language that has recently become widely used.

In this section, first we give an overview of the main approaches to database programming. Then we discuss some of the problems that occur when trying to access a database from a general-purpose programming language, and the typical sequence of commands for interacting with a database from a software program.

10.1.1 Approaches to Database Programming

Several techniques exist for including database interactions in application programs. The main approaches for database programming are the following:

1. Embedding database commands in a general-purpose programming language. In this approach, database statements are embedded into the host

programming language, but they are identified by a special prefix. For example, the prefix for embedded SQL is the string EXEC SQL, which precedes all SQL commands in a host language program.¹ A precompiler or preprocessor scans the source program code to identify database statements and extract them for processing by the DBMS. They are replaced in the program by function calls to the DBMS-generated code. This technique is generally referred to as embedded SQL.

2. Using a library of database functions or classes. A library of functions is made available to the host programming language for database calls. For example, there could be functions to connect to a database, prepare a query, execute a query, execute an update, loop over the query result on record at a time, and so on. The actual database query and update commands and any

1

Other prefixes are sometimes used, but this is the most common.

342 Chapter 10 Introduction to SQL Programming Techniques

other necessary information are included as parameters in the function calls. This approach provides what is known as an application programming interface (API) for accessing a database from application programs. For object-oriented programming languages (OOPs), a class library is used. For example, Java has the JDBC class library, which can generate various types of objects such as: connection objects to a particular database, query objects, and query result objects. Each type of object has a set of operations associated with the class corresponding to the object.

3. Designing a brand-new language. A database programming language is designed from scratch to be compatible with the database model and query language. Additional programming structures such as loops and conditional statements are added to the database language to convert it into a full-fledged programming language. An example of this approach is Oracle's PL/SQL. The SQL standard has the SQL/PSM language for specifying stored procedures. In practice, the first two approaches are more common, since many applications are already written in general-purpose programming languages but require some database access. The third approach is more appropriate for applications that have intensive database interaction. One of the main problems with the first two approaches is impedance mismatch, which does not occur in the third approach.

10.1.2 Impedance Mismatch

Impedance mismatch is the term used to refer to the problems that occur because of differences between the database model and the programming language model. For example, the practical relational model has three main constructs: columns (attributes) and their data types, rows (also referred to as tuples or records), and tables (sets or multisets of records). The first problem that may occur is that the data types of the programming language differ from the attribute data types that are

available in the data model. Hence, it is necessary to have a binding for each host programming language that specifies for each attribute type the compatible programming language types. A different binding is needed for each programming language because different languages have different data types. For example, the data types available in C/C++ and Java are different, and both differ from the SQL data types, which are the standard data types for relational databases.

Another problem occurs because the results of most queries are sets or multisets of tuples (rows), and each tuple is formed of a sequence of attribute values. In the program, it is often necessary to access the individual data values within individual tuples for printing or processing. Hence, a binding is needed to map the query result data structure, which is a table, to an appropriate data structure in the programming language. A mechanism is needed to loop over the tuples in a query result in order to access a single tuple at a time and to extract individual values from the tuple. The extracted attribute values are typically copied to appropriate program variables for further processing by the program. A cursor or iterator variable is typically used to loop over the tuples in a query result. Individual values within each tuple are then extracted into distinct program variables of the appropriate type.

10.1 Overview of Database Programming Techniques and Issues 343

Impedance mismatch is less of a problem when a special database programming language is designed that uses the same data model and data types as the database model. One example of such a language is Oracle's PL/SQL. The SQL standard also has a proposal for such a database programming language, known as SQL/PSM. For object databases, the object data model (see Chapter 12) is quite similar to the data model of the Java programming language, so the impedance mismatch is greatly reduced when Java is used as the host language for accessing a Java-compatible object database. Several database programming languages have been implemented as research prototypes (see the Selected Bibliography).

10.1.3 Typical Sequence of Interaction in Database Programming

When a programmer or software engineer writes a program that requires access to a database, it is quite common for the program to be running on one computer system while the database is installed on another. Recall from Section 2.5 that a common architecture for database access is the three-tier client/server model, where a top-tier client program handles display of information on a laptop or mobile device usually as a Web client or mobile app, a middle-tier application program implements the logic of a business software application but includes some calls to one or more database servers at the bottom tier to access or update the data.² When writing such an application program, a common sequence of interaction is the following:

1. When the application program requires access to a particular database, the program must first establish or open a connection to the database server.

Typically, this involves specifying the Internet address (URL) of the machine where the database server is located, plus providing a login account name and password for database access.

2. Once the connection is established, the program can interact with the database by submitting queries, updates, and other database commands. In general, most types of SQL statements can be included in an application program.

3. When the program no longer needs access to a particular database, it should terminate or close the connection to the database.

A program can access multiple databases if needed. In some database programming approaches, only one connection can be active at a time, whereas in other approaches multiple connections can be established simultaneously.

In the next three sections, we discuss examples of each of the three main approaches to database programming. Section 10.2 describes how SQL is embedded into a programming language. Section 10.3 discusses how function calls and class libraries are used to access the database using SQL/CLI (similar to ODBC) and JDBC, and Section 10.4 discusses an extension to SQL called SQL/PSM that allows general-purpose

2

As we discussed in Section 2.5, there are two-tier and three-tier architectures; to keep our discussion

simple, we will assume a two-tier client/server architecture here.

344 Chapter 10 Introduction to SQL Programming Techniques

programming constructs for defining modules (procedures and functions) that are stored within the database system.³ Section 10.5 compares these approaches.

10.2 Embedded SQL, Dynamic SQL, and SQL J

In this section, we give an overview of the techniques for embedding SQL statements in a general-purpose programming language. We focus on two languages: C and Java. The examples used with the C language, known as embedded SQL, are presented in Sections 10.2.1 through 10.2.3, and can be adapted to other similar programming languages. The examples using Java, known as SQLJ, are presented in Sections 10.2.4 and 10.2.5. In this embedded approach, the programming language is called the host language. Most SQL statements—including data or constraint definitions, queries, updates, or view definitions—can be embedded in a host language program.

10.2.1 Retrieving Single Tuples with Embedded SQL

To illustrate the concepts of embedded SQL, we will use C as the host programming language.⁴ In a C program, an embedded SQL statement is distinguished from programming language statements by prefixing it with the keywords EXEC SQL so that a preprocessor (or precompiler) can separate embedded SQL statements from the host language source code. The SQL statements within a program are terminated by a matching END-EXEC or by a semicolon (;). Similar rules apply to embedding SQL in other programming languages.

Within an embedded SQL command, the programmer can refer to specially declared C program variables; these are called shared variables because they are used in both the C program and the embedded SQL statements. Shared variables are prefixed by a colon (:) when they appear in an SQL statement. This distinguishes program variable names from the names of database schema constructs such as attributes (column names) and relations (table names). It also allows program variables to have the same names as attribute names, since they are distinguishable by the colon (:) prefix in the SQL statement. Names of database schema constructs—such as attributes and relations—can only be used within the SQL commands, but shared program variables can be used elsewhere in the C program without the colon (:) prefix.

Suppose that we want to write C programs to process the COMPANY database in Figure 5.5. We need to declare program variables to match the types of the database attributes that the program will process. The programmer can choose the names of the program variables; they may or may not have names that are identical to their

3

SQL/PSM illustrates how typical general-purpose programming language constructs—such as loops and conditional structures—can be incorporated into SQL.

4

Our discussion here also applies to the C++ or C# programming languages, since we do not use any

of the object-oriented features, but focus on the database programming mechanism.

10.2 Embedded SQL, Dynamic SQL, and SQL J 345

0) int loop ;

1) EXEC SQL BEGIN DECLARE SECTION ;

2) varchar dname [16], fname [16], lname [16], address [31] ;

3) char ssn [10], bdate [11], sex [2], minit [2] ;

4) float salary, raise ;

5) int dno, dnumber ; Figure 10.1

6) int SQLCODE ; char SQLSTATE [6] ; C program variables used in the

7) EXEC SQL END DECLARE SECTION ; embedded SQL examples E1 and E2.

corresponding database attributes. We will use the C program variables declared in Figure 10.1 for all our examples and show C program segments without variable declarations. Shared variables are declared within a declare section in the program, as shown in Figure 10.1 (lines 1 through 7).⁵ A few of the common bindings of C types to SQL types are as follows. The SQL types INTEGER, SMALLINT, REAL, and DOUBLE are mapped to the C data types long , short , float , and double , respectively. Fixed-length and varying-length strings (CHAR [i] , VARCHAR [i]) in SQL can be mapped to arrays of characters (char [i+1] , varchar [i+1]) in C that are one character longer than the SQL type because strings in C are terminated by a NULL character (\0), which is not part of the

character string itself.⁶ Although varchar is not a standard C data type, it is permitted when C is used for SQL database programming.

Notice that the only embedded SQL commands in Figure 10.1 are lines 1 and 7, which tell the precompiler to take note of the C variable names between BEGIN DECLARE and END DECLARE because they can be included in embedded SQL statements—as long as they are preceded by a colon (:). Lines 2 through 5 are regular C program declarations. The C program variables declared in lines 2 through 5 correspond to the attributes of the EMPLOYEE and DEPARTMENT tables from the COMPANY database in Figure 5.5 that was declared by the SQL DDL in Figure 6.1. The variables declared in line 6—SQLCODE and SQLSTATE—are called SQL communication variables; they are used to communicate errors and exception conditions between the database system and the executing program. Line 0 shows a program variable loop that will not be used in any embedded SQL statement, so it is declared outside the SQL declare section.

Connecting to the Database. The SQL command for establishing a connection to a database has the following form:

```
CONNECT TO <server name>AS <connection name>
AUTHORIZATION <user account name and password> ;
```

In general, since a user or program can access several database servers, several connections can be established, but only one connection can be active at any point in

5

We use line numbers in our code segments for easy reference; these numbers are not part of the actual code.

6

SQL strings can also be mapped to char* types in C.

346 Chapter 10 Introduction to SQL Programming Techniques

time. The programmer or user can use the <connection name> to change from the currently active connection to a different one by using the following command:

```
SET CONNECTION <connection name> ;
```

Once a connection is no longer needed, it can be terminated by the following command:

```
DISCONNECT <connection name> ;
```

In the examples in this chapter, we assume that the appropriate connection has already been established to the COMPANY database, and that it is the currently active connection.

Communication variables SQLCODE and SQLSTATE. The two special communication variables that are used by the DBMS to communicate exception or error conditions to the program are SQLCODE and SQLSTATE. The SQLCODE variable shown in Figure 10.1 is an integer variable. After each database command is executed, the DBMS returns a value in SQLCODE. A value of 0 indicates that the

statement was executed successfully by the DBMS. If `SQLCODE > 0` (or, more specifically, if `SQLCODE = 100`), this indicates that no more data (records) are available in a query result. If `SQLCODE < 0`, this indicates some error has occurred. In some systems—for example, in the Oracle RDBMS—`SQLCODE` is a field in a record structure called `SQLCA` (SQL communication area), so it is referenced as `SQLCA.SQLCODE`. In this case, the definition of `SQLCA` must be included in the C program by including the following line:

```
EXEC SQL include SQLCA ;
```

In later versions of the SQL standard, a communication variable called `SQLSTATE` was added, which is a string of five characters. A value of `'00000'` in `SQLSTATE` indicates no error or exception; other values indicate various errors or exceptions. For example, `'02000'` indicates 'no more data' when using `SQLSTATE`. Currently, both `SQLSTATE` and `SQLCODE` are available in the SQL standard. Many of the error and exception codes returned in `SQLSTATE` are supposed to be standardized for all SQL vendors and platforms,⁷ whereas the codes returned in `SQLCODE` are not standardized but are defined by the DBMS vendor. Hence, it is generally better to use `SQLSTATE` because this makes error handling in the application programs independent of a particular DBMS. As an exercise, the reader should rewrite the examples given later in this chapter using `SQLSTATE` instead of `SQLCODE`.

Example of Embedded SQL Programming. Our first example to illustrate embedded SQL programming is a repeating program segment (loop) that takes as input a Social Security number of an employee and prints some information from the corresponding `EMPLOYEE` record in the database. The C program code is shown as program segment E1 in Figure 10.2. The program reads (inputs) an `Ssn` value

7

In particular, `SQLSTATE` codes starting with the characters 0 through 4 or A through H are supposed to

be standardized, whereas other values can be implementation-defined.

10.2 Embedded SQL, Dynamic SQL, and SQL J 347

//Program Segment E1: Figure 10.2

```
0) loop = 1 ; Program segment E1,
1) while (loop) { a C program segment
2) prompt("Enter a Social Security Number: ", ssn) ; with embedded SQL.
3) EXEC SQL
4) SELECT Fname, Minit, Lname, Address, Salary
5) INTO :fname, :minit, :lname, :address, :salary
6) FROM EMPLOYEE WHERE Ssn = :ssn ;
7) if (SQLCODE == 0) printf(fname, minit, lname, address, salary)
8) else printf("Social Security Number does not exist: ", ssn) ;
9) prompt("More Social Security Numbers (enter 1 for Yes, 0 for No): ", loop) ;
10) }
```

and then retrieves the EMPLOYEE tuple with that Ssn from the database via the embedded SQL command. The INTO clause (line 5) specifies the program variables into which attribute values from the database record are retrieved. C program variables in the INTO clause are prefixed with a colon (:), as we discussed earlier. The INTO clause can be used in this manner only when the query result is a single record; if multiple records are retrieved, an error will be generated. We will see how multiple records are handled in Section 10.2.2.

Line 7 in E1 illustrates the communication between the database and the program through the special variable SQLCODE. If the value returned by the DBMS in SQLCODE is 0, the previous statement was executed without errors or exception conditions. Line 7 checks this and assumes that if an error occurred, it was because no EMPLOYEE tuple existed with the given Ssn; therefore it outputs a message to that effect (line 8).

When a single record is retrieved as in example E1, the programmer can assign its attribute values directly to C program variables in the INTO clause, as in line 5. In general, an SQL query can retrieve many tuples. In that case, the C program will typically loop through the retrieved tuples and process them one at a time. The concept of a cursor is used to allow tuple-at-a-time processing of a query result by the host language program. We describe cursors next.

10.2.2 Processing Query Results Using Cursors

A cursor is a variable that refers to a single tuple (row) from a query result that retrieves a collection of tuples. It is used to loop over the query result, one record at a time. The cursor is declared when the SQL query is declared. Later in the program, an OPEN CURSOR command fetches the query result from the database and sets the cursor to a position before the first row in the result of the query. This becomes the current row for the cursor. Subsequently, FETCH commands are issued in the program; each FETCH moves the cursor to the next row in the result of the query, making it the current row and copying its attribute values into the C (host language) program variables specified in the FETCH command by an INTO

348 Chapter 10 Introduction to SQL Programming Techniques

clause. The cursor variable is basically an iterator that iterates (loops) over the tuples in the query result—one tuple at a time.

To determine when all the tuples in the result of the query have been processed, the communication variable SQLCODE (or, alternatively, SQLSTATE) is checked. If a FETCH command is issued that results in moving the cursor past the last tuple in the result of the query, a positive value ($\text{SQLCODE} > 0$) is returned in SQLCODE, indicating that no data (tuple) was found (or the string '02000' is returned in SQLSTATE). The programmer uses this to terminate the loop over the tuples in the query result. In general, numerous cursors can be opened at the same time. A CLOSE CURSOR command is issued to indicate that we are done with processing the result of the query associated with that cursor.

An example of using cursors to process a query result with multiple records is shown in Figure 10.3, where a cursor called EMP is declared in line 4. The EMP cursor is associated with the SQL query declared in lines 5 through 6, but the query is not executed until the OPEN EMP command (line 8) is processed. The OPEN <cursor name> command executes the query and fetches its result as a table into the program workspace, where the program can loop through the individual rows (tuples) by subsequent FETCH <cursor name> commands (line 9). We assume

Figure 10.3

Program segment E2, a C program segment that uses cursors with embedded SQL for update purposes.

//Program Segment E2:

```
0) prompt("Enter the Department Name: ", dname) ;
1) EXEC SQL
2) SELECT Dnumber INTO :dnumber
3) FROM DEPARTMENT WHERE Dname = :dname ;
4) EXEC SQL DECLARE EMP CURSOR FOR
5) SELECT Ssn, Fname, Minit, Lname, Salary
6) FROM EMPLOYEE WHERE Dno = :dnumber
7) FOR UPDATE OF Salary ;
8) EXEC SQL OPEN EMP ;
9) EXEC SQL FETCH FROM EMP INTO :ssn, :fname, :minit, :lname, :salary ;
10) while (SQLCODE == 0) {
11) printf("Employee name is:", Fname, Minit, Lname) ;
12) prompt("Enter the raise amount: ", raise) ;
13) EXEC SQL
14) UPDATE EMPLOYEE
15) SET Salary = Salary + :raise
16) WHERE CURRENT OF EMP ;
17) EXEC SQL FETCH FROM EMP INTO :ssn, :fname, :minit, :lname, :salary ;
18) }
19) EXEC SQL CLOSE EMP ;
```

10.2 Embedded SQL, Dynamic SQL, and SQL J 349

that appropriate C program variables have been declared as in Figure 10.1. The program segment in E2 reads (inputs) a department name (line 0), retrieves the matching department number from the database (lines 1 to 3), and then retrieves the employees who work in that department via the declared EMP cursor. A loop (lines 10 to 18) iterates over each record in the query result, one at a time, and prints the employee name, then reads (inputs) a raise amount for that employee (line 12) and updates the employee's salary in the database by the raise amount (lines 14 to 16).

This example also illustrates how the programmer can update database records.

When a cursor is defined for rows that are to be modified (updated), we must add the clause `FOR UPDATE OF` in the cursor declaration and list the names of any attributes that will be updated by the program. This is illustrated in line 7 of code segment E2. If rows are to be deleted, the keywords `FOR UPDATE` must be added without specifying any attributes. In the embedded `UPDATE` (or `DELETE`) command, the condition `WHERE CURRENT OF <cursor name>` specifies that the current tuple referenced by the cursor is the one to be updated (or deleted), as in line 16 of E2. There is no need to include the `FOR UPDATE OF` clause in line 7 of E2 if the results of the query are to be used for retrieval purposes only (no update or delete).

General Options for a Cursor Declaration. Several options can be specified when declaring a cursor. The general form of a cursor declaration is as follows:

```
DECLARE <cursor name> [ INSENSITIVE ] [ SCROLL ] CURSOR
[ WITH HOLD ] FOR <query specification>
[ ORDER BY <ordering specification> ]
[ FOR READ ONLY | FOR UPDATE [ OF <attribute list> ] ] ;
```

We already briefly discussed the options listed in the last line. The default is that the query is for retrieval purposes (`FOR READ ONLY`). If some of the tuples in the query result are to be updated, we need to specify `FOR UPDATE OF <attribute list>` and list the attributes that may be updated. If some tuples are to be deleted, we need to specify `FOR UPDATE` without any attributes listed.

When the optional keyword `SCROLL` is specified in a cursor declaration, it is possible to position the cursor in other ways than for purely sequential access. A fetch orientation can be added to the `FETCH` command, whose value can be one of `NEXT`, `PRIOR`, `FIRST`, `LAST`, `ABSOLUTE i`, and `RELATIVE i`. In the latter two commands, `i` must evaluate to an integer value that specifies an absolute tuple position within the query result (for `ABSOLUTE i`), or a tuple position relative to the current cursor position (for `RELATIVE i`). The default fetch orientation, which we used in our examples, is `NEXT`. The fetch orientation allows the programmer to move the cursor around the tuples in the query result with greater flexibility, providing random access by position or access in reverse order. When `SCROLL` is specified on the cursor, the general form of a `FETCH` command is as follows, with the parts in square brackets being optional:

```
FETCH [ [ <fetch orientation> ] FROM ] <cursor name> INTO <fetch target list>;
```

350 Chapter 10 Introduction to SQL Programming Techniques

The `ORDER BY` clause orders the tuples so that the `FETCH` command will fetch them in the specified order. It is specified in a similar manner to the corresponding clause for SQL queries (see Section 6.3.6). The last two options when declaring a cursor (`INSENSITIVE` and `WITH HOLD`) refer to transaction characteristics of database programs, which we will discuss in Chapter 20.

10.2.3 Specifying Queries at Runtime Using Dynamic SQL

In the previous examples, the embedded SQL queries were written as part of the

host program source code. Hence, anytime we want to write a different query, we must modify the program code and go through all the steps involved (compiling, debugging, testing, and so on). In some cases, it is convenient to write a program that can execute different SQL queries or updates (or other operations) dynamically at runtime. For example, we may want to write a program that accepts an SQL query typed from the monitor, executes it, and displays its result, such as the interactive interfaces available for most relational DBMSs. Another example is when a user-friendly interface generates SQL queries dynamically for the user based on user input through a Web interface or mobile App. In this section, we give a brief overview of dynamic SQL, which is one technique for writing this type of database program, by giving a simple example to illustrate how dynamic SQL can work. In Section 10.3, we will describe another approach for dealing with dynamic queries using function libraries or class libraries.

Program segment E3 in Figure 10.4 reads a string that is input by the user (that string should be an SQL update command in this example) into the string program variable `sqlupdatestring` in line 3. It then prepares this as an SQL command in line 4 by associating it with the SQL variable `sqlcommand`. Line 5 then executes the command. Notice that in this case no syntax check or other types of checks on the command are possible at compile time, since the SQL command is not available until runtime. This contrasts with our previous examples of embedded SQL, where the query could be checked at compile time because its text was in the program source code.

In E3, the reason for separating PREPARE and EXECUTE is that if the command is to be executed multiple times in a program, it can be prepared only once. Preparing the command generally involves syntax and other types of checks by the system, as

//Program Segment E3: Figure 10.4

0) EXEC SQL BEGIN DECLARE SECTION ; Program segment E3, a C program segment

1) `varchar sqlupdatestring [256]` ; that uses dynamic SQL for updating a table.

2) EXEC SQL END DECLARE SECTION ;

3) `prompt("Enter the Update Command: ", sqlupdatestring)` ;

4) EXEC SQL PREPARE `sqlcommand` FROM `:sqlupdatestring` ;

5) EXEC SQL EXECUTE `sqlcommand` ;

10.2 Embedded SQL, Dynamic SQL, and SQL J 351

well as generating the code for executing it. It is possible to combine the PREPARE and EXECUTE commands (lines 4 and 5 in E3) into a single statement by writing EXEC SQL EXECUTE IMMEDIATE `:sqlupdatestring` ;

This is useful if the command is to be executed only once. Alternatively, the programmer can separate the two statements to catch any errors after the PREPARE statement as in E3.

Although including a dynamic update command is relatively straightforward in dynamic SQL, a dynamic retrieval query is much more complicated. This is because

the programmer does not know the types or the number of attributes to be retrieved by the SQL query when writing the program. A complex data structure is needed to allow for different numbers and types of attributes in the query result if no prior information is known about the dynamic query. Techniques similar to those that we shall discuss in Section 10.3 can be used to assign retrieval query results (and query parameters) to host program variables.

10.2.4 SQLJ: Embedding SQL Commands in Java

In the previous subsections, we gave an overview of how SQL commands can be embedded in a traditional programming language, using the C language in our examples. We now turn our attention to how SQL can be embedded in an object-oriented programming language,⁸ in particular, the Java language. SQLJ is a standard that has been adopted by several vendors for embedding SQL in Java.

Historically, SQLJ was developed after JDBC, which is used for accessing SQL databases from Java using class libraries and function calls. We discuss JDBC in Section 10.3.2. In this section, we focus on SQLJ as it is used in the Oracle RDBMS. An SQLJ translator will generally convert SQL statements into Java, which can then be executed through the JDBC interface. Hence, it is necessary to install a JDBC driver when using SQLJ.⁹ In this section, we focus on how to use SQLJ concepts to write embedded SQL in a Java program.

Before being able to process SQLJ with Java in Oracle, it is necessary to import several class libraries, shown in Figure 10.5. These include the JDBC and IO classes (lines 1 and 2), plus the additional classes listed in lines 3, 4, and 5. In addition, the program must first connect to the desired database using the function call `getConnection`, which is one of the methods of the oracle class in line 5 of Figure 10.5. The format of this function call, which returns an object of type `default context`,¹⁰ is as follows:

```
public static DefaultContext
getConnection(String url, String user, String password,
Boolean autoCommit)
throws SQLException ;
```

8

This section assumes familiarity with object-oriented concepts (see Chapter 12) and basic Java concepts.

9

We discuss JDBC drivers in Section 10.3.2.

10

A default context, when set, applies to subsequent commands in the program until it is changed.

352 Chapter 10 Introduction to SQL Programming Techniques

1) import java.sql.* ; Figure 10.5

2) import java.io.* ; Importing classes needed for including

3) import sqlj.runtime.* ; SQLJ in Java programs in Oracle, and

4) import sqlj.runtime.ref.* ; establishing a connection

and default context.

5) import oracle.sqlj.runtime.* ;

6) DefaultContext cntxt =

7) oracle.getConnection("<url name>", "<user name>", "<password>", true) ;

8) DefaultContext.setDefaultContext(cntxt) ;

For example, we can write the statements in lines 6 through 8 in Figure 10.5 to connect to an Oracle database located at the url <url name> using the login of <user name> and <password> with automatic commitment of each command,¹¹ and then set this connection as the default context for subsequent commands.

In the following examples, we will not show complete Java classes or programs since it is not our intention to teach Java. Rather, we will show program segments that illustrate the use of SQLJ. Figure 10.6 shows the Java program variables used in our examples. Program segment J1 in Figure 10.7 reads an employee's Ssn and prints some of the employee's information from the database.

Notice that because Java already uses the concept of exceptions for error handling, a special exception called `SQLException` is used to return errors or exception conditions after executing an SQL database command. This plays a similar role to `SQLCODE` and `SQLSTATE` in embedded SQL. Java has many types of predefined exceptions. Each Java operation (function) must specify the exceptions that can be thrown—that is, the exception conditions that may occur while executing the Java code of that operation. If a defined exception occurs, the system transfers control to the Java code specified for exception handling. In J1, exception handling for an `SQLException` is specified in lines 7 and 8. In Java, the following structure

```
try {<operation>} catch (<exception>) {<exception handling
code>} <continuation code>
```

Figure 10.6 1) string dname, ssn , fname, fn, lname, ln,

Java program variables bdate, address ;

used in SQLJ examples 2) char sex, minit, mi ;

J1 and J2. 3) double salary, sal ;

4) integer dno, dnumber ;

11

Automatic commitment roughly means that each command is applied to the database after it is executed. The alternative is that the programmer wants to execute several related database commands

and then commit them together. We discuss commit concepts in Chapter 20 when we describe database

transactions.

10.2 Embedded SQL, Dynamic SQL, and SQL J 353

//Program Segment J1: Figure 10.7

1) ssn = readEntry("Enter a Social Security Number: ") ; Program segment J1,

```

2) try { a Java program
3) #sql { SELECT Fname, Minit, Lname, Address, Salary segment with SQLJ.
4) INTO :fname, :minit, :lname, :address, :salary
5) FROM EMPLOYEE WHERE Ssn = :ssn} ;
6) } catch (SQLException se) {
7) System.out.println("Social Security Number does not exist: " + ssn) ;
8) Return ;
9) }
10) System.out.println(fname + " " + minit + " " + lname + " " + address
+ " " + salary)

```

is used to deal with exceptions that occur during the execution of <operation>. If no exception occurs, the <continuation code> is processed directly. Exceptions that can be thrown by the code in a particular operation should be specified as part of the operation declaration or interface—for example, in the following format:

```

<operation return type> <operation name> (<parameters>)
throws SQLException, IOException ;

```

In SQLJ, the embedded SQL commands within a Java program are preceded by #sql, as illustrated in J1 line 3, so that they can be identified by the preprocessor. The #sql is used instead of the keywords EXEC SQL that are used in embedded SQL with the C programming language (see Section 10.2.1). SQLJ uses an INTO clause—similar to that used in embedded SQL—to return the attribute values retrieved from the database by an SQL query into Java program variables. The program variables are preceded by colons (:) in the SQL statement, as in embedded SQL.

In J1 a single tuple is retrieved by the embedded SQLJ query; that is why we are able to assign its attribute values directly to Java program variables in the INTO clause in line 4 in Figure 10.7. For queries that retrieve many tuples, SQLJ uses the concept of an iterator, which is similar to a cursor in embedded SQL.

10.2.5 Processing Query Results in SQLJ Using Iterators

In SQLJ, an iterator is a type of object associated with a collection (set or multiset) of records in a query result.¹² The iterator is associated with the tuples and attributes that appear in a query result. There are two types of iterators:

1. A named iterator is associated with a query result by listing the attribute names and types that appear in the query result. The attribute names must correspond to appropriately declared Java program variables, as shown in Figure 10.6.
2. A positional iterator lists only the attribute types that appear in the query result.

12

We shall discuss iterators in more detail in Chapter 12 when we present object database concepts.

354 Chapter 10 Introduction to SQL Programming Techniques

In both cases, the list should be in the same order as the attributes that are listed in

the SELECT clause of the query. However, looping over a query result is different for the two types of iterators. First, we show an example of using a named iterator in Figure 10.8, program segment J2A. Line 9 in Figure 10.8 shows how a named iterator type Emp is declared. Notice that the names of the attributes in a named iterator type must match the names of the attributes in the SQL query result. Line 10 shows how an iterator object e of type Emp is created in the program and then associated with a query (lines 11 and 12).

When the iterator object is associated with a query (lines 11 and 12 in Figure 10.8), the program fetches the query result from the database and sets the iterator to a position before the first row in the result of the query. This becomes the current row for the iterator. Subsequently, next operations are issued on the iterator object; each next moves the iterator to the next row in the result of the query, making it the current row. If the row exists, the operation retrieves the attribute values for that row into the corresponding program variables. If no more rows exist, the next operation returns NULL, and can thus be used to control the looping. Notice that the named iterator does not need an INTO clause, because the program variables corresponding to the retrieved attributes are already specified when the iterator type is declared (line 9 in Figure 10.8).

In Figure 10.8, the command (e.next()) in line 13 performs two functions: It gets the next tuple in the query result and controls the WHILE loop. Once the

Figure 10.8

Program segment J2A, a Java program segment that uses a named iterator to print employee information in a particular department.

//Program Segment J2A:

```
0) dname = readEntry("Enter the Department Name: ") ;
1) try {
2) #sql { SELECT Dnumber INTO :dnumber
3) FROM DEPARTMENT WHERE Dname = :dname} ;
4) } catch (SQLException se) {
5) System.out.println("Department does not exist: " + dname) ;
6) Return ;
7) }
8) System.out.println("Employee information for Department: " + dname) ;
9) #sql iterator Emp(String ssn, String fname, String minit, String lname,
double salary) ;
10) Emp e = null ;
11) #sql e = { SELECT ssn, fname, minit, lname, salary
12) FROM EMPLOYEE WHERE Dno = :dnumber} ;
13) while (e.next()) {
14) System.out.println(e.ssn + " " + e.fname + " " + e.minit + " " +
```

```
e.lname + " " + e.salary) ;
15) } ;
16) e.close() ;
```

10.2 Embedded SQL, Dynamic SQL, and SQL J 355

program is done with processing the query result, the command `e.close()` (line 16) closes the iterator.

Next, consider the same example using positional iterators as shown in Figure 10.9 (program segment J2B). Line 9 in Figure 10.9 shows how a positional iterator type `Emppos` is declared. The main difference between this and the named iterator is that there are no attribute names (corresponding to program variable names) in the positional iterator—only attribute types. This can provide more flexibility, but it makes the processing of the query result slightly more complex. The attribute types must still be compatible with the attribute types in the SQL query result and in the same order. Line 10 shows how a positional iterator object `e` of type `Emppos` is created in the program and then associated with a query (lines 11 and 12).

The positional iterator behaves in a manner that is more similar to embedded SQL (see Section 10.2.2). A `FETCH <iterator variable> INTO <program variables>` command is needed to get the next tuple in a query result. The first time fetch is executed, it gets the first tuple (line 13 in Figure 10.9). Line 16 gets the next tuple until no more tuples exist in the query result. To control the loop, a positional iterator function `e.endFetch()` is used. This function is automatically set to a value of `TRUE` when the iterator is initially associated with an SQL query (line 11), and is set to `FALSE` each time a fetch command returns a valid tuple from the query result. It is set to `TRUE` again when a fetch command does not find any more tuples. Line 14 shows how the looping is controlled by negation.

Figure 10.9

Program segment J2B, a Java program segment that uses a positional iterator to print employee information in a particular department.

//Program Segment J2B:

```
0) dname = readEntry("Enter the Department Name: ") ;
1) try {
2) #sql { SELECT Dnumber INTO :dnumber
3) FROM DEPARTMENT WHERE Dname = :dname} ;
4) } catch (SQLException se) {
5) System.out.println("Department does not exist: " + dname) ;
6) Return ;
7) }
8) System.out.println("Employee information for Department: " + dname) ;
9) #sql iterator Emppos(String, String, String, String, double) ;
10) Emppos e = null ;
```

```

11) #sql e = { SELECT ssn, fname, minit, lname, salary
12) FROM EMPLOYEE WHERE Dno = :dnumber} ;
13) #sql { FETCH :e INTO :ssn, :fn, :mi, :ln, :sal} ;
14) while (!e.endFetch()) {
15) System.out.println(ssn + " " + fn + " " + mi + " " + ln + " " + sal) ;
16) #sql { FETCH :e INTO :ssn, :fn, :mi, :ln, :sal} ;
17) } ;
18) e.close() ;

```

356 Chapter 10 Introduction to SQL Programming Techniques

10.3 Database Programming with Function

Calls and Class Libraries: SQL/CLI
and JDBC

Embedded SQL (see Section 10.2) is sometimes referred to as a static database programming approach because the query text is written within the program source code and cannot be changed without recompiling or reprocessing the source code. The use of function calls is a more dynamic approach for database programming than embedded SQL. We already saw one dynamic database programming technique—dynamic SQL—in Section 10.2.3. The techniques discussed here provide another approach to dynamic database programming. A library of functions, also known as an application programming interface (API), is used to access the database. Although this provides more flexibility because no preprocessor is needed, one drawback is that syntax and other checks on SQL commands have to be done at runtime. Another drawback is that it sometimes requires more complex programming to access query results because the types and numbers of attributes in a query result may not be known in advance.

In this section, we give an overview of two function call interfaces. We first discuss the SQL Call Level Interface (SQL/CLI), which is part of the SQL standard. This was developed as a standardization of the popular library of functions known as ODBC (Open Database Connectivity). We use C as the host language in our SQL/CLI examples. Then we give an overview of JDBC, which is the call function interface for accessing databases from Java. Although it is commonly assumed that JDBC stands for Java Database Connectivity, JDBC is just a registered trademark of Sun Microsystems (now Oracle), not an acronym.

The main advantage of using a function call interface is that it makes it easier to access multiple databases within the same application program, even if they are stored under different DBMS packages. We discuss this further in Section 10.3.2 when we discuss Java database programming with JDBC, although this advantage also applies to database programming with SQL/CLI and ODBC (see Section 10.3.1).

10.3.1 Database Programming with SQL/CLI Using C as the Host Language

Before using the function calls in SQL/CLI, it is necessary to install the appropriate library packages on the database server. These packages are obtained from the vendor of the DBMS being used. We now give an overview of how SQL/CLI can be used in a C program.¹³ We will illustrate our presentation with the sample program segment CLI1 shown in Figure 10.10.

13

Our discussion here also applies to the C++ and C# programming languages, since we do not use any of the object-oriented features but focus on the database programming mechanism.

10.3 Database Programming with Function Calls and Class Libraries: SQL/CLI and JDBC 357

//Program CLI1:

```
0) #include sqlcli.h ;
1) void printSal() {
2) SQLHSTMT stmt1 ;
3) SQLHDBC con1 ;
4) SQLHENV env1 ;
5) SQLRETURN ret1, ret2, ret3, ret4 ;
6) ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &env1) ;
7) if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, env1, &con1) else exit ;
8) if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz",
SQL_NTS) else exit ;
9) if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &stmt1) else exit ;
10) SQLPrepare(stmt1, "select Lname, Salary from EMPLOYEE where Ssn = ?",
SQL_NTS) ;
11) prompt("Enter a Social Security Number: ", ssn) ;
12) SQLBindParameter(stmt1, 1, SQL_CHAR, &ssn, 9, &fetchlen1) ;
13) ret1 = SQLExecute(stmt1) ;
14) if (!ret1) {
15) SQLBindCol(stmt1, 1, SQL_CHAR, &lname, 15, &fetchlen1) ;
16) SQLBindCol(stmt1, 2, SQL_FLOAT, &salary, 4, &fetchlen2) ;
17) ret2 = SQLFetch(stmt1) ;
18) if (!ret2) printf(ssn, lname, salary)
19) else printf("Social Security Number does not exist: ", ssn) ;
20) }
21) }
```

Figure 10.10

Program segment CLI1, a C program segment with SQL/CLI.

Handles to environment, connection, statement, and description records. When using SQL/CLI, the SQL statements are dynamically created and passed as string parameters in the function calls. Hence, it is necessary to keep track of the information about host program interactions with the database in runtime data structures because the database commands are processed at runtime. The

information is kept in four types of records, represented as structs in C data types. An environment record is used as a container to keep track of one or more database connections and to set environment information. A connection record keeps track of the information needed for a particular database connection. A statement record keeps track of the information needed for one SQL statement. A description record keeps track of the information about tuples or parameters—for example, the number of attributes and their types in a tuple, or the number and types of parameters in a function call. This is needed when the programmer does not know this information about the query when writing the program. In our examples, we assume that the programmer knows the exact query, so we do not show any description records.

358 Chapter 10 Introduction to SQL Programming Techniques

Each record is accessible to the program through a C pointer variable—called a handle to the record. The handle is returned when a record is first created. To create a record and return its handle, the following SQL/CLI function is used:

```
SQLAllocHandle(<handle_type>, <handle_1>, <handle_2>)
```

In this function, the parameters are as follows:

■ ■ **<handle_type>** indicates the type of record being created. The possible values for this parameter are the keywords `SQL_HANDLE_ENV`, `SQL_HANDLE_DBC`, `SQL_HANDLE_STMT`, or `SQL_HANDLE_DESC`, for an environment, connection, statement, or description record, respectively.

■ ■ **<handle_1>** indicates the container within which the new handle is being created. For example, for a connection record this would be the environment within which the connection is being created, and for a statement record this would be the connection for that statement.

■ ■ **<handle_2>** is the pointer (handle) to the newly created record of type **<handle_type>**.

Steps in a database program. When writing a C program that will include database calls through SQL/CLI, the following are the typical steps that are taken. We illustrate the steps by referring to the example CLI1 in Figure 10.10, which reads a Social Security number of an employee and prints the employee's last name and salary.

1. Including the library of functions. The library of functions comprising SQL/CLI must be included in the C program. This is called `sqlcli.h`, and is included using line 0 in Figure 10.10.
2. Declaring handle variables. Declare handle variables of types `SQLHSTMT`, `SQLHDBC`, `SQLHENV`, and `SQLHDESC` for the statements, connections, environments, and descriptions needed in the program, respectively (lines 2 to 4).¹⁴ Also declare variables of type `SQLRETURN` (line 5) to hold the return codes from the SQL/CLI function calls. A return code of 0 (zero) indicates successful execution of the function call.

3. Environment record. An environment record must be set up in the program using `SQLAllocHandle`. The function to do this is shown in line 6. Because an environment record is not contained in any other record, the parameter `<handle_1>` is the NULL handle `SQL_NULL_HANDLE` (NULL pointer) when creating an environment. The handle (pointer) to the newly created environment record is returned in variable `env1` in line 6.

4. Connecting to the database. A connection record is set up in the program using `SQLAllocHandle`. In line 7, the connection record created has the handle `con1` and is contained in the environment `env1`. A connection is then established in `con1` to a particular server database using the `SQLConnect`

14

To keep our presentation simple, we will not show description records here.

10.3 Database Programming with Function Calls and Class Libraries: SQL/CLI and JDBC 359

function of SQL/CLI (line 8). In our example, the database server name we are connecting to is `dbms` and the account name and password for login are `js` and `xyz`, respectively.

5. Statement record. A statement record is set up in the program using `SQLAllocHandle`. In line 9, the statement record created has the handle `stmt1` and uses the connection `con1`.

6. Preparing an SQL statement and statement parameters. The SQL statement is prepared using the SQL/CLI function `SQLPrepare`. In line 10, this assigns the SQL statement string (the query in our example) to the statement handle `stmt1`. The question mark (?) symbol in line 10 represents a statement parameter, which is a value to be determined at runtime—typically by binding it to a C program variable. In general, there could be several parameters in a statement string. They are distinguished by the order of appearance of the question marks in the statement string (the first ? represents parameter 1, the second ? represents parameter 2, and so on). The last parameter in `SQLPrepare` should give the length of the SQL statement string in bytes, but if we enter the keyword `SQL_NTS`, this indicates that the string holding the query is a NULL-terminated string so that SQL can calculate the string length automatically. This use of `SQL_NTS` also applies to other string parameters in the function calls in our examples.

7. Binding the statement parameters. Before executing the query, any parameters in the query string should be bound to program variables using the SQL/CLI function `SQLBindParameter`. In Figure 10.10, the parameter (indicated by ?) to the prepared query referenced by `stmt1` is bound to the C program variable `ssn` in line 12. If there are `n` parameters in the SQL statement, we should have `n` `SQLBindParameter` function calls, each with a different parameter position (1, 2, ..., `n`).

8. Executing the statement. Following these preparations, we can now execute the SQL statement referenced by the handle stmt1 using the function SQLExecute (line 13). Notice that although the query will be executed in line 13, the query results have not yet been assigned to any C program variables.

9. Processing the query result. In order to determine where the result of the query is returned, one common technique is the bound columns approach. Here, each column in a query result is bound to a C program variable using the SQLBindCol function. The columns are distinguished by their order of appearance in the SQL query. In Figure 10.10 lines 15 and 16, the two columns in the query (Lname and Salary) are bound to the C program variables lname and salary, respectively.¹⁵

15

An alternative technique known as unbound columns uses different SQL/CLI functions, namely SQLGetCol or SQLGetData, to retrieve columns from the query result without previously binding them;

these are applied after the SQLFetch command in line 17.

360 Chapter 10 Introduction to SQL Programming Techniques

10. Retrieving column values. Finally, in order to retrieve the column values into the C program variables, the function SQLFetch is used (line 17). This function is similar to the FETCH command of embedded SQL. If a query result has a collection of tuples, each SQLFetch call gets the next tuple and returns its column values into the bound program variables. SQLFetch returns an exception (nonzero) code if there are no more tuples in the query result.¹⁶

As we can see, using dynamic function calls requires a lot of preparation to set up the SQL statements and to bind statement parameters and query results to the appropriate program variables.

In CLI1 a single tuple is selected by the SQL query. Figure 10.11 shows an example of retrieving multiple tuples. We assume that appropriate C program variables have been declared as in Figure 10.1. The program segment in CLI2 reads (inputs) a

//Program Segment CLI2: Figure 10.11

```
0) #include sqlcli.h ; Program segment CLI2, a C program segment
1) void printDepartmentEmps() { that uses SQL/CLI for a query with a collection
2) SQLHSTMT stmt1 ; of tuples in its result.
3) SQLHDBC con1 ;
4) SQLHENV env1 ;
5) SQLRETURN ret1, ret2, ret3, ret4 ;
6) ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &env1) ;
7) if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, env1, &con1) else exit ;
8) if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz",
SQL_NTS) else exit ;
```

```

9) if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &stmt1) else exit ;
10) SQLPrepare(stmt1, "select Lname, Salary from EMPLOYEE where Dno = ?",
SQL_NTS) ;
11) prompt("Enter the Department Number: ", dno) ;
12) SQLBindParameter(stmt1, 1, SQL_INTEGER, &dno, 4, &fetchlen1) ;
13) ret1 = SQLExecute(stmt1) ;
14) if (!ret1) {
15) SQLBindCol(stmt1, 1, SQL_CHAR, &lname, 15, &fetchlen1) ;
16) SQLBindCol(stmt1, 2, SQL_FLOAT, &salary, 4, &fetchlen2) ;
17) ret2 = SQLFetch(stmt1) ;
18) while (!ret2) {
19) printf(lname, salary) ;
20) ret2 = SQLFetch(stmt1) ;
21) }
22) }
23) }
16

```

If unbound program variables are used, SQLFetch returns the tuple into a temporary program area. Each subsequent SQLGetCol (or SQLGetData) returns one attribute value in order. Basically, for each

row in the query result, the program should iterate over the attribute values (columns) in that row. This is

useful if the number of columns in the query result is variable.

10.3 Database Programming with Function Calls and Class Libraries: SQL/CLI and JDBC 361

department number and then retrieves the employees who work in that department. A loop then iterates over each employee record, one at a time, and prints the employee's last name and salary.

10.3.2 JDBC: SQL Class Library for Java Programming

We now turn our attention to how SQL can be called from the Java object-oriented programming language.¹⁷ The class libraries and associated function calls for this access are known as JDBC.¹⁸ The Java programming language was designed to be platform independent—that is, a program should be able to run on any type of computer system that has a Java interpreter installed. Because of this portability, many RDBMS vendors provide JDBC drivers so that it is possible to access their systems via Java programs.

JDBC drivers. A JDBC driver is basically an implementation of the classes and associated objects and function calls specified in JDBC for a particular vendor's RDBMS. Hence, a Java program with JDBC objects and function calls can access any RDBMS that has a JDBC driver available.

Because Java is object-oriented, its function libraries are implemented as classes.

Before being able to process JDBC function calls with Java, it is necessary to import

the JDBC class libraries, which are called `java.sql.*`. These can be downloaded and installed via the Web.¹⁹

JDBC is designed to allow a single Java program to connect to several different databases. These are sometimes called the data sources accessed by the Java program, and could be stored using RDBMSs from different vendors residing on different machines. Hence, different data source accesses within the same Java program may require JDBC drivers from different vendors. To achieve this flexibility, a special JDBC class called the driver manager class is employed, which keeps track of the installed drivers. A driver should be registered with the driver manager before it is used. The operations (methods) of the driver manager class include `getDriver`, `registerDriver`, and `deregisterDriver`. These can be used to add and remove drivers for different systems dynamically. Other functions set up and close connections to data sources.

To load a JDBC driver explicitly, the generic Java function for loading a class can be used. For example, to load the JDBC driver for the Oracle RDBMS, the following command can be used:

```
Class.forName("oracle.jdbc.driver.OracleDriver")
```

17

This section assumes familiarity with object-oriented concepts (see Chapter 11) and basic Java concepts.

18

As we mentioned earlier, JDBC is a registered trademark of Sun Microsystems, although it is commonly

thought to be an acronym for Java Database Connectivity.

19

These are available from several Web sites—for example, at <http://industry.java.sun.com/products/jdbc/drivers>.

362 Chapter 10 Introduction to SQL Programming Techniques

This will register the driver with the driver manager and make it available to the program. It is also possible to load and register the driver(s) needed in the command line that runs the program, for example, by including the following in the command line:

```
-Djdbc.drivers = oracle.jdbc.driver
```

JDBC programming steps. The following are typical steps that are taken when writing a Java application program with database access through JDBC function calls. We illustrate the steps by referring to the example JDBC1 in Figure 10.12, which reads a Social Security number of an employee and prints the employee's last name and salary.

1. Import the JDBC class library. The JDBC library of classes must be imported into the Java program. These classes are called `java.sql.*`, and can be imported using line 1 in Figure 10.12. Any additional Java class

libraries needed by the program must also be imported.

//Program JDBC1: Figure 10.12

```
0) import java.io.* ; Program segment JDBC1,
1) import java.sql.* a Java program segment
... with JDBC.
2) class getEmplInfo {
3) public static void main (String args []) throws SQLException, IOException {
4) try { Class.forName("oracle.jdbc.driver.OracleDriver")
5) } catch (ClassNotFoundException x) {
6) System.out.println ("Driver could not be loaded") ;
7) }
8) String dbacct, passwd, ssn, lname ;
9) Double salary ;
10) dbacct = readentry("Enter database account:") ;
11) passwd = readentry("Enter password:") ;
12) Connection conn = DriverManager.getConnection
13) ("jdbc:oracle:oci8:" + dbacct + "/" + passwd) ;
14) String stmt1 = "select Lname, Salary from EMPLOYEE where Ssn = ?" ;
15) PreparedStatement p = conn.prepareStatement(stmt1) ;
16) ssn = readentry("Enter a Social Security Number: ") ;
17) p.clearParameters() ;
18) p.setString(1, ssn) ;
19) ResultSet r = p.executeQuery() ;
20) while (r.next()) {
21) lname = r.getString(1) ;
22) salary = r.getDouble(2) ;
23) system.out.println(lname + salary) ;
24) } }
25) }
```

10.3 Database Programming with Function Calls and Class Libraries: SQL/CLI and JDBC 363

2. Load the JDBC driver. This is shown in lines 4 to 7. The Java exception in line 5 occurs if the driver is not loaded successfully.

3. Create appropriate variables. These are the variables needed in the Java program (lines 8 and 9).

4. The Connection object. A connection object is created using the getConnection function of the DriverManager class of JDBC. In lines 12 and 13, the Connection object is created by using the function call getConnection(urlstring), where urlstring has the form
jdbc:oracle:<driverType>:<dbaccount>/<password>

An alternative form is

getConnection(url, dbaccount, password)

Various properties can be set for a connection object, but they are mainly related to transactional properties, which we discuss in Chapter 21.

5. The Prepared Statement object. A statement object is created in the program. In JDBC, there is a basic statement class, `Statement`, with two specialized subclasses: `PreparedStatement` and `CallableStatement`. The example in Figure 10.12 illustrates how `PreparedStatement` objects are created and used. The next example (Figure 10.13) illustrates the other type of `Statement` objects. In line 14 in Figure 10.12, a query string with a single parameter—indicated by the `?` symbol—is created in the string variable `stmt1`. In line 15, an object `p` of type `PreparedStatement` is created based on the query string in `stmt1` and using the connection object `conn`. In general, the programmer should use `PreparedStatement` objects if a query is to be executed multiple times, since it would be prepared, checked, and compiled only once, thus saving this cost for the additional executions of the query.

6. Setting the statement parameters. The question mark (`?`) symbol in line 14 represents a statement parameter, which is a value to be determined at run-time, typically by binding it to a Java program variable. In general, there could be several parameters, distinguished by the order of appearance of the question marks within the statement string (first `?` represents parameter 1, second `?` represents parameter 2, and so on), as we discussed previously.

7. Binding the statement parameters. Before executing a `PreparedStatement` query, any parameters should be bound to program variables. Depending on the type of the parameter, different functions such as `setString`, `setInteger`, `setDouble`, and so on are applied to the `PreparedStatement` object to set its parameters. The appropriate function should be used to correspond to the data type of the parameter being set. In Figure 10.12, the parameter (indicated by `?`) in object `p` is bound to the Java program variable `ssn` in line 18. The function `setString` is used because `ssn` is a string variable. If there are `n` parameters in the SQL statement, we should have `n` set ... functions, each with a different parameter position (1, 2, ..., `n`). Generally, it is advisable to clear all parameters before setting any new values (line 17).

364 Chapter 10 Introduction to SQL Programming Techniques

//Program Segment JDBC2: Figure 10.13

0) import java.io.* ; Program segment JDBC2, a Java program

1) import java.sql.* segment that uses JDBC for a query with a ... collection of tuples in its result.

2) class printDepartmentEmps {

3) public static void main (String args [])

throws SQLException, IOException {

4) try { Class.forName("oracle.jdbc.driver.OracleDriver")

```

5) } catch (ClassNotFoundException x) {
6) System.out.println ("Driver could not be loaded") ;
7) }
8) String dbacct, passwr, lname ;
9) Double salary ;
10) Integer dno ;
11) dbacct = readentry("Enter database account:") ;
12) passwr = readentry("Enter password:") ;
13) Connection conn = DriverManager.getConnection
14) ("jdbc:oracle:oci8:" + dbacct + "/" + passwr) ;
15) dno = readentry("Enter a Department Number: ") ;
16) String q = "select lname, salary from EMPLOYEE where Dno = " +
dno.toString() ;
17) Statement s = conn.createStatement() ;
18) ResultSet r = s.executeQuery(q) ;
19) while (r.next()) {
20) lname = r.getString(1) ;
21) salary = r.getDouble(2) ;
22) system.out.println(lname + salary) ;
23) } }
24) }

```

8. Executing the SQL statement. Following these preparations, we can now execute the SQL statement referenced by the object `p` using the function `executeQuery` (line 19). There is a generic function `execute` in JDBC, plus two specialized functions: `executeUpdate` and `executeQuery`. `executeUpdate` is used for SQL insert, delete, or update statements, and returns an integer value indicating the number of tuples that were affected. `executeQuery` is used for SQL retrieval statements, and returns an object of type `ResultSet`, which we discuss next.

9. Processing the `ResultSet` object. In line 19, the result of the query is returned in an object `r` of type `ResultSet`. This resembles a two-dimensional array or a table, where the tuples are the rows and the attributes returned are the columns. A `ResultSet` object is similar to a cursor in embedded SQL and an iterator in SQLJ. In our example, when the query is executed, `r` refers to a tuple before the first tuple in the query result. The `r.next()` function (line 20) moves to the next tuple (row) in the `ResultSet` object and returns `NULL` if there are no more objects. This is used to control the looping. The

10.4 Database Stored Procedures and SQL/PSM 365

programmer can refer to the attributes in the current tuple using various `get ...` functions that depend on the type of each attribute (for example, `getString`, `getInteger`, `getDouble`, and so on). The programmer can

either use the attribute positions (1, 2) or the actual attribute names ("Lname", "Salary") with the get ... functions. In our examples, we used the positional notation in lines 21 and 22.

In general, the programmer can check for SQL exceptions after each JDBC function call. We did not do this to simplify the examples.

Notice that JDBC does not distinguish between queries that return single tuples and those that return multiple tuples, unlike some of the other techniques. This is justifiable because a single tuple result set is just a special case.

In example JDBC1, a single tuple is selected by the SQL query, so the loop in lines 20 to 24 is executed at most once. The example shown in Figure 10.13 illustrates the retrieval of multiple tuples. The program segment in JDBC2 reads (inputs) a department number and then retrieves the employees who work in that department. A loop then iterates over each employee record, one at a time, and prints the employee's last name and salary. This example also illustrates how we can execute a query directly, without having to prepare it as in the previous example. This technique is preferred for queries that will be executed only once, since it is simpler to program. In line 17 of Figure 10.13, the programmer creates a Statement object (instead of PreparedStatement, as in the previous example) without associating it with a particular query string. The query string *q* is passed to the statement object *s* when it is executed in line 18.

This concludes our brief introduction to JDBC. The interested reader is referred to the Web site <http://java.sun.com/docs/books/tutorial/jdbc/>, which contains many further details about JDBC.

10.4 Database Stored Procedures and SQL/PSM

This section introduces two additional topics related to database programming. In Section 10.4.1, we discuss the concept of stored procedures, which are program modules that are stored by the DBMS at the database server. Then in Section 10.4.2 we discuss the extensions to SQL that are specified in the standard to include general-purpose programming constructs in SQL. These extensions are known as SQL/PSM (SQL/Persistent Stored Modules) and can be used to write stored procedures. SQL/PSM also serves as an example of a database programming language that extends a database model and language—namely, SQL—with programming language constructs, such as conditional statements and loops.

10.4.1 Database Stored Procedures and Functions

In our presentation of database programming techniques so far, there was an implicit assumption that the database application program was running on a client

366 Chapter 10 Introduction to SQL Programming Techniques

machine, or more likely at the application server computer in the middle-tier of a three-tier client-server architecture (see Section 2.5.4 and Figure 2.7). In either case, the machine where the program is executing is different from the machine on which

the database server—and the main part of the DBMS software package—is located. Although this is suitable for many applications, it is sometimes useful to create database program modules—procedures or functions—that are stored and executed by the DBMS at the database server. These are historically known as database stored procedures, although they can be functions or procedures. The term used in the SQL standard for stored procedures is persistent stored modules because these programs are stored persistently by the DBMS, similarly to the persistent data stored by the DBMS.

Stored procedures are useful in the following circumstances:

- ■ If a database program is needed by several applications, it can be stored at the server and invoked by any of the application programs. This reduces duplication of effort and improves software modularity.
- ■ Executing a program at the server can reduce data transfer and communication cost between the client and server in certain situations.
- ■ These procedures can enhance the modeling power provided by views by allowing more complex types of derived data to be made available to the database users via the stored procedures. Additionally, they can be used to check for complex constraints that are beyond the specification power of assertions and triggers.

In general, many commercial DBMSs allow stored procedures and functions to be written in a general-purpose programming language. Alternatively, a stored procedure can be made of simple SQL commands such as retrievals and updates. The general form of declaring stored procedures is as follows:

```
CREATE PROCEDURE <procedure name> (<parameters>)
<local declarations>
<procedure body> ;
```

The parameters and local declarations are optional, and are specified only if needed.

For declaring a function, a return type is necessary, so the declaration form is:

```
CREATE FUNCTION <function name> (<parameters>)
RETURNS <return type>
<local declarations>
<function body> ;
```

If the procedure (or function) is written in a general-purpose programming language, it is typical to specify the language as well as a file name where the program code is stored. For example, the following format can be used:

```
CREATE PROCEDURE <procedure name> (<parameters>)
LANGUAGE <programming language name>
EXTERNAL NAME <file path name> ;
```

10.4 Database Stored Procedures and SQL/PSM 367

In general, each parameter should have a parameter type that is one of the SQL data types. Each parameter should also have a parameter mode, which is one of IN,

OUT, or INOUT. These correspond to parameters whose values are input only, output (returned) only, or both input and output, respectively.

Because the procedures and functions are stored persistently by the DBMS, it should be possible to call them from the various SQL interfaces and programming techniques. The CALL statement in the SQL standard can be used to invoke a stored procedure—either from an interactive interface or from embedded SQL or SQLJ.

The format of the statement is as follows:

```
CALL <procedure or function name> (<argument list>) ;
```

If this statement is called from JDBC, it should be assigned to a statement object of type CallableStatement (see Section 10.3.2).

10.4.2 SQL/PSM: Extending SQL for Specifying Persistent

Stored Modules

SQL/PSM is the part of the SQL standard that specifies how to write persistent stored modules. It includes the statements to create functions and procedures that we described in the previous section. It also includes additional programming constructs to enhance the power of SQL for the purpose of writing the code (or body) of stored procedures and functions.

In this section, we discuss the SQL/PSM constructs for conditional (branching) statements and for looping statements. These will give a flavor of the type of constructs that SQL/PSM has incorporated;20 then we give an example to illustrate how these constructs can be used.

The conditional branching statement in SQL/PSM has the following form:

```
IF <condition> THEN <statement list>
ELSEIF <condition> THEN <statement list>
ELSEIF <condition> THEN <statement list>
ELSE <statement list>
END IF ;
```

Consider the example in Figure 10.14, which illustrates how the conditional branch structure can be used in an SQL/PSM function. The function returns a string value (line 1) describing the size of a department within a company based on the number of employees. There is one IN integer parameter, deptno, which gives a department number. A local variable NoOfEmps is declared in line 2. The query in lines 3 and 4 returns the number of employees in the department, and the conditional

We only give a brief introduction to SQL/PSM here. There are many other features in the SQL/PSM standard.

368 Chapter 10 Introduction to SQL Programming Techniques

```
//Function PSM1:
```

```
0) CREATE FUNCTION Dept_size(IN deptno INTEGER)
1) RETURNS VARCHAR [7]
2) DECLARE No_of_emp INTEGER ;
```

```

3) SELECT COUNT(*) INTO No_of_emps
4) FROM EMPLOYEE WHERE Dno = deptno ;
5) IF No_of_emps > 100 THEN RETURN "HUGE"
6) ELSEIF No_of_emps > 25 THEN RETURN "LARGE"
Figure 10.14 7) ELSEIF No_of_emps > 10 THEN RETURN "MEDIUM"

```

Declaring a function in 8) ELSE RETURN "SMALL"

SQL/PSM. 9) END IF ;

branch in lines 5 to 8 then returns one of the values {'HUGE', 'LARGE', 'MEDIUM', 'SMALL'} based on the number of employees.

SQL/PSM has several constructs for looping. There are standard while and repeat looping structures, which have the following forms:

WHILE <condition> DO

<statement list>

END WHILE ;

REPEAT

<statement list>

UNTIL <condition>

END REPEAT ;

There is also a cursor-based looping structure. The statement list in such a loop is executed once for each tuple in the query result. This has the following form:

FOR <loop name> AS <cursor name> CURSOR FOR <query> DO

<statement list>

END FOR ;

Loops can have names, and there is a LEAVE <loop name> statement to break a loop when a condition is satisfied. SQL/PSM has many other features, but they are outside the scope of our presentation.

10.5 Comparing the Three Approaches

In this section, we briefly compare the three approaches for database programming and discuss the advantages and disadvantages of each approach.

4. Embedded SQL Approach. The main advantage of this approach is that the query text is part of the program source code itself, and hence can be checked for syntax errors and validated against the database schema at compile time. This also makes the program quite readable, as the queries are readily visible

10.6 Summary 369

in the source code. The main disadvantages are the loss of flexibility in changing the query at runtime, and the fact that all changes to queries must go through the whole recompilation process. In addition, because the queries are known beforehand, the choice of program variables to hold the query results is a simple task, and so the programming of the application is generally easier. However, for complex applications where queries have to be generated at runtime, the function call approach will be more suitable.

5. Library of Classes and Function Calls Approach. This approach provides more flexibility in that queries can be generated at runtime if needed. However, this leads to more complex programming, as program variables that match the columns in the query result may not be known in advance. Because queries are passed as statement strings within the function calls, no checking can be done at compile time. All syntax checking and query validation has to be done at runtime by preparing the query, and the programmer must check and account for possible additional runtime errors within the program code.
6. Database Programming Language Approach. This approach does not suffer from the impedance mismatch problem, as the programming language data types are the same as the database data types. However, programmers must learn a new programming language rather than use a language they are already familiar with. In addition, some database programming languages are vendor-specific, whereas general-purpose programming languages can easily work with systems from multiple vendors.

10.6 Summary

In this chapter we presented additional features of the SQL database language. In particular, we presented an overview of the most important techniques for database programming in Section 10.1. Then we discussed the various approaches to database application programming in Sections 10.2 to 10.4.

In Section 10.2, we discussed the general technique known as embedded SQL, where the queries are part of the program source code. A precompiler is typically used to extract SQL commands from the program for processing by the DBMS, and replacing them with function calls to the DBMS compiled code. We presented an overview of embedded SQL, using the C programming language as host language in our examples. We also discussed the SQLJ technique for embedding SQL in Java programs. The concepts of cursor (for embedded SQL) and iterator (for SQLJ) were presented and illustrated by examples to show how they are used for looping over the tuples in a query result, and extracting the attribute value into program variables for further processing.

In Section 10.3, we discussed how function call libraries can be used to access SQL databases. This technique is more dynamic than embedding SQL, but requires more complex programming because the attribute types and number in a query result may be determined at runtime. An overview of the SQL/CLI standard was

370 Chapter 10 Introduction to SQL Programming Techniques

presented, with examples using C as the host language. We discussed some of the functions in the SQL/CLI library, how queries are passed as strings, how query parameters are assigned at runtime, and how results are returned to program variables. We then gave an overview of the JDBC class library, which is used with Java, and discussed some of its classes and operations. In particular, the `ResultSet` class

is used to create objects that hold the query results, which can then be iterated over by the next() operation. The get and set functions for retrieving attribute values and setting parameter values were also discussed.

In Section 10.4, we gave a brief overview of stored procedures, and discussed SQL/PSM as an example of a database programming language. Finally, we briefly compared the three approaches in Section 10.5. It is important to note that we chose to give a comparative overview of the three main approaches to database programming, since studying a particular approach in depth is a topic that is worthy of its own textbook.

Review Questions

- 10.1. Mention the two ways in which we can run SQL commands?
- 10.2. What are the different sequences of interaction in Database Application Programming?
- 10.3. What do you mean by a shared variable in embedded SQL? In which part of the program is a shared variable declared?
- 10.4. What is the impedance mismatch problem? Which of the three programming approaches minimizes this problem?
- 10.5. Describe the concept of a cursor and how it is used in embedded SQL.
- 10.6. What is SQLJ used for? Describe the two types of iterators available in SQLJ.

Exercises

- 10.7. Consider the database shown in Figure 1.2, whose schema is shown in Figure 2.1. Write a program segment to read a student's name and print his or her grade point average, assuming that A = 4, B = 3, C = 2, and D = 1 points. Use embedded SQL with C as the host language.
- 10.8. Repeat Exercise 10.7, but use SQLJ with Java as the host language.
- 10.9. Consider the library relational database schema in Figure 6.6. Write a program segment that retrieves the list of books that became overdue yesterday and that prints the book title and borrower name for each. Use embedded SQL with C as the host language.
- 10.10. Repeat Exercise 10.9, but use SQLJ with Java as the host language.

Selected Bibliography 371

- 10.11. Repeat Exercises 10.7 and 10.9, but use SQL/CLI with C as the host language.
- 10.12. Repeat Exercises 10.7 and 10.9, but use JDBC with Java as the host language.
- 10.13. Repeat Exercise 10.7, but write a function in SQL/PSM.
- 10.14. Create a function in PSM that computes the median salary for the EMPLOYEE table shown in Figure 5.5.

Selected Bibliography

There are many books that describe various aspects of SQL database programming. For example, Sunderraman (2007) describes programming on the Oracle 10g

DBMS and Reese (1997) focuses on JDBC and Java programming. Many Web resources are also available.

Paper 3 of 8 — CSC 421 PQ 2025

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES EFFURUN
DEPARTMENT OF COMPUTER SCIENCE
2nd SEMESTER UNDERGRADUATE EXAMINATION (2024/2025 SESSION)
CSC 421: DATA MANAGEMENT II (3 UNITS)

INSTRUCTIONS:

1. Attempt Question 1 and Any Other 3 Questions.
2. Time allowed: 2½ hours.
3. Answer Each Question on a New Page.

Question 1 [Compulsory].

- i) What are the major abstractions used to describe the entity relationship analysis? (2½mks).
- ii) Draw E-R diagram with attributes, cardinality and identifiers for the following:

A company with a unique COMPANY-NAME have COMPANY-ID, LOCATION, and OPERATION purchase 12 cars identified by REG-NO and have TYPE, COLOUR and CHASSIS-NO. The cars can be used to carry several items with a unique ITEM-NO and have ITEM-NAME and DESTINATION. The DATE and TIME for the cars to convey items is recorded. A car can be driven by 2 drivers with a unique DRIVER-ID and have DRIVER-NAME, and RANK. Drivers can win several prizes with a unique PRIZE-NAME, and have PRIZE-NO and AMOUNT. A company can partner with several insurance with unique INSURANCE-NO have INSURANCE-TYPE and DUR. (6mks).

- iii) Convert the segment (DRIVERS and PRIZES) of the E-R diagram in (ii) to relations using any arbitrary data items. (3mks).
- iv) Briefly explain normalization. (2½mks).
- v) Identify the most popular approach to normalizing a relational database schema. (1mks).
- vi) What are the likely anomalies that may exist in a relation scheme? (2mks).
- vii) Consider the following relation.

LOANS				
LOAN-NO.	APPLICANT	APPLICANT-ADDRESS	LOAN TYPE	LOAN-DUR
L1	John	Lagos	Mortgage	6
L2	Lily	Kosofe	Bank	4
L3	John	Lagos	Personal	8
L4	John	Ibadan	Personal	12
L5	John	Port	Adventure	4
L6	Lily	Ibadan	Bank	12
L7	Lily	Kosofe	Personal	6

- a) Identify the relation key and state the current normal form of loans. (2mks).
- b) Show the relations after one or more iteration(a) and state the highest normal form. (6mks).

Question 2.

- i) Briefly explain data integrity in relational database environment. (3mks).
- ii) Identify any five integrity constraints you know. (3mks).
- iii) A given system may support multiple host languages and multiple subordinate languages. If we have multiple users as follows: Users A1 and A2; users B1, B2, and B3; users C1 and C2, respectively. You are required to:
 - (a) Explain the underlined words. (4mks).
 - (b) Show a detailed system architecture. (5mks).

Question 3.

- i) What is a relation? (3mks).
- ii) What are the properties of relations? (2mks).
- iii) What is a transaction? Briefly explain the ACID property a transaction can enforce. (5mks).
- iv) Differentiate between primary and foreign keys. (4mks).

Question 4.

- i) What is concurrency control in DBMS? (4mks).
- ii) Outline the objectives of concurrency control. (3mks).
- iii) Why is concurrency control important? (4mks).
- iv) Outline the main problems in using concurrency control in DBMS and the various techniques you know in addressing concurrency control. (4mks).

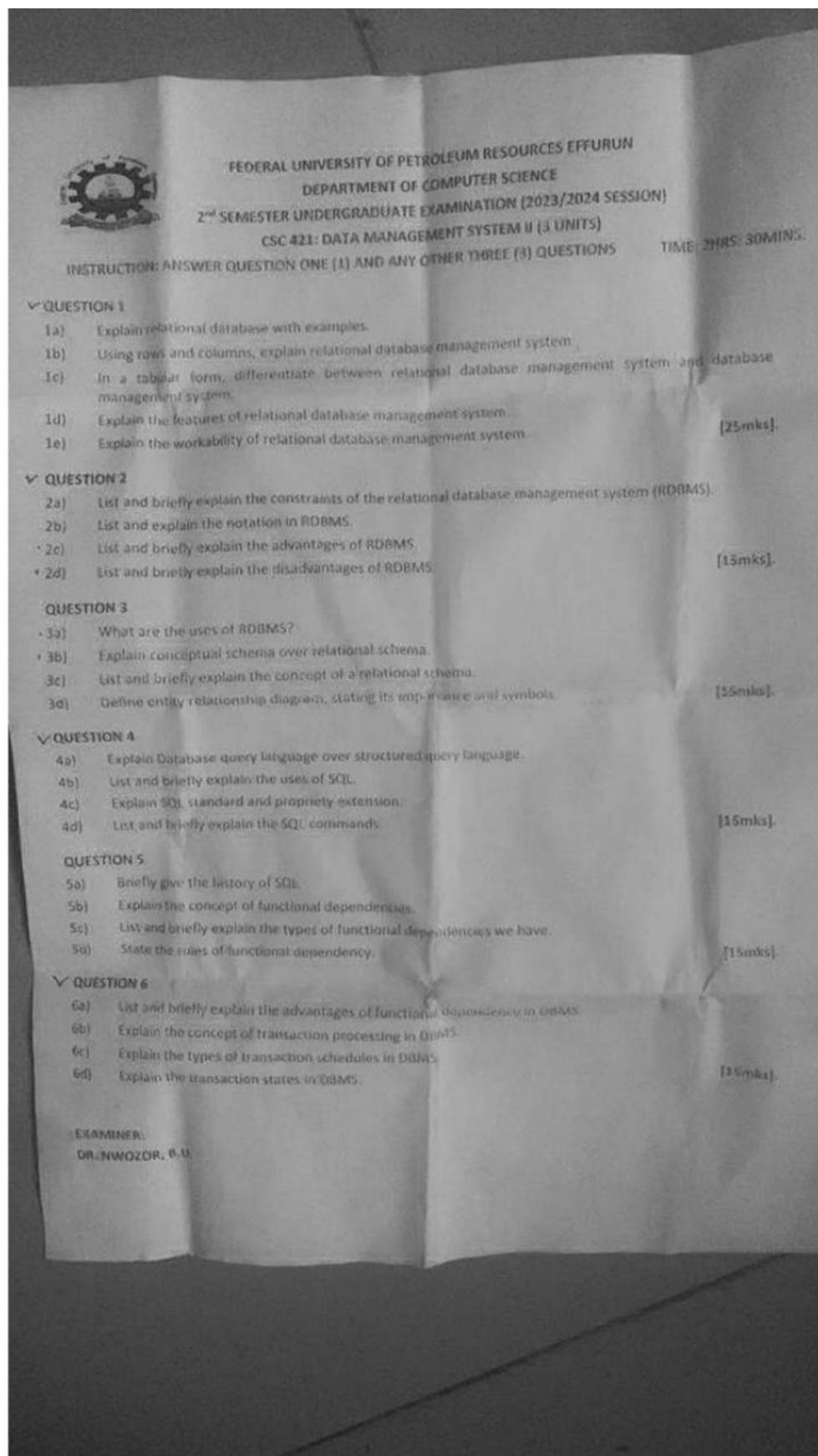
Question 5.

- i) Briefly explain distributed database and outline its functions. (5mks).
- ii) What are the distributed DBMS Design strategies? Hence briefly outline the advantages of any of the distributed DBMS Design strategy you know. (5mks).
- iii) What is distributed DBMS distribution transparency all about? Identify the dimensions of distribution transparency. (5mks).

Paper 4 of 8 — CSC 421 PQ DATA MANAGEMENT 2024

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



Paper 5 of 8 — CSC 421 SQL Lecture Notes 2026

Level 400 • Second Semester • Handout

This source paper is a scanned document; its pages are reproduced below.

CSC421 SQL Lecture Notes

SELECT, WHERE, ORDER BY, COUNT, GROUP BY, HAVING, UPDATE and CRUD

Database Used in the Lecture

The examples in this note assume a PostgreSQL table named students. The table appears to have the following columns:

```
students(
  student_id,
  matric,
  name,
  department,
  level,
  state,
  lga,
  dob,
  gender,
  email,
  phone,
  admission_year,
  attendance_percentage,
  score,
  student_cgpa,
  grade
);
```

In the examples, students is the table name. Columns such as matric, name, state, score and grade are fields stored in the table. Each student record is one row in the table.

Stage	Concept	Classroom Purpose
1	SELECT	Read data from the table.
2	WHERE	Filter rows that meet a condition.
3	ORDER BY	Sort query results.
4	COUNT	Count records.
5	GROUP BY	Summarize records by category.
6	HAVING	Filter summarized groups.
7	UPDATE	Modify existing records.
8	CRUD	Understand the four major data operations: Create, Read, Update and Delete.

1. SELECT

The SELECT statement is used to retrieve data from a database table. It is the basic command for reading data in SQL. When students write SELECT, they are telling the database the columns they want to see and the table where the data should come from.

The SELECT statement does not change the data in the table. It only displays data. This makes it a safe command for beginners.

Basic Syntax

```
SELECT column_name(s)
FROM table_name;
```

CSC421-SQL Practice Notes

1

Example 1: Display All Columns

```
SELECT *  
FROM students;
```

The asterisk means all columns. This command displays every column and every record in the students table. It is useful when students first want to inspect the content of the table.

Example 2: Display Selected Columns

```
SELECT matric, name, score  
FROM students;
```

This displays only the matric number, name and score. It is better than `SELECT *` when the table contains many columns and the lecturer wants the result to be clear.

Example 3: Display Contact Details

```
SELECT matric, name, email, phone  
FROM students;
```

This retrieves the basic contact information of all students in the table.

Classroom Explanation

Students should understand that `SELECT` controls what appears in the output. The `FROM` clause tells PostgreSQL the table where the data is stored. A query is not complete if PostgreSQL does not know the table to read from.

2. WHERE

The `WHERE` clause is used to filter records. Without `WHERE`, SQL returns all rows from the selected table. With `WHERE`, SQL returns only rows that satisfy the stated condition.

`WHERE` is important because real databases may contain thousands or millions of records. It is not practical to inspect all records every time. Filtering helps us focus on the exact records needed.

Basic Syntax

```
SELECT column_name(s)  
FROM table_name  
WHERE condition;
```

Example 1: Students Who Scored Below 40

```
SELECT matric, name, score  
FROM students  
WHERE score < 40;
```

This query displays only students whose score is less than 40. The condition `score < 40` is tested for every row. Rows that satisfy the condition are returned.

Example 2: Students From Delta State

```
SELECT matric, name, state  
FROM students  
WHERE state = 'Delta';
```

This query displays students whose state column contains Delta. Text values in SQL are written inside single quotation marks.

Example 3: Female Students

```
SELECT matric, name, gender
FROM students
WHERE gender = 'Female';
```

This displays all female students in the table.

Example 4: Students With A Grade

```
SELECT matric, name, score, grade
FROM students
WHERE grade = 'A';
```

This displays students who have grade A.

Example 5: Combine Conditions With AND

```
SELECT matric, name, state, score
FROM students
WHERE state = 'Delta'
AND score >= 70;
```

The AND operator means that both conditions must be true. This query displays students from Delta State who also scored 70 or above.

Example 6: Combine Conditions With OR

```
SELECT matric, name, state
FROM students
WHERE state = 'Delta'
OR state = 'Edo';
```

The OR operator means that at least one of the conditions must be true. This query displays students from either Delta State or Edo State.

Example 7: Search for a Name

```
SELECT matric, name, score
FROM students
WHERE name ILIKE '%Dean%';
```

ILIKE is useful in PostgreSQL because it performs case insensitive matching. This means Dean, dean and DEAN can all be found. The percent sign before and after Dean means the word can appear anywhere in the name.

3. ORDER BY

ORDER BY is used to sort the result of a query. Sorting makes the output easier to understand. For example, a lecturer may want to see the highest scores first, the lowest scores first or students arranged alphabetically by name.

Basic Syntax

```
SELECT column_name(s)
FROM table_name
ORDER BY column_name ASC;
```

ASC means ascending order. DESC means descending order. If ASC or DESC is not stated, SQL uses ascending order by default.

Example 1: Highest Scores First

```
SELECT matric, name, score
FROM students
ORDER BY score DESC;
```

This query displays students from the highest score to the lowest score.

Example 2: Lowest Scores First

```
SELECT matric, name, score
FROM students
ORDER BY score ASC;
```

This query displays students from the lowest score to the highest score.

Example 3: Sort by Name

```
SELECT matric, name
FROM students
ORDER BY name ASC;
```

This displays students in alphabetical order by name.

Example 4: Sort by State Then Score

```
SELECT matric, name, state, score
FROM students
ORDER BY state ASC, score DESC;
```

This sorts the records first by state. Within each state, students are sorted from highest score to lowest score.

Classroom Explanation

ORDER BY should normally be placed near the end of the query. It does not choose which records to display. It only arranges the records after they have been selected.

4. COUNT

COUNT is an aggregate function used to count records. It is useful when we want to know how many rows satisfy a condition. COUNT does not list all the records. It returns a number.

Basic Syntax

```
SELECT COUNT(*)
FROM table_name;
```

Example 1: Count All Students

```
SELECT COUNT(*)
FROM students;
```

This counts every row in the students table.

Example 2: Count Students Who Scored Below 40

```
SELECT COUNT(*)
FROM students
WHERE score < 40;
```

This counts students whose score is below 40.

Example 3: Count Female Students

```
SELECT COUNT(*)
FROM students
WHERE gender = 'Female';
```

This counts all female students in the table.

Example 4: Count Students From Delta State

```
SELECT COUNT(*)
FROM students
WHERE state = 'Delta';
```

This counts students from Delta State.

Example 5: Give the Count a Clear Name

```
SELECT COUNT(*) AS total_students
FROM students;
```

The AS keyword gives the output column a more meaningful heading. This is called an alias.

5. GROUP BY

GROUP BY is used to group rows that have the same value in a column. It is often used with aggregate functions such as COUNT, AVG, MIN, MAX and SUM. GROUP BY helps us move from listing individual records to producing summaries.

For example, instead of listing all students, a lecturer may want to know how many students come from each state or the average score for each state.

Basic Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Example 1: Count Students by State

```
SELECT state, COUNT(*)
FROM students
GROUP BY state;
```

This groups students by state and counts how many students are in each state.

Example 2: Count Students by Grade

```
SELECT grade, COUNT(*)
FROM students
GROUP BY grade;
```

This groups students by grade and counts how many students obtained each grade.

Example 3: Count Students by Gender

```
SELECT gender, COUNT(*)
FROM students
GROUP BY gender;
```

This counts how many male and female students are in the table.

Example 4: Average Score by State

```
SELECT state,
ROUND(AVG(score), 2) AS average_score
FROM students
GROUP BY state
ORDER BY average_score DESC;
```

This calculates the average score of students from each state. ROUND is used to display the average score to two decimal places. ORDER BY displays the state with the highest average score first.

Example 5: Highest Score by State

```
SELECT state,
       MAX(score) AS highest_score
FROM students
GROUP BY state
ORDER BY highest_score DESC;
```

This shows the highest score recorded for each state.

When GROUP BY is used, every selected column must either appear in the GROUP BY clause or be used inside an aggregate function. This is a common rule students must understand clearly.

6. HAVING

HAVING is used to filter grouped results. It is similar to WHERE, but WHERE filters individual rows before grouping while HAVING filters groups after grouping.

Students often confuse WHERE and HAVING. A simple way to explain it is this: use WHERE before GROUP BY to filter raw rows, and use HAVING after GROUP BY to filter summarized groups.

Basic Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Example 1: States With More Than 10 Students

```
SELECT state, COUNT(*)
FROM students
GROUP BY state
HAVING COUNT(*) > 10;
```

This groups students by state, counts them and then displays only states that have more than 10 students.

Example 2: Grades Awarded to at Least 20 Students

```
SELECT grade, COUNT(*)
FROM students
GROUP BY grade
HAVING COUNT(*) >= 20;
```

This displays only grades that occur 20 times or more.

Example 3: States With Average Score Above 70

```
SELECT state,
       ROUND(AVG(score), 2) AS average_score
FROM students
GROUP BY state
HAVING AVG(score) > 70
ORDER BY average_score DESC;
```

This displays only states whose average score is greater than 70.

WHERE Versus HAVING

Clause	Used For	Works On	Example
WHERE	Filtering rows before grouping	Individual records	WHERE score < 40
HAVING	Filtering groups after grouping	Grouped summary results	HAVING COUNT(*) > 10

7. UPDATE

UPDATE is used to modify existing records in a table. Unlike SELECT, UPDATE changes the content of the database. Students must learn to use UPDATE with care, especially with the WHERE clause.

Basic Syntax

```
UPDATE table_name
SET column_name = new_value
WHERE condition;
```

Example 1: Add 10 Marks to Students Who Scored Below 40

```
UPDATE students
SET score = score + 10
WHERE score < 40;
```

This increases the score of every student who scored below 40 by 10 marks. The WHERE clause ensures that only students below 40 are affected.

Example 2: Preview Before Updating

```
SELECT matric, name, score,
       score + 10 AS new_score
FROM students
WHERE score < 40;
```

This preview shows what the new score would be before running the UPDATE command. This is a good professional practice.

Example 3: Update One Student by Matric Number

```
UPDATE students
SET score = 65
WHERE matric = 'FUPRE/12345/2022';
```

This changes the score of one specific student identified by matric number.

Example 4: Avoid Scores Above 100

```
UPDATE students
SET score = LEAST(score + 10, 100)
WHERE score < 40;
```

The LEAST function ensures that the score does not go above 100. It returns the smaller of the two values supplied.

Very Important Warning

```
UPDATE students
SET score = 50;
```

This command is dangerous because there is no WHERE clause. It sets the score of every student in the table to 50. Students should be taught never to run UPDATE without checking the WHERE condition unless they truly intend to update every row.

8. CRUD Operations in SQL

CRUD stands for Create, Read, Update and Delete. These are the four main operations performed on data stored in a relational database. A complete database application usually performs all four operations.

CRUD Operation	SQL Command	Meaning	Effect on Database
Create	INSERT	Adds a new record	Changes the database by adding data.
Read	SELECT	Retrieves records	Does not change the database.

Update	UPDATE	Modifies existing records	Changes existing data.
Delete	DELETE	Removes records	Removes data from the database.

Create: INSERT

The INSERT statement is used to add a new record to a table.

```
INSERT INTO students (
    matric,
    name,
    department,
    level,
    state,
    lga,
    dob,
    gender,
    email,
    phone,
    admission_year,
    attendance_percentage,
    score,
    student_cgpa,
    grade
)
VALUES (
    'FUPRE/12345/2022',
    'NEW Student Example',
    'Computer Science',
    400,
    'Delta',
    'Uvwie',
    '2002-01-01',
    'Male',
    'new.student@example.com',
    '08000000000',
    2022,
    75.00,
    60.00,
    3.50,
    'B'
);
```

This command inserts a new student into the students table. The column list tells PostgreSQL the columns receiving values. The VALUES section supplies the actual data.

Read: SELECT

The SELECT statement retrieves existing records from the table.

```
SELECT matric, name, score
FROM students;
```

This reads the matric number, name and score of all students.

Update: UPDATE

The UPDATE statement changes existing records.

```
UPDATE students
SET score = 65
WHERE matric = 'FUPRE/12345/2022';
```

This changes the score of the student with the specified matric number to 65.

CSC421-SQL Practice Notes

Delete: DELETE

The DELETE statement removes records from a table.

```
DELETE FROM students  
WHERE matric = 'FUPRE/12345/2022';
```

This removes the specified student from the students table. DELETE must be used with great caution.

Dangerous DELETE Example

```
DELETE FROM students;
```

This command removes all students from the table because there is no WHERE clause. It should not be used unless the intention is to empty the table.

Practice the following

1. Write a query to display all students.
2. Write a query to display only matric, name and score.
3. Write a query to list students from Edo State.
4. Write a query to list students who scored below 40.
5. Write a query to list students who scored 70 and above.
6. Write a query to arrange students from highest score to lowest score.
7. Write a query to arrange students alphabetically by name.
8. Write a query to count all students in the table.
9. Write a query to count students who obtained grade A.
10. Write a query to count students from Delta State.
11. Write a query to count students by state.
12. Write a query to count students by grade.
13. Write a query to calculate the average score for each state.
14. Write a query to display states with more than 10 students.
15. Write a query to add 10 marks to students who scored below 40, but preview the affected students first.

Paper 6 of 8 — CSC 421 note 2025

Level 400 • Second Semester • Handout

By bright

CSC 421 09/07/2025

The architecture of a system defines its structure. It has three classifications.

1. The internal
2. The conceptual
3. The external

- The Internal level: the one closest to the physical level and it is concerned with the way data is stored.
- The External level: is the one closest to the users and its concerned with the way data is stored.
- The Conceptual level: it is a level of indirection between the internal and the external. It is concerned with the global view of the database+.

DIAGRAM

Host Lang. + Host Lang. + Host Lang. + Host Lang. + Host Lang. +
OSL DSL DSL DSL DSL

External View

B

External View

A

Conceptual / External Mapping

Conceptual Database

View manageme

nt system.

(DBMS)

Conceptual or Internal Mapping

Internal View.

Detailed Architectural Database System

EXTERNAL LEVEL

The external level is the individual user level, each user have a language at his/her disposal. For the application programmer, that language at the disposal of the user could be either one of the conventional programing language such as Dbase, COBOL, Pascal, C++ or a proprietary language e.g.

Dbase.

By bright

For the end user, the language will either be a query language or a special purpose language mainly

driven and online application program. Note that such language will include Data Sub Language

(DSL). It is specifically with database objects and operations. The DSL is embedded within the corresponding host language. The host language is charged with responsibility of providing various non-database facilities such as local variables, computational operations, if then else login. A given system may support multiple host language and multiple if the else language.

Data Sub Language

It is the combination of two languages: DDL (Data definition language) and DML (Data manipulation language). DDL provides definitions of database audits.

DML supports the manipulation or processing of objects. The DSL is defined in terms of external records. Each external view is defined by means of external schema which consists basically of definitions of each of various types of external records in that external view. The external schema is written using the DDL portion of the DSL

CONCEPTUAL LEVEL

The conceptual level is a representation of the entire information context of the database. It consists

of multiple occurrences of multiple types of conceptual records. A conceptual record is necessary the same as either an external or internal record.

The conceptual view is defined by means of the conceptual schema which includes definition of each

of the various types of conceptual schema. The conceptual schema is written using the conceptual DDL. Note: the conceptual schemas are not mere sets of definitions only many additional features are included such as security and integrity checks.

INTERNAL LEVEL

The internal level is a low level representation of the entire database, it comprises multiple occurrences of multiple types of internal records. The internal view is described by means of internal

schema which defines the various types of stored records and specifies what indexes exist, how stored fields are represented the internal schema is written using another yet internal DDL.

Mapping: there are two levels of mapping

A. The conceptual/external mapping

B. Conceptual/internal

External/conceptual mapping: this mapping defines the correspondence between a particular external view and the conceptual view. It specifies how external records are represented for the various conceptual views.

Conceptual/internal mapping: it describes the correspondence between the conceptual view and the

stored database (internal view). It specifies how conceptual records and fields are represented at the

internal view. If a change is made on the internal schema, then the conceptual internal mapping must also be changed accordingly.

DATABASE MANAGEMENT SYSTEM (DBMS): is the software that handles all access to the database.

Conceptually, what happens is:

1. A user issues an access request using some particular sub language e.g. SQL

By bright

2. The DBMS intercepts that request from the user and analyses it
3. The DBMS inspects in turn the external schema for that user, the corresponding external/conceptual mapping, the conceptual schema, the corresponding mapping and the external schema.
4. The database management system executes the necessary instructions on the internal view or the stored database.

Next class: Data model

CSC 421 29/7/25

Data Models

There are different types of data

Hierarchical model

Network model

Relational model.

- The Hierarchical model: the basic construct of the hierarchical data model is a parent child type of relationship between pairs of record types. It is an ordered set consisting of multiple occurrences of a single type of tree. (Diagram).

Limitations of hierarchical model

1. Since each record may have only one parent, many to many relationship may not be directly represented.
2. Partial information is difficult to represent since it is only the explicit linkage to its parents that renders the record accessible, which means ghost/imaginary records has to be created when information is available only about the child and not the parent.

- The network data model can be regarded as an extension of the hierarchical data model, in this case, where a child record can have more than one parent.

A B C

D E F G

Limitations of the Network Model

By bright

1. The model does not clearly separates the logical representation from its storage structure
2. Processing is record at a time and not set based.

- Relational data model: to overcome the major limitations of the hierarchical and the network data models, the relational model was introduced as a tool to free users from having to deal with the complexities of storage structure and access strategy. A relational database is a collection of relations. A relation is a table or a matrix, each row of a relation is referred to as a

tuple and each column is referred to as attribute.

Properties of Relations

1. There are no duplicate tuples, (no repetitions or duplications).
2. Tuples are unordered top to bottom.
3. Attributes are unordered from left to right.
4. All attribute values are atomic

Integrity in relational database

Integrity of data has to do with ensuring that what a user is attempting to do is valid (correct). In other words, it has to do with accuracy or validity. Integrity checks can be ensured by specifying integrity constraints as part of the database definition. There are several integrity constraints that are available in performing integrity checks. These constraints are done during data entry.

1. Format constraints: a format constraint might specify that the value of a field conforms to the format 99-99-99.
2. Range constraints: a range constraints might specify that the value of a field/attribute are required to be in the range 0 – 20,000 or 0 – 100, whichever is preferred.
3. The Numeric, character constraints.
4. Functional, multi-values, and joined dependencies.
5. Primary and Foreign Key constraints.

Keys in Relational Database

Primary keys are usually a special case of more general constructs called candidate keys, a primary key is a unique identifier that is used to order a relation in ascending or descending order.

S

S# Name Age

S1

S2

S3

P

P# Item Amt

P1

P2

P3

SP

By bright

SP# P# Qty

A Relational Database.

A foreign key is an attribute in one relation, relation R2, whose values are required to match those of

the primary key of some relation, (R1). There are two integrity rules:

1. Entity Integrity
2. Referential Integrity

Data Modelling

This is made up of two levels

1. The first level of data modelling: Analysts develop a conceptual modelling, these model represent the major data objects and relationships between them.
2. The second level of data modelling: this level organizes the model into a proper shape usually by removing redundancies by a method or technique known as normalization.

The Conceptual Modelling: a conceptual model describes the essential features of system data, in this course, we're going to use symbols from a modelling model called an Entity relationship analysis.

Entity Relationship analysis uses three major abstractions to describe data, this includes

1. Entities: these are distinct things in the enterprise
2. Relationship: these are meaningful interactions between the objects or entities
3. Attributes: these are the properties of the entities and relationships.

By bright

We can give an ideal entity relation analysis model as follows:

Person Set

Work Set

Project Set

Occurrence Diagram

By bright

The E.R diagram does not represent individual entities and relationships, rather they represent the entity and relationship sets. The E.R Diagram uses some symbols during representation.

□ This is used to represent entities and the name is inscribed inside

□ This is used to represent the relationship inscribed inside.

Next class: Building Entity relationship models.

By bright

CSC 421 05/08/2025

Building Entity Relationship models

Person ID Persons

Name

Age

Person ID

Project ID Works

Project ID

Budget Projects

The person entity can have a relationship (works) with another entity (Projects), while at the same time, have another relationship with a different entity. An entity can have multiple relationships with other entities.

Choosing Object set name: in an E.R Diagram, Entity sets are normally labelled by nouns and relationships are labelled by verbs.

Abstractions of Entity relationship analysis: (Attributes): they are mainly identifiers.

Modelling Relationship cardinality: on the E.R diagram, if we show the number of relationships in which an entity can appear, this value is called relationship cardinality.

Adding Cardinality in E.R Diagram: by convention, the letter N and M are used in E.R diagram to represent many/several respectively

Exercise

Draw E.R Diagram with attributes, cardinality and identifiers as follows: Customers identified by a CUSTOMER_NAME and with an ADDRESS buys Items. Item are identified by an ITEM_NO and have a COLOR. The QTY_BOUGHT of an item by each customer is recorded. An item can be bought by many

customers. (The underlined are entities and the Upper Case words are attributes).

By bright

Solution

CUSTOMER_NAME

Customers ADDRESS

N

CUSTOMER_NAME

ITEM_NO

Buy QTY_BOUGHT

M

ITEM_NO

Items COLOR

Relationships: The above E.R diagram can be converted to relations:

Customer

Customer Address

Name

Buy

Customer Item Qty

Name No Bought

Items

Item No Color

Normalization

Many relations are non-normal, so they goal is to make the relation good, to sharpen the relation to

the proper shape. Normalization is a step by step reversible process of replacing a given collection of relations by successive collection in which relations have a progressively simpler and more regular structure. By bright

structure. The aim of normalization is to eliminate various anomalies of a relation in order to obtain better relations. The problems associated with relations are as follows

1. Repetition anomaly
2. Update anomaly
3. Insertion anomaly
4. Deletion anomaly.

There are several approaches but the most popular approach to normalizing a relational database scheme is the decomposition approach/techniques. This approach begins with a single relation which is normally called the universal relation which contains all attributes and probably anomalies and can undergo several iterations. At each iteration, a relation is split into two or more relations of a higher normal form

NORMAL RELATION

A number of normal form have been defined for relation.

1NF

2NF Codd

3NF

BCNF Boyce-Codd

4NF

5NF Fagin

The first, second, and third normal forms are defined by a researcher called CODD, every normalized relation is in first normal form.

Next class: First Normal Form – Fifth Normal Form.

By bright

CSC 421 06/08/2025

Normal –Form Relatives

Normalization: it is a step by step reversible process of replacing a given collection of relations by successive collective in which relation have a progressively simple and more regular structure. The aim of normalization is to eliminate various anomalies of a relation in order to obtain better relations.

The problems associated with relations are:

1. Repetition Anomaly
2. Update anomaly
3. Insertion anomaly

4. Deletion anomaly

The most popular approach to normalizing a relation database scheme is the decomposition approach or technique that begins with a single relation is normally called “a universal relation” a relation is split into two or more relations of a higher normal form. A relation must have a name and could have anomalies. If anomalies exists, the need to decompose arises to split that relation into two relations to remove anomalies, if the anomalies still exists after split, split further until anomalies are completely removed giving rise to a completely normal form.

N.B. Each split is called an iteration and each iteration results in a better form of a relation.

Diagram

Normal Form Database

A number of normal form relations has been identified (defined) for relations.

From the first normal form to the fifth normal form

1NF (First normal form)

2NF (Second normal form)

3 NF (Third normal form)

Boyce – Codd NF (BCNF)

4NF (Fourth normal form)

5NF (Fifth normal form)

The first, second and third normal forms are defined by a researcher “Codd”, the BCNF was defined by “Boyce-Codd” and is an extension of the third normal form and the fourth and fifth normal form were defined by Fagin. There exists a hierarchy relationship amongst these normal forms. The higher

normal forms have better relations and properties than others with respect to the 4 anomalies mentioned (i.e. 5NF > 4NF > 3NF > 2NF > 1NF) every normalized relation is in the first normal form.

By bright

Normal Form and Non-Normal Form

The difference between the two will be illustrated with the following relation.

ORDER_NO ORDER_DATE ORDER_LINE

PART_NO QTY_ORDERED

Ord 1 4th august 2025 P1 10

P6 30

Ord 2 5th August, 2025 P3 10

P6 50

P2 30

This is a non – normal form relation, you can convert it to a normal relation as follows:

ORDERS

ORDER_NO ORDER_DATE PART_NO QTY_ORDERED

Ord 1 7th August 2025 P1 10

Ord 1 7th August 2025 P6 30

Ord 2 5th August 2025 P3 10

Ord 2 5th August 2025 P6 50

Ord 2 5th August 2025 P2 30

Any non- normal form relation converted to a normal form in the first normal (1NF)

Dependency Structures

The normal forms are based on certain dependency structures. The:

- 1NF – BCNF are based on functional dependencies
- 4NF is based on multivalued dependencies
- 5NF is based on projection joint dependencies

Functional dependencies describes some of the rules that holds between attributes in a system, they

state whether a value of one attribute x in a relation determines a value/ particular value in another attributes Y for that relation. The symbol “ $\rightarrow$ ” is used to represent functional dependency and is expressed as “ $x \rightarrow y$ ” i.e. x functionally determines y.

Keywords functionality determines non keywords ($KY \rightarrow NKW$)

In “ORDERS”, we can say $ORDER_NO \rightarrow \{ORDER_DATE, PART_NO, \text{ and } QTY_ORDERED\}$

Also derived functional dependencies is not allowed.

If

$X \rightarrow Y$

$Y \rightarrow Z$

Therefore $X \rightarrow Z$

This is not permitted.

Relation keys & Normal form Selections

By bright

The idea of normal form relations revolves around functional dependencies and relation keys

Second Normal Form

Relations where subsets/parts of key attributes determine non-key attributes may be in the first normal form but cannot be in the second normal form. To be in the second normal form (2NF), a relation must not have a non-key attribute that is functionally dependent on parts of the relation key, relations that are not in the second normal form can always be decomposed into the second normal form

Consider relation “ORDERS”, it is in the 1NF

FD: $ORDER_NO \rightarrow ORDER_DATE$

REST: $ORDER_NO | PART_NO | QTY_ORDERED$

$ORDER_1$

ORDER_NO ORDER_DATE

Ord 1 7th August

Ord 2 5th August

ORDER_NO PART_NO QTY_ORDERED

Ord 1 P1 10

Ord 1 P6 30

Ord 2 P3 10

Ord 2 P6 50

--- ---- ---

By bright

CSC 421 13/08/2025

Third normal form (3NF)

It is possible to have relations where non key attributes are functionally dependent on other non-key

attributes. Let's consider a database that models an engineering company and the entities to be modelled are the engineers and the projects. Consider the following relations.

Relation Name: ENG

ENG_2

ENG ENAME TITLE SAL PNO RESP DUR

NO

E1 P. Mary Electrical engr. 4000 P1 Manager 12

E2 J. Jones Analyst 3400 P1 Analyst 24

E2 J. Jones Analyst 3400 P2 Analyst 6

E3 A. Ngozi Mechanical engr. 2700 P3 Consultant 10

E3 A. Ngozi Mechanical engr. 2700 P4 Engineer 48

E4 L. Abia Programmer 2400 P2 Programmer 18

E5 B. Chukwu Analyst 3400 P2 Manager 24

E6 L. Billy Electrical Engr. 4000 P4 Manager 48

E7 R. Davis Mechanical Engr. 2700 P3 Engineer 36

E8 J. Jonny Analyst 3400 P3 Manager 40

This is a normal form relation. The key of this relation is ENO.

Title is a non-key relation that behaves like a key relation so we pull it out to form its own relation:

ENG_1

TITLE SAL

Electrical Engr. 4000

Analyst 3400

Mechanical Engr. 2700

Programmer 2400

FDs: ENO → (ENAME, TITLE)

REST: (ENO|PNO|RESP|DUR)

By bright

ENO ENAME TITLE

E1 P. Mary Electrical Engr.

E2 J. Jones Analyst

E3 A. Ngozi Mechanical Engr.

E4 L. Abia Programmer

E5 B. Chukwu Analyst

E6 L. Billy Electrical Engr.

E7 R. Davis Mechanical engr.

E8 J. Jonny Analyst

The relations are: ENG_1, ENG_2, ENG_2_1 and ENG_2_2 are in the 3NF

OPTIMAL NORMAL FORM (BCNF)

All the relations discussed so far from the first to third normal form had one relation key however normal forms must also cover relations that have more than one relation key. The particular problem with relations that have more than one key is that part of one key may be functionally dependent on part of another key. Consider the following relations:

Relation key: WORK

WORK

PROJECT_ID PERSON_ID MANAGER TIME_SPENT

Proj1 J1 Vick 30

Proj2 J2 Joe 12

Proj1 J2 Vick 11

Proj2 J2 Joe 79

Proj3 J2 Ovie 17

Proj2 J2 Joe 14

PROJECT_ID and PERSON_ID are composite keys in this relation and they are both key attributes.

Functional dependencies (FDs): PROJECT_ID → MANAGER

REST: (PROJECT_ID|PERSON_ID|TIME_SPENT)

WORK_1

PROJECT_ID MANAGER

Proj1 Vick

Proj2 Joe

Proj3 Ovie

The Fourth Normal Form

Consider the relation PERSONS as follows:

By bright

PERSONS

PERSON_ID SKILL PROJECT_ID

Jill Computing Proj1
 Jill French Proj1
 Jill Computing Proj3
 Jill French Proj3
 Mary French Proj1
 Mary Economics Proj1
 John Computing Proj1
 John Computing Proj2

The relation here has some redundancies in the sense that we can store all of Jill's skills once on which each of the project Jill works. This redundancy is not possible in the 4th normal form, however

relation in the 4th normal form should not contain more than one independent multivalued dependency or one independent multivalued dependency together with a functional dependency.

Multi Valued dependency: this exists in a relation when a value of one column or set of attributes X determines a set of values in another attributes or column Y. multi valued dependencies are usually expressed by the symbol ($\twoheadrightarrow$). E.g. $X \twoheadrightarrow Y$.

So to say: $PERSON_ID \twoheadrightarrow SKILL$: this means that a value of $PERSON_ID$ always determine a set of values of $SKILL$.

NEXT CLASS: revisit this class.

By bright

CSC 421 19/08/2025

REVISITING PREVIOUS TOPIC

A multivalued dependency occurs in a relational database when a single attribute has multiple independent values for a given set of other attributes, in other words, a single tuple in a table can have multiple values for a particular attribute and these values are not dependent on each other.

Consider the following relation:

Employee

EMPLOYEE_ID EMPLOYEE_NAME SKILL

1 Mary Computing
 1 Mary Networking
 1 Mary Marketing
 2 Peter Programming
 2 Peter Drawing

In this example, the $SKILL$ attribute has a multivalued dependency on the $EMPLOYEE_ID$ attribute.

Decomposing this relation:

FDs:

Employee_1

EMPLOYEE_ID EMPLOYEE_NAME

1 Mary

2 Peter

Employee_2

EMPLOYEE_ID SKILL

1 Computing

1 Networking

1 Marketing

2 Programmer

2 Drawing

E.g. 2

Suppose we had a relation that stores information about ORDERS, and the products included in each

ORDERS are as follows:

Orders

ORDER_ID CUSTOMER_NAME PRODUCT_ID PRODUCT_NAME

1 Mary A Product A

1 Mary B Product B

2 Peter A Product A

2 Peter C

By bright

In this example, the PRODUCT_ID and the PRODUCT_NAME attributes have a multivalued dependency on the ORDER_ID.

Decomposing:

FDs:

Orders_1

ORDER_ID CUSTOMER_NAME

1 Mary

2 Peter

Orders_1_2

ORDER_ID PRODUCT_ID

1 A

1 B

2 A

2 B

Orders_2_2

ORDER_ID PRODUCT_NAME

1 Product A

1 Product B

2 Product A

2 Product C

The Fifth Normal Form

The fifth normal form is a database normalization technique that deals with joined dependencies. A relation is in fifth normal form if it satisfies the following condition:

1. It is in the fourth normal form. This means the relation is normalized to fourth normal form.
2. There is no joined dependencies. That means the relation does not have any joined dependencies that can be removed by decomposition.

Joined Dependencies: a joined dependency occurs when a relation can be reconstructed from smaller relations by joining them on common attributes. In other words, a joined dependency exists when a relation can be decomposed into smaller relations without losing any information. It is only in the fifth normal form that you can have several decomposition. For example:

BB

AGENT COMPANY PRODUCT

A X P1

A Y P2

B X P1

B Y P2

By bright

BB_1

AGENT COMPANY

A X

A Y

B X

B Y

BB_2

AGENT PRODUCT

A P1

A P2

B P1

B P2

BB_3

COMPANY PRODUCT

X P1

Y P2

By bright

CSC 421 20/08/2025

Database integrity

There are two concepts associated with database integrity:

1. Integrity control/constraints
2. Transaction processing

Integrity control/constraint is a condition or restriction that is applied to a particular set of data, in other words, it's a restriction applied to a given set of data used to minimize data entry error. They are used to minimize data inconsistencies caused when users or application program make errors in

entering or changing data within the database. In relational models, integrity controls may apply to:

1. Individual attributes
2. Relationship between two different attributes
3. The relationship between two tuples of one or more relations

Ideally, the enforcement of integrity control would be carried out by the DBMS automatically, as each new data item is entered.

Transaction processing: a transaction is the execution of program that accesses or changes the contents of the database. It is also an action or a series of actions carried out by a single user or an application program which reads or updates the contents of the database. In other words, a transaction is a logical unit of work that must be either entirely completed or aborted. No intermediate states are accepted. A consistent database state is one in which all integrity constraints are satisfied.

Example of a transaction:

Amount in stock = y

Y = 30 initial state <Consistent state>

Y + 20 □ Transaction T1

Read → modify → write <Modifies database>

Y = 50 finish state <Consistent state>

Properties of Transactions

To ensure consistency completeness of database, there must be some properties that would guide transaction and they are called ACID properties:

By bright

1. Atomicity
2. Consistency
3. Isolation
4. Durability

Atomicity:

- i. This property states that all of the instruction within the transaction must be executed or none of the instruction should be executed.
- ii. All transaction execution must be Atomic
- iii. It involves the following tool operations: ABORT i.e. if a transaction aborts, it means changes made to the database are not visible. COMMIT, i.e. if a transaction commits, changes made in a database is visible.

Consider the following transaction, T1 and T2, transfer of 100 from account X to account Y.

Before $x = 50$ $Y = 200$

Transaction T

T1 T2

Read(X) Read (Y)

$X = X - 100$ $Y = Y + 100$

Write (X) Write (Y)

Acc: $x = 400$ $Y: 300$

If the transaction fails after completion of T1 but before completion of T2, say after write (X) but before write (Y), then amount has been deducted from X but not added to Y. This action results in an inconsistent database state, therefore the transaction must be executed in its entirety in order to ensure correctness of database state.

Consistency: The database must remain in a consistent state even after performing any kind of transaction ensuring correctness of the database, if we execute a particular transaction in isolation or together with other transaction in multi programming environment, the transaction should give same result in any case, each transaction run by itself with no concurrent execution of other transaction must preserve the consistency of the database. This property is called consistency. In other words, consistency means that all data in the database must ultimately be consistent. Even though there might be temporary inconsistencies while a transaction is being processed, in the end the database must be returned to a consistent state.

Isolation: when executing multiple transactions concurrently and trying to access shared resources, the system should create an order that the only one transaction can access the shared resources at the same time and release it after completion of its execution for other transaction, in other words, isolation means that concurrent access problems are prevented. That means changes by one transaction do not result to errors in another transaction.

Durability: this property states that once after the transaction is completed, the changes that have been made should be permanent and should be recoverable even after system failure or crash. These property ensure that once transaction has completed execution, the updates and modifications to the database are stored in and written to disk, and they would remain even if there is a system failure.

By bright

Concurrency control: this is the coordination of simultaneous execution of transactions in a multiprocessing database system. Concurrency control is required to maintain consistency of data. For example, if we take ATM machines and do not adopt concurrency, multiple persons can draw money at a time in different places. The objectives of concurrency control is to ensure the serialization of transactions in a multiuser transaction database environment.

Advantages of concurrency control

1. Waiting time will be decreased
2. Response time will decrease
3. Resource utilization will increase
4. System performance will improve

5. Efficiency will increase.

Importance of concurrency control: Concurrency control is important because the simultaneous execution of transaction over a shared database can create several data integrity and consistency problems.

There are three main problems associated with concurrency controls, these are:

1. Lost updates
2. Uncommitted data
3. Inconsistent retrievals

Next Class: Concurrency Control Techniques

By bright

CSC 421 26/08/2025

Problems Associated With Concurrency Control Techniques

Lost Updates: When two or more transaction read the same piece of data and updates accordingly to the real value, the final value represents only the most recent update. Thereby losing the earlier updates since no transaction can observe the updates performed by previous transaction

Uncommitted Data: When a transaction accesses data that has been modified by another transaction but has not yet been committed, this can lead to uncommitted data issues, this is also referred to as dirty reads. The data that was read by the first transaction becomes invalid if the transaction that made the modification is rolled back, a second transaction might be working with uncommitted data that could be rolled back, for example, if it accesses a record that has been updated by the first transaction before it commits.

Inconsistent retrievals: When a transaction reads the same data again and receives different outputs/results each time due to another transaction modifying the data in between the reads, this is known as inconsistent retrieval or non-repeatable reads. The operation of the transaction may become an anomaly as a result of that transaction. For instance if a transaction reads a collection of

data, another transaction modifies that data and then the first transaction reads the data again and observes different values. This causes an inconsistent retrieval.

Inconsistent data: when multiple users are accessing and modifying the data in a transaction it may lead to inconsistent data states. E.g. if there is inadequate concurrency management and two transactions are modifying the same record at the same time, the final state of the data may only reflect one of the updates which would cause inconsistent results.

Various concurrency Control Techniques

These includes:

1. Two-phase locking (2PL) control protocol
2. Time stamp ordering protocol.
3. Multi-version concurrency control.
4. Validation concurrency control.

1. Two-phase locking protocol: locking is an operation which secures permission to read or

permission to write a data item, two phase locking is a process used to gain ownership of shared resources without creating the possibility of deadlocks. Three activities exist in two-phase update algorithm,

These include:

1. Lock acquisition
2. Modification of data
3. Release lock.

Two phase locking prevents deadlock by guaranteeing a process releases all held locks if it cannot access all needed resources without waiting. This avoids situations where processes wait on each other preventing deadlock. A transaction in two phase locking can either be of the following two phases.

By bright

1. Growing phase: in this phase, a transaction can only acquire locks, but cannot release any locks, the point when a transaction acquires all the locks it needs is referred to as lock wealth.
2. Shrinking phase: in this phase, a transaction can only release lock but cannot acquire any
2. The time stamp ordering protocol: a time stamp is a tag showing when a transaction or data item was last used. It can be assigned using the system clock or a logical counter. Each data item has two time stamps, one for the last read and the other for the last write:

WriteTimeStamp(x) implies the latest time when the data item x has been written into.

ReadTimeStamp(x) implies the latest time when the data item x has been read from.

These two Timestamps are updated each time a successful read/write operation is performed on the data item x.

3. Multi-version concurrency control: this improves concurrency by keeping multiple versions of data. Each write creates a new version with a timestamp, reads access the version matching the transaction timestamp

4. Validation concurrency control: this is an optimistic approach which assumes that conflicts between transactions are rare so it allows transaction to execute freely without using locks or timestamps. Each transaction passes three phases. The first is the read phase, second is the validation phase, and the third phase is the write phase.

- a. The read phase: the transaction reads data and performs all operations using a private copy, any updates are stored in a temporary area not visible to other transactions,
- b. Validation phase: before committing, the transaction is checked to ensure that applying its changes would violate database concurrency.
 - i. If the check passes, it proceeds to the next phase, otherwise if the check fails, the transaction is rolled back and restarted
- c. The write phase: the validated changes are now written permanently to the database.

By bright

Deadlock: deadlock arises when two or more processes have placed locks on data and are waiting

for the other's data.

Process 1

1. Lock data A
3. Wait for data B

Data A Data B

Process 2

2. Lock Data B
4. Wait for Data A

Distributed database management system: a distributed database is a collection of multiple interconnected databases which are spread physically across various locations that communicate via computer networks.

Advantages of distributed Database

1. There is fast data processing as several sites participate in request processing
2. Reliability and availability of these systems is high, that means when failures occur in centralized databases, the system comes to a complete stop, when a component fails in a distributed database system, the system will continue to function at a reduced performance until the error is fixed.
3. It offers reduced operating cost
4. It is easier to expand the system by adding more files thereby encouraging modular development
5. It has increased sharing ability and local autonomy.

Disadvantages of distributed database

1. The system becomes complex to manage and control
2. The security issues must be carefully managed
3. The system require deadlock handling during the transaction processing otherwise the entire system may be in inconsistent state.

By bright

4. It comes with processing overhead, even simple operations may require a large number of communications and additional computation to provide uniformity in data across the sites.
5. Responsiveness of queries is largely dependent upon proper data distribution.
6. The need for updating in multiple sites pose problems of data integrity

(*)Functions of distributed database systems

1. Keeping track of data.
2. Distributed query processing
3. Replicated data management
4. Distributed database recovery
5. Security
6. Distributed directory management

7. Distributed transaction management

Types of distributed database.

Distributed Database Environment

Homogenous Heterogeneous

Autonomous Non-Autonomous Federated Un-federate

Homogenous Database: in homogenous distributed database, all the physical sites have the same underlining hardware and run the same operating system and database applications. All the different sites store database identically, this means the data structure used are the same at all sites.

Homogenous distributed database systems appear to the user as a single system and they can be much easier to design and manage. The database is accessed through a single interface as if it's a single database. There are two types of homogenous database:

1. **Autonomous:** each database is independent and functions on its own, they are integrated by a controlling application and use message parsing to share data updates.
2. **Non-autonomous:** data is distributed across the homogenous nodes, and a central or master DBMS co-ordinates data updates across the sites

Heterogeneous database: in a heterogeneous distributed database, different sites can use different schema and software that can lead to problems in query processing and transactions, also similarly a

particular site might be completely unaware of the other sites. Different computers may use different operating systems, different database application, etc. they may even use different data models for the database. Hence translations are required for different sites to communicate.

Types of heterogeneous database are:

By bright

1. **Federated:** the heterogeneous database systems are independent in nature and integrated together so that they function as a single database system
2. **Un-federated:** the database systems employ a central coordinating module through which the databases are accessed.

Distributed DBMS architectures

There are several different architectures for distributed database management systems (DDBMS), these architectures are as follows:

1. **Client-Server Architecture:** this is a 2 – level architecture where the functionality is divided into servers and clients. In this architecture, the clients connect to a central server which manages the distributed database system. The server is responsible for coordinating data management, query processing, optimization, transactions, managing data storage and providing access controls. The responsibility of the client is mainly user interface. There are two types of client-server architecture:
 - a. Single server multiple clients.
 - b. Multiple servers, multiple clients.

2. Peer to Peer Architecture: in this architecture, each site in the distributed database system is connected to all other sites. Each site acts both as a client and a server and each site is responsible for managing its own data and coordinating transactions with other sites. The peers share their resources with other pairs and coordinates their activities.

Next lecture: distributed database management design strategies

By bright

CSC 421 03/09/2025

Distributed Database Management Design Strategies.

It is divided into two strategies:

1. Data replication: this is the process of storing separate copies of the database at two or more sites or locations. It is a popular fault tolerant technique of distributed databases. Replicated data is used to create instances of data in different parts of database, by using replicated data, distributed databases can access identical data locally thereby avoiding traffic. Replicated data can be divided into two categories: read only and writeable data. The read only version allows revisions only to the first instance. Subsequent enterprise data are then adjusted. Writeable data can be altered but the first instance is immediately changed. Database replication ensures that the data in distributed database remains up to date.

Advantages of Data Replication.

- a. Reliability: in case of failure of any site, the database system continue to work since a copy is available at another site or sites.
- b. Reduction in network load: since local copies of data are available, query processing can be done with reduced network usage.
- c. Quicker response: availability of local copies of data ensures quick query processing and consequently quick response time.
- d. Simpler transactions: transactions require less number of joins of tables/relations located at different sites and minimal coordination across the network

Disadvantages

- a. Increased storage requirements: maintaining multiple copies of data is associated with increased storage cost. The storage space required is in multiples of the storage required for a centralized system.
 - b. Increased cost and complexity of data updating: each time a data item is updated, the update needs to be reflected in all the copies of the data at the different sites and this requires complex synchronization techniques and protocols.
 - c. Undesirable application database coupling: if complex update mechanisms are not used, removing data inconsistency requires complex coordination at application level. This results in undesirable application database coupling.
2. Fragmentation: this is the task of dividing a table or relation into a set of smaller tables or relations. The subsets of the relations or table are called fragments. Three categories of fragmentation exists namely: horizontal, vertical and hybrid.

Advantages

- a. Since data is stored close to the site of usage, efficiency of database system is increased.
- b. Local query optimization techniques are sufficient for most queries since data is locally available
- c. Since irrelevant data is not available at the sites, security and privacy of the database system can be ensured.

Disadvantages

- a. When data from different fragments are required, the access speeds may be very low.
- b. In case of recursive fragmentations, the job of reconstruction will need expensive techniques.

By bright

- c. Lack of backup copies of data in different sites may render the database ineffective in case of failure of a site.

Distributed Database Management system Distribution transparency

Distribution transparency is the property of distributed databases that concerns how the internal details of a distribution are hidden from the users. There are three dimensions of distribution transparency, these include location transparency, fragmentation transparency and replication transparency.

Location transparency: ensures that the user can query on any table(s) or fragments of a table as if they were stored locally in the user's site. The fact that the table or its fragments are stored at remote sites in the Distributed Database system should be completely oblivious to the end user. The

address of the remote sites and the ... are hidden from the user in order to incorporate location transparency DDBMS should have access to updated and accurate data dictionary and DDMS directory which contains the details of locations of data.

Fragmentation Transparency: this enables users to query upon any table as if it were un-fragmented

therefore it hides the fact that the table the user is querying on is actually a fragment or union of some fragment. It also conceals the fact that the fragments are concealed at diverse sites. This approach is similar to users of SQL views where the user may not know that they are using a view of a table instead of the table itself.

Replication transparency: ensures that replication of databases are hidden from the users. It enables

users to query upon a table as if only a single copy of the table exists. Replication transparency is associated with concurrency and failure transparency whenever a user updates a data item, the data

is reflected in all the copies of the table. This is concurrency transparency. Also in case of failure of a site, the user can still proceed with his query using replicated copies without any knowledge of

failure. This is failure transparency.

Combination of the transparencies (Hybrid): in any distributed database system, the designer should

ensure that all the stated transparencies are observed or maintained to a considerate extent. The designer may choose to fragment tables, replicate them and store them in different sites. All these facts are hidden to the end users

Distributed DBMS Database Control

Database control refers to the task of enforcing regulations so as to provide correct data to authentic/genuine users and applications of database. In order that correct data is available to users,

all data should conform to the integrity constraints defined in the database. Database control is one of the primary tasks of the database administrator (DBA). Three dimensions of database control exists, these include:

1. Authentication
2. Access rights
3. Integrity constraints

Authentication: in a Distributed Database System, authentication is a process through which only legitimate users can gain access to the data resources. Authentication can be enforced in two levels:

1. Controlling access to client's computer; at this level, users access is restricted while login to the client's computer that provides user interface to the database server. The most common

By bright

methods of approach is username or password or the combination of that. Other is through biometrics

2. Controlling access to the database software: at this level, the database software or administrator assigns some credentials to the user, the user gains access to the database using these credentials.

Access Rights: a use's access rights refers to the privileges that the user is granted regarding DBMS operations such as the right to create, query, drop, add or delete or update tables

Integrity Control: defines and enforces the integrity constraints of the database system. The integrity

constraints are as follows among others: data type, entity and referential integrity constraints, etc.

Distributed DBMS Transaction Processing Systems

A transaction is a program including a collection of database operations executed as a logical unit of

data processing. Any transaction must maintain the ACID properties which includes atomicity, consistency, isolation and durability.

Paper 7 of 8 — CSC 421 NOTE

Level 400 • Second Semester • Handout

CSC 421: DATA MANAGEMENT 11 (3 UNITS)

COURSE OUTLINE

Rational database: mapping conceptual schema to rational schema; Database query languages (SQL) concept of functional dependencies & multi valued dependencies. Transaction processing; Distributed databases.

RELATIONAL DATABASES

A relational database is a collection of information that organizes data in predefined relationships where data is stored in one or more tables (or "relations") of columns and rows, making it easy to see and understand how different data structures relate to each other.

A relational database management system (RDBMS) is a collection of programs and capabilities that enable IT teams and others to create, update, administer and otherwise interact with a relational database. A relational database is a type of database that stores related data points.

RDBMSes store data in the form of tables, with most commercial relational database management systems using Structured Query Language (SQL) to access the database. However, since SQL was invented after the initial development of the relational model, it isn't necessary for RDBMS use.

The RDBMS is the most popular database system among organizations. It provides a dependable method of storing and retrieving large amounts of data while offering a combination of system performance and ease of implementation. It's also the basis for modern database systems like MySQL.

RDBMS vs. DBMS

In general, databases store sets of data that can be queried for use in other applications. A database management system supports the development, administration and use of database platforms.

An RDBMS is a type of database management system (DBMS) that stores data in a row-based table structure that connects related data elements. An RDBMS includes functions that maintain the security, accuracy, integrity and consistency of the data. This is different than the file storage used in a DBMS.

Other differences between database management systems and relational database management systems include the following:

□ THE RELATIONAL MODEL: RDBMSes use a relational model to

map relationships between data elements, while a DBMS can support different data models.

- SQL: RDBMSes use SQL as a standard language for managing and querying data, while DBMSes: don't have a standard language and can support different programming languages.

- NUMBER OF ALLOWED USERS: While a DBMS can only accept one user at a time, an RDBMS can operate with multiple users.

- HARDWARE AND SOFTWARE REQUIREMENTS: A DBMS needs less software and hardware than an RDBMS.

- AMOUNT OF DATA: RDBMSes can handle any amount of data, from small to large, while a DBMS can only manage small amounts.

- DATABASE STRUCTURE: In a DBMS, data is kept in a hierarchical form, whereas an RDBMS utilizes a table where the headers are used as column names and the rows contain the corresponding values.

- ACID IMPLEMENTATION: DBMSes don't use the atomicity, consistency, isolation and durability (ACID) model for storing data. On the other hand, RDBMSes base the structure of their data on the ACID model to ensure consistency.

- DISTRIBUTED DATABASES: While an RDBMS offers complete support for distributed databases, a DBMS won't provide support.

- TYPES OF PROGRAMS MANAGED: While an RDBMS helps manage the relationships between its incorporated tables of data, a DBMS focuses on maintaining databases that are present within the computer network and system hard disks.

- SUPPORT OF DATABASE NORMALIZATION: An RDBMS can be normalized, but a DBMS can't.

- SCHEMAS: RDBMSes have a rigid schema, which limits the types of data they can store and manage, while DBMSes can accommodate flexible schemas.

RDBMS features typically include the following:

- ACID support.

- Multi-user access.

- Data durability.

- Data consistency.

- Data flexibility.

- Hierarchical relationship.

How an RDBMS works

As mentioned previously, an RDBMS stores data in the form of a

table. Each system will have varying numbers of tables with each table possessing its own unique primary key. The primary key is then used to identify each table. Within the table are rows and columns. The rows are known as records or horizontal entities; they contain the information for the individual entry. The columns are known as vertical entities and possess information about the specific field. Before creating these tables, the RDBMS must check the following constraints:

- Primary keys identify each row in the table. One table can only contain one primary key. The key must be unique and without null values.
- Foreign keys are used to link two tables. The foreign key is stored in one table and refers to the primary key associated with another table.
- Not null ensures that every column doesn't have a null value, such as an empty cell.
- Check confirms that each entry in a column or row satisfies a precise condition and that every column holds unique data.
- Data integrity ensures the integrity of the data is confirmed before the data is created.

RDBMSes also consist of the following notations:

- SQL. This is the domain-specific language used for storing and retrieving data.
- SQL query. This is a data request from an RDBMS system.
- Index. This is a data structure used to accelerate database retrieval.
- View. This is a table that shows a data output figured from underlying tables.

Ensuring the integrity of data includes several specific tests, including entity, domain, referential and user-defined integrity. Entity integrity confirms that the rows aren't duplicated in the table. Domain integrity ensures that data is entered into the table based on specific conditions, such as file format or range of values. Referential integrity ensures that any row that's relinked to a different table can't be deleted. Finally, user-defined integrity confirms that the table will satisfy all user-defined conditions.

Advantages of a relational database management system

The use of an RDBMS can be beneficial to most organizations; the systematic view of raw data helps companies better understand and execute the information while enhancing the decision-making process. Using tables to store data also improves the security of

information stored in the databases. Users can customize access and set barriers to limit the content that's made available. This feature makes the RDBMS particularly useful to organizations in which the manager decides what data is provided to employees and customers.

Furthermore, RDBMSes make it easy to add new data to the system or alter existing tables while ensuring consistency with the previously available content.

Other advantages of the RDBMS include the following:

- Flexibility. Updating data is more efficient, as the changes only need to be made in one place.
- Maintenance. DBAs can easily maintain, control and update data in the database. Backups also become easier, as automation tools included in the RDBMS automate these tasks.
- Data structure. The table format used in RDBMSes is easy to understand and provides an organized and structural manner through which entries are matched by firing queries.
- ACID properties. These properties increase data consistency, isolation and durability.
- Security. RDBMS systems can include security features such as encryption, access controls and user authentication.
- Scalability. RDBMS systems can horizontally distribute data across different servers.

Disadvantages of a relational database management system

On the other hand, relational database management systems also have some disadvantages. For example, to implement an RDBMS, special software must be purchased. This introduces an additional cost for execution. Once the software is obtained, the setup process can be tedious, as it requires millions of lines of content to be transferred into the RDBMS tables. This process might require the help of a programmer or a team of data entry specialists. Special attention must be paid to the data during entry to ensure sensitive information isn't placed into the wrong hands.

Some other drawbacks of the RDBMS include the character limit placed on certain fields in the tables and the inability to fully understand new forms of data -- such as complex numbers, designs and images.

Furthermore, while isolated databases can be created using an RDBMS, the process requires large chunks of information be separated from each other. Connecting these large amounts of data to form the isolated database can be complicated.

Uses of RDBMS

Relational database management systems are frequently used in disciplines such as manufacturing, human resources and banking. The system is also useful for airlines that need to store ticket service and passenger documentation information, as well as universities that maintain student databases.

Other examples of RDBMS uses include the following:

- Business systems. Business applications can use RDBMSes to store, manage and process transaction data.
- E-commerce. An RDBMS can be used to manage data related to inventory management, orders, transactions and customer data.
- Healthcare. RDBMSes are used to manage data related to healthcare, medical records, lab results and electronic health record systems.
- Education systems. RDBMSes can be used to manage student data and academic records.

Examples of RDBMS types

There are many different types of DBMSes, including a varying set of options for RDBMSes. Examples of different RDBMSes include the following:

- Oracle Database. This RDBMS system produced and marketed by Oracle is known for its varied feature set, scalability and security.
- MySQL. This widely used open source RDBMS system excels in speed, reliability and usability.
- Azure SQL. This Microsoft-provided cloud-based RDBMS system is used for small database applications.
- SQL Server. This Microsoft-provided RDBMS system is more complex than Azure SQL and offers full control.
- IBM Db2. This IBM-offered RDBMS system was also extended to support object-relational and non-relational structures such as JavaScript Object Notation and Extensible Markup Language.

CONCEPTUAL SCHEMA VS RELATIONAL SCHEMA

A relational schema is a set of relational tables and associated items that are related to one another. All of the base tables, views, indexes, domains, user roles, stored modules, and other items that a user creates to fulfill the data needs of a particular enterprise or set of applications belong to one schema.

Relation schema defines the design and structure of the relation or table in the database. It is the way of representation of relation states in such a way that every relation database state fulfills the integrity constraints set (Like Primary key, Foreign Key, Not null, Unique

constraints) on a relational schema.

It consists of the relation name, set of attributes/field names/column names.

Components of a Relation Schema

□ Relation Name: Name of the table that is stored in the database.

It should be unique and related to the data that is stored in the table. For example- The name of the table can be Employee store the data of the employee.

□ Attributes Name: Attributes specify the name of each column within the table. Each attribute has a specific data type.

□ Domains: The set of possible values for each attribute. It specifies the type of data that can be stored in each column or attribute, such as integer, string, or date.

□ Primary Key: The primary key is the key that uniquely identifies each tuple. It should be unique and not be null.

□ Foreign Key: The foreign key is the key that is used to connect two tables. It refers to the primary key of another table.

□ Constraints: Rules that ensure the integrity and validity of the data. Common constraints include NOT NULL, UNIQUE, CHECK, and DEFAULT.

Example of Relation Schema

There is a student named EMMA, he is pursuing B.SC, in the 4th year, and belongs to the computer science department (department no. 1) and has roll number 1601347. If we want to represent this using databases we would have to create a student table with name, sex, degree, year, department, department number, roll number, and proctor (adviser) as the attributes.

Student Table

NAME REG DEPT SEX AGE LEVEL PROGRAM

NO TYPE

EMMA 1156 CSC M 21 400 REGULAR

CONCEPTUAL SCHEMA IN DATABASE

A conceptual schema is a map of concepts and their relationships used for databases. This describes the semantics of an organization and represents a series of assertions about its nature. Specifically, it describes the things of significance to an organization (entity classes), about which it is inclined to collect information, and their characteristics (attributes) and the associations between pairs of those things of significance (relationships). It describes the structure of the whole database for a group of users. The conceptual model is

also known as the data model that can be used to describe the conceptual schema when a database system is implemented. It hides the internal details of physical storage and targets the description of entities, datatypes, relationships and constraints.

Because a conceptual schema represents the semantics of an organization, and not a database design, it may exist on various levels of abstraction.

A conceptual schema is a high-level description of information needs that are to be addressed by a certain database or information system. It is typically the first phase for designing of a database schema from scratch. By “high-level” we mean here that it (1) attempts to abstract from which exact type of DBMS we are designing for and (2) uses a notation that is more likely to be understandable for the stakeholders of the database or information system that is being designed.

The most common forms for conceptual schemas are Entity-Relationship diagrams and UML Class diagrams.

ENTITY RELATIONSHIPS DIAGRAM

The Entity Relationship Diagram explains the relationship among the entities present in the database. ER models are used to model real-world objects like a person, a car, or a company and the relation between these real-world objects. In short, the ER Diagram is the structural format of the database.

THE IMPORTANCE OF ENTITY RELATIONSHIP DIAGRAM

ER diagrams are used to represent the E-R model in a database, which makes them easy to convert into relations (tables).

ER diagrams provide the purpose of real-world modeling of objects which makes them intently useful.

ER diagrams require no technical knowledge and no hardware support.

These diagrams are very easy to understand and easy to create even for a naive user.

It gives a standard solution for visualizing the data logically.

Symbols Used in ER Model

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

- Rectangles: Rectangles represent Entities in the ER Model.
- Ellipses: Ellipses represent Attributes in the ER Model.
- Diamond: Diamonds represent Relationships among Entities.
- Lines: Lines represent attributes to entities and entity sets with other relationship types.
- Double Ellipse: Double Ellipses represent Multi-Valued

Attributes.

□ Double Rectangle: Double Rectangle represents a Weak Entity.

DATABASE QUERY LANGUAGES

A query language, also known as data query language or database query language (DQL), is a computer language used to make queries in databases and information systems. In database systems, query languages rely on strict theory to retrieve information. A well-known example is the Structured Query Language (SQL).

STRUCTURED QUERY LANGUAGE (SQL)

Structured query language (SQL) is a standard language for database creation and manipulation. MySQL is a relational database program that uses SQL queries. While SQL commands are defined by international standards, the MySQL software undergoes continual upgrades and improvements.

Structured Query Language (SQL) is a standardized programming language that is used to manage relational databases and perform various operations on the data in them. Initially created in the 1970s, SQL is regularly used not only by database administrators but also by developers writing data integration scripts and data analysts looking to set up and run analytical queries.

SQL is used for the following:

- Modifying database table and index structures.
- Adding, updating and deleting rows of data.
- Retrieving subsets of information from within relational database management systems (RDBMSes). This information can be used for transaction processing, analytics applications and other applications that require communicating with a relational database.

A table is the most basic unit of a database and consists of rows and columns of data. A single table holds records, and each record is stored in a row of the table. Tables are the most used type of database objects or structures that hold or reference data in a relational database.

The SQL query language can be used for relational or non-relational databases, but it offers advantages for relational databases.

Why is SQL important?

SQL is a great language to learn because it's the primary database language used for data processing tasks and is used across various industries.

The following reasons highlight the importance of SQL:

□ Backbone of the data industry. SQL is considered the backbone of the data industry. It's widely used by data-centric professionals including data analysts, data scientists, business analysts and database developers.

□ Universal language. SQL is a universal language that is transferable to other disciplines and languages. Learning SQL can help one understand the workings of other languages such as Python and Java. It also makes collaboration easy, as it has a large supportive community.

□ In-demand skill. SQL knowledge is one of the most in-demand skills in the field of data science. It appears in a significant percentage of data science job postings, making it a prized skill for professionals in this field.

□ Data manipulation. SQL is well-suited for data manipulation. It enables users to easily test and manipulate data, making it efficient for tasks such as filtering, sorting and aggregating data.

□ Rapid query processing. SQL enables rapid query processing, enabling users to retrieve, manipulate or store data quickly and efficiently. However, optimizing queries for rapid processing involves a combination of proper indexing, query optimization and database design considerations.

□ Security features. SQL provides various security features such as authentication, access control, audit trails and encryption, making it easy to manage permissions and ensure the security of data.

□ Commonality and compatibility. SQL is widely used in various IT systems and is compatible with multiple other languages. Its commonality benefits beginners in the profession, as they are likely to use SQL throughout their careers. It also contributes to ease of application and improves the production and efficiency of businesses.

□ Scalability. SQL is suitable for organizations of any size. It can easily scale up to accommodate future growth, making it a versatile choice for small and large businesses alike.

□ Open source and community support. SQL is an open source language with a vibrant community of developers that regularly provide updates and troubleshooting assistance to SQL users.

□ Cost-effective. Due to its open source nature, SQL is more cost-effective than proprietary solutions, making it ideal for organizations with budget constraints.

SQL standard and proprietary extensions

An official SQL standard was adopted by the American National Standards Institute (ANSI) in 1986, with the International Organization for Standardization (ISO) adopting the standard in 1987. New versions of the SQL standard are published every few years, the most recent in 2022.

ISO/IEC 9075 is the ISO SQL standard developed jointly by ISO and the International Electro Technical Commission. The standard way of referring to an ISO standard version is to use the standards organizations -- ISO/IEC -- followed by the ISO standard number, a colon and the publication year. The current ISO standard for SQL is ISO/IEC 9075:2022.

Both proprietary and open source RDBMSes built around SQL are available for use by organizations. SQL-compliant database server products include the following:

- Microsoft SQL Server.
- Oracle Database.
- IBM Db2.
- SAP HANA.
- SAP Adaptive Server.
- Oracle MySQL.
- Open source PostgreSQL.
- Azure SQL Database.
- Google Cloud SQL.

Some versions of SQL include proprietary extensions to the standard language for procedural programming and other functions. For example, Microsoft offers a set of extensions called Transact-SQL, while Oracle's extended version of the standard is Procedural Language for SQL. Commercial vendors offer proprietary extensions to differentiate their product offerings by giving customers additional features and functions. As a result, the different variants of extended SQL offered by vendors are not fully compatible with one another.

SQL commands and syntax

SQL is, fundamentally, a programming language designed for accessing, modifying and extracting information from relational databases. As a programming language, SQL has commands and a syntax for issuing those commands.

SQL commands are divided into several different types, including the following:

- Data Definition Language (DDL) commands are also called data definition commands because they are used to define data

tables.

□ Data Manipulation Language (DML) commands are used to manipulate data in existing tables by adding, changing or removing data. Unlike DDL commands that define how data is stored, DML commands operate in the tables defined with DDL commands.

□ Data Query Language consists of just one command, SELECT, used to get specific data from tables. This command is sometimes grouped with the DML commands.

□ Data Control Language commands are used to grant or revoke user access privileges.

□ Transaction Control Language commands are used to change the state of some data -- for example, to COMMIT transaction changes or to ROLLBACK transaction changes.

History of SQL

With roots going back to the early 1970s, SQL continues to make key milestones as one of the most successful ideas in computing history. The following is a timeline of the SQL language and its key milestones:

□ 1970. "A Relational Model of Data for Large Shared Data Banks" by E.F. Codd is published in Communications of the ACM, laying the basis for RDBMSes.

□ 1974. IBM researchers publish an article introducing Structured Query Language, first called SEQUEL or Structured English Query Language. The name was changed for trademark purposes.

□ 1977. Relational Software Inc., the company that eventually became Oracle, begins building a commercial RDBMS.

□ 1979. Oracle ships the first commercially available RDBMS for Digital Equipment Corp.'s minicomputer systems.

□ 1982. IBM ships SQL/Data System, a SQL RDBMS for IBM mainframes.

□ 1985. IBM ships Database 2, a SQL RDBMS for IBM's Multiple Virtual Storage mainframe operating system.

□ 1986. An ANSI committee and then ISO adopt SQL as a standard.

□ 1989. The first revision of the ISO SQL standard, SQL-89, is published.

□ 1992. The first major revision of the ISO SQL standard, SQL-92, is published.

□ 1999. The first version to be named in accordance with ISO naming standards, ISO/IEC SQL:1999, adds programming functionality and support for Java.

□ 2003. ISO/IEC SQL:2003 adds support for a predefined data type

for Extensible Markup Language (XML) objects.

- 2006. ISO/IEC SQL:2006 expands XML-related functionality.
- 2008. ISO/IEC SQL:2008 adds support for partitioned JOINS, a method for linking two or more tables that treats the joined tables as a single table.
- 2011. ISO/IEC SQL:2011 improves support for relational databases containing time-related data.
- 2016. ISO/IEC SQL:2016 adds optional new features, including JavaScript Object Notation-related changes, support for polymorphic table functions and row pattern matching.
- 2017. ISO/IEC SQL:2017. SQL Server 2017 provides choices of development languages, data types, on-premises cloud and OS systems by adding the SQL Server power to Linux, Windows and Linux-based containers.
- 2019. ISO/IEC SQL:2019. SQL Server 2019 introduces Big Data Clusters for SQL and also offers additional options and improvements for the SQL Server database engine, SQL Server Analysis Services, SQL Server Machine Learning Services, SQL Server on Linux and SQL Server Master Data Services.
- 2022. ISO/IEC SQL:2022. SQL Server 2022 continues to build on the previous versions and their capabilities with additional features such as Azure Synapse Link, which is a real-time analytics feature, as well as enhanced security through features such as Microsoft Defender.

CONCEPT OF FUNCTIONAL DEPENDENCIES

A functional dependency means that when the value of one attribute is known, one can always determine the value of another attribute. A Functional Dependency in DBMS is a fundamental concept that describes the relationship between attributes (columns) in a table. It shows how the values in one or more attributes determine the value in another. In layperson's terms, it describes how data in one column or set of columns can relate to data in another column. It helps to maintain the quality of the data in DBMS.

Functional Dependency is represented in the form of an equation. Here, you have a set of attributes (A, B, C, etc.) and an arrow (->) denoting the Dependency. For example, if we have a table of employee data with columns "Employee ID," "First Name," and "Last Name," we can express a functional dependency like this:

Employee ID -> First Name, Last Name.

How to Denote a Functional Dependency in DBMS?

In DBMS, you denote functional dependencies using a notation. It contains two main components: the left-hand side (LHS) and the right-hand side (RHS) of an arrow ($\rightarrow$).

For example, if we have a table with attributes "A," "B," and "C," and attribute "A" determines the values of attributes "B" and "C," you would denote it as

$A \rightarrow B, C$

This notation indicates that the value(s) in attribute "A" determines the value(s) in attributes "B" and "C." In other words, if you know the value of "A," you can determine the values of "B" and "C."

Types of Functional Dependencies in DBMS

Here are some of the important types of Functional Dependency In DBMS

Trivial Functional Dependency

A trivial functional dependency in DBMS occurs when an attribute or set of attributes (columns) on the left-hand side (LHS) of a functional dependency arrow ($\rightarrow$) already determines the attributes on the right-hand side (RHS) without any extra information.

Suppose we have a table of students with attributes "Student ID" and "Student Name." In this case, if we state the functional dependency as

$\text{Student ID} \rightarrow \text{Student Name},$

This is a trivial functional dependency. Because within a single "Student ID," there can be only one corresponding "Student Name." In other words, the value of "Student ID" determines the value of "Student Name" without any more information or conditions.

Non-trivial Functional Dependency

A non-trivial functional dependency is a specific type of dependency between attributes (columns) in a table. Here, the relationship is not obvious or trivial. It conveys meaningful information about how the values in one set of attributes determine the values in another.

To illustrate this concept, let's consider an example with a table of student data:

Student Id	Student Name	Student DOB	Class
------------	--------------	-------------	-------

101	Emma	1995-05-15	10A
-----	------	------------	-----

102	Udo	2000-03-20	10B
-----	-----	------------	-----

103	Sonia	1999-07-10	10A
-----	-------	------------	-----

104	David	2000-01-05	10B
-----	-------	------------	-----

We want to express a functional dependency based on the student's

birthdate (StudentDOB) and class (Class). A non-trivial functional dependency in this case would be

StudentDOB, Class -> StudentName.

This functional dependency means that given a combination of a student's date of birth and class, you can uniquely determine their name. It's non-trivial because it provides valuable information about the relationship between attributes in the table.

Multivalued Functional Dependency

A multivalued functional dependency in a database occurs when one or more attributes determine multiple unrelated sets of values in another attribute. It shows that changes in the determining attributes can lead to various combinations of values in the dependent attribute, indicating complex relationships within the data.

Here's a simple example:

Student_ID Student_Name Courses_Enrolled

1 Alice {Math, English}

2 Bob {Science, History}

3 Carol {Math, Science}

In this case, the multivalued dependency in DBMS holds:

- Alice is Student 1 enrolled in {Math, English}.
- Bob is Student 2 enrolled in {Science, History}.
- Carol is Student 3 enrolled in {Math, Science}.

Transitive Functional Dependency

A transitive functional dependency in DBMS is a relational database table's relationship between attributes (columns). It occurs when one attribute's value determines another's value through an intermediary (a third) attribute.

Example:

Consider a database table called "Student_Info" with the following attributes:

Student_ID (unique identifier for each student),
Student_Name,
Student_Address
Student_City.

In this example, we can assume that Student_Address depends on Student_City, and Student_City depends on Student_ID. This creates a transitive dependency in DBMS, where Student_ID indirectly determines Student_Address.

RULES OF FUNCTIONAL DEPENDENCY

There are three functional dependency rules, known as Armstrong's axioms, you use to infer functional dependency within relational databases.

Reflexive rule: This rule states that if X is a set of attributes and Y is its subset, then X holds a value of Y .

Augmentation rule: This rule adds attributes, though they rarely change the basic dependencies and states if $X \rightarrow Y$ holds and C is an attribute set, then $XC \rightarrow YC$ also holds.

Transitivity rule: Closely aligned with algebra's transitive rule, this rule states if $X \rightarrow Y$ holds and $Y \rightarrow Z$ holds, then $X \rightarrow Z$ must also hold. With this rule, $X \rightarrow Y$ is the functional dependent that determines Y .

ADVANTAGES OF FUNCTIONAL DEPENDENCY IN DBMS

Here are the key advantages of functional dependencies in DBMS:

Data Integrity

Functional dependencies help ensure data integrity in a database. By defining rules that govern the relationships between attributes, DBMS can enforce constraints to prevent inconsistent or incorrect data from being entered into the database.

Normalization

Functional dependencies are used in database normalization. By identifying and applying functional dependencies, database designers can break down large tables into smaller, more manageable ones, reducing data duplication and anomalies.

Efficient Storage

Normalizing a database through functional dependencies can lead to more efficient data storage. Smaller, normalized tables need less storage space, which can be especially beneficial for large databases.

Ease of Maintenance

Databases that adhere to functional dependencies are easier to maintain. When changes are required in the database structure or schema, the impact of those changes is more localized, reducing the risk of introducing errors or inconsistencies in the data.

Query Optimization

With the help of functional dependencies, we are able to decide the connectivity between the tables and the necessary attributes need to be projected to retrieve the required data from the tables. This helps in query optimization and improves performance.

Data Quality Improvement

Functional dependencies ensure that the data in the database to be

accurate, complete and updated. This helps to improve the overall quality of the data, as well as it eliminates errors and inaccuracies that might occur during data analysis and decision making, thus functional dependency helps in improving the quality of data in database.

DISADVANTAGES OF FUNCTIONAL DEPENDENCY IN DBMS

Overly restrictive functional dependencies can result in slow query performance or data inconsistencies, as data that should be related may not be properly linked.

Functional dependencies do not take into account the semantic meaning of data, and may not always reflect the true relationships between data elements.

Conclusion

Functional dependency in DBMS is the blueprint for organizing and ensuring data accuracy. Identifying these relationships simplifies database design, reduces redundancy, and enhances query efficiency.

They are the building blocks for maintaining structured and reliable databases, essential for modern data management and information retrieval. All the functional dependencies are useful tools to make the Database Management System more efficient for the user.

TRANSACTION PROCESSING IN DBMS.

DBMS transaction processing is followed as a set of operations accomplished on the data records stored in the server database that results in an alteration in the database state. This DBMS transaction is completed through the "ACID" principles, which include: Atomicity, Consistency, Isolation and Durability. When the data of users is stored in a database, that data needs to be accessed and modified from time to time. This task should be performed with a specified set of rules and in a systematic way to maintain the consistency and integrity of the data present in a database. In DBMS, this task is called a transaction. It is similar to a bank transaction, where the user requests to withdraw some amount of money from his account. Subsequently, several operations take place such as fetching the user's balance from the database, subtracting the desired amount from it, and updating the user's account balance. This series of operations can be called a transaction. Transactions are very common in DBMS.

Transaction in Database Management Systems (DBMS) can be defined as a set of logically related operations. It is the result of a request made by the user to access the contents of the database and perform

operations on it. It consists of various operations and has various states in its completion journey. It also has some specific properties that must be followed to keep the database consistent.

Operations of Transaction

A user can make different types of requests to access and modify the contents of a database. So, we have different types of operations relating to a transaction. They are discussed as follows:

i) Read(X)

A read operation is used to read the value of X from the database and store it in a buffer in the main memory for further actions such as displaying that value. Such an operation is performed when a user wishes just to see any content of the database and not make any changes to it. For example, when a user wants to check his/her account's balance, a read operation would be performed on user's account balance from the database.

ii) Write(X)

A write operation is used to write the value to the database from the buffer in the main memory. For a write operation to be performed, first a read operation is performed to bring its value in buffer, and then some changes are made to it, e.g. some set of arithmetic operations are performed on it according to the user's request, then to store the modified value back in the database, a write operation is performed. For example, when a user requests to withdraw some money from his account, his account balance is fetched from the database using a read operation, then the amount to be deducted from the account is subtracted from this value, and then the obtained value is stored back in the database using a write operation.

iii) Commit

This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction. To ensure that further operations of any other transaction are performed only after work of the current transaction is done, a commit operation is performed to the changes made by a transaction permanently to the database.

iv) Rollback

This operation is performed to bring the database to the last saved state when any transaction is interrupted in between due to any power, hardware, or software failure. In simple words, it can be said that a rollback operation does undo the operations of transactions

that were performed before its interruption to achieve a safe state of the database and avoid any kind of ambiguity or inconsistency.

TRANSACTION SCHEDULES

When multiple transaction requests are made at the same time, we need to decide their order of execution. Thus, a transaction schedule can be defined as a chronological order of execution of multiple transactions.

There are broadly two types of transaction schedules discussed as follows,

i) Serial Schedule

In this kind of schedule, when multiple transactions are to be executed, they are executed serially, i.e. at one time only one transaction is executed while others wait for the execution of the current transaction to be completed. This ensures consistency in the database as transactions do not execute simultaneously. But, it increases the waiting time of the transactions in the queue, which in turn lowers the throughput of the system, i.e. number of transactions executed per time. To improve the throughput of the system, another kind of schedule are used which has some more strict rules which help the database to remain consistent even when transactions execute simultaneously.

ii) Non-Serial Schedule

To reduce the waiting time of transactions in the waiting queue and improve the system efficiency, we use non-serial schedules which allow multiple transactions to start before a transaction is completely executed. This may sometimes result in inconsistency and errors in database operation. So, these errors are handled with specific algorithms to maintain the consistency of the database and improve CPU throughput as well. Serial Schedules are also sometimes referred to as parallel schedules as transactions execute in parallel in this kind of schedules.

TRANSACTION STATES IN DBMS.

i) Active state

It is the first stage of any transaction when it has begun to execute. The execution of the transaction takes place in this state. Operations such as insertion, deletion, or updating are performed during this state. During this state, the data records are under manipulation and they are not saved to the database, rather they remain somewhere in a buffer in the main memory.

ii) Partially Committed

This state of transaction is achieved when it has completed most of

the operations and is executing its final operation. It can be a signal to the commit operation, as after the final operation of the transaction completes its execution, the data has to be saved to the database through the commit operation. If some kind of error occurs during this state, the transaction goes into a failed state, else it goes into the Committed state.

iii) Committed

This state of transaction is achieved when all the transaction-related operations have been executed successfully along with the Commit operation, i.e. data is saved into the database after the required manipulations in this state. This marks the successful completion of a transaction.

iv) Failed

If any of the transaction-related operations cause an error during the active or partially committed state, further execution of the transaction is stopped and it is brought into a failed state. Here, the database recovery system makes sure that the database is in a consistent state.

v) Aborted

If the error is not resolved in the failed state, then the transaction is aborted and a rollback operation is performed to bring database to the the last saved consistent state. When the transaction is aborted, the database recovery module either restarts the transaction or kills it.

The illustration below shows the various states that a transaction may encounter in its completion journey.

PROPERTIES OF TRANSACTION

As transactions deal with accessing and modifying the contents of the database, they must have some basic properties which help maintain the consistency and integrity of the database before and after the transaction. Transactions follow 4 properties, namely, Atomicity, Consistency, Isolation, and Durability. Generally, these are referred to as ACID properties of transactions in DBMS. ACID is the acronym used for transaction properties. A brief description of each property of the transaction is as follows.

1) Atomicity

This property ensures that either all operations of a transaction are executed or it is aborted. In any case, a transaction can never be completed partially. Each transaction is treated as a single unit (like an atom). Atomicity is achieved through commit and rollback operations, i.e. changes are made to the database only if all

operations related to a transaction are completed, and if it gets interrupted, any changes made are rolled back using rollback operation to bring the database to its last saved state.

ii) Consistency

This property of a transaction keeps the database consistent before and after a transaction is completed. Execution of any transaction must ensure that after its execution, the database is either in its prior stable state or a new stable state. In other words, the result of a transaction should be the transformation of a database from one consistent state to another consistent state. Consistency, here means, that the changes made in the database are a result of logical operations only which the user desired to perform and there is not any ambiguity.

iii) Isolation

This property states that two transactions must not interfere with each other, i.e. if some data is used by a transaction for its execution, then any other transaction cannot concurrently access that data until the first transaction has completed. It ensures that the integrity of the database is maintained and we don't get any ambiguous values. Thus, any two transactions are isolated from each other. This property is enforced by the concurrency control subsystem of DBMS.

iv) Durability

This property ensures that the changes made to the database after a transaction is completely executed, are durable. It indicates that permanent changes are made by the successful execution of a transaction. In the event of any system failures or crashes, the consistent state achieved after the completion of a transaction remains intact. The recovery subsystem of DBMS is responsible for enforcing this property.

DISTRIBUTED DATABASES

A distributed database is a database that runs and stores data across multiple computers, as opposed to doing everything on a single machine. Typically, distributed databases operate on two or more interconnected servers on a computer network.

A distributed database system is located on various sites that don't share physical components. This may be required when a particular database needs to be accessed by various users globally. It needs to be managed such that for the users it looks like one single database.

Examples of distributed databases

Though there are many distributed databases to choose from, some

examples of distributed databases include Apache Ignite, Apache Cassandra, Apache HBase, Couchbase Server, Amazon SimpleDB, Clusterpoint, and FoundationDB.

TYPES OF DISTRIBUTED DATABASE

1. Homogeneous Database:

In a homogeneous database, all different sites store database identically. The operating system, database management system, and the data structures used – all are the same at all sites. Hence, they're easy to manage.

2. Heterogeneous Database:

In a heterogeneous distributed database, different sites can use different schema and software that can lead to problems in query processing and transactions. Also, a particular site might be completely unaware of the other sites. Different computers may use a different operating system, different database application. They may even use different data models for the database. Hence, translations are required for different sites to communicate.

DISTRIBUTED DATA STORAGE:

There are 2 ways in which data can be stored on different sites. These are:

1. Replication –

In this approach, the entire relationship is stored redundantly at 2 or more sites. If the entire database is available at all sites, it is a fully redundant database. Hence, in replication, systems maintain copies of data.

This is advantageous as it increases the availability of data at different sites. Also, now query requests can be processed in parallel.

However, it has certain disadvantages as well. Data needs to be constantly updated. Any change made at one site needs to be recorded at every site that relation is stored or else it may lead to inconsistency. This is a lot of overhead. Also, concurrency control becomes way more complex as concurrent access now needs to be checked over a number of sites.

2. Fragmentation –

In this approach, the relations are fragmented (i.e., they're divided into smaller parts) and each of the fragments is stored in different sites where they're required. It must be made sure that the fragments are such that they can be used to reconstruct the original relation (i.e., there isn't any loss of data).

Fragmentation is advantageous as it doesn't create copies of data,

consistency is not a problem.

FRAGMENTATION OF RELATIONS CAN BE DONE IN TWO WAYS:

Horizontal fragmentation – Splitting by rows –

The relation is fragmented into groups of tuples so that each tuple is assigned to at least one fragment.

Vertical fragmentation – Splitting by columns –

The schema of the relation is divided into smaller schemas. Each fragment must contain a common candidate key so as to ensure a lossless join.

In certain cases, an approach that is hybrid of fragmentation and replication is used.

APPLICATIONS OF DISTRIBUTED DATABASE:

It is used in Corporate Management Information System.

It is used in multimedia applications.

Used in Military's control system, Hotel chains etc.

It is also used in manufacturing control system.

THERE ARE SEVERAL DIFFERENT ARCHITECTURES FOR DISTRIBUTED DATABASE SYSTEMS, INCLUDING:

Client-server architecture: In this architecture, clients connect to a central server, which manages the distributed database system. The server is responsible for coordinating transactions, managing data storage, and providing access control.

Peer-to-peer architecture: In this architecture, each site in the distributed database system is connected to all other sites. Each site is responsible for managing its own data and coordinating transactions with other sites.

Federated architecture: In this architecture, each site in the distributed database system maintains its own independent database, but the databases are integrated through a middleware layer that provides a common interface for accessing and querying the data.

ADVANTAGES OF DISTRIBUTED DATABASE SYSTEM:

- 1) There is fast data processing as several sites participate in request processing.
- 2) Reliability and availability of this system is high.
- 3) It possess reduced operating cost.
- 4) It is easier to expand the system by adding more sites.
- 5) It has improved sharing ability and local autonomy.

Disadvantages of Distributed Database System:

- 1) The system becomes complex to manage and control.

- 2) The security issues must be carefully managed.
- 3) The system requires deadlock handling during the transaction processing otherwise the entire system may be in inconsistent state.
- 4) There is need of some standardization for processing of distributed database system.

Paper 8 of 8 — CSC 421

Level 400 • Second Semester • Handout

CSC 421: DATA MANAGEMENT 11 (3 UNITS)

COURSE OUTLINE

Rational database: mapping conceptual schema to rational schema; Database query languages (SQL) concept of functional dependencies & multi valued dependencies. Transaction processing; Distributed databases.

RELATIONAL DATABASES

A relational database is a collection of information that organizes data in predefined relationships where data is stored in one or more tables (or "relations") of columns and rows, making it easy to see and understand how different data structures relate to each other.

A relational database management system (RDBMS) is a collection of programs and capabilities that enable IT teams and others to create, update, administer and otherwise interact with a relational database. A relational database is a type of database that stores related data points.

RDBMSes store data in the form of tables, with most commercial relational database management systems using Structured Query Language (SQL) to access the database. However, since SQL was invented after the initial development of the relational model, it isn't necessary for RDBMS use.

The RDBMS is the most popular database system among organizations. It provides a dependable method of storing and retrieving large amounts of data while offering a combination of system performance and ease of implementation. It's also the basis for modern database systems like MySQL.

RDBMS vs. DBMS

In general, databases store sets of data that can be queried for use in other applications. A database management system supports the development, administration and use of database platforms.

An RDBMS is a type of database management system (DBMS) that stores data in a row-based table structure that connects related data elements. An RDBMS includes functions that maintain the security, accuracy, integrity and consistency of the data. This is different than the file storage used in a DBMS.

Other differences between database management systems and relational database management systems include the following:

□ THE RELATIONAL MODEL: RDBMSes use a relational model to

map relationships between data elements, while a DBMS can support different data models.

- SQL: RDBMSes use SQL as a standard language for managing and querying data, while DBMSes: don't have a standard language and can support different programming languages.

- NUMBER OF ALLOWED USERS: While a DBMS can only accept one user at a time, an RDBMS can operate with multiple users.

- HARDWARE AND SOFTWARE REQUIREMENTS: A DBMS needs less software and hardware than an RDBMS.

- AMOUNT OF DATA: RDBMSes can handle any amount of data, from small to large, while a DBMS can only manage small amounts.

- DATABASE STRUCTURE: In a DBMS, data is kept in a hierarchical form, whereas an RDBMS utilizes a table where the headers are used as column names and the rows contain the corresponding values.

- ACID IMPLEMENTATION: DBMSes don't use the atomicity, consistency, isolation and durability (ACID) model for storing data. On the other hand, RDBMSes base the structure of their data on the ACID model to ensure consistency.

- DISTRIBUTED DATABASES: While an RDBMS offers complete support for distributed databases, a DBMS won't provide support.

- TYPES OF PROGRAMS MANAGED: While an RDBMS helps manage the relationships between its incorporated tables of data, a DBMS focuses on maintaining databases that are present within the computer network and system hard disks.

- SUPPORT OF DATABASE NORMALIZATION: An RDBMS can be normalized, but a DBMS can't.

- SCHEMAS: RDBMSes have a rigid schema, which limits the types of data they can store and manage, while DBMSes can accommodate flexible schemas.

RDBMS features typically include the following:

- ACID support.

- Multi-user access.

- Data durability.

- Data consistency.

- Data flexibility.

- Hierarchical relationship.

How an RDBMS works

As mentioned previously, an RDBMS stores data in the form of a

table. Each system will have varying numbers of tables with each table possessing its own unique primary key. The primary key is then used to identify each table. Within the table are rows and columns. The rows are known as records or horizontal entities; they contain the information for the individual entry. The columns are known as vertical entities and possess information about the specific field. Before creating these tables, the RDBMS must check the following constraints:

- Primary keys identify each row in the table. One table can only contain one primary key. The key must be unique and without null values.
- Foreign keys are used to link two tables. The foreign key is stored in one table and refers to the primary key associated with another table.
- Not null ensures that every column doesn't have a null value, such as an empty cell.
- Check confirms that each entry in a column or row satisfies a precise condition and that every column holds unique data.
- Data integrity ensures the integrity of the data is confirmed before the data is created.

RDBMSes also consist of the following notations:

- SQL. This is the domain-specific language used for storing and retrieving data.
- SQL query. This is a data request from an RDBMS system.
- Index. This is a data structure used to accelerate database retrieval.
- View. This is a table that shows a data output figured from underlying tables.

Ensuring the integrity of data includes several specific tests, including entity, domain, referential and user-defined integrity. Entity integrity confirms that the rows aren't duplicated in the table. Domain integrity ensures that data is entered into the table based on specific conditions, such as file format or range of values. Referential integrity ensures that any row that's relinked to a different table can't be deleted. Finally, user-defined integrity confirms that the table will satisfy all user-defined conditions.

Advantages of a relational database management system

The use of an RDBMS can be beneficial to most organizations; the systematic view of raw data helps companies better understand and execute the information while enhancing the decision-making process. Using tables to store data also improves the security of

information stored in the databases. Users can customize access and set barriers to limit the content that's made available. This feature makes the RDBMS particularly useful to organizations in which the manager decides what data is provided to employees and customers.

Furthermore, RDBMSes make it easy to add new data to the system or alter existing tables while ensuring consistency with the previously available content.

Other advantages of the RDBMS include the following:

- Flexibility. Updating data is more efficient, as the changes only need to be made in one place.
- Maintenance. DBAs can easily maintain, control and update data in the database. Backups also become easier, as automation tools included in the RDBMS automate these tasks.
- Data structure. The table format used in RDBMSes is easy to understand and provides an organized and structural manner through which entries are matched by firing queries.
- ACID properties. These properties increase data consistency, isolation and durability.
- Security. RDBMS systems can include security features such as encryption, access controls and user authentication.
- Scalability. RDBMS systems can horizontally distribute data across different servers.

Disadvantages of a relational database management system

On the other hand, relational database management systems also have some disadvantages. For example, to implement an RDBMS, special software must be purchased. This introduces an additional cost for execution. Once the software is obtained, the setup process can be tedious, as it requires millions of lines of content to be transferred into the RDBMS tables. This process might require the help of a programmer or a team of data entry specialists. Special attention must be paid to the data during entry to ensure sensitive information isn't placed into the wrong hands.

Some other drawbacks of the RDBMS include the character limit placed on certain fields in the tables and the inability to fully understand new forms of data -- such as complex numbers, designs and images.

Furthermore, while isolated databases can be created using an RDBMS, the process requires large chunks of information be separated from each other. Connecting these large amounts of data to form the isolated database can be complicated.

Uses of RDBMS

Relational database management systems are frequently used in disciplines such as manufacturing, human resources and banking. The system is also useful for airlines that need to store ticket service and passenger documentation information, as well as universities that maintain student databases.

Other examples of RDBMS uses include the following:

- Business systems. Business applications can use RDBMSes to store, manage and process transaction data.
- E-commerce. An RDBMS can be used to manage data related to inventory management, orders, transactions and customer data.
- Healthcare. RDBMSes are used to manage data related to healthcare, medical records, lab results and electronic health record systems.
- Education systems. RDBMSes can be used to manage student data and academic records.

Examples of RDBMS types

There are many different types of DBMSes, including a varying set of options for RDBMSes. Examples of different RDBMSes include the following:

- Oracle Database. This RDBMS system produced and marketed by Oracle is known for its varied feature set, scalability and security.
- MySQL. This widely used open source RDBMS system excels in speed, reliability and usability.
- Azure SQL. This Microsoft-provided cloud-based RDBMS system is used for small database applications.
- SQL Server. This Microsoft-provided RDBMS system is more complex than Azure SQL and offers full control.
- IBM Db2. This IBM-offered RDBMS system was also extended to support object-relational and non-relational structures such as JavaScript Object Notation and Extensible Markup Language.

CONCEPTUAL SCHEMA VS RELATIONAL SCHEMA

A relational schema is a set of relational tables and associated items that are related to one another. All of the base tables, views, indexes, domains, user roles, stored modules, and other items that a user creates to fulfill the data needs of a particular enterprise or set of applications belong to one schema.

Relation schema defines the design and structure of the relation or table in the database. It is the way of representation of relation states in such a way that every relation database state fulfills the integrity constraints set (Like Primary key, Foreign Key, Not null, Unique

constraints) on a relational schema.

It consists of the relation name, set of attributes/field names/column names.

Components of a Relation Schema

□ **Relation Name:** Name of the table that is stored in the database.

It should be unique and related to the data that is stored in the table. For example- The name of the table can be Employee store the data of the employee.

□ **Attributes Name:** Attributes specify the name of each column within the table. Each attribute has a specific data type.

□ **Domains:** The set of possible values for each attribute. It specifies the type of data that can be stored in each column or attribute, such as integer, string, or date.

□ **Primary Key:** The primary key is the key that uniquely identifies each tuple. It should be unique and not be null.

□ **Foreign Key:** The foreign key is the key that is used to connect two tables. It refers to the primary key of another table.

□ **Constraints:** Rules that ensure the integrity and validity of the data. Common constraints include NOT NULL, UNIQUE, CHECK, and DEFAULT.

Example of Relation Schema

There is a student named EMMA, he is pursuing B.SC, in the 4th year, and belongs to the computer science department (department no. 1) and has roll number 1601347. If we want to represent this using databases we would have to create a student table with name, sex, degree, year, department, department number, roll number, and proctor (adviser) as the attributes.

Student Table

NAME REG DEPT SEX AGE LEVEL PROGRAM

NO TYPE

EMMA 1156 CSC M 21 400 REGULAR

CONCEPTUAL SCHEMA IN DATABASE

A conceptual schema is a map of concepts and their relationships used for databases. This describes the semantics of an organization and represents a series of assertions about its nature. Specifically, it describes the things of significance to an organization (entity classes), about which it is inclined to collect information, and their characteristics (attributes) and the associations between pairs of those things of significance (relationships). It describes the structure of the whole database for a group of users. The conceptual model is

also known as the data model that can be used to describe the conceptual schema when a database system is implemented. It hides the internal details of physical storage and targets the description of entities, datatypes, relationships and constraints.

Because a conceptual schema represents the semantics of an organization, and not a database design, it may exist on various levels of abstraction.

A conceptual schema is a high-level description of information needs that are to be addressed by a certain database or information system. It is typically the first phase for designing of a database schema from scratch. By “high-level” we mean here that it (1) attempts to abstract from which exact type of DBMS we are designing for and (2) uses a notation that is more likely to be understandable for the stakeholders of the database or information system that is being designed.

The most common forms for conceptual schemas are Entity-Relationship diagrams and UML Class diagrams.

ENTITY RELATIONSHIPS DIAGRAM

The Entity Relationship Diagram explains the relationship among the entities present in the database. ER models are used to model real-world objects like a person, a car, or a company and the relation between these real-world objects. In short, the ER Diagram is the structural format of the database.

THE IMPORTANCE OF ENTITY RELATIONSHIP DIAGRAM

ER diagrams are used to represent the E-R model in a database, which makes them easy to convert into relations (tables).

ER diagrams provide the purpose of real-world modeling of objects which makes them intently useful.

ER diagrams require no technical knowledge and no hardware support.

These diagrams are very easy to understand and easy to create even for a naive user.

It gives a standard solution for visualizing the data logically.

Symbols Used in ER Model

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

- Rectangles: Rectangles represent Entities in the ER Model.
- Ellipses: Ellipses represent Attributes in the ER Model.
- Diamond: Diamonds represent Relationships among Entities.
- Lines: Lines represent attributes to entities and entity sets with other relationship types.
- Double Ellipse: Double Ellipses represent Multi-Valued

Attributes.

□ Double Rectangle: Double Rectangle represents a Weak Entity.

DATABASE QUERY LANGUAGES

A query language, also known as data query language or database query language (DQL), is a computer language used to make queries in databases and information systems. In database systems, query languages rely on strict theory to retrieve information. A well-known example is the Structured Query Language (SQL).

STRUCTURED QUERY LANGUAGE (SQL)

Structured query language (SQL) is a standard language for database creation and manipulation. MySQL is a relational database program that uses SQL queries. While SQL commands are defined by international standards, the MySQL software undergoes continual upgrades and improvements.

Structured Query Language (SQL) is a standardized programming language that is used to manage relational databases and perform various operations on the data in them. Initially created in the 1970s, SQL is regularly used not only by database administrators but also by developers writing data integration scripts and data analysts looking to set up and run analytical queries.

SQL is used for the following:

- Modifying database table and index structures.
- Adding, updating and deleting rows of data.
- Retrieving subsets of information from within relational database management systems (RDBMSes). This information can be used for transaction processing, analytics applications and other applications that require communicating with a relational database.

A table is the most basic unit of a database and consists of rows and columns of data. A single table holds records, and each record is stored in a row of the table. Tables are the most used type of database objects or structures that hold or reference data in a relational database.

The SQL query language can be used for relational or non-relational databases, but it offers advantages for relational databases.

Why is SQL important?

SQL is a great language to learn because it's the primary database language used for data processing tasks and is used across various industries.

The following reasons highlight the importance of SQL:

□ Backbone of the data industry. SQL is considered the backbone of the data industry. It's widely used by data-centric professionals including data analysts, data scientists, business analysts and database developers.

□ Universal language. SQL is a universal language that is transferable to other disciplines and languages. Learning SQL can help one understand the workings of other languages such as Python and Java. It also makes collaboration easy, as it has a large supportive community.

□ In-demand skill. SQL knowledge is one of the most in-demand skills in the field of data science. It appears in a significant percentage of data science job postings, making it a prized skill for professionals in this field.

□ Data manipulation. SQL is well-suited for data manipulation. It enables users to easily test and manipulate data, making it efficient for tasks such as filtering, sorting and aggregating data.

□ Rapid query processing. SQL enables rapid query processing, enabling users to retrieve, manipulate or store data quickly and efficiently. However, optimizing queries for rapid processing involves a combination of proper indexing, query optimization and database design considerations.

□ Security features. SQL provides various security features such as authentication, access control, audit trails and encryption, making it easy to manage permissions and ensure the security of data.

□ Commonality and compatibility. SQL is widely used in various IT systems and is compatible with multiple other languages. Its commonality benefits beginners in the profession, as they are likely to use SQL throughout their careers. It also contributes to ease of application and improves the production and efficiency of businesses.

□ Scalability. SQL is suitable for organizations of any size. It can easily scale up to accommodate future growth, making it a versatile choice for small and large businesses alike.

□ Open source and community support. SQL is an open source language with a vibrant community of developers that regularly provide updates and troubleshooting assistance to SQL users.

□ Cost-effective. Due to its open source nature, SQL is more cost-effective than proprietary solutions, making it ideal for organizations with budget constraints.

SQL standard and proprietary extensions

An official SQL standard was adopted by the American National Standards Institute (ANSI) in 1986, with the International Organization for Standardization (ISO) adopting the standard in 1987. New versions of the SQL standard are published every few years, the most recent in 2022.

ISO/IEC 9075 is the ISO SQL standard developed jointly by ISO and the International Electro Technical Commission. The standard way of referring to an ISO standard version is to use the standards organizations -- ISO/IEC -- followed by the ISO standard number, a colon and the publication year. The current ISO standard for SQL is ISO/IEC 9075:2022.

Both proprietary and open source RDBMSes built around SQL are available for use by organizations. SQL-compliant database server products include the following:

- Microsoft SQL Server.
- Oracle Database.
- IBM Db2.
- SAP HANA.
- SAP Adaptive Server.
- Oracle MySQL.
- Open source PostgreSQL.
- Azure SQL Database.
- Google Cloud SQL.

Some versions of SQL include proprietary extensions to the standard language for procedural programming and other functions. For example, Microsoft offers a set of extensions called Transact-SQL, while Oracle's extended version of the standard is Procedural Language for SQL. Commercial vendors offer proprietary extensions to differentiate their product offerings by giving customers additional features and functions. As a result, the different variants of extended SQL offered by vendors are not fully compatible with one another.

SQL commands and syntax

SQL is, fundamentally, a programming language designed for accessing, modifying and extracting information from relational databases. As a programming language, SQL has commands and a syntax for issuing those commands.

SQL commands are divided into several different types, including the following:

- Data Definition Language (DDL) commands are also called data definition commands because they are used to define data

tables.

□ Data Manipulation Language (DML) commands are used to manipulate data in existing tables by adding, changing or removing data. Unlike DDL commands that define how data is stored, DML commands operate in the tables defined with DDL commands.

□ Data Query Language consists of just one command, SELECT, used to get specific data from tables. This command is sometimes grouped with the DML commands.

□ Data Control Language commands are used to grant or revoke user access privileges.

□ Transaction Control Language commands are used to change the state of some data -- for example, to COMMIT transaction changes or to ROLLBACK transaction changes.

History of SQL

With roots going back to the early 1970s, SQL continues to make key milestones as one of the most successful ideas in computing history. The following is a timeline of the SQL language and its key milestones:

□ 1970. "A Relational Model of Data for Large Shared Data Banks" by E.F. Codd is published in Communications of the ACM, laying the basis for RDBMSes.

□ 1974. IBM researchers publish an article introducing Structured Query Language, first called SEQUEL or Structured English Query Language. The name was changed for trademark purposes.

□ 1977. Relational Software Inc., the company that eventually became Oracle, begins building a commercial RDBMS.

□ 1979. Oracle ships the first commercially available RDBMS for Digital Equipment Corp.'s minicomputer systems.

□ 1982. IBM ships SQL/Data System, a SQL RDBMS for IBM mainframes.

□ 1985. IBM ships Database 2, a SQL RDBMS for IBM's Multiple Virtual Storage mainframe operating system.

□ 1986. An ANSI committee and then ISO adopt SQL as a standard.

□ 1989. The first revision of the ISO SQL standard, SQL-89, is published.

□ 1992. The first major revision of the ISO SQL standard, SQL-92, is published.

□ 1999. The first version to be named in accordance with ISO naming standards, ISO/IEC SQL:1999, adds programming functionality and support for Java.

□ 2003. ISO/IEC SQL:2003 adds support for a predefined data type

for Extensible Markup Language (XML) objects.

- 2006. ISO/IEC SQL:2006 expands XML-related functionality.
- 2008. ISO/IEC SQL:2008 adds support for partitioned JOINS, a method for linking two or more tables that treats the joined tables as a single table.
- 2011. ISO/IEC SQL:2011 improves support for relational databases containing time-related data.
- 2016. ISO/IEC SQL:2016 adds optional new features, including JavaScript Object Notation-related changes, support for polymorphic table functions and row pattern matching.
- 2017. ISO/IEC SQL:2017. SQL Server 2017 provides choices of development languages, data types, on-premises cloud and OS systems by adding the SQL Server power to Linux, Windows and Linux-based containers.
- 2019. ISO/IEC SQL:2019. SQL Server 2019 introduces Big Data Clusters for SQL and also offers additional options and improvements for the SQL Server database engine, SQL Server Analysis Services, SQL Server Machine Learning Services, SQL Server on Linux and SQL Server Master Data Services.
- 2022. ISO/IEC SQL:2022. SQL Server 2022 continues to build on the previous versions and their capabilities with additional features such as Azure Synapse Link, which is a real-time analytics feature, as well as enhanced security through features such as Microsoft Defender.

CONCEPT OF FUNCTIONAL DEPENDENCIES

A functional dependency means that when the value of one attribute is known, one can always determine the value of another attribute. A Functional Dependency in DBMS is a fundamental concept that describes the relationship between attributes (columns) in a table. It shows how the values in one or more attributes determine the value in another. In layperson's terms, it describes how data in one column or set of columns can relate to data in another column. It helps to maintain the quality of the data in DBMS.

Functional Dependency is represented in the form of an equation. Here, you have a set of attributes (A, B, C, etc.) and an arrow (->) denoting the Dependency. For example, if we have a table of employee data with columns "Employee ID," "First Name," and "Last Name," we can express a functional dependency like this:

Employee ID -> First Name, Last Name.

How to Denote a Functional Dependency in DBMS?

In DBMS, you denote functional dependencies using a notation. It contains two main components: the left-hand side (LHS) and the right-hand side (RHS) of an arrow ($\rightarrow$).

For example, if we have a table with attributes "A," "B," and "C," and attribute "A" determines the values of attributes "B" and "C," you would denote it as

$A \rightarrow B, C$

This notation indicates that the value(s) in attribute "A" determines the value(s) in attributes "B" and "C." In other words, if you know the value of "A," you can determine the values of "B" and "C."

Types of Functional Dependencies in DBMS

Here are some of the important types of Functional Dependency In DBMS

Trivial Functional Dependency

A trivial functional dependency in DBMS occurs when an attribute or set of attributes (columns) on the left-hand side (LHS) of a functional dependency arrow ($\rightarrow$) already determines the attributes on the right-hand side (RHS) without any extra information.

Suppose we have a table of students with attributes "Student ID" and "Student Name." In this case, if we state the functional dependency as

$\text{Student ID} \rightarrow \text{Student Name},$

This is a trivial functional dependency. Because within a single "Student ID," there can be only one corresponding "Student Name." In other words, the value of "Student ID" determines the value of "Student Name" without any more information or conditions.

Non-trivial Functional Dependency

A non-trivial functional dependency is a specific type of dependency between attributes (columns) in a table. Here, the relationship is not obvious or trivial. It conveys meaningful information about how the values in one set of attributes determine the values in another.

To illustrate this concept, let's consider an example with a table of student data:

Student Id Student Name Student DOB Class

101 Emma 1995-05-15 10A

102 Udo 2000-03-20 10B

103 Sonia 1999-07-10 10A

104 David 2000-01-05 10B

We want to express a functional dependency based on the student's

birthdate (StudentDOB) and class (Class). A non-trivial functional dependency in this case would be

StudentDOB, Class $\rightarrow$ StudentName.

This functional dependency means that given a combination of a student's date of birth and class, you can uniquely determine their name. It's non-trivial because it provides valuable information about the relationship between attributes in the table.

Multivalued Functional Dependency

A multivalued functional dependency in a database occurs when one or more attributes determine multiple unrelated sets of values in another attribute. It shows that changes in the determining attributes can lead to various combinations of values in the dependent attribute, indicating complex relationships within the data.

Here's a simple example:

Student_ID Student_Name Courses_Enrolled

1 Alice {Math, English}

2 Bob {Science, History}

3 Carol {Math, Science}

In this case, the multivalued dependency in DBMS holds:

- Alice is Student 1 enrolled in {Math, English}.
- Bob is Student 2 enrolled in {Science, History}.
- Carol is Student 3 enrolled in {Math, Science}.

Transitive Functional Dependency

A transitive functional dependency in DBMS is a relational database table's relationship between attributes (columns). It occurs when one attribute's value determines another's value through an intermediary (a third) attribute.

Example:

Consider a database table called "Student_Info" with the following attributes:

Student_ID (unique identifier for each student),

Student_Name,

Student_Address

Student_City.

In this example, we can assume that Student_Address depends on Student_City, and Student_City depends on Student_ID. This creates a transitive dependency in DBMS, where Student_ID indirectly determines Student_Address.

RULES OF FUNCTIONAL DEPENDENCY

There are three functional dependency rules, known as Armstrong's axioms, you use to infer functional dependency within relational databases.

Reflexive rule: This rule states that if X is a set of attributes and Y is its subset, then X holds a value of Y .

Augmentation rule: This rule adds attributes, though they rarely change the basic dependencies and states if $X \rightarrow Y$ holds and C is an attribute set, then $XC \rightarrow YC$ also holds.

Transitivity rule: Closely aligned with algebra's transitive rule, this rule states if $X \rightarrow Y$ holds and $Y \rightarrow Z$ holds, then $X \rightarrow Z$ must also hold. With this rule, $X \rightarrow Y$ is the functional dependent that determines Y .

ADVANTAGES OF FUNCTIONAL DEPENDENCY IN DBMS

Here are the key advantages of functional dependencies in DBMS:

Data Integrity

Functional dependencies help ensure data integrity in a database. By defining rules that govern the relationships between attributes, DBMS can enforce constraints to prevent inconsistent or incorrect data from being entered into the database.

Normalization

Functional dependencies are used in database normalization. By identifying and applying functional dependencies, database designers can break down large tables into smaller, more manageable ones, reducing data duplication and anomalies.

Efficient Storage

Normalizing a database through functional dependencies can lead to more efficient data storage. Smaller, normalized tables need less storage space, which can be especially beneficial for large databases.

Ease of Maintenance

Databases that adhere to functional dependencies are easier to maintain. When changes are required in the database structure or schema, the impact of those changes is more localized, reducing the risk of introducing errors or inconsistencies in the data.

Query Optimization

With the help of functional dependencies, we are able to decide the connectivity between the tables and the necessary attributes need to be projected to retrieve the required data from the tables. This helps in query optimization and improves performance.

Data Quality Improvement

Functional dependencies ensure that the data in the database to be

accurate, complete and updated. This helps to improve the overall quality of the data, as well as it eliminates errors and inaccuracies that might occur during data analysis and decision making, thus functional dependency helps in improving the quality of data in database.

DISADVANTAGES OF FUNCTIONAL DEPENDENCY IN DBMS

Overly restrictive functional dependencies can result in slow query performance or data inconsistencies, as data that should be related may not be properly linked.

Functional dependencies do not take into account the semantic meaning of data, and may not always reflect the true relationships between data elements.

Conclusion

Functional dependency in DBMS is the blueprint for organizing and ensuring data accuracy. Identifying these relationships simplifies database design, reduces redundancy, and enhances query efficiency.

They are the building blocks for maintaining structured and reliable databases, essential for modern data management and information retrieval. All the functional dependencies are useful tools to make the Database Management System more efficient for the user.

TRANSACTION PROCESSING IN DBMS.

DBMS transaction processing is followed as a set of operations accomplished on the data records stored in the server database that results in an alteration in the database state. This DBMS transaction is completed through the "ACID" principles, which include: Atomicity, Consistency, Isolation and Durability. When the data of users is stored in a database, that data needs to be accessed and modified from time to time. This task should be performed with a specified set of rules and in a systematic way to maintain the consistency and integrity of the data present in a database. In DBMS, this task is called a transaction. It is similar to a bank transaction, where the user requests to withdraw some amount of money from his account. Subsequently, several operations take place such as fetching the user's balance from the database, subtracting the desired amount from it, and updating the user's account balance. This series of operations can be called a transaction. Transactions are very common in DBMS.

Transaction in Database Management Systems (DBMS) can be defined as a set of logically related operations. It is the result of a request made by the user to access the contents of the database and perform

operations on it. It consists of various operations and has various states in its completion journey. It also has some specific properties that must be followed to keep the database consistent.

Operations of Transaction

A user can make different types of requests to access and modify the contents of a database. So, we have different types of operations relating to a transaction. They are discussed as follows:

i) Read(X)

A read operation is used to read the value of X from the database and store it in a buffer in the main memory for further actions such as displaying that value. Such an operation is performed when a user wishes just to see any content of the database and not make any changes to it. For example, when a user wants to check his/her account's balance, a read operation would be performed on user's account balance from the database.

ii) Write(X)

A write operation is used to write the value to the database from the buffer in the main memory. For a write operation to be performed, first a read operation is performed to bring its value in buffer, and then some changes are made to it, e.g. some set of arithmetic operations are performed on it according to the user's request, then to store the modified value back in the database, a write operation is performed. For example, when a user requests to withdraw some money from his account, his account balance is fetched from the database using a read operation, then the amount to be deducted from the account is subtracted from this value, and then the obtained value is stored back in the database using a write operation.

iii) Commit

This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction. To ensure that further operations of any other transaction are performed only after work of the current transaction is done, a commit operation is performed to the changes made by a transaction permanently to the database.

iv) Rollback

This operation is performed to bring the database to the last saved state when any transaction is interrupted in between due to any power, hardware, or software failure. In simple words, it can be said that a rollback operation does undo the operations of transactions

that were performed before its interruption to achieve a safe state of the database and avoid any kind of ambiguity or inconsistency.

TRANSACTION SCHEDULES

When multiple transaction requests are made at the same time, we need to decide their order of execution. Thus, a transaction schedule can be defined as a chronological order of execution of multiple transactions.

There are broadly two types of transaction schedules discussed as follows,

i) Serial Schedule

In this kind of schedule, when multiple transactions are to be executed, they are executed serially, i.e. at one time only one transaction is executed while others wait for the execution of the current transaction to be completed. This ensures consistency in the database as transactions do not execute simultaneously. But, it increases the waiting time of the transactions in the queue, which in turn lowers the throughput of the system, i.e. number of transactions executed per time. To improve the throughput of the system, another kind of schedule are used which has some more strict rules which help the database to remain consistent even when transactions execute simultaneously.

ii) Non-Serial Schedule

To reduce the waiting time of transactions in the waiting queue and improve the system efficiency, we use non-serial schedules which allow multiple transactions to start before a transaction is completely executed. This may sometimes result in inconsistency and errors in database operation. So, these errors are handled with specific algorithms to maintain the consistency of the database and improve CPU throughput as well. Serial Schedules are also sometimes referred to as parallel schedules as transactions execute in parallel in this kind of schedules.

TRANSACTION STATES IN DBMS.

i) Active state

It is the first stage of any transaction when it has begun to execute. The execution of the transaction takes place in this state. Operations such as insertion, deletion, or updating are performed during this state. During this state, the data records are under manipulation and they are not saved to the database, rather they remain somewhere in a buffer in the main memory.

ii) Partially Committed

This state of transaction is achieved when it has completed most of

the operations and is executing its final operation. It can be a signal to the commit operation, as after the final operation of the transaction completes its execution, the data has to be saved to the database through the commit operation. If some kind of error occurs during this state, the transaction goes into a failed state, else it goes into the Committed state.

iii) Committed

This state of transaction is achieved when all the transaction-related operations have been executed successfully along with the Commit operation, i.e. data is saved into the database after the required manipulations in this state. This marks the successful completion of a transaction.

iv) Failed

If any of the transaction-related operations cause an error during the active or partially committed state, further execution of the transaction is stopped and it is brought into a failed state. Here, the database recovery system makes sure that the database is in a consistent state.

v) Aborted

If the error is not resolved in the failed state, then the transaction is aborted and a rollback operation is performed to bring database to the the last saved consistent state. When the transaction is aborted, the database recovery module either restarts the transaction or kills it.

The illustration below shows the various states that a transaction may encounter in its completion journey.

PROPERTIES OF TRANSACTION

As transactions deal with accessing and modifying the contents of the database, they must have some basic properties which help maintain the consistency and integrity of the database before and after the transaction. Transactions follow 4 properties, namely, Atomicity, Consistency, Isolation, and Durability. Generally, these are referred to as ACID properties of transactions in DBMS. ACID is the acronym used for transaction properties. A brief description of each property of the transaction is as follows.

1) Atomicity

This property ensures that either all operations of a transaction are executed or it is aborted. In any case, a transaction can never be completed partially. Each transaction is treated as a single unit (like an atom). Atomicity is achieved through commit and rollback operations, i.e. changes are made to the database only if all

operations related to a transaction are completed, and if it gets interrupted, any changes made are rolled back using rollback operation to bring the database to its last saved state.

ii) Consistency

This property of a transaction keeps the database consistent before and after a transaction is completed. Execution of any transaction must ensure that after its execution, the database is either in its prior stable state or a new stable state. In other words, the result of a transaction should be the transformation of a database from one consistent state to another consistent state. Consistency, here means, that the changes made in the database are a result of logical operations only which the user desired to perform and there is not any ambiguity.

iii) Isolation

This property states that two transactions must not interfere with each other, i.e. if some data is used by a transaction for its execution, then any other transaction cannot concurrently access that data until the first transaction has completed. It ensures that the integrity of the database is maintained and we don't get any ambiguous values. Thus, any two transactions are isolated from each other. This property is enforced by the concurrency control subsystem of DBMS.

iv) Durability

This property ensures that the changes made to the database after a transaction is completely executed, are durable. It indicates that permanent changes are made by the successful execution of a transaction. In the event of any system failures or crashes, the consistent state achieved after the completion of a transaction remains intact. The recovery subsystem of DBMS is responsible for enforcing this property.

DISTRIBUTED DATABASES

A distributed database is a database that runs and stores data across multiple computers, as opposed to doing everything on a single machine. Typically, distributed databases operate on two or more interconnected servers on a computer network.

A distributed database system is located on various sites that don't share physical components. This may be required when a particular database needs to be accessed by various users globally. It needs to be managed such that for the users it looks like one single database.

Examples of distributed databases

Though there are many distributed databases to choose from, some

examples of distributed databases include Apache Ignite, Apache Cassandra, Apache HBase, Couchbase Server, Amazon SimpleDB, Clusterpoint, and FoundationDB.

TYPES OF DISTRIBUTED DATABASE

1. Homogeneous Database:

In a homogeneous database, all different sites store database identically. The operating system, database management system, and the data structures used – all are the same at all sites. Hence, they're easy to manage.

2. Heterogeneous Database:

In a heterogeneous distributed database, different sites can use different schema and software that can lead to problems in query processing and transactions. Also, a particular site might be completely unaware of the other sites. Different computers may use a different operating system, different database application. They may even use different data models for the database. Hence, translations are required for different sites to communicate.

DISTRIBUTED DATA STORAGE:

There are 2 ways in which data can be stored on different sites. These are:

1. Replication –

In this approach, the entire relationship is stored redundantly at 2 or more sites. If the entire database is available at all sites, it is a fully redundant database. Hence, in replication, systems maintain copies of data.

This is advantageous as it increases the availability of data at different sites. Also, now query requests can be processed in parallel.

However, it has certain disadvantages as well. Data needs to be constantly updated. Any change made at one site needs to be recorded at every site that relation is stored or else it may lead to inconsistency. This is a lot of overhead. Also, concurrency control becomes way more complex as concurrent access now needs to be checked over a number of sites.

2. Fragmentation –

In this approach, the relations are fragmented (i.e., they're divided into smaller parts) and each of the fragments is stored in different sites where they're required. It must be made sure that the fragments are such that they can be used to reconstruct the original relation (i.e., there isn't any loss of data).

Fragmentation is advantageous as it doesn't create copies of data,

consistency is not a problem.

FRAGMENTATION OF RELATIONS CAN BE DONE IN TWO WAYS:

Horizontal fragmentation – Splitting by rows –

The relation is fragmented into groups of tuples so that each tuple is assigned to at least one fragment.

Vertical fragmentation – Splitting by columns –

The schema of the relation is divided into smaller schemas. Each fragment must contain a common candidate key so as to ensure a lossless join.

In certain cases, an approach that is hybrid of fragmentation and replication is used.

APPLICATIONS OF DISTRIBUTED DATABASE:

It is used in Corporate Management Information System.

It is used in multimedia applications.

Used in Military's control system, Hotel chains etc.

It is also used in manufacturing control system.

THERE ARE SEVERAL DIFFERENT ARCHITECTURES FOR DISTRIBUTED DATABASE SYSTEMS, INCLUDING:

Client-server architecture: In this architecture, clients connect to a central server, which manages the distributed database system. The server is responsible for coordinating transactions, managing data storage, and providing access control.

Peer-to-peer architecture: In this architecture, each site in the distributed database system is connected to all other sites. Each site is responsible for managing its own data and coordinating transactions with other sites.

Federated architecture: In this architecture, each site in the distributed database system maintains its own independent database, but the databases are integrated through a middleware layer that provides a common interface for accessing and querying the data.

ADVANTAGES OF DISTRIBUTED DATABASE SYSTEM:

- 1) There is fast data processing as several sites participate in request processing.
- 2) Reliability and availability of this system is high.
- 3) It possess reduced operating cost.
- 4) It is easier to expand the system by adding more sites.
- 5) It has improved sharing ability and local autonomy.

Disadvantages of Distributed Database System:

- 1) The system becomes complex to manage and control.

- 2) The security issues must be carefully managed.
- 3) The system requires deadlock handling during the transaction processing otherwise the entire system may be in inconsistent state.
- 4) There is need of some standardization for processing of distributed database system.